

I/O Systems

e9

9.1 INTRODUCTION

Input/Output (I/O) systems are used to connect a computer with external devices called peripherals. In a personal computer, the devices typically include keyboards, monitors, printers, and wireless networks. In embedded systems, devices could include a toaster's heating element, a doll's speech synthesizer, an engine's fuel injector, a satellite's solar panel positioning motors, and so forth. A processor accesses an I/O device using the address and data busses in the same way that it accesses memory.

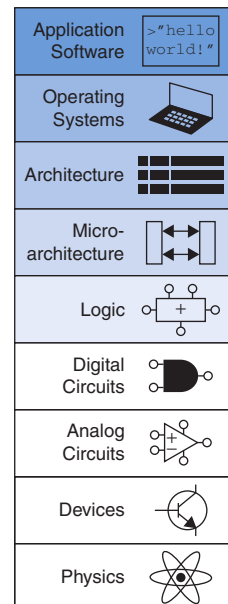
This chapter provides concrete examples of I/O devices. [Section 9.2](#) shows the basic principles of interfacing an I/O device to a processor and accessing it from a program. [Section 9.3](#) examines I/O in the context of embedded systems, showing how to use an ARM-based Raspberry Pi single-board computer to access on-board peripherals including general-purpose, serial, and analog I/O as well as timers. [Section 9.4](#) gives examples of interfacing with other common devices such as character LCDs, VGA monitors, Bluetooth radios, and motors. [Section 9.5](#) describes bus interfaces and illustrates the popular AHB-Lite bus. [Section 9.6](#) surveys the major I/O systems used in PCs.

9.2 MEMORY-MAPPED I/O

Recall from [Section 6.5.1](#) that a portion of the address space is dedicated to I/O devices rather than memory. For example, suppose that physical addresses in the range 0x20000000 to 0x20FFFFFF are used for I/O. Each I/O device is assigned one or more memory addresses in this range. A store to the specified address sends data to the device. A load receives data from the device. This method of communicating with I/O devices is called *memory-mapped I/O*.

In a system with memory-mapped I/O, a load or store may access either memory or an I/O device. [Figure e9.1](#) shows the hardware needed to support two memory-mapped I/O devices. An address decoder determines which device communicates with the processor. It uses the *Address* and *MemWrite* signals to generate control signals for the rest of the hardware. The *ReadData* multiplexer selects between memory and the various I/O devices. Write-enabled registers hold the values written to the I/O devices.

9.1	Introduction
9.2	Memory-Mapped I/O
9.3	Embedded I/O Systems
9.4	Other Microcontroller Peripherals
9.5	Bus Interfaces
9.6	PC I/O Systems
9.7	Summary



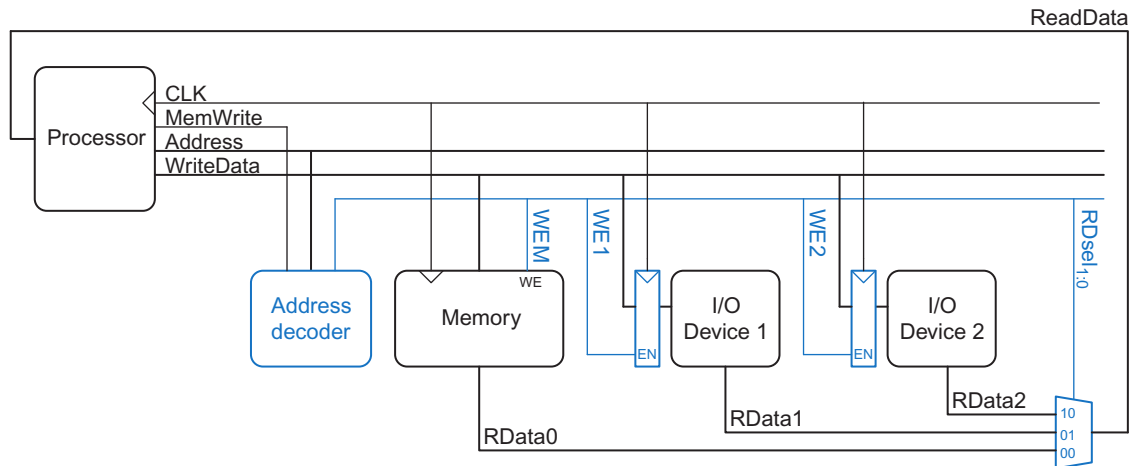


Figure e9.1 Support hardware for memory-mapped I/O

Example e9.1 COMMUNICATING WITH I/O DEVICES

Suppose I/O Device 1 in Figure e9.1 is assigned the memory address 0x20001000. Show the ARM assembly code for writing the value 7 to I/O Device 1 and for reading the output value from I/O Device 1.

Solution: The following assembly code writes the value 7 to I/O Device 1.

```
MOV R1, #7
LDR R2, =ioadr
STR R1, [R2]
ioadr DCD 0x20001000
```

The address decoder asserts *WE1* because the address is 0x20001000 and *MemWrite* is TRUE. The value on the *WriteData* bus, 7, is written into the register connected to the input pins of I/O Device 1.

To read from I/O Device 1, the processor executes the following assembly code.

```
LDR R1, [R2]
```

The address decoder sets *RDsel_{1,0}* to 01, because it detects the address 0x20001000 and *MemWrite* is FALSE. The output of I/O Device 1 passes through the multiplexer onto the *ReadData* bus and is loaded into R1 in the processor.

The addresses associated with I/O devices are often called *I/O registers* because they may correspond with physical registers in the I/O device like those shown in [Figure e9.1](#).

Software that communicates with an I/O device is called a *device driver*. You have probably downloaded or installed device drivers for your printer or other I/O device. Writing a device driver requires detailed knowledge about the I/O device hardware including the addresses and behavior of the memory-mapped I/O registers. Other programs call functions in the device driver to access the device without having to understand the low-level device hardware.

9.3 EMBEDDED I/O SYSTEMS

Embedded systems use a processor to control interactions with the physical environment. They are typically built around microcontroller units (MCUs) which combine a microprocessor with a set of easy-to-use peripherals such as general-purpose digital and analog I/O pins, serial ports, timers, etc. Microcontrollers are generally inexpensive and are designed to minimize system cost and size by integrating most of the necessary components onto a single chip. Most are smaller and lighter than a dime, consume milliwatts of power, and range in cost from a few dimes up to several dollars. Microcontrollers are classified by the size of data that they operate upon. 8-bit microcontrollers are the smallest and least expensive, while 32-bit microcontrollers provide more memory and higher performance.

For the sake of concreteness, this section will illustrate embedded system I/O in the context of a real system. Specifically, we will focus on the popular and inexpensive Raspberry Pi board, which contains a Broadcom BCM2835 system-on-chip (SoC) with a 700 MHz 32-bit ARM1176JZ-F processor implementing the ARMv6 instruction set. The principles in each subsection will be followed by specific examples that run on the Pi. All of the examples have been tested on a Pi running NOOBS Raspbian Linux in 2014.

[Figure e9.2](#) shows a photograph of a Raspberry Pi Model B+ board, which is a complete Linux computer about the size of a credit card that sells for \$35. The Pi draws up to 1 A from a 5 V USB power supply. It has 512 MB of onboard RAM and an SD card socket for a memory card that contains the operating system and user files. Connectors provide video and audio output, USB ports for a mouse and keyboard, and an Ethernet (Local Area Network) port, along with 40 general-purpose I/O (GPIO) pins that are the main subject of this chapter.

While the BCM2835 SoC has many capabilities beyond those in a typical inexpensive microcontroller, the general-purpose I/O is very similar. This chapter begins by describing the BCM2835 on the Raspberry Pi and describing a device driver for memory-mapped I/O. The remainder of this chapter will illustrate how embedded systems perform general-purpose digital, analog, and serial I/O. Timers are also commonly used to generate or measure precise time intervals.

Some architectures, notably x86, use specialized instructions instead of memory-mapped I/O to communicate with I/O devices. These instructions are of the following form, where *device1* and *device2* are the unique IDs of the peripheral device:

```
LDRIO R1, device1
STRIO R2, device2
```

This type of communication with I/O devices is called *programmed I/O*.

Approximately \$19B of microcontrollers were sold in 2014, and the market is forecast to reach \$27B by 2020. The average price of a microcontroller is less than \$1, and an 8-bit microcontroller can be integrated on a system-on-chip for less than a penny. Microcontrollers have become ubiquitous and nearly invisible, with an estimated 150 in each home and 50 in each automobile in 2010. The 8051 is a classic 8-bit microcontroller originally developed by Intel in 1980 and now sold by a host of manufacturers. Microchip's PIC16 and PIC18-series are 8-bit market leaders. The Atmel AVR series of microcontrollers has been popularized among hobbyists as the brain of the Arduino platform. Among 32-bit microcontrollers, Renesas leads the overall market. Freescale, Samsung, Texas Instruments, and Infineon are other major microcontroller manufacturers. ARM processors are found in nearly all smart phones and tablets today and are usually part of a system-on-chip containing the multi-core applications processor, a graphics processing unit, and extensive I/O.

The Raspberry Pi was developed in 2011-12 by the nonprofit Raspberry Pi Foundation in the UK to promote teaching computer science. Built around the brain of an inexpensive smartphone, the computer has become wildly popular, selling more than 3 million units by 2014. The name pays homage to early home computers including Apple, Apricot, and Tangerine. Pi is derived from Python, a programming language often used in education. Documentation and purchasing information can be found at

raspberrypi.org

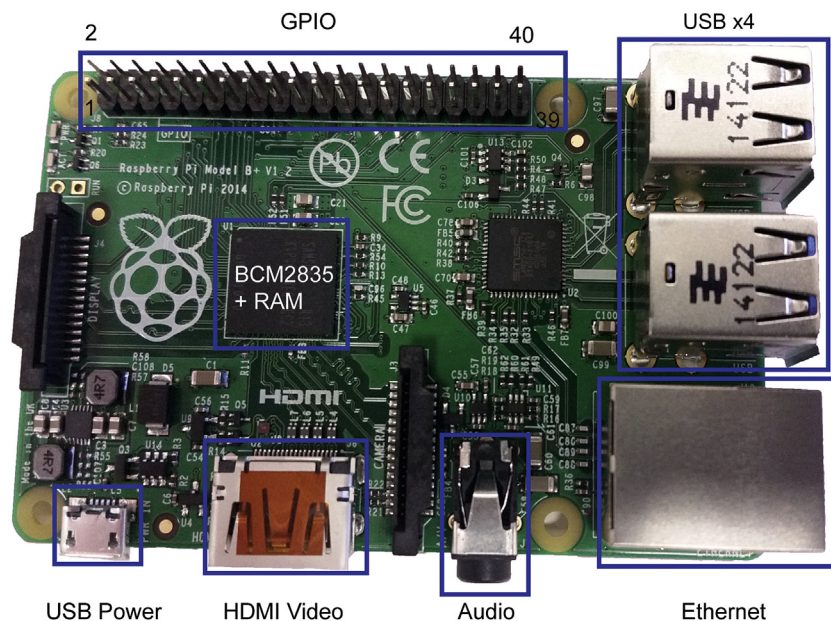


Figure e9.2 Raspberry Pi Model B+

Eben Upton (1978-) is the architect of the Raspberry Pi and a founder of the Raspberry Pi Foundation. He received his Bachelor's and Ph.D. from the University of Cambridge before joining Broadcom Corporation as a chip architect.



(Photograph © Eben Upton. Reproduced with permission.)

9.3.1 BCM2835 System-on-Chip

The BCM2835 SoC is a powerful yet inexpensive chip designed by Broadcom for mobile devices and other multimedia applications. The SoC includes an ARM microprocessor known as the *applications processor*, a VideoCore processor for graphics, video, and cameras, and many I/O peripherals. The BCM2835 is packaged in a plastic ball grid array with tiny solder balls underneath; it is best soldered by a robot that aligns the package to matching copper pads on a printed circuit board and applies heat. Broadcom does not publish a complete datasheet, but an abbreviated datasheet is available on the Raspberry Pi site describing how to access peripherals from the ARM processor. The datasheet describes many features and I/O registers that are omitted in this chapter for simplicity.

www.raspberrypi.org/documentation/hardware/

Figure e9.3 shows a simplified schematic of the Raspberry Pi model board. The board receives 5 V power from a USB power supply and regulators produce 3.3, 2.5, and 1.8 V levels for I/O, analog, and

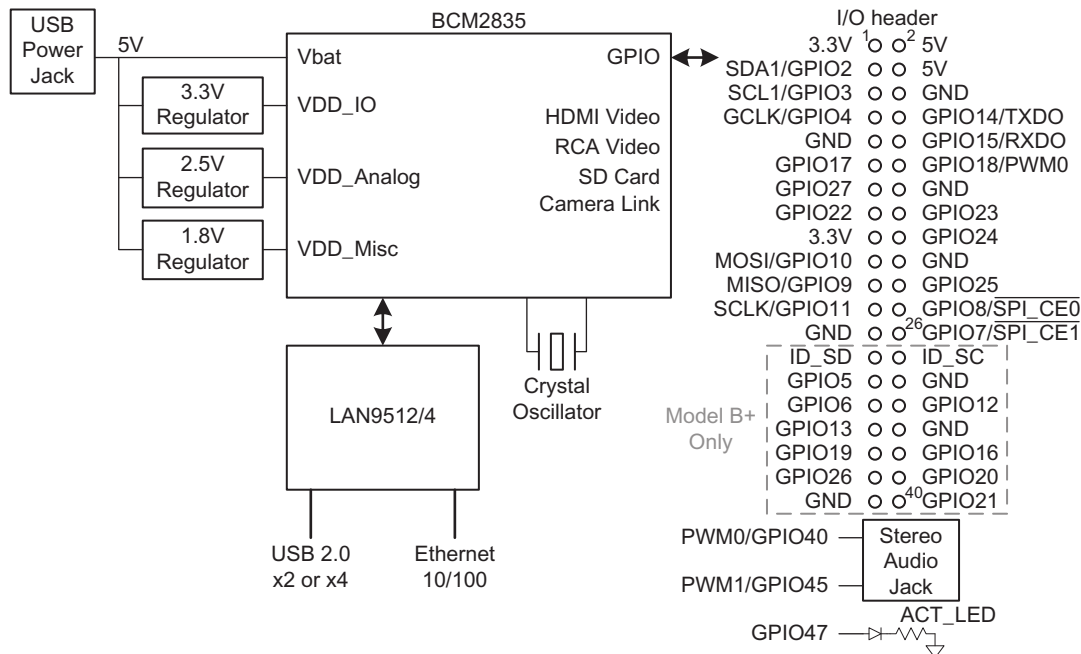


Figure e9.3 Raspberry Pi I/O schematic

miscellaneous functions. The BCM2835 also has an internal switching regulator that produces a variable lower voltage for the power-efficient SoC. The BCM2835 connects to a USB/Ethernet controller and also directly outputs video. It also has 54 configurable I/O signals but, for space reasons, only a fraction of these are accessible to the user via header pins. The header also provides 3.3 and 5 V and ground to conveniently power small devices attached to the Pi, but the maximum total current is 50 mA from 3.3 V and ~300 mA from 5 V. The model B and B+ are similar, but B+ boosts the number of I/O header pins from 26 to 40 and the number of USB ports from 2 to 4. Various cables including the Adafruit Pi Cobbler are available to connect these header pins to a breadboard.

The Raspberry Pi uses an SD card as a Flash memory disk. The card is typically preloaded with Raspbian Linux, a small version of Linux that fits on an 8 GB SD card. You can work with the Pi either by attaching an HDMI monitor and USB mouse and keyboard to turn it into a full computer, or by connecting to it from another computer over an Ethernet cable.

The Raspberry Pi continues to advance and by the time you read this, a newer model might be available with a more advanced processor and a different set of embedded I/O. Nevertheless, the same principles will apply, and the principles also apply to other types of microcontrollers. You can expect to find the same types of I/O peripherals. You will need to consult the data sheet to look up the mapping between the peripheral, the pin on the chip, and the pin on the board, as well as the addresses of the memory-mapped I/O registers associated with each peripheral. You'll write configuration registers to initialize the peripheral and read and write data registers to access the peripheral.

As this book was going to press, the Raspberry Pi Foundation released the Raspberry Pi 2 Model B with a BCM2836 SoC containing a quad Cortex-A7 processor and 1 GB of RAM. The Pi 2 runs about 6 times faster than the B+ but has the same I/O as the B+ described in this chapter. The peripheral base address has moved from 0x20000000 to 0x3F000000. An updated EasyPIO supporting both models is posted to the textbook website.

Caution: the I/O connector pinout has changed between Raspberry Pi board revisions.

Caution: connecting 5 V to one of the 3.3 V I/Os will damage the I/O and possibly the entire Raspberry Pi. If you probe the I/O pins with a voltmeter, beware that you do not accidentally make contact between the 5 V pins and a nearby pin!

Pin 1 of the I/O header is labeled in Figure e9.3. When you make connections, be sure you have properly identified it and aren't rotated by 180 degrees. This is an easy mistake that could cause you to accidentally damage the Pi.

EasyPIO and the code examples in this chapter can be downloaded from the textbook website: <http://booksite.elsevier.com/9780128000564>. The WiringPi driver and documentation is at wiringpi.com.

9.3.2 Device Drivers

Programmers can manipulate I/O devices directly by reading or writing the memory-mapped I/O registers. However, it is better programming practice to call functions that access the memory-mapped I/O. These functions are called *device drivers*. Some of the benefits of using device drivers include:

- ▶ The code is easier to read when it involves a clearly named function call rather than a write to bit fields at an obscure memory address.
- ▶ Somebody who is familiar with the deep workings of the I/O devices can write the device driver and casual users can call it without having to understand the details.
- ▶ The code is easier to port to another processor with different memory mapping or I/O devices because only the device driver must change.
- ▶ If the device driver is part of the operating system, the OS can control access to physical devices shared among multiple programs running on the system and can manage security (e.g. so a malicious program can't read the keyboard while you are typing your password into a web browser).

This section will develop a simple device driver called EasyPIO to access BCM2835 devices so that you can understand what is happening under the hood in a device driver. Casual users are likely to prefer WiringPi, an open-source I/O library for the Pi, which has functions similar to but not exactly matching those in EasyPIO.

The memory-mapped I/O on the BCM2835 is found at physical addresses 0x20000000-0x20FFFFFF. The physical base addresses used by various peripherals are summarized in Table e9.1. Peripherals have multiple I/O registers starting at their base address. For example, reading address 0x20200034 will return the values of GPIO (general-purpose I/O) pins 31:0. The peripherals in bold will be discussed further in subsequent sections.

The Raspberry Pi typically runs a Linux operating system using virtual memory, which further complicates memory-mapped I/O. Loads and stores in a program refer to virtual addresses, not physical, so a program cannot immediately access memory-mapped I/O. Instead, it must begin by asking the operating system to map the physical addresses of interest to the program's virtual address space. The `pioInit` function from EasyPIO in Example e9.2 performs this task. The code involves some heavy duty pointer manipulation in C. The general principle is to open `/dev/mem`, which is a Linux method of accessing physical memory. Then the `mmap` function is used to set `gpio` as a pointer to physical address 0x20200000, the beginning of the GPIO registers. The pointer is declared

Table e9.1 Memory mapped I/O addresses

Physical Base Address	Peripheral
0x20003000	System Timer
0x2000B200	Interrupts
0x2000B400	ARM Timer
0x20200000	GPIO
0x20201000	UART0
0x20203000	PCM Audio
0x20204000	SPI0
0x20205000	I ² C Master #1
0x2020C000	PWM
0x20214000	I ² C Slave
0x20215000	miniUART1, SPI1, SPI2
0x20300000	SD Card Controller
0x20804000	I ² C Master #2
0x20805000	I ² C Master #3

`volatile`, telling the compiler that the memory-mapped I/O value might change on its own, so the program should always read the register directly instead of relying on an old value. GPLEV0 accesses the I/O register 13 words past GPIO, e.g. at 0x20200034, which contains the values of GPIO 31:0. For brevity, this example omits error checking that takes place in the actual EasyPIO library. Subsequent subsections define more registers and functions to access I/O devices.

Example e9.2 INITIALIZING MEMORY-MAPPED I/O

```
#include <sys/mman.h>
#define BCM2835_PERI_BASE 0x20000000
#define GPIO_BASE (BCM2835_PERI_BASE + 0x200000)
volatile unsigned int *gpio; //Pointer to base of gpio
#define GPLEV0 (* (volatile unsigned int *) (gpio + 13))
#define BLOCK_SIZE (4*1024)
```

For security reasons, Linux only grants the *superuser* access to memory-mapped hardware. To run a program as the superuser, type `sudo` before the Linux command. The next section will give an example.

```

void pioInit(){
    int mem_fd;
    void *reg_map;

    // /dev/mem is a psuedo-driver for accessing memory in Linux
    mem_fd = open("/dev/mem", O_RDWR|O_SYNC);
    reg_map = mmap(
        NULL,                      // Address at which to start local mapping (null = don't-care)
        BLOCK_SIZE,                // 4KB mapped memory block
        PROT_READ|PROT_WRITE,      // Enable both reading and writing to the mapped memory
        MAP_SHARED,                // Nonexclusive access to this memory
        mem_fd,                    // Map to /dev/mem
        GPIO_BASE);                // Offset to GPIO peripheral

    gpio = (volatile unsigned *)reg_map;
    close(mem_fd);
}

```

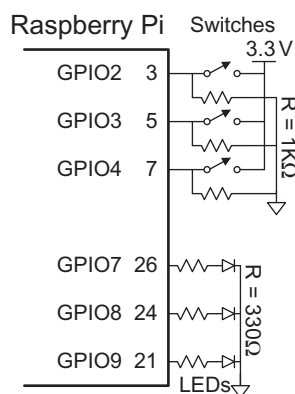


Figure e9.4 LEDs and switches connected to GPIO pins

In the context of bit manipulation, “setting” means writing to 1 and “clearing” means writing to 0.

9.3.3 General-Purpose Digital I/O

General-purpose I/O (GPIO) pins are used to read or write digital signals. For example, [Figure e9.4](#) shows three light-emitting diodes (LEDs) and three switches connected to six GPIO pins. The LEDs are wired to glow when driven with a 1 and to turn off when driven with a 0. The current-limiting resistors are placed in series with the LEDs to set the brightness and avoid overloading the current capability of the GPIO. The switches are wired to produce a 1 when closed and a 0 when open. The schematic indicates the pin name as well as the corresponding header pin number.

At a minimum, any GPIO pin requires registers to read input pin values, write output pin values, and set the direction of the pin. In many embedded systems, the GPIO pins can be shared with one or more special-purpose peripherals, so additional configuration registers are necessary to determine whether the pin is general or special-purpose. Furthermore, the processor may generate interrupts when an event such as a rising or falling edge occurs on an input pin, and configuration registers may be used to specify the conditions for an interrupt.

Recall that the BCM2835 has 54 GPIOs. They are controlled by the GPFSEL, GPLEV, GPSET, and GPCLR registers. [Figure e9.5](#) shows a memory map for these GPIO registers. GPFSEL5...0 determine whether each pin is a general-purpose input, output, or special-purpose I/O. Each of these function select registers uses 3 bits to specify each pin and thus

Table e9.2 GPFSEL register bit field to GPIO mapping

	GPFSEL0	GPFSEL1	GPFSEL2	GPFSEL3	GPFSEL4	GPFSEL5
[2:0]	GPIO0	GPIO10	GPIO20	GPIO30	GPIO40	GPIO50
[5:3]	GPIO1	GPIO11	GPIO21	GPIO31	GPIO41	GPIO51
[8:6]	GPIO2	GPIO12	GPIO22	GPIO32	GPIO42	GPIO52
[11:9]	GPIO3	GPIO13	GPIO23	GPIO33	GPIO43	GPIO53
[14:12]	GPIO4	GPIO14	GPIO24	GPIO34	GPIO44	
[17:15]	GPIO5	GPIO15	GPIO25	GPIO35	GPIO45	
[20:18]	GPIO6	GPIO16	GPIO26	GPIO36	GPIO46	
[23:21]	GPIO7	GPIO17	GPIO27	GPIO37	GPIO47	
[26:24]	GPIO8	GPIO18	GPIO28	GPIO38	GPIO48	
[29:27]	GPIO9	GPIO19	GPIO29	GPIO39	GPIO49	

each 32-bit register controls 10 GPIOs as given in Table e9.2 and six GPFSEL registers are necessary to control all 54 GPIOs. For example, GPIO13 is configured by GPFSEL1[11:9]. The configurations are summarized in Table e9.3; many pins have multiple special-purpose functions that will be discussed in subsequent sections; ALT0 is most commonly used. Reading GPLEV1...0 returns the values of the pins. For example, GPIO14 is read as GPLEV0[14] and GPIO34 is read as GPLEV1[2]. The pins cannot be directly written; instead, bits are forced high or low by asserting the corresponding bit of GPSET1...0 or GPCLR1...0. For example, GPIO14 is forced to 1 by writing GPSET0[14] = 1 and forced to 0 by writing GPCLR0[14] = 1.

The BCM2835 datasheet does not specify the logic levels or output current capability of the GPIOs. However, users have determined empirically that one should not try to draw more than 16 mA from any single I/O or 50 mA total from all the I/Os. Thus, a GPIO pin is suitable for driving a small LED but not a motor. The I/Os are generally compatible with other 3.3 V chips but are not 5 V-tolerant.

...
0x20200038
0x20200034
0x20200030
0x2020002C
0x20200028
0x20200024
0x20200020
0x2020001C
0x20200018
0x20200014
0x20200010
0x2020000C
0x20200008
0x20200004
0x20200000
...

Figure e9.5 GPIO memory map**Table e9.3** GPFSEL configuration

GPFSEL	Pin Function
000	Input
001	Output
010	ALT5
011	ALT4
100	ALT0
101	ALT1
110	ALT2
111	ALT3

The BCM2835 has unusually complex GPIO access. Some microcontrollers use a single register to configure whether each pin is input or output and another register to read and write the pins.

Example e9.3 GPIO FOR SWITCHES AND LEDS

Enhance EasyPIO with `pinMode`, `digitalRead`, and `digitalWrite` functions to configure a pin's direction and read or write it. Write a C program using these functions to read the three switches and turn on the corresponding LEDs using the hardware in [Figure e9.4](#).

Solution: The additional EasyPIO code is given below. Because multiple registers are used to control the I/O, the functions must compute which register to access and what bit offset to use within the register. `pinMode` then clears the 0 bits and sets the 1 bits for the intended 3-bit function. `digitalWrite` handles writing either 1 or 0 by using GPSET or GPCLR. `digitalRead` pulls out the value of the desired pin and masks off the others.

```
#define GPFSEL ((volatile unsigned int *) (gpio + 0))
#define GPSET  ((volatile unsigned int *) (gpio + 7))
#define GPCLR  ((volatile unsigned int *) (gpio + 10))
#define GPLEV  ((volatile unsigned int *) (gpio + 13))
#define INPUT  0
#define OUTPUT  1

...
void pinMode(int pin, int function) {
    int reg      = pin/10;
    int offset   = (pin%10)*3;
    GPFSEL[reg] &= ~((0b111 & ~function) << offset);
    GPFSEL[reg] |=  ((0b111 & function) << offset);
}

void digitalWrite(int pin, int val) {
    int reg      = pin / 32;
    int offset   = pin % 32;

    if (val) GPSET[reg] = 1 << offset;
    else    GPCLR[reg] = 1 << offset;
}

int digitalRead(int pin) {
    int reg      = pin / 32;
    int offset   = pin % 32;

    return (GPLEV[reg] >> offset) & 0x00000001;
}
```

The program to read switches and write LEDs is given below. It initializes GPIO access, then sets pins 2–4 as inputs for the switches and pins 7–9 as outputs for the LEDs. It then continuously reads the switches and writes their values to the corresponding LEDs.

```
#include "EasyPIO.h"

void main(void) {
    pioInit();
```

```
// Set GPIO 4:2 as inputs
pinMode(2, INPUT);
pinMode(3, INPUT);
pinMode(4, INPUT);

// Set GPIO 9:7 as an output
pinMode(7, OUTPUT);
pinMode(8, OUTPUT);
pinMode(9, OUTPUT);

while (1) { // Read each switch and write corresponding LED
    digitalWrite(7, digitalRead(2));
    digitalWrite(8, digitalRead(3));
    digitalWrite(9, digitalRead(4));
}
```

Assuming the program is in a file named `dip2led.c` and that `EasyPIO.h` is in the same directory, you can compile and run the program using the following commands on the Raspberry Pi command line. `gcc` is the C compiler. Note that `sudo` is required so that the program can access the protected I/O memory. To stop a running program, press Ctrl-C.

```
gcc dip2led.c -o dip2led
sudo ./dip2led
```

9.3.4 Serial I/O

If a microcontroller needs to send more bits than the number of free GPIO pins, it must break the message into multiple smaller transmissions. In each step, it can send either one bit or several bits. The former is called serial I/O and the latter is called parallel I/O. Serial I/O is popular because it uses few wires and is fast enough for many applications. Indeed, it is so popular that many standards for serial I/O have been established and microcontrollers offer dedicated hardware to easily send data via these standards. This section describes the Serial Peripheral Interface (SPI) and Universal Asynchronous Receiver/Transmitter (UART) standard serial interfaces.

Other common serial standards include Inter-Integrated Circuit (I²C), Universal Serial Bus (USB), and Ethernet. I²C (pronounced “I squared C”) is a 2-wire interface with a clock and a bidirectional data pin; it is used in a fashion similar to SPI. USB and Ethernet are more complex, high-performance standards described in [Sections 9.6.1 and 9.6.4](#), respectively. All five of these standards are supported on the Raspberry Pi.

SPI always sends data in both directions on each transfer. If the system only needs unidirectional communication, it can ignore the unwanted data. For example, if the master only needs to send data to the slave, the byte received from the slave can be ignored. If the master only needs to receive data from the slave, it must still trigger the SPI communication by sending an arbitrary byte that the slave will ignore. It can then read the data received from the slave. The SPI clock only toggles while the master is transmitting data.

9.3.4.1 Serial Peripheral Interface (SPI)

SPI (pronounced “S-P-I”) is a simple synchronous serial protocol that is easy to use and relatively fast. The physical interface consists of three pins: Serial Clock (SCK), Master Out Slave In (MOSI, also known as SDO), and Master In Slave Out (MISO, also known as SDI). SPI connects a *master* device to a *slave* device, as shown in Figure e9.6(a). The master produces the clock. It initiates communication by sending a series of clock pulses on SCK. If it wants to send data to the slave, it puts the data on MOSI, starting with the most significant bit. The slave may simultaneously respond by putting data on MISO. Figure e9.6(b) shows the SPI waveforms for an 8-bit data transmission. Bits change on the falling edge of SCK and are stable to sample on the rising edge. The SPI interface may also send an active-low chip enable to alert the receiver that data is coming.

The BCM2835 has three SPI master ports and one slave port. This section describes SPI Master Port 0, which is readily accessible on the Raspberry Pi on GPIO pins 11:9. To use these pins for SPI rather than GPIO, their GPFSEL must be set to ALT0. The Pi must then configure the port. When the Pi writes to the SPI, the data is transmitted serially to the slave. Simultaneously, data received from the slave is collected and the Pi can read it when the transfer is complete.

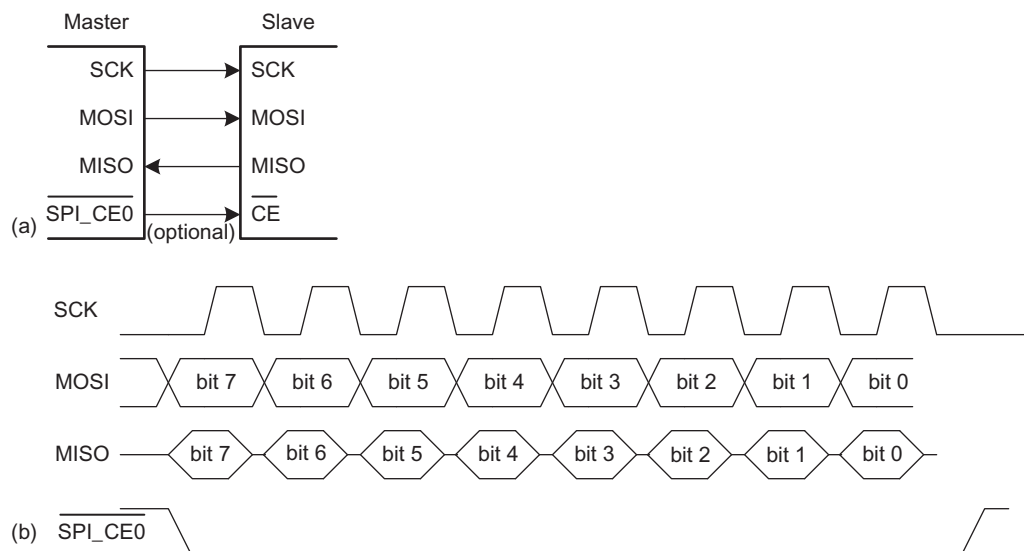


Figure e9.6 SPI connection and waveforms

Table e9.4 SPI0CS register fields

Bit	Name	Function	Meaning for 0	Meaning for 1
16	DONE	Transfer Done	Transfer in progress	Transfer complete
7	TA	Transfer Active	SPI disabled	SPI enabled
3	CPOL	Clock Polarity	Clock idles low	Clock idles high
2	CPHA	Clock Phase	First SCK transition at middle of data bit	First SCK transition at beginning of data bit

SPI Master Port 0 is associated with three registers, given in the memory map in [Figure e9.7](#). SPI0CS is the control register. It is used to turn the SPI on and set attributes such as the polarity of the clock. [Table e9.4](#) lists the names and functions of some of the bits in SPI0CS that are relevant to this discussion. All have a default value of 0 on reset. Most of the functions, such as chip selects and interrupts, are not used in this section but can be found in the datasheet. SPI0FIFO is written to transmit a byte and read to get the byte received back. SPI0CLK configures the SPI clock frequency by dividing the 250 MHz peripheral clock by a power of two specified in the register. Thus, the SPI clock frequency is summarized in [Table e9.5](#).

Example e9.4 SENDING AND RECEIVING BYTES OVER SPI

Design a system to communicate between a Raspberry Pi master and an FPGA slave over SPI. Sketch a schematic of the interface. Write the C code for the Pi to send the character 'A' and receive a character back. Write HDL code for an SPI slave on the FPGA. How could the slave be simplified if it only needs to receive data?

Solution: [Figure e9.8](#) shows the connection between the devices using SPI Master Port 0. The pin numbers are obtained from the component datasheets (e.g., [Figure e9.3](#)). Notice that both the pin numbers and signal names are shown on the diagram to indicate both the physical and logical connectivity. When the SPI is enabled, these pins cannot be used for GPIO.

	...
0x20204008	SPI0CLK
0x20204004	SPI0FIFO
0x20204000	SPI0CS
	...

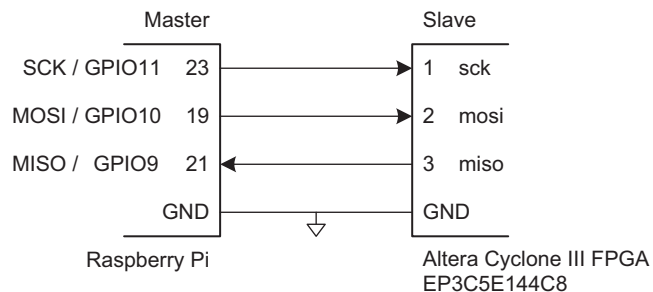
Figure e9.7 SPI Master Port 0 registers

If the frequency is too high (>~1 MHz on a breadboard or tens of MHz on an unterminated printed circuit board), the SPI may become unreliable due to reflections, crosstalk, or other signal integrity issues.

Table e9.5 SPI0CLK frequencies

SPI0CLK	SPI Frequency (kHz)
2	125000
4	62500
8	31250
16	15625
32	7812
64	3906
128	1953
256	976
512	488
1024	244
2048	122

Figure e9.8 SPI connection between Pi and FPGA



The following code from EasyPIO.h is used to initialize the SPI and to send and receive a character. The code to set up the memory map and define the register addresses is similar to that for GPIO and is not reprinted here.

```
void spiInit(int freq, int settings) {
    pinMode(8, ALT0);          // CE0b
    pinMode(9, ALT0);          // MISO
    pinMode(10, ALT0);         // MOSI
    pinMode(11, ALT0);         // SCLK

    SPIOCLK = 250000000/freq; // Set SPI clock divider to desired
                             // freq
    SPIOCS = settings;
    SPIOCSbits.TA = 1;        // Turn SPI on
}

char spiSendReceive(char send){
    SPIOFIFO = send;          // Send data to slave
    while (!SPIOCSbits.DONE); // Wait until SPI complete
    return SPIOFIFO;          // Return received data
}
```

The C code below initializes the SPI and then sends and receives a character. It sets the SPI clock to 244 kHz.

```
#include "EasyPIO.h"

void main(void) {
    char received;

    pioInit();
    spiInit(244000, 0); // Initialize the SPI:
                        // 244 kHz clk, default settings
    received = spiSendReceive('A'); // Send letter A and receive byte
}
```

The HDL code for the FPGA is listed below. [Figure e9.9](#) shows a block diagram and timing. The FPGA uses a shift register to hold the bits that have been received from the master and the bits that remain to be sent to the master.

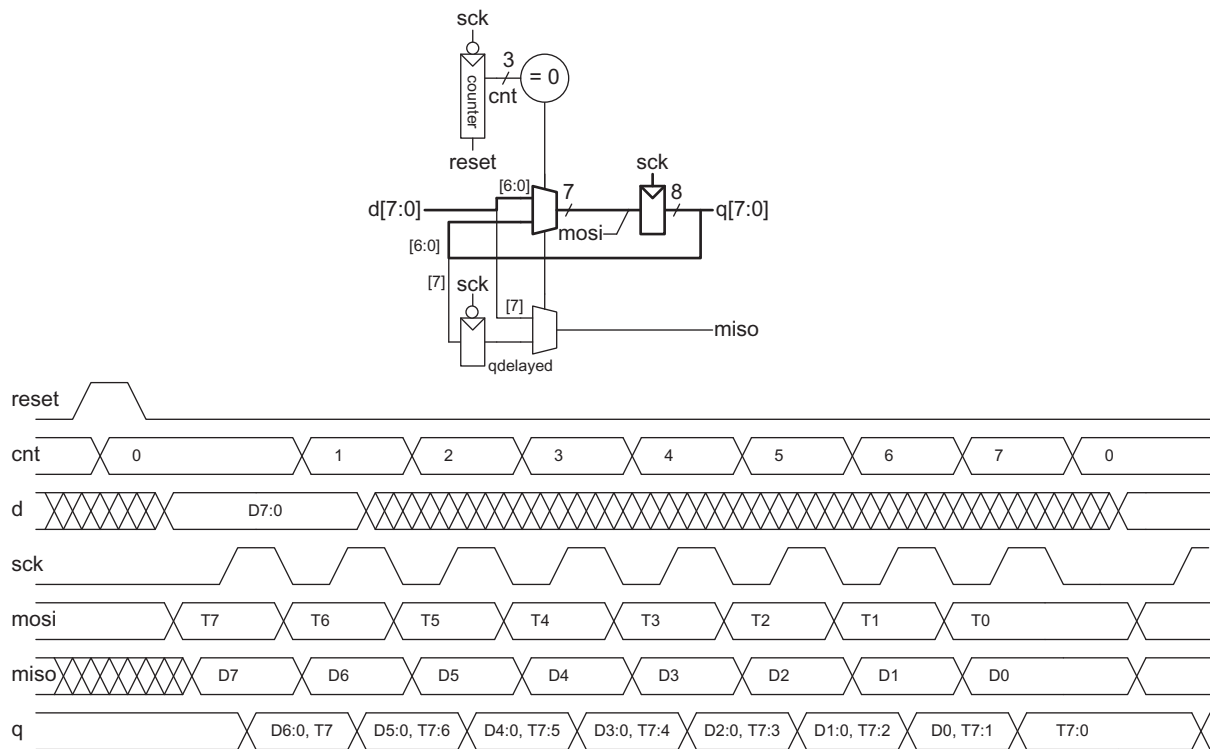


Figure e9.9 SPI slave circuitry and timing

On the first rising *sck* edge after reset and each 8 cycles thereafter, a new byte from *d* is loaded into the shift register. On each subsequent cycle, a bit is shifted in on *mosi* and a bit is shifted out of *miso*. *miso* is delayed until the falling edge of *sck* so that it can be sampled by the master on the next rising edge. After 8 cycles, the byte received can be found in *q*.

```

module spi_slave(input logic sck, // From master
                 input logic mosi, // From master
                 output logic miso, // To master
                 input logic reset, // System reset
                 input logic [7:0] d, // Data to send
                 output logic [7:0] q); // Data received

    logic [2:0] cnt;
    logic qdelayed;

    // 3-bit counter tracks when full byte is transmitted
    always_ff @(negedge sck, posedge reset)
        if (reset) cnt = 0;
        else cnt = cnt + 3'b1;

```

```

// Loadable shift register
// Loads d at the start, shifts mosi into bottom on each step
always_ff @(posedge sck)
    q <= (cnt == 0) ? {d[6:0], mosi} : {q[6:0], mosi};

// Align miso to falling edge of sck
// Load d at the start
always_ff @(negedge sck)
    qdelayed = q[7];
assign miso = (cnt == 0) ? d[7] : qdelayed;
endmodule

```

If the slave only needs to receive data from the master, it reduces to a simple shift register given in the following HDL code.

```

module spi_slave_receive_only(input logic    sck, //From master
                              input logic    mosi, //From master
                              output logic [7:0] q); //Data received

    always_ff @(posedge sck)
        q <= {q[6:0], sdi}; // shift register
endmodule

```

SPI ports are highly configurable so that they can talk to a wide variety of serial devices. Unfortunately, this leads to the possibility of incorrectly configuring the port and garbling the data transmission. Sometimes it is necessary to change the configuration bits to communicate with a device that expects different timing. When $CPOL = 1$, SCK is inverted. When $CPHA = 1$, the clocks toggle half a cycle earlier relative to the data. These modes are shown in Figure e9.10. Be aware that different SPI products may use different names and polarities for these options; check the waveforms carefully for your device. It can also be helpful to examine

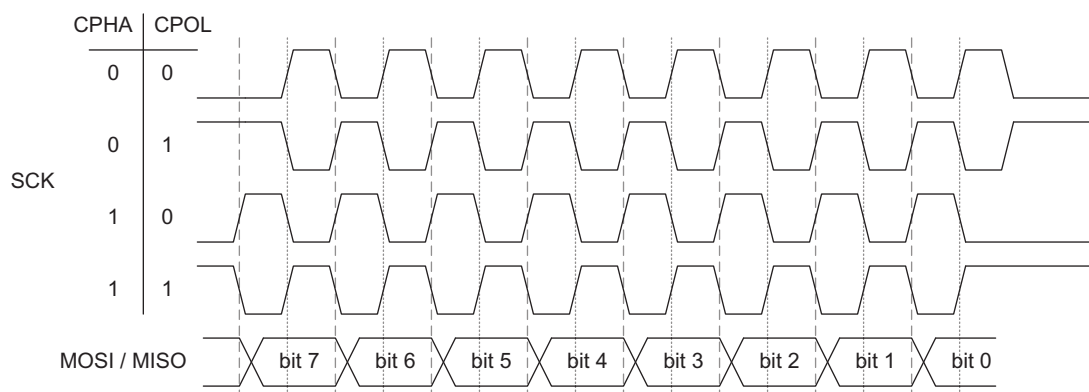


Figure e9.10 SPI clock and data timing configurations

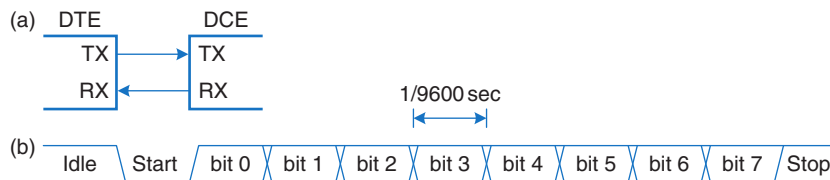


Figure e9.11 Asynchronous serial link

SCK, MOSI, and MISO on an oscilloscope if you are having communication difficulties.

9.3.4.2 Universal Asynchronous Receiver/Transmitter (UART)

A UART (pronounced “you-art”) is a serial I/O peripheral that communicates between two systems without sending a clock. Instead, the systems must agree in advance about what data rate to use and must each locally generate its own clock. Hence, the transmission is asynchronous because the clocks are not synchronized. Although these system clocks may have a small frequency error and an unknown phase relationship, the UART manages reliable asynchronous communication. UARTs are used in protocols such as RS-232 and RS-485. For example, old computer serial ports use the RS-232C standard, introduced in 1969 by the Electronics Industries Associations. The standard originally envisioned connecting *Data Terminal Equipment* (DTE) such as a mainframe computer to *Data Communication Equipment* (DCE) such as a modem. Although a UART is relatively slow compared to SPI and prone to misconfiguration issues, the standards have been around for so long that they remain important today.

Figure e9.11(a) shows an asynchronous serial link. The DTE sends data to the DCE over the TX line and receives data back over the RX line. Figure e9.11(b) shows one of these lines sending a character at a data rate of 9600 baud. The lines idle at a logic ‘1’ when not in use. Each character is sent as a start bit (0), 7 or 8 data bits, an optional parity bit, and one or more stop bits (1’s). The UART detects the falling transition from idle to start to lock on to the transmission at the appropriate time. Although seven data bits is sufficient to send an ASCII character, eight bits are normally used because they can convey an arbitrary byte of data.

The optional parity bit allows the system to detect if a bit was corrupted during transmission. It can be configured as *even* or *odd*; even parity means that the parity bit is chosen such that the total collection of data and parity has an even number of 1’s; in other words, the parity bit is the XOR of the data bits. The receiver can then check if an even number of 1’s was received and signal an error if not. Odd parity is the reverse.

Baud rate gives the signaling rate, measured in symbols per second, whereas bit rate gives the data rate, measured in bits per second. The signaling we’ve discussed in this text is 2-level signaling, where each symbol represents a bit. However, multi-level signaling can send multiple bits per symbol; for example, 4-level signaling sends two bits per symbol. In that case, the bit rate is twice the baud rate. In a simple system like SPI where each symbol is a bit and each symbol represents data, the baud rate is equal to the bit rate. UARTs and some other signaling conventions require overhead bits in addition to the data. For example, a two-level signaling system that adds start and stop bits for each 8 bits of data and operates at a baud rate of 9600 has a bit rate of $(9600 \text{ symbols/second}) \times (8 \text{ bits/10 symbols}) = 7680 \text{ bits/second} = 960 \text{ characters/second}$.

In the 1950s through 1970s, early hackers calling themselves phone phreaks learned to control the phone company switches by whistling appropriate tones. A 2600 Hz tone produced by a toy whistle from a Cap'n Crunch cereal box e9.12 could be exploited to place free long-distance and international calls.



Figure e9.12 Cap'n Crunch Boson Whistle

(Photograph by Evrim Sen, reprinted with permission.)

Handshaking refers to the negotiation between two systems; typically, one system signals that it is ready to send or receive data, and the other system acknowledges that request.

A common choice is 8 data bits, no parity, and 1 stop bit, making a total of 10 symbols to convey an 8-bit character of information. Hence, signaling rates are referred to in units of baud rather than bits/sec. For example, 9600 baud indicates 9600 symbols/sec, or 960 characters/sec. Both systems must be configured for the appropriate baud rate and number of data, parity, and stop bits or the data will be garbled. This is a hassle, especially for nontechnical users, which is one of the reasons that the Universal Serial Bus (USB) has replaced UARTs in personal computer systems.

Typical baud rates include 300, 1200, 2400, 9600, 14400, 19200, 38400, 57600, and 115200. The lower rates were used in the 1970's and 1980's for modems that sent data over the phone lines as a series of tones. In contemporary systems, 9600 and 115200 are two of the most common baud rates; 9600 is encountered where speed doesn't matter, and 115200 is the fastest standard rate, though still slow compared to other modern serial I/O standards.

The RS-232 standard defines several additional signals. The Request to Send (RTS) and Clear to Send (CTS) signals can be used for *hardware handshaking*. They can be operated in either of two modes. In *flow control* mode, the DTE clears RTS to 0 when it is ready to accept data from the DCE. Likewise, the DCE clears CTS to 0 when it is ready to receive data from the DTE. Some datasheets use an overbar to indicate that they are active-low. In the older *simplex* mode, the DTE clears RTS to 0 when it is ready to transmit. The DCE replies by clearing CTS when it is ready to receive the transmission.

Some systems, especially those connected over a telephone line, also use Data Terminal Ready (DTR), Data Carrier Detect (DCD), Data Set Ready (DSR), and Ring Indicator (RI) to indicate when equipment is connected to the line.

The original standard recommended a massive 25-pin DB-25 connector, but PCs streamlined to a male 9-pin DE-9 connector with the pinout shown in Figure e9.13(a). The cable wires normally connect straight across as shown in Figure e9.13(b). However, when directly connecting two DTEs, a *null modem* cable shown in Figure e9.13(c) may be needed to swap RX and TX and complete the handshaking. As a final insult, some connectors are male and some are female. In summary, it can take a large box of cables and a certain amount of guess-work to connect two systems over RS-232, again explaining the shift to USB. Fortunately, embedded systems typically use a simplified 3- or 5-wire setup consisting of GND, TX, RX, and possibly RTS and CTS.

RS-232 represents a 0 electrically with 3 to 15 V and a 1 with -3 to -15 V; this is called *bipolar* signaling. A transceiver converts the digital logic levels of the UART to the positive and negative levels expected by RS-232, and also provides electrostatic discharge protection to protect the serial port from getting zapped when the user plugs in a cable. The

MAX3232E is a popular transceiver compatible with both 3.3 and 5 V digital logic. It contains a charge pump that, in conjunction with external capacitors, generates ± 5 V outputs from a single low-voltage power supply. Some serial peripherals intended for embedded systems omit the transceiver and just use 0 V for a 0 and 3.3 or 5 V for a 1; check the datasheet!

The BCM2835 has two UARTs named UART0 and UART1. Either can be configured to communicate on pins 14 and 15, but UART0 is more fully featured and is described here. To use these pins for UART0 rather than GPIO, their GPFSEL must be set to ALT0. As with SPI, the Pi must first configure the port. Unlike SPI, reading and writing can occur independently because either system may transmit without receiving and vice versa. UART0's registers are shown in Figure e9.14.

To configure the UART, first set the baud rate. The UART has an internal 3 MHz clock that must be divided down to produce a clock that is 16x the desired baud rate. Hence, the appropriate divisor, BRD, is

$$\text{BRD} = 3000000 / (16 \times \text{baud rate})$$

BRD is represented with a 16-bit integer portion in UART_IBRD and a 6-bit fractional portion in UART_FBRD: $\text{BRD} = \text{IBRD} + \text{FBRD}/64$. Table e9.6 shows these settings for popular baud rates.¹

Table e9.6 BRD settings

Target Baud Rate	UART_IBRD	UART_FBRD	Actual Baud Rate	Error (%)
300	625	0	300	0
1200	156	16	1200	0
2400	78	8	2400	0
9600	19	34	9600	0
19200	9	49	19200	0
38400	4	56	38461	0.16
57600	3	16	57692	0.16
115200	1	40	115384	0.16

¹ The baud rates do not all evenly divide 3 MHz, so some divisors produce a frequency error. The UART, by its asynchronous nature, accommodates this error so long as it is small enough.

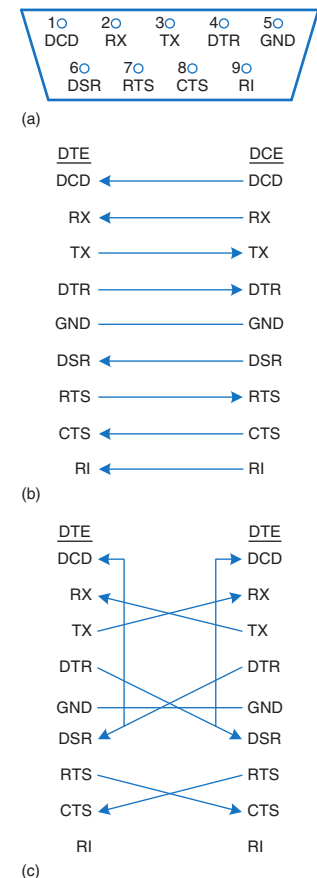


Figure e9.13 DE-9 male cable (a) pinout, (b) standard wiring, and (c) null modem wiring

	...
0x20201030	UART_CR
0x2020102C	UART_LCRH
0x20201028	UART_FBRD
0x20201024	UART_IBRD
	...
0x20201000	UART_DR
	...

Figure e9.14 UART0 registers

Next, set the number of data, stop, and parity bits using the UART_LCRH line control register. By default, the UART has 1 stop bit and no parity, but strangely only transmits and receives 5-bit words. Hence, the WLEN field (bits 6:5) of UART_LCRH must be set to 3 to handle 8-bit words. Finally, enable the UART by turning on bit 0 (UARTEN) of the UART_CR control register.

Data is transmitted and received using the UART_DR data register and UART_FR framing register. To transmit data, wait until bit 7 (TXFE) of UART_FR is 1 to indicate that the transmitter is not busy, then write a byte to UART_DR. To receive data, wait until bit 4 (RXFE) of UART_FR is 0 to indicate that the receiver has data, then read the byte from UART_DR.

Example e9.5 SERIAL COMMUNICATION WITH A PC

Develop a circuit and a C program for a Raspberry Pi to communicate with a PC over a serial port at 115200 baud with 8 data bits, 1 stop bit, and no parity. The PC should be running a console program such as PuTTY² to read and write over the serial port. The program should ask the user to type a string. It should then tell the user what she typed.

Solution: Figure e9.15(a) shows a basic schematic of the serial link illustrating the issues of level conversion and cabling. Because few PCs still have physical serial ports, we use a Plugable USB to RS-232 DB9 Serial Adapter from plugable.com shown in Figure e9.16 to provide a serial connection to the PC. The adapter connects to a female DE-9 connector soldered to wires that feed a transceiver, which converts the voltages from the bipolar RS-232 levels to the Pi's 3.3 V level. The Pi and PC are both Data Terminal Equipment, so the TX and RX pins must be cross-connected in the circuit. The RTS/CTS handshaking from the Pi is not used, and the RTS and CTS on the DE9 connector are tied together so that the PC will shake its own hand. Figure e9.15(b) shows an easier approach with an Adafruit 954 USB to TTL serial cable. The cable is directly compatible with 3.3 V levels and has female header pins that plug directly into the Raspberry Pi male headers.

To configure PuTTY to work with the serial link, set *Connection type* to Serial and *Speed* to 115200. Set *Serial line* to the COM port assigned by the operating system to the Serial to USB Adapter. In Windows, this can be found

² PuTTY is available for free download at www.putty.org.

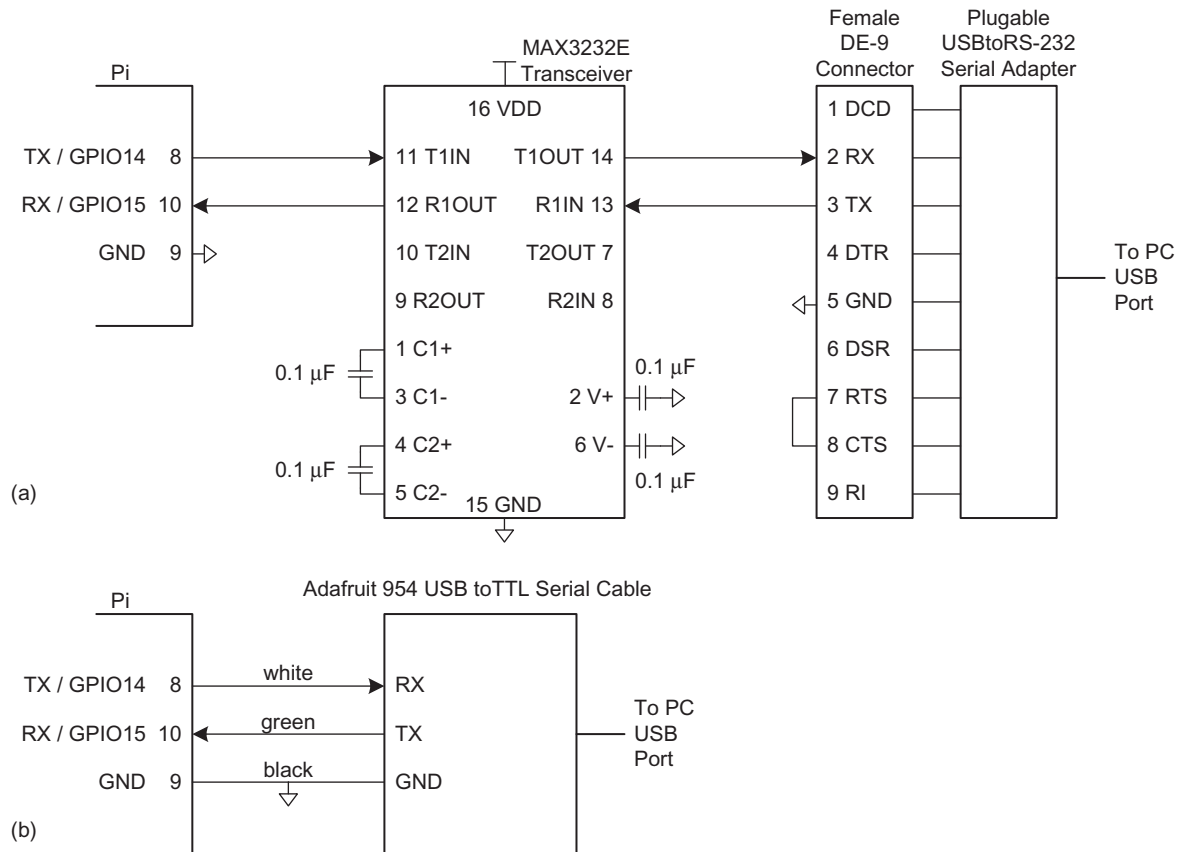


Figure e9.15 Raspberry Pi to PC serial link (a) Plugable cable, (b) Adafruit cable

in the Device Manager; for example, it might be COM3. Under the *Connection Serial* tab, set flow control to NONE or RTS/CTS. Under the Terminal tab, set Local Echo to Force On to have characters appear in the terminal as you type them.

The serial port device driver code in EasyPIO.h is listed below. The Enter key in the terminal program corresponds to a carriage return character represented as '\r' in C with an ASCII code of 0x0D. To advance to the beginning of the next line when printing, send both the '\n' and '\r' (new line and carriage return) characters.³ The `uartInit` function configures the UART as described above. Similarly, `getCharSerial` and `putCharSerial`

Note that the operating system also prints a log-in prompt to the serial port. You may see some interesting interactions between the OS and your program when both use the port.



Figure e9.16 Pluggable USB to RS-232 DB9 serial adapter
(© 2012 Pluggable Technologies; reprinted with permission.)

wait until the UART is ready and then read or write a byte from the data register.

```
void uartInit(int baud) {
    uint fb = 12000000/ baud;           // 3 MHz UART clock

    pinMode(14, ALT0);                  // TX
    pinMode(15, ALT0);                  // RX
    UART_IBRD = fb >> 6;                 // 6 Fract, 16 Int bits of BRD
    UART_FBRD = fb & 63;
    UART_LCRHbits.WLEN = 3;             // 8 Data, 1 Stop, no Parity, no FIFO, no Flow
    UART_CRbits.UARTEN = 1;             // Enable uart
}

char getCharSerial(void) {
    while (UART_FRbits.RXFE);           // Wait until data is available
    return UART_DRbits.DATA;            // Return char from serial port
}

void putCharSerial(char c) {
    while (!UART_FRbits.TXFE);          // Wait until ready to transmit
    UART_DRbits.DATA = c;               // Send char to serial port
}
```

The main function demonstrates printing to the console and reading from the console using the `putStrSerial` and `getStrSerial` functions.

```
#include "EasyPIO.h"
#define MAX_STR_LEN 80

void getStrSerial(char *str) {
    int i = 0;
    do {
        str[i] = getCharSerial();        // Read an entire string until
    } while ((str[i++] != '\r') && (i < MAX_STR_LEN)); // Carriage return
    str[i-1] = 0;                        // Look for carriage return
    str[i-1] = 0;                        // Null-terminate the string
}

void putStrSerial(char *str) {
    int i = 0;
    while (str[i] != 0) {
        putCharSerial(str[i++]);        // Iterate over string
    }                                    // Send each character
}

int main(void) {
    char str[MAX_STR_LEN];
```

³ PuTTY prints correctly even if the `\r` is omitted.


```
pioInit();
uartInit(115200);           // Initialize UART with baud rate

while (1) {
    putStrSerial("Please type something: \r\n");
    getStrSerial(str);
    putStrSerial("You typed: ");
    putStrSerial(str);
    putStrSerial("\r\n");
}
```

Communicating with the serial port from a C program on a PC is a bit of a hassle because serial port driver libraries are not standardized across operating systems. Other programming environments such as Python, Matlab, or LabVIEW make serial communication painless.

9.3.5 Timers

Embedded systems commonly need to measure time. For example, a microwave oven needs a timer to keep track of the time of day and another to measure how long to cook. It might use yet another to generate pulses to the motor spinning the platter, and a fourth to control the power setting by only activating the microwave’s energy for a fraction of every second.

The BCM2835 has a system timer with a 64-bit free-running counter that increments every microsecond (i.e. at 1 MHz) and four 32-bit timer compare channels. Figure e9.17 shows the memory map for the system timer. SYS_TIMER_CLO and CHI contain the lower and upper 32 bits of the 64-bit counter value. SYS_TIMER_C0...C3 are 32-bit compare channels. When any of the compare channels match SYS_TIMER_CLO, the corresponding match bit (M0-M3) in the bottom four bits of SYS_TIMER_CS is set. A match bit is cleared by writing a 1 to that bit of SYS_TIMER_CS. This may seem counterintuitive, but it prevents inadvertently clearing other match bits. One can measure a particular number of microseconds by adding that time to CLO and putting it in C1, clearing SYS_TIMER_CS.M1, then waiting until SYS_TIMER_CS.M1 is set.

Unfortunately, Linux is a multitasking operating system that may switch between processes without warning. If your program is waiting for a timer match and then another process begins executing, your program may not resume until long after the match occurs and

The graphics processing unit and operating system may use channels 0, 2, and 3, so user code should check SYSTEM_TIMER_C1.

	...
0x20003018	SYS_TIMER_C3
0x20003014	SYS_TIMER_C2
0x20003010	SYS_TIMER_C1
0x2000300C	SYS_TIMER_C0
0x20003008	SYS_TIMER_CHI
0x20003004	SYS_TIMER_CLO
0x20003000	SYS_TIMER_CS
	...

Figure e9.17 System timer registers

you may measure the wrong amount of time. To avoid this, your program can turn off interrupts during critical timing loops so that Linux will not switch processes. Be sure to turn the interrupts back on when you are done. EasyPIO defines `noInterrupts` and `interrupts` functions to disable and enable interrupts, respectively. While interrupts are disabled, the Pi will not switch between processes and cannot even respond to the user pressing Ctrl-C to kill a program. If your program hangs, you'll need to turn off power and reboot your Pi to recover.

Example e9.6 BLINKING LED

Write a program that blinks the status LED on the Raspberry Pi 5 times per second for 4 seconds.

Solution: The `delayMicros` function in EasyPIO creates a delay of a specified number of microseconds using the timer compare channel 1.

```
void delayMicros(int micros) {
    SYS_TIMER_C1 = SYS_TIMER_C10 + micros; // Set the compare register
    SYS_TIMER_CSbits.M1 = 1;               // Reset match flag to 0
    while (SYS_TIMER_CSbits.M1 == 0);      // Wait until match flag is set
}

void delayMillis(int millis) {
    delayMicros(millis*1000);              // 1000 µs per ms
}
```

GPIO47 drives the activity LED on the Pi B+. The program sets this pin to be an output and disables interrupts. It then turns the LED OFF and ON through a series of digital writes with a 200 ms repetition rate (5 Hz). The program finally reenables interrupts.

```
#include "EasyPIO.h"

void main(void) {
    int i;

    pioInit();

    pinMode(47, OUTPUT); // Status led as output
    noInterrupts();       // Disable interrupts

    for (i=0; i<20; i++) {
        delayMillis(150);
        digitalWrite(47, 0); // Turn led off
        delayMillis(50);
        digitalWrite(47, 1); // Turn led on
    }
    interrupts();         // Re-enable interrupts
}
```

9.3.6 Analog I/O

The real world is an analog place. Many embedded systems need analog inputs and outputs to interface with the world. They use analog-to-digital converters (ADCs) to quantize analog signals into digital values, and digital-to-analog-converters (DACs) to do the reverse. Figure e9.18 shows symbols for these components. Such converters are characterized by their resolution, dynamic range, sampling rate, and accuracy. For example, an ADC might have $N=12$ -bit resolution over a range V_{ref}^- to V_{ref}^+ of 0–5 V with a sampling rate of $f_s = 44$ kHz and an accuracy of ± 3 least significant bits (lsbs). Sampling rates are also listed in samples per second (sps), where 1 sps = 1 Hz. The relationship between the analog input voltage $V_{in}(t)$ and the digital sample $X[n = t / f_s]$ is

$$X[n] = 2^N \frac{V_{in}(t) - V_{ref}^-}{V_{ref}^+ - V_{ref}^-}$$

For example, an input voltage of 2.5 V (half of full scale) would correspond to an output of $100000000000_2 = 800_{16}$, with an uncertainty of up to 3 lsbs.

Similarly, a DAC might have $N=16$ -bit resolution over a full-scale output range of $V_{ref} = 2.56$ V. It produces an output of

$$V_{out}(t) = \frac{X[n]}{2^N} V_{ref}$$

Many microcontrollers have built-in ADCs of moderate performance. For higher performance (e.g., 16-bit resolution or sampling rates in excess of 1 MHz), it is often necessary to use a separate ADC connected to the microcontroller. Fewer microcontrollers have built-in DACs, so separate chips may also be used. However, microcontrollers often produce analog outputs using a technique called pulse-width modulation (PWM).

9.3.6.1 D/A Conversion

The BCM2835 has a specialized DAC for composite video output, but no general-purpose converter. This section describes D/A conversion using external DACs and illustrates interfacing the Raspberry Pi to other chips over parallel and serial ports. The next section achieves the same result using pulse-width modulation.

Some DACs accept the N -bit digital input on a parallel interface with N wires, while others accept it over a serial interface such as SPI. Some DACs require both positive and negative power supply voltages, while others operate off of a single supply. Some support a flexible range of supply voltages, while others demand a specific voltage. The input logic levels should be compatible with the digital source. Some DACs produce a voltage output proportional to the digital input, while others produce a

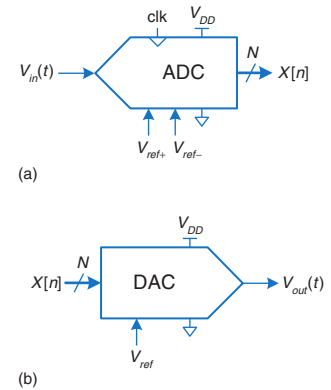


Figure e9.18 ADC and DAC symbols

current output; an operational amplifier may be needed to convert this current to a voltage in the desired range.

In this section, we use the Analog Devices AD558 8-bit parallel DAC and the Linear Technology LTC1257 12-bit serial DAC. Both produce voltage outputs, run off a single 5–15 V power supply, use $V_{IH} = 2.4$ V such that they are compatible with 3.3 V I/O, come in DIP packages that make them easy to breadboard, and are easy to use. The AD558 produces an output on a scale of 0–2.56 V, consumes 75 mW, comes in a 16-pin package, and has a 1 μ s settling time permitting an output rate of 1 Msamples/sec. The datasheet is at analog.com. The LTC1257 produces an output on a scale of 0–2.048 V, consumes less than 2 mW, comes in an 8-pin package, and has a 6 μ s settling time. Its SPI operates at a maximum of 1.4 MHz. The datasheet is at linear.com.

Example e9.7 ANALOG OUTPUT WITH EXTERNAL DACS

Sketch a circuit and write the software for a simple signal generator producing sine and triangle waves using a Raspberry Pi, an AD558, and an LTC1257.

Solution: The circuit is shown in Figure e9.19. The AD558 connects to the Pi via GPIO14, 15, 17, 18, 22, 23, 24, and 25. It connects *Vout Sense* and *Vout Select* to *Vout* to set the 2.56 V full-scale output range. The LTC1257 connects

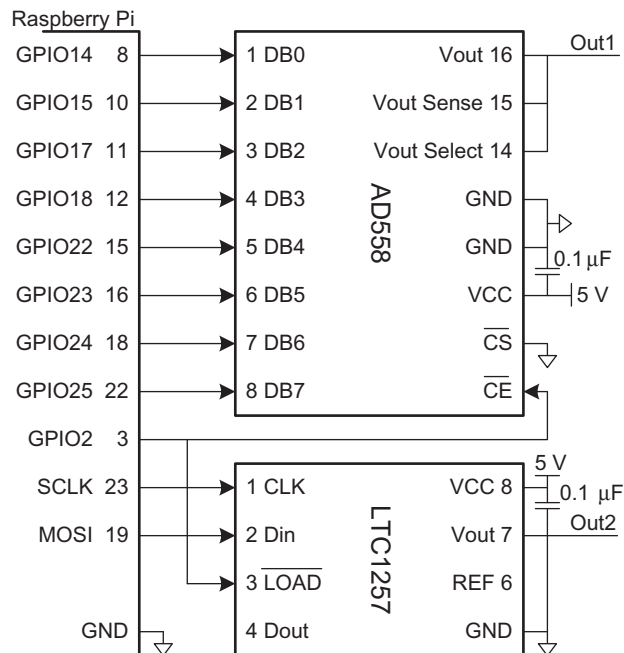


Figure e9.19 DAC parallel and serial interfaces to a Raspberry Pi

to the Pi via SPI0. Both ADCs use a 5 V power supply and have a 0.1 μ F decoupling capacitor to reduce power supply noise. The active-low chip enable and load signals on the DACs indicate when to convert the next digital input. They are driven high while a new input is being loaded.

The program is listed below. `pinsMode` and `digitalWrites` are similar to `pinMode` and `digitalWrite` but operate on an array of pins. The program sets the 8 parallel port pins to be outputs and also configures GPIO2 as an output to drive the chip enable and load signals. It initializes the SPI to 1.4 MHz. `initWaveTables` precomputes an array of sample values for the sine and triangle waves. The sine wave is set to a 12-bit scale and the triangle to an 8-bit scale. There are 64 points per period of each wave; changing this value trades precision for frequency. `genWaves` cycles through the samples. It disables interrupts to avoid switching processes and garbling the waves. For each sample, it disables the CE and LOAD signals to the DACs, sends the new sample over the parallel and serial ports, reenables the DACs, and then waits until the timer indicates that it is time for the next sample. `spiSendReceive16` transmits two bytes, but the LTC1257 only cares about the last 12 bits sent. The maximum frequency of somewhat more than 1000 Hz (64 Ksamples/sec) is set by the time to send each point in the `genWaves` function, of which the SPI transmission is a major component.

```
#include "EasyPI0.h"
#include math.h> // required to use the sine function

#define NUMPTS 64
int sine[NUMPTS], triangle[NUMPTS];
int parallelPins[8] = {14,15,17,18,22,23,24,25};

void initWaveTables(void) {
    int i;
    for (i=0; i<NUMPTS; i++) {
        sine[i] = 2047*(sin(2*3.14159*i/NUMPTS) + 1); // 12-bit scale
        if (i<NUMPTS/2) triangle[i] = i*511/NUMPTS; // 8-bit scale
        else triangle[i] = 510-i*511/NUMPTS;
    }
}

void genWaves(int freq) {
    int i, j;
    int microPeriod = 1000000/(NUMPTS*freq);

    noInterrupts(); // disable interrupts to get accurate timing
    for (i=0; i<2000; i++){
        for (j=0; j<NUMPTS; j++) {
            SYS_TIMER_C1 = SYS_TIMER_C10 + microPeriod; // Set time between samples
            SYS_TIMER_CSbits.M1 = 1; // Clear timer match
            digitalWrite(2,1); // No load while changing inputs
            spiSendReceive16(sine[j]);
            digitalWrites(parallelPins, 8, triangle[j]);
            digitalWrite(2,0); // Load new points into DACs
            while (!SYS_TIMER_CSbits.M1); // Wait until timer matches
        }
    }
}
```

```

    }
    interrupts();
}

void main(void) {
    pioInit();

    pinsMode(parallelPins, 8, OUTPUT); // Set pins connected to the AD558 as outputs
    pinMode(2, OUTPUT);                // Make pin 2 an output to control LOAD and CE
    spiInit(1400000, 0);                // 1.4MHz SPI clock, default settings
    initWaveTables();
    genWaves(1000);
}

```

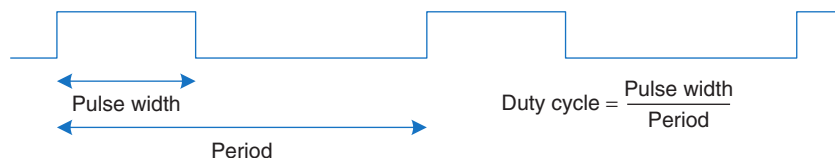
9.3.6.2 Pulse-Width Modulation

Another way for a digital system to generate an analog output is with *pulse-width modulation* (PWM), in which a periodic output is pulsed high for part of the period and low for the remainder. The duty cycle is the fraction of the period for which the pulse is high, as shown in Figure e9.20. The average value of the output is proportional to the duty cycle. For example, if the output swings between 0 and 3.3 V and has a duty cycle of 25%, the average value will be $0.25 \times 3.3 = 0.825$ V. Low-pass filtering a PWM signal eliminates the oscillation and leaves a signal with the desired average value. Thus, PWM is an effective way to produce an analog output if the pulse rate is much higher than the analog output frequencies of interest.

The BCM2835 has a PWM controller capable of producing two simultaneous outputs. PWM0 is available at GPIO18 as pin function ALT5, while both PWM outputs are available on the stereo audio jack. Figure e9.21 shows the memory map for the PWM unit and for the clock manager that it depends upon.

The PWM_CTL register is used to turn on pulse width modulation. Bit 0 (PWEN1) must be set to enable the output. Bit 7 (MSEN1: mark-space enable) should also be set to produce pulse width modulation of the form of Figure e9.20 in which the output is HIGH for part of the period and LOW for the remainder.

Figure e9.20 Pulse-width modulated (PWM) signal



The PWM signals are derived from a PWM clock generated by the BCM2835 clock manager. The PWM_RNG1 and PWM_DAT1 registers control the period and duty cycle, respectively, by specifying the number of PWM clock ticks for the overall waveform and for the HIGH portion. For example, if the clock manager produces a 25 MHz clock and PWM_RNG1 = 1000 and PWM_DAT1 = 300, the PWM output will operate at $(25 \text{ MHz} / 1000) = 25 \text{ kHz}$ and the duty cycle will be $300 / 1000 = 30\%$.

The clock manager is configured using the CM_PWMCTL and the frequency is set using the CM_PWMDIV register. Table e9.7 summarizes the bit fields of the CM_PWMCTL register. The maximum frequency of the PWM clock is 25 MHz. It can be obtained from the 500 MHz PLLD clock on the Pi as follows:

- ▶ CM_PWMCTL: Write 0x5A to PASSWD and 1 to KILL to stop the clock generator
- ▶ CM_PWMCTL: Wait for BUSY to clear to indicate the clock is stopped
- ▶ CM_PWMCTL: Write 0x5A to PASSWD, 1 to MASH, and 6 to SRC to select PLLD with no audio noise shaping
- ▶ CM_PWMDIV: Write 0x5A to PASSWD and 20 to bits 23:12 to divide PLLD by 20 from 500 MHz down to 25 MHz
- ▶ CM_PWMCTL: Write 0x5A to PASSWD and 1 to ENAB to restart the clock generator
- ▶ CM_PWMCTL: Wait for BUSY to set to indicate the clock is running

...	...
0x2020C014	PWM_DAT1
0x2020C010	PWM_RNG1
...	...
0x2020C000	PWM_CTL
...	...
0x201010A4	CM_PWMDIV
0x201010A0	CM_PWMCTL
...	...

Figure e9.21 PWM and clock manager registers

The CM_PWM registers are not documented in the BCM2835 datasheet. You may find information on them by searching the Internet for “BCM2835 Audio & PWM Clocks” by G.J. van Loo.

Table e9.7 CM_PWMCTL register fields

Bit	Name	Description
31:24	PASSWD	Must be set to 5A when writing
10:9	MASH	Audio noise shaping
7	BUSY	Clock generator running
5	KILL	Write a 1 to stop the clock generator
4	ENAB	Write a 1 to start the clock generator
3:0	SRC	Clock source

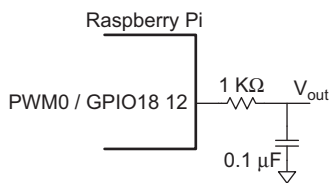


Figure e9.22 Analog output using PWM and low-pass filter

Example e9.8 ANALOG OUTPUT WITH PWM

Write an `analogWrite(val)` function to generate an analog output voltage using PWM and an external RC filter. The function should accept an input between 0 (for 0 V output) and 255 (for full 3.3 V output).

Solution: Use PWM0 to produce a 78.125 kHz signal on GPIO18. The low pass filter in Figure e9.22 has a corner frequency of

$$f_c = \frac{1}{2\pi RC} = 1.6 \text{ kHz}$$

to eliminate the high-speed oscillations and pass the average value.

The PWM functions in EasyPIO are given below. `pwmInit` initializes the PWM module on GPIO18 as described above. `setPWM` sets the frequency and duty cycle of the PWM output. Duty should be between 0 (always OFF) and 1 (always ON). The `analogWrite` function sets the duty cycle based on a full scale of 255.

```
// Default PLLD value is 500 [MHz]
#define PLL_FREQUENCY 500000000
// Max pwm clk is 25 [MHz]
#define CM_FREQUENCY 25000000
#define PLL_CLOCK_DIVISOR (PLL_FREQUENCY / CM_FREQUENCY)

void pwmInit() {
    pinMode(18, ALT5);

    // Configure the clock manager to generate a 25 MHz PWM clock.
    // Documentation on the clock manager is missing in the datasheet
    // but found in "BCM2835 Audio and PWM Clocks" by G.J. van Loo 6 Feb 2013.
    // Maximum operating frequency of PWM clock is 25 MHz.
    // Writes to the clock manager registers require simultaneous writing
    // a "password" of 5A to the top bits to reduce the risk of accidental writes.

    CM_PWMCTL = 0; // Turn off PWM before changing
    CM_PWMCTL = PWM_CLK_PASSWORD|0x20; // Turn off clock generator
    while (CM_PWMCTLbits.BUSY); // Wait for generator to stop
    CM_PWMCTL = PWM_CLK_PASSWORD|0x206; // Src = unfiltered 500 MHz CLKD
    CM_PWMDIV = PWM_CLK_PASSWORD|(PLL_CLOCK_DIVISOR << 12); // 25 MHz
    CM_PWMCTL = CM_PWMCTL|PWM_CLK_PASSWORD|0x10; // Enable PWM clock
    while (!CM_PWMCTLbits.BUSY); // Wait for generator to start
    PWM_CTLbits.MSEN1 = 1; // Channel 1 in mark/space mode
    PWM_CTLbits.PWEN1 = 1; // Enable PWM
}

void setPWM(float freq, float duty) {
    PWM_RNG1 = (int)(CM_FREQUENCY / freq);
    PWM_DAT1 = (int)(duty * (CM_FREQUENCY / freq));
}
```



```
void analogWrite(int val) {
    setPWM(78125, val/255.0);
}
```

The main function tests the PWM by setting the output to half scale (1.65 V).

```
#include "EasyPI0.h"

void main(void) {
    pioInit();
    pwmInit();
    analogWrite(128);
}
```

9.3.6.3 A/D Conversion

The BCM2835 has no built-in ADC, so this section describes A/D conversion using an external converter similar to the external DAC.

Example e9.9 ANALOG INPUT WITH AN EXTERNAL ADC

Interface a 10-bit MCP3002 A/D converter to a Raspberry Pi using SPI and print the input value. Set a full scale voltage of 3.3 V. Search for the datasheet on the Web for full details of operation.

Solution: Figure e9.23 shows a schematic of the connection. The MCP3002 uses VDD as its full scale reference. It accepts a 3.3–5.5 V supply and we

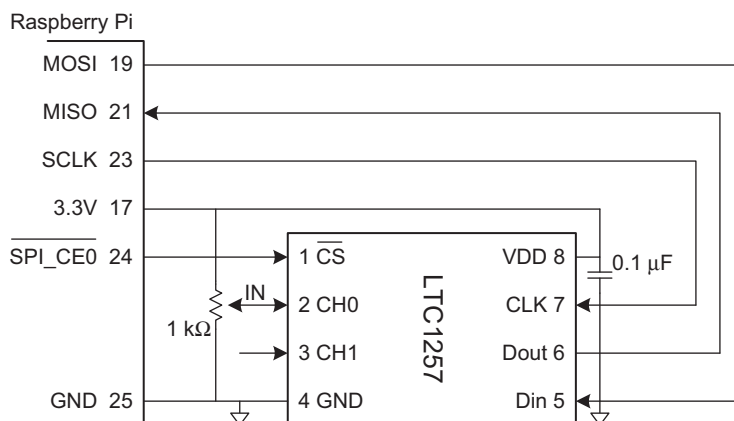


Figure e9.23 Analog input using external ADC

choose 3.3 V. The ADC has two input channels, and we connect channel 0 to a potentiometer that we can rotate to adjust the input voltage between 0 and 3.3 V.

The Pi code initializes the SPI and repeatedly reads and prints samples. According to the datasheet, the Raspberry Pi must send the 16-bit quantity 0x6000 over SPI to read CH0 and will receive the 10-bit result back in the bottom 10 bits of the 16-bit result. The converter also requires a chip select signal, conveniently provided by the SPI chip enable.

```
#include "EasyPIO.h"

void main(void) {
    int sample;

    pioInit();
    spiInit(200000, 0); // 200 kHz SPI clock, default settings

    while (1){
        sample = spiSendReceive16(0x6000);
        printf("Read %d\n", sample);
    }
}
```

9.3.7 Interrupts

So far, we have relied on *polling*, in which the program continually checks until an event occurs such as data arriving on a UART or a timer reaching its compare value. This can be a waste of the processor's power and makes it difficult to write programs that do interesting work while simultaneously waiting for events to occur.

Most microcontrollers support *interrupts*. When an event occurs, the microcontroller can stop regular program execution and jump to an interrupt handler that responds to the interrupt, then return seamlessly to where it left off.

The Raspberry Pi normally runs Linux, which intercepts interrupts before they get to the program. Therefore, it is presently not straightforward to write interrupt-based programs and this text does not provide examples on the Pi.

9.4 OTHER MICROCONTROLLER PERIPHERALS

Microcontrollers frequently interface with other external peripherals. This section describes a variety of common examples, including character-mode liquid crystal displays (LCDs), VGA monitors, Bluetooth wireless links, and motor control. Standard communication interfaces including USB and Ethernet are described in [Sections 9.6.1 and 9.6.4](#).

9.4.1 Character LCDs

A character LCD is a small liquid crystal display capable of showing one or a few lines of text. They are commonly used in the front panels of appliances such as cash registers, laser printers, and fax machines that need to display a limited amount of information. They are easy to interface with a microcontroller over parallel, RS-232, or SPI interfaces. Crystallfontz America sells a wide variety of character LCDs ranging from 8 columns $\times$ 1 row to 40 columns $\times$ 4 rows with choices of color, back-light, 3.3 or 5 V operation, and daylight visibility. Their LCDs can cost \$20 or more in small quantities, but prices come down to under \$5 in high volume.

This section gives an example of interfacing a Raspberry Pi to a character LCD over an 8-bit parallel interface. The interface is compatible with the industry-standard HD44780 LCD controller originally developed by Hitachi. Figure e9.24 shows a Crystallfontz CFAH2002A-TMI-JT 20 $\times$ 2 parallel LCD.

Figure e9.25 shows the LCD connected to a Pi over an 8-bit parallel interface. The logic operates at 5 V but is compatible with 3.3 V inputs from the Pi. The LCD contrast is set by a second voltage produced with a potentiometer; it is usually most readable at a setting of 4.2–4.8 V. The LCD receives three control signals: RS (1 for characters, 0 for instructions), $R/\overline{W}$ (1 to read from the display, 0 to write), and E (pulsed high for at least 250 ns to enable the LCD when the next byte is ready). When the instruction is read, bit 7 returns the busy flag, indicating 1 when busy and 0 when the LCD is ready to accept another instruction.

To initialize the LCD, the Pi must write a sequence of instructions to the LCD as given in Table e9.8. The instructions are written by holding $RS = 0$ and $R/\overline{W} = 0$, putting the value on the eight data lines, and pulsing E. After each instruction, it must wait for at least a specified amount of time (or sometimes until the busy flag is clear).



Figure e9.24 Crystallfontz CFAH2002A-TMI 20 $\times$ 2 character LCD
(© 2012 Crystallfontz America; reprinted with permission.)

Figure e9.25 Parallel LCD interface

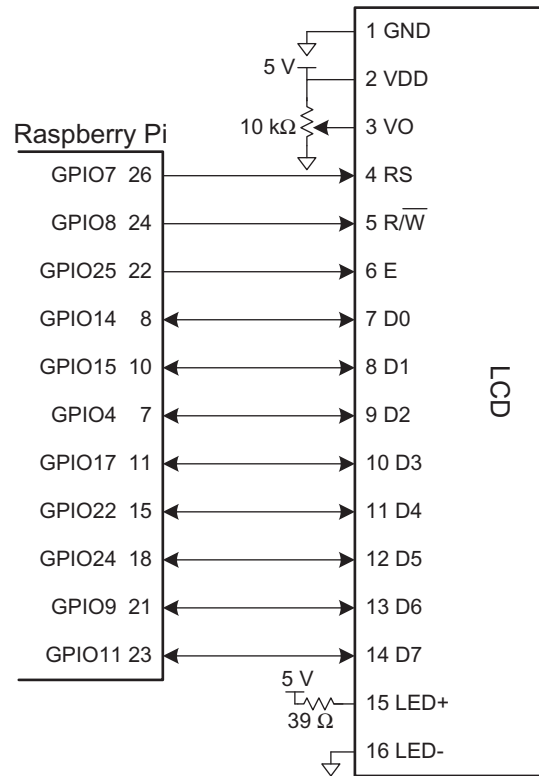


Table e9.8 LCD initialization sequence

Write	Purpose	Wait (μ s)
(apply V_{DD})	Allow device to turn on	15000
0x30	Set 8-bit mode	4100
0x30	Set 8-bit mode again	100
0x30	Set 8-bit mode yet again	Until busy flag is clear
0x3C	Set 2 lines and 5 × 8 dot font	Until busy flag is clear
0x08	Turn display OFF	Until busy flag is clear
0x01	Clear display	1530
0x06	Set entry mode to increment cursor after each character	Until busy flag is clear
0x0C	Turn display ON with no cursor	

Then, to write text to the LCD, the Pi can send a sequence of ASCII characters. After each character, it must wait for the busy bit to clear. It may also send the instruction 0x01 to clear the display or 0x02 to return to the home position in the upper left.

Example e9.10 LCD CONTROL

Write a program to print “I love LCDs” to a character display.

Solution: The following program writes “I love LCDs” to the display by initializing the display and then sending the characters.

```
#include "EasyPIO.h"

int LCD_IO_Pins[] = {14, 15, 4, 17, 22, 24, 9, 11};

typedef enum {INSTR, DATA} mode;
#define RS 7
#define RW 8
#define E 25

char lcdRead(mode md) {
    char c;
    pinsMode(LCD_IO_Pins, 8, INPUT);
    digitalWrite(RS, (md == DATA)); // Set instr/data mode
    digitalWrite(RW, 1); // Read mode
    digitalWrite(E, 1); // Pulse enable
    delayMicros(10); // Wait for LCD response
    c = digitalReads(LCD_IO_Pins, 8); // Read a byte from parallel port
    digitalWrite(E, 0); // Turn off enable
    delayMicros(10);
    return c;
}

void lcdBusyWait(void) {
    char state;
    do {
        state = lcdRead(INSTR);
    } while (state & 0x80);
}

void lcdWrite(char val, mode md) {
    pinsMode(LCD_IO_Pins, 8, OUTPUT);
    digitalWrite(RS, (md == DATA)); // Set instr/data mode. OUTPUT=1, INPUT=0
    digitalWrite(RW, 0); // Set RW pin to write (aka: 0)
    digitalWrite(LCD_IO_Pins, 8, val); // Write the char to the parallel port
    digitalWrite(E, 1); delayMicros(10); // Pulse E
    digitalWrite(E, 0); delayMicros(10);
}

void lcdClear(void) {
    lcdWrite(0x01, INSTR); delayMicros(1530);
}
```

```

void lcdPrintString(char* str) {
    while (*str != 0) {
        lcdWrite(*str, DATA); lcdBusyWait();
        str++;
    }
}

void lcdInit(void) {
    pinMode(RS, OUTPUT); pinMode(RW, OUTPUT); pinMode(E, OUTPUT);
    // send initialization routine:
    delayMicros(15000);
    lcdWrite(0x30, INSTR); delayMicros(4100);
    lcdWrite(0x30, INSTR); delayMicros(100);
    lcdWrite(0x30, INSTR); lcdBusyWait();
    lcdWrite(0x3C, INSTR); lcdBusyWait();
    lcdWrite(0x08, INSTR); lcdBusyWait();
    lcdClear();
    lcdWrite(0x06, INSTR); lcdBusyWait();
    lcdWrite(0x0C, INSTR); lcdBusyWait();
}

void main(void) {
    pioInit();
    lcdInit();
    lcdPrintString("I love LCDs!");
}

```

9.4.2 VGA Monitor

A more flexible display option is to drive a computer monitor. The Raspberry Pi has built-in support for HDMI and Composite video output. This section explains the low-level details of driving a VGA monitor directly from an FPGA.

The *Video Graphics Array* (VGA) monitor standard was introduced in 1987 for the IBM PS/2 computers, with a 640×480 pixel resolution on a *cathode ray tube* (CRT) and a 15-pin connector conveying color information with analog voltages. Modern LCD monitors have higher resolution but remain backward compatible with the VGA standard.

In a cathode ray tube, an electron gun scans across the screen from left to right exciting fluorescent material to display an image. Color CRTs use three different phosphors for red, green, and blue, and three electron beams. The strength of each beam determines the intensity of each color in the pixel. At the end of each scanline, the gun must turn off for a *horizontal blanking interval* to return to the beginning of the next line. After all of the scanlines are complete, the gun must turn off again for a *vertical blanking interval* to return to the upper left corner. The process repeats about 60–75 times per second to refresh the fluorescence and give the visual illusion of a steady image. A liquid crystal display doesn't require the same electron scan gun, but uses the same VGA interface timing for compatibility.

In a 640×480 pixel VGA monitor refreshed at 59.94 Hz, the pixel clock operates at 25.175 MHz, so each pixel is 39.72 ns wide. The full screen can be viewed as 525 horizontal scanlines of 800 pixels each, but only 480 of the scanlines and 640 pixels per scan line actually convey the image, while the remainder are black. A scanline begins with a *back porch*, the blank section on the left edge of the screen. It then contains 640 pixels, followed by a blank *front porch* at the right edge of the screen and a horizontal sync (hsync) pulse to rapidly move the gun back to the left edge.

Figure e9.26(a) shows the timing of each of these portions of the scanline, beginning with the active pixels. The entire scan line is 31.778 μs long. In the vertical direction, the screen starts with a back porch at the top, followed by 480 active scan lines, followed by a front porch at the bottom and a vertical sync (vsync) pulse to return to the top to start the next frame. A new frame is drawn 60 times per second.

Figure e9.26(b) shows the vertical timing; note that the time units are now scan lines rather than pixel clocks. Higher resolutions use a faster pixel clock, up to 388 MHz at 2048×1536 at 85 Hz. For example, 1024×768 at 60 Hz can be achieved with a 65 MHz pixel clock.

The horizontal timing involves a front porch of 16 clocks, hsync pulse of 96 clocks, and back porch of 48 clocks. The vertical timing involves a front porch of 11 scan lines, vsync pulse of 2 lines, and back porch of 32 lines.

Figure e9.27 shows the pinout for a female connector coming from a video source. Pixel information is conveyed with three analog voltages for red, green, and blue. Each voltage ranges from 0–0.7 V, with more positive indicating brighter. The voltages should be 0 during the front and

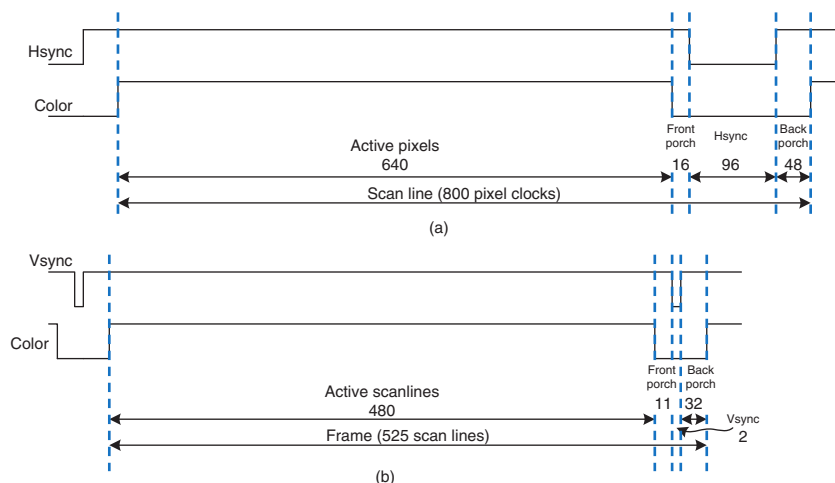


Figure e9.26 VGA timing: (a) horizontal, (b) vertical

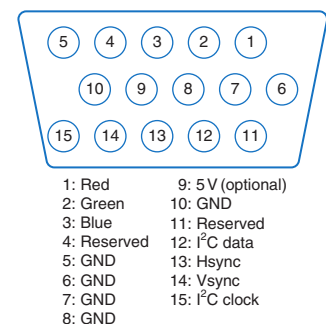
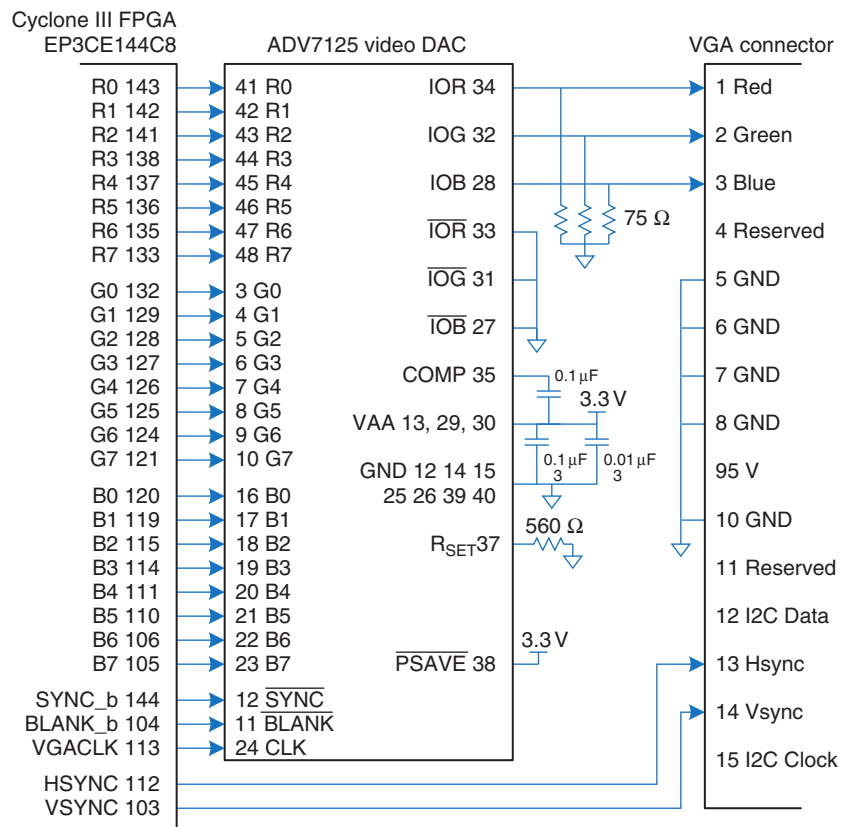


Figure e9.27 VGA connector pinout

back porches. The video signal must be generated in real time at high speed, which is difficult on a microcontroller but easy on an FPGA. A simple black and white display could be produced by driving all three color pins with either 0 or 0.7 V using a voltage divider connected to a digital output pin. A color monitor, on the other hand, uses a *video DAC* with three separate D/A converters to independently drive the three color pins. [Figure e9.28](#) shows an FPGA driving a VGA monitor through an ADV7125 triple 8-bit video DAC. The DAC receives 8 bits of R, G, and B from the FPGA. It also receives a SYNC_b signal that is driven active low whenever HSYNC or VSYNC are asserted. The video DAC produces three output currents to drive the red, green, and blue analog lines, which are normally 75 Ω transmission lines parallel terminated at both the video DAC and the monitor. The R_{SET} resistor sets the scale of the output current to achieve the full range of color. The clock rate depends on the resolution and refresh rate; it may be as high as 330 MHz with a fast-grade ADV7125JSTZ330 model DAC.

Figure e9.28 FPGA driving VGA cable through video DAC



Example e9.11 VGA MONITOR DISPLAY

Write HDL code to display text and a green box on a VGA monitor using the circuitry from [Figure e9.28](#).

Solution: The code assumes a system clock frequency of 40 MHz and uses a phase-locked loop (PLL) on the FPGA to generate the 25.175 MHz VGA clock. PLL configuration varies among FPGAs; for the Cyclone III, the frequencies are specified with Altera's megafunction wizard. Alternatively, the VGA clock could be provided directly from a signal generator.

The VGA controller counts through the columns and rows of the screen, generating the hsync and vsync signals at the appropriate times. It also produces a blank_b signal that is asserted low to draw black when the coordinates are outside the 640×480 active region.

The video generator produces red, green, and blue color values based on the current (x, y) pixel location. (0, 0) represents the upper left corner. The generator draws a set of characters on the screen, along with a green rectangle. The character generator draws an 8×8-pixel character, giving a screen size of 80×60 characters. It looks up the character from a ROM, where it is encoded in binary as 6 columns by 8 rows. The other two columns are blank. The bit order is reversed by the SystemVerilog code because the leftmost column in the ROM file is the most significant bit, while it should be drawn in the least significant x-position.

[Figure e9.29](#) shows a photograph of the VGA monitor while running this program. The rows of letters alternate red and blue. A green box overlays part of the image.

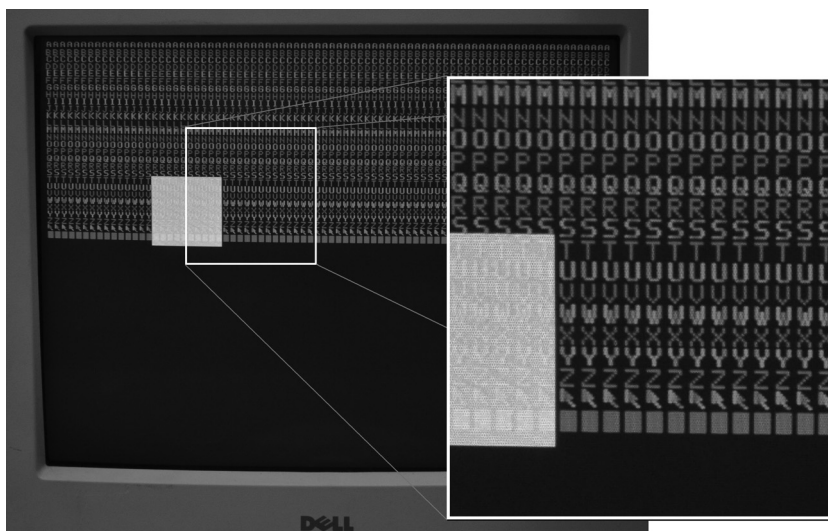


Figure e9.29 VGA output

vga.sv

```

module vga(input logic      clk,
           output logic     vgaclk,           // 25.175 MHz VGA clock
           output logic     hsync, vsync,
           output logic     sync_b, blank_b, // To monitor & DAC
           output logic [7:0] r, g, b);      // To video DAC

    logic [9:0] x, y;

    // Use a PLL to create the 25.175 MHz VGA pixel clock
    // 25.175 MHz clk period = 39.772 ns
    // Screen is 800 clocks wide by 525 tall, but only 640 x 480 used
    // HSync = 1/(39.772 ns * 800) = 31.470 kHz
    // Vsync = 31.474 kHz / 525 = 59.94 Hz (~60 Hz refresh rate)
    pll vgapll(.inclk0(clk), .c0(vgaclk));

    // Generate monitor timing signals
    vgaController vgaCont(vgaclk, hsync, vsync, sync_b, blank_b, x, y);

    // User-defined module to determine pixel color
    videoGen videoGen(x, y, r, g, b);
endmodule

module vgaController #(parameter HACTIVE = 10'd640,
                                HFP      = 10'd16,
                                HSYN     = 10'd96,
                                HBP      = 10'd48,
                                HMAX     = HACTIVE + HFP + HSYN + HBP,
                                VBP      = 10'd32,
                                VACTIVE  = 10'd480,
                                VFP      = 10'd11,
                                VSYN     = 10'd2,
                                VMAX     = VACTIVE + VFP + VSYN + VBP)
    (input logic      vgaclk,
     output logic     hsync, vsync, sync_b, blank_b,
     output logic [9:0] x, y);
    // counters for horizontal and vertical positions
    always @(posedge vgaclk) begin
        x++;
        if (x == HMAX) begin
            x = 0;
            y++;
            if (y == VMAX) y = 0;
        end
    end

    // Compute sync signals (active low)
    assign hsync = ~(hcnt >= HACTIVE + HFP & hcnt < HACTIVE + HFP + HSYN);
    assign vsync = ~(vcnt >= VACTIVE + VFP & vcnt < VACTIVE + VFP + VSYN);
    assign sync_b = hsync & vsync;

    // Force outputs to black when outside the legal display area
    assign blank_b = (hcnt < HACTIVE) & (vcnt < VACTIVE);
endmodule

module videoGen(input logic [9:0] x, y, output logic [7:0] r, g, b);

```

```

    logic    pixel, inrect;

    // Given y position, choose a character to display
    // then look up the pixel value from the character ROM
    // and display it in red or blue. Also draw a green rectangle.
    chargenrom chargenromb(y[8:3]+8'd65, x[2:0], y[2:0], pixel);
    rectgen    rectgen(x, y, 10'd120, 10'd150, 10'd200, 10'd230, inrect);
    assign {r, b} = (y[3]==0) ? {{8{pixel}}, 8'h00} : {8'h00, {8{pixel}}};
    assign g =      inrect ? 8'hFF : 8'h00;
endmodule

module chargenrom(input  logic [7:0] ch,
                  input  logic [2:0] xoff, yoff,
                  output logic      pixel);

    logic [5:0] charrom[2047:0]; // character generator ROM
    logic [7:0] line;           // a line read from the ROM

    // Initialize ROM with characters from text file
    initial
        $readmemb("charrom.txt", charrom);

    // Index into ROM to find line of character
    assign line = charrom[yoff+{ch-65, 3'b000}]; // Subtract 65 because A
                                                // is entry 0

    // Reverse order of bits
    assign pixel = line[3'd7-xoff];
endmodule

module rectgen(input  logic [9:0] x, y, left, top, right, bot,
               output logic      inrect);

    assign inrect = (x >= left & x < right & y >= top & y < bot);
endmodule

```

charrom.txt

```

// A ASCII 65
011100
100010
100010
111110
100010
100010
100010
100010
000000
//B ASCII 66
111100
100010
100010
111100
100010
100010
111100
000000

```

Bluetooth is named for King Harald Bluetooth of Denmark, a 10th century monarch who unified the warring Danish tribes. This wireless standard is only partially successful at unifying a host of competing wireless protocols!

Table e9.9 Bluetooth classes

Class	Transmitter Power (mW)	Range (m)
1	100	100
2	2.5	10
3	1	5

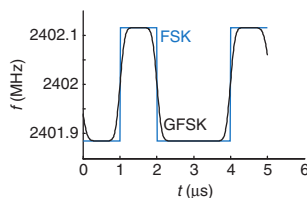


Figure e9.30 FSK and GFSK waveforms

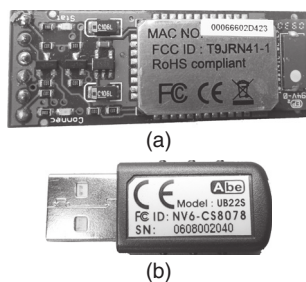


Figure e9.31 BlueSMiRF module and USB dongle

```
//C ASCII 67
011100
100010
100000
100000
100000
100010
011100
000000
...
```

9.4.3 Bluetooth Wireless Communication

There are many standards now available for wireless communication, including Wi-Fi, ZigBee, and Bluetooth. The standards are elaborate and require sophisticated integrated circuits, but a growing assortment of modules abstract away the complexity and give the user a simple interface for wireless communication. One of these modules is the BlueSMiRF, which is an easy-to-use Bluetooth wireless interface that can be used instead of a serial cable.

Bluetooth is a wireless standard initially developed by Ericsson in 1994 for low-power, moderate speed communication over distances of 5–100 meters, depending on the transmitter power level. It is commonly used to connect an earpiece to a cellphone or a keyboard to a computer. Unlike infrared communication links, it does not require a direct line of sight between devices.

Bluetooth operates in the 2.4 GHz unlicensed industrial-scientific-medical (ISM) band. It defines 79 radio channels spaced at 1 MHz intervals starting at 2402 MHz. It hops between these channels in a pseudo-random pattern to avoid consistent interference with other devices, such as wireless routers operating in the same band. As given in Table e9.9, Bluetooth transmitters are classified at one of three power levels, which dictate the range and power consumption. In the basic rate mode, it operates at 1 Mbit/sec using Gaussian frequency shift keying (FSK). In ordinary FSK, each bit is conveyed by transmitting a frequency of $f_c \pm f_d$, where f_c is the center frequency of the channel and f_d is an offset of at least 115 kHz. The abrupt transition in frequencies between bits consumes extra bandwidth. In Gaussian FSK, the change in frequency is smoothed to make better use of the spectrum. Figure e9.30 shows the frequencies being transmitted for a sequence of 0's and 1's on a 2402 MHz channel using FSK and GFSK.

A BlueSMiRF Silver module, shown in Figure e9.31(a), contains a Class 2 Bluetooth radio, modem, and interface circuitry on a small card with a serial interface. It communicates with another Bluetooth device

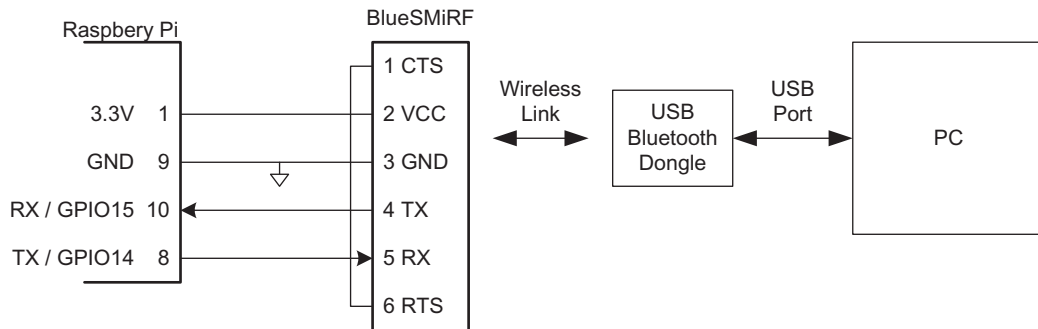


Figure e9.32 BlueSMiRF Raspberry Pi to PC link

such as a Bluetooth USB dongle connected to a PC. Thus, it can provide a wireless serial link between a Pi and a PC similar to the link from Figure e9.15 but without the cable. The wireless link is compatible with the same software as is the wired link.

Figure e9.32 shows a schematic for such a link. The TX pin of the BlueSMiRF connects to the RX pin of the Pi, and vice versa. The RTS and CTS pins are connected so that the BlueSMiRF shakes its own hand.

The BlueSMiRF defaults to 115.2 k baud with 8 data bits, 1 stop bit, and no parity or flow control. It operates at 3.3 V digital logic levels, so no RS-232 transceiver is necessary to connect with another 3.3 V device.

To use the interface, plug a USB Bluetooth dongle into a PC. Power up the Pi and BlueSMiRF. The red STAT light will flash on the BlueSMiRF indicating that it is waiting to make a connection. Open the Bluetooth icon in the PC system tray and use the Add Bluetooth Device Wizard to pair the dongle with the BlueSMiRF. The default passkey for the BlueSMiRF is 1234. Take note of which COM port is assigned to the dongle. Then communication can proceed just as it would over a serial cable. Note that the dongle typically operates at 9600 baud and that PuTTY must be configured accordingly.

9.4.4 Motor Control

Another major application of microcontrollers is to drive actuators such as motors. This section describes three types of motors: DC motors, servo motors, and stepper motors. *DC motors* require a high drive current, so a powerful driver such as an *H-bridge* must be connected between the microcontroller and the motor. They also require a *shaft encoder* if the

user wants to know the current position of the motor. *Servo motors* accept a pulse-width modulated signal to specify their position over a limited range of angles. They are very easy to interface, but are not as powerful and are not suited to continuous rotation. *Stepper motors* accept a sequence of pulses, each of which rotates the motor by a fixed angle called a step. They are more expensive and still need an H-bridge to drive the high current, but the position can be precisely controlled.

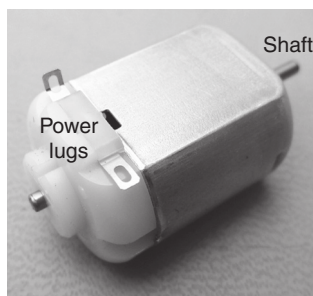
Motors can draw a substantial amount of current and may introduce glitches on the power supply that disturb digital logic. One way to reduce this problem is to use a different power supply or battery for the motor than for the digital logic.

9.4.4.1 DC Motors

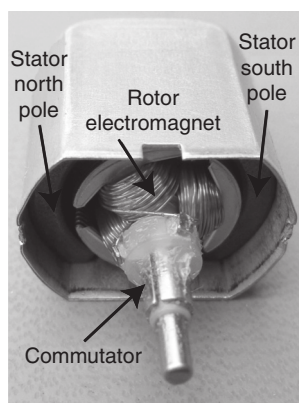
Figure e9.33 shows the structure of a brushed DC motor. The motor is a two terminal device. It contains permanent stationary magnets called the *stator* and a rotating electromagnet called the *rotor* or *armature* connected to the shaft. The front end of the rotor connects to a split metal ring called a *commutator*. Metal brushes attached to the power lugs (input terminals) rub against the commutator, providing current to the rotor's electromagnet. This induces a magnetic field in the rotor that causes the rotor to spin to become aligned with the stator field. Once the rotor has spun part way around and approaches alignment with the stator, the brushes touch the opposite sides of the commutator, reversing the current flow and magnetic field and causing it to continue spinning indefinitely.

DC motors tend to spin at thousands of rotations per minute (RPM) at very low torque. Most systems add a gear train to reduce the speed to a more reasonable level and increase the torque. Look for a gear train designed to mate with your motor. Pittman manufactures a wide range of high quality DC motors and accessories, while inexpensive toy motors are popular among hobbyists.

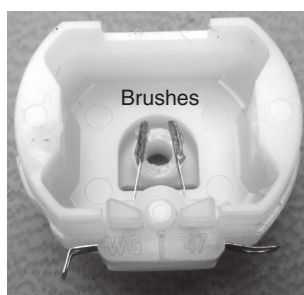
A DC motor requires substantial current and voltage to deliver significant power to a load. The current should be reversible so the motor can spin in both directions. Most microcontrollers cannot produce enough current to drive a DC motor directly. Instead, they use an H-bridge, which conceptually contains four electrically controlled switches, as shown in Figure e9.34(a). If switches A and D are closed, current flows from left to right through the motor and it spins in one direction. If B and C are closed, current flows from right to left through the motor and it spins in the other direction. If A and C or B and D are closed, the voltage across the motor is forced to 0, causing the motor to actively brake. If none of the switches are closed, the motor will coast to a stop. The switches in an H-bridge are power transistors. The H-bridge also contains some digital logic to conveniently control the switches.



(a)



(b)



(c)

Figure e9.33 DC motor

When the motor current changes abruptly, the inductance of the motor's electromagnet will induce a large voltage spike that could damage the power transistors. Therefore, many H-bridges also have protection diodes in parallel with the switches, as shown in Figure e9.34(b). If the inductive kick drives either terminal of the motor above V_{motor} or below ground, the diodes will turn ON and clamp the voltage at a safe level. H-bridges can dissipate large amounts of power so a heat sink may be necessary to keep them cool.

Example e9.12 AUTONOMOUS VEHICLE

Design a system in which a Raspberry Pi controls two drive motors for a robot car. Write a library of functions to initialize the motor driver and to make the car drive forward and back, turn left or right, and stop. Use PWM to control the speed of the motors.

Solution: Figure e9.35 shows a pair of DC motors controlled by a Pi via a Texas Instruments SN754410 dual H-bridge. The H-bridge requires a 5 V logic supply V_{CC1} and a 4.5–36 V motor supply V_{CC2} ; it has $V_{IH} = 2$ V and is hence compatible with the 3.3 V I/O from the Pi. It can deliver up to 1 A of current to each of two motors. V_{motor} should come from a separate battery pack; the 5 V output of the Pi cannot supply enough current to drive most motors and the Pi could be damaged.

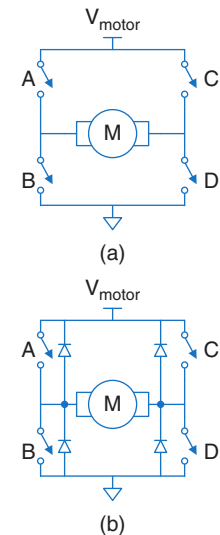


Figure e9.34 H-bridge

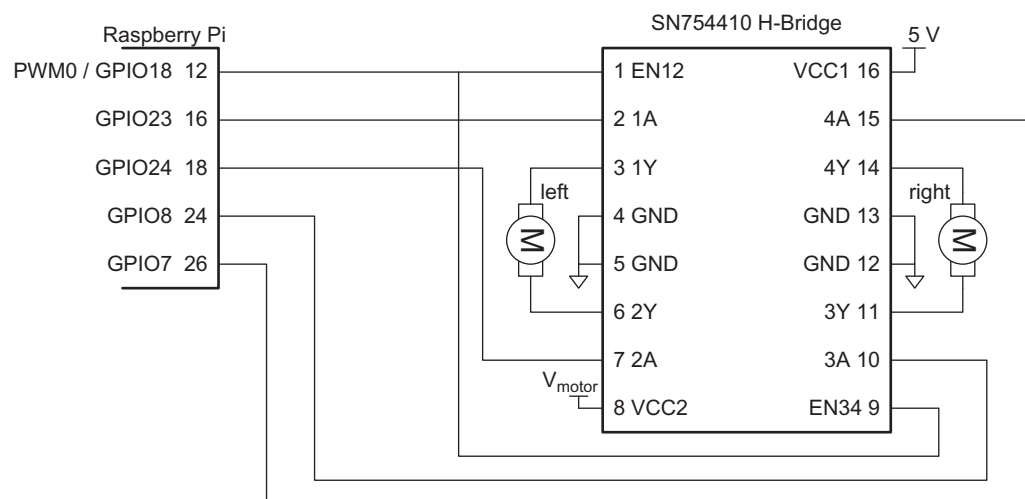


Figure e9.35 Motor control with dual H-bridge

Table e9.10 H-Bridge control

EN12	1A	2A	Motor
0	X	X	Coast
1	0	0	Brake
1	0	1	Reverse
1	1	0	Forward
1	1	1	Brake

Table e9.10 describes how the inputs to each H-bridge control a motor. The microcontroller drives the enable signals with a PWM signal to control the speed of the motors. It drives the four other pins to control the direction of each motor.

The PWM is configured to work at about 5 kHz with a duty cycle ranging from 0 to 100%. Any PWM frequency far higher than the motor's bandwidth will give the effect of smooth movement. Note that the relationship between duty cycle and motor speed is nonlinear and that below some duty cycle, the motor will not move at all.

```
#include "EasyPIO.h"

// Motor Constants
#define MOTOR_1A 23
#define MOTOR_2A 24
#define MOTOR_3A 8
#define MOTOR_4A 7

void setSpeed(float dutycycle) {           // pwmInit() must be called first.
    setPWM(5000, dutycycle);
}

void setMotorLeft(int dir) {               // dir of 1 = forward, 0 = backward
    digitalWrite(MOTOR_1A, dir);
    digitalWrite(MOTOR_2A, !dir);
}

void setMotorRight(int dir) {             // dir of 1 = forward, 0 = backward
    digitalWrite(MOTOR_3A, dir);
    digitalWrite(MOTOR_4A, !dir);
}

void forward(void) {
    setMotorLeft(1); setMotorRight(1); // Both motors drive forward
}

void backward(void) {
    setMotorLeft(0); setMotorRight(0); // Both motors drive backward
}

void left(void) {
    setMotorLeft(0); setMotorRight(1); // Left back, right forward
}

}
```



```

void right(void) {
    setMotorLeft(1); setMotorRight(0); // Right back, left forward
}

void halt(void) {                                // Turn both motors off
    digitalWrite(MOTOR_1A, 0);
    digitalWrite(MOTOR_2A, 0);
    digitalWrite(MOTOR_3A, 0);
    digitalWrite(MOTOR_4A, 0);
}

void initMotors(void) {
    pinMode(MOTOR_1A, OUTPUT);
    pinMode(MOTOR_2A, OUTPUT);
    pinMode(MOTOR_3A, OUTPUT);
    pinMode(MOTOR_4A, OUTPUT);
    halt();                                     // Ensure motors are not spinning
    pwmInit();                                 // Turn on PWM
    setSpeed(0.75);                             // Default to partial power
}

main(void) {
    pioInit();
    initMotors();
    forward(); delayMillis(5000);
    backward(); delayMillis(5000);
    left(); delayMillis(5000);
    right(); delayMillis(5000);
    halt();
}

```

In the previous example, there is no way to measure the position of each motor. Two motors are unlikely to be exactly matched, so one is likely to turn slightly faster than the other, causing the robot to veer off course. To solve this problem, some systems add shaft encoders. [Figure e9.36\(a\)](#) shows a simple shaft encoder consisting of a disk with slots attached to the motor shaft. An LED is placed on one side and a light sensor is placed on the other side. The shaft encoder produces a pulse every time the gap rotates past the LED. A microcontroller can count these pulses to measure the total angle that the shaft has turned. By using two LED/sensor pairs spaced half a slot width apart, an improved shaft encoder can produce quadrature outputs shown in [Figure e9.36\(b\)](#) that indicate the direction the shaft is turning as well as the angle by which it has turned. Sometimes shaft encoders add another hole to indicate when the shaft is at an index position.

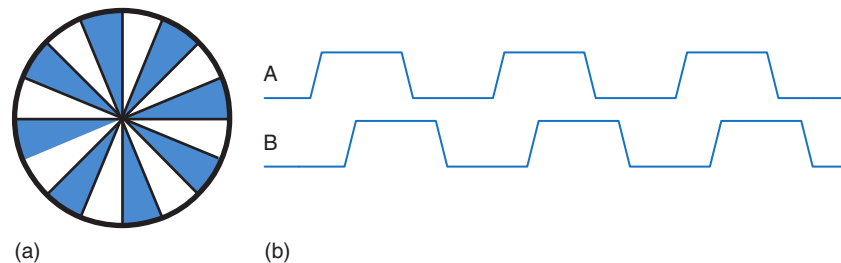


Figure e9.36 Shaft encoder (a) disk, (b) quadrature outputs

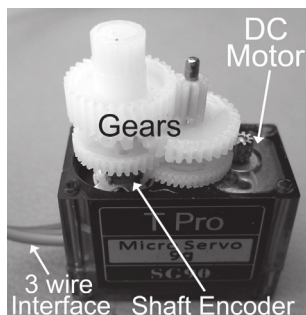


Figure e9.37 SG90 servo motor

9.4.4.2 Servo Motor

A servo motor is a DC motor integrated with a gear train, a shaft encoder, and some control logic so that it is easier to use. They have a limited rotation, typically 180° . [Figure e9.37](#) shows a servo with the lid removed to reveal the gears. A servo motor has a 3-pin interface with power (typically 5 V), ground, and a control input. The control input is typically a 50 Hz pulse-width modulated signal. The servo's control logic drives the shaft to a position determined by the duty cycle of the control input. The servo's shaft encoder is typically a rotary potentiometer that produces a voltage dependent on the shaft position.

In a typical servo motor with 180 degrees of rotation, a pulse width of 0.5 ms drives the shaft to 0° , 1.5 ms to 90° , and 2.5 ms to 180° . For example, [Figure e9.38](#) shows a control signal with a 1.5 ms pulse width. Driving the servo outside its range may cause it to hit mechanical stops and be damaged. The servo's power comes from the power pin rather than the control pin, so the control can connect directly to a microcontroller without an H-bridge. Servo motors are commonly used in remote-control model airplanes and small robots because they are small, light, and convenient. Finding a motor with an adequate datasheet can be difficult. The center pin with a red wire is normally power, and the black or brown wire is normally ground.

Example e9.13 SERVO MOTOR

Design a system in which a Raspberry Pi drives a servo motor to a desired angle.

Solution: [Figure e9.39](#) shows a diagram of the connection to an SG90 servo motor, including the colors of the wires on the servo cable. The servo operates off of a 4.0–7.2 V power supply. It can draw as much as 0.5 A if it must deliver a large amount of force, but may run directly off the Raspberry Pi power supply if the load is light. A single wire carries the PWM signal, which can be provided at 5 or 3.3 V logic levels. The code configures the PWM generation and computes

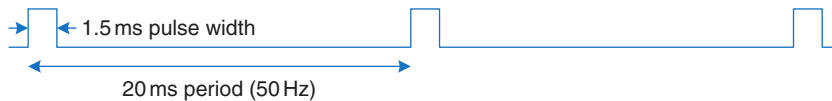


Figure e9.38 Servo control waveform

the appropriate duty cycle for the desired angle. It cycles through positioning the servo at 0, 90, and 180 degrees.

```
#include "EasyPIO.h"
void setServo(float angle) {
    setPWM(50.0, 0.025 + (0.1 * (angle / 180)));
}

void main(void) {
    pioInit();
    pwmInit();
    while (1) {
        setServo(0.0);    // Left
        delayMillis(1000);
        setServo(90.0);   // Center
        delayMillis(1000);
        setServo(180.0);  // Right
        delayMillis(1000);
    }
}
```

It is also possible to convert an ordinary servo into a continuous rotation servo by carefully disassembling it, removing the mechanical stop, and replacing the potentiometer with a fixed voltage divider. Many websites show detailed directions for particular servos. The PWM will then control the velocity rather than position, with 1.5 ms indicating stop, 2.5 ms indicating full speed forward, and 0.5 ms indicating full speed backward. A continuous rotation servo may be more convenient and less expensive than a simple DC motor combined with an H-bridge and gear train.

9.4.4.3 Stepper Motor

A stepper motor advances in discrete steps as pulses are applied to alternate inputs. The step size is usually a few degrees, allowing precise positioning and continuous rotation. Small stepper motors generally come with two sets of coils called *phases* wired in *bipolar* or *unipolar* fashion. Bipolar motors are more powerful and less expensive for a given size but require an H-bridge driver, while unipolar motors can be driven with transistors acting as switches. This section focuses on the more efficient bipolar stepper motor.

Figure e9.40(a) shows a simplified two-phase bipolar motor with a 90° step size. The rotor is a permanent magnet with one north and one south pole. The stator is an electromagnet with two pairs of coils comprising

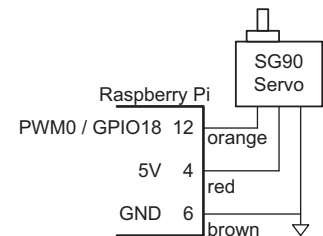


Figure e9.39 Servo motor control

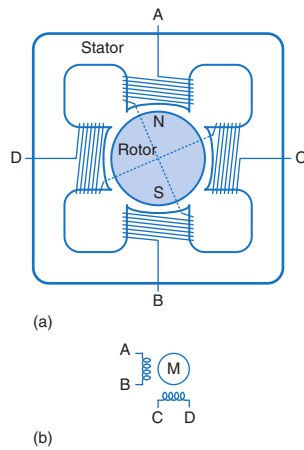


Figure e9.40 Two-phase bipolar motor: (a) simplified diagram, (b) symbol

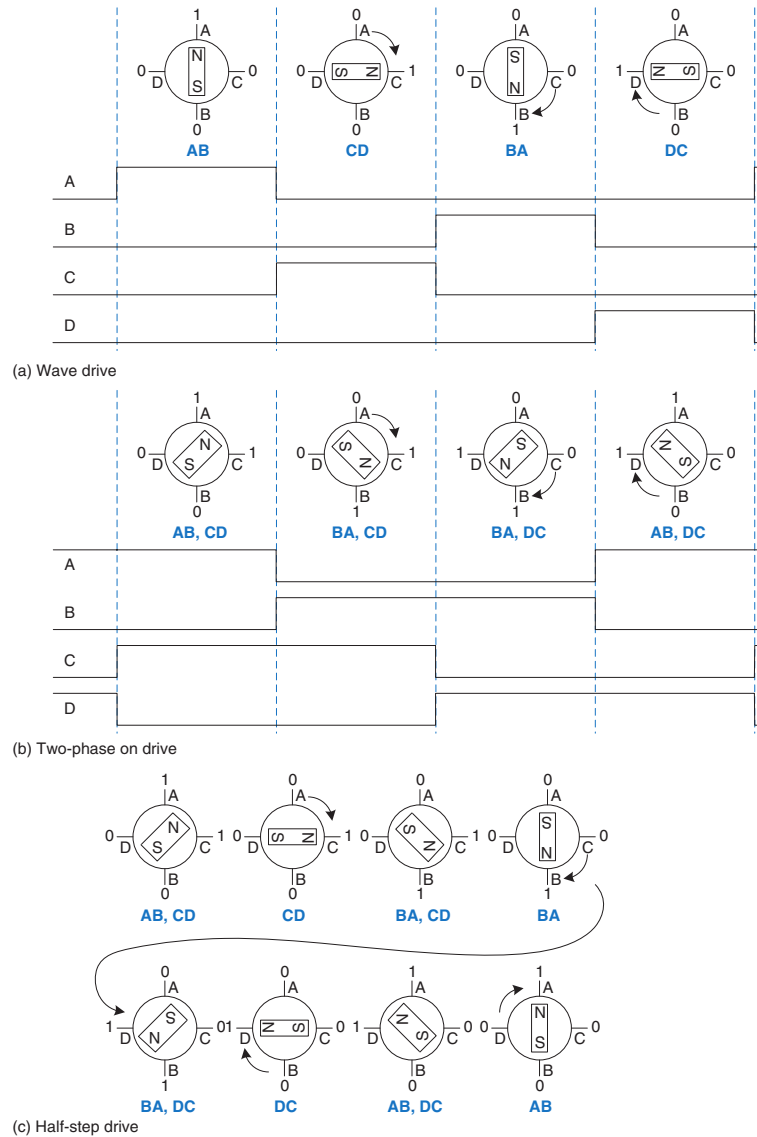


Figure e9.41 Bipolar motor drive

the two phases. Two-phase bipolar motors thus have four terminals. Figure e9.40(b) shows a symbol for the stepper motor modeling the two coils as inductors. Practical motors add gearing to reduce the output step size and increase torque.

Figure e9.41 shows three common drive sequences for a two phase bipolar motor. Figure e9.41(a) illustrates *wave drive*, in which the coils

are energized in the sequence AB – CD – BA – DC. Note that BA means that the winding AB is energized with current flowing in the opposite direction; this is the origin of the name bipolar. The rotor turns by 90 degrees at each step. Figure e9.41(b) illustrates *two-phase-on drive*, following the pattern (AB, CD) – (BA, CD) – (BA, DC) – (AB, DC). (AB, CD) indicates that both coils AB and CD are energized simultaneously. The rotor again turns by 90 degrees at each step, but aligns itself halfway between the two pole positions. This gives the highest torque operation because both coils are delivering power at once. Figure e9.41(c) illustrates *half-step drive*, following the pattern (AB, CD) – CD – (BA, CD) – BA – (BA, DC) – DC – (AB, DC) – AB. The rotor turns by 45 degrees at each half-step. The rate at which the pattern advances determines the speed of the motor. To reverse the motor direction, the same drive sequences are applied in the opposite order.

In a real motor, the rotor has many poles to make the angle between steps much smaller. For example, Figure e9.42 shows an AIRPAX LB82773-M1 bipolar stepper motor with a 7.5 degree step size. The motor operates off 5 V and draws 0.8 A through each coil.

The torque in the motor is proportional to the coil current. This current is determined by the voltage applied and by the inductance L and resistance R of the coil. The simplest mode of operation is called *direct voltage drive* or *L/R drive*, in which the voltage V is directly applied to the coil. The current ramps up to $I = V/R$ with a time constant set by L/R , as shown in Figure e9.43(a). This works well for slow speed operation. However, at higher speed, the current doesn't have enough time to ramp up to the full level, as shown in Figure e9.43(b), and the torque drops off.

A more efficient way to drive a stepper motor is by pulse-width modulating a higher voltage. The high voltage causes the current to ramp up to full current more rapidly, then it is turned off (PWM) to avoid overloading the motor. The voltage is then modulated or *chopped* to maintain the current near the desired level. This is called *chopper constant current drive* and is shown in Figure e9.43(c). The controller uses a small resistor in series with the motor to sense the current being applied by measuring the voltage drop, and applies an enable signal to the H-bridge to turn off the drive when the current reaches the desired level. In principle, a microcontroller could generate the right waveforms, but it is easier to use a stepper motor controller. The L297 controller from ST Microelectronics is a convenient choice, especially when coupled with the L298 dual H-bridge with current sensing pins and a 2 A peak power capability. Unfortunately, the L298 is not available in a DIP package so it is harder to breadboard. ST's application notes AN460 and AN470 are valuable references for stepper motor designers.

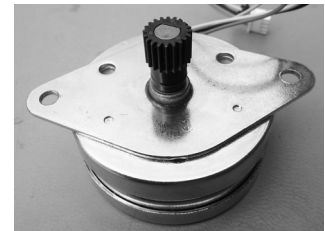


Figure e9.42 AIRPAX LB82773-M1 bipolar stepper motor

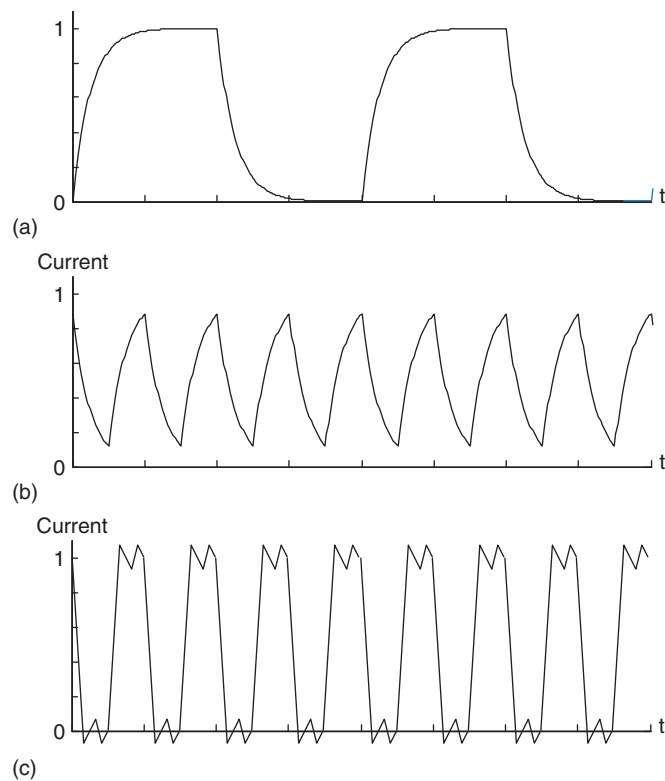


Figure e9.43 Bipolar stepper motor direct drive current: (a) slow rotation, (b) fast rotation, (c) fast rotation with chopper drive

Example e9.14 BIPOLAR STEPPER MOTOR DIRECT WAVE DRIVE

Design a system to drive an AIRPAX bipolar stepper motor at a specified speed and direction using direct wave drive.

Solution: Figure e9.44 shows the bipolar stepper motor driven directly by an H-bridge with the same interface as the DC motor. Note that VCC2 must supply enough voltage and current to meet the motor's demands or else the motor may skip steps as the rotation rate increases.

```
#include "EasyPIO.h"

#define STEPSIZE 7.5
#define SECS_PER_MIN 60
#define MICROS_PER_SEC 1000000
#define DEG_PER_REV 360

int stepperPins[] = {18, 8, 7, 23, 24};
```

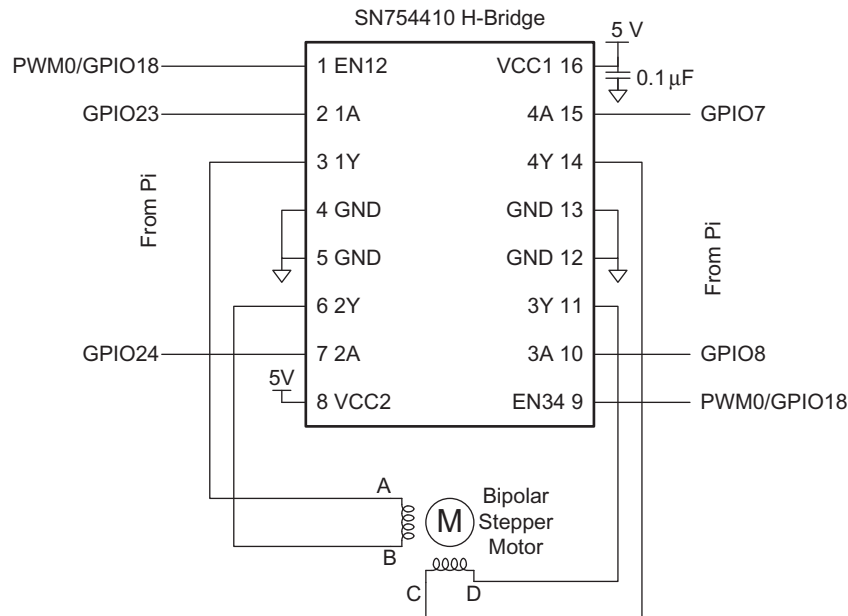


Figure e9.44 Bipolar stepper motor direct drive with H-bridge

```

int curStepState; // Keep track of the current position of stepper motor

void stepperInit(void) {
    pinsMode(stepperPins, 5, OUTPUT);
    curStepState = 0;
}

void stepperSpin(int dir, int steps, float rpm) {
    int sequence[4] = {0b00011, 0b01001, 0b00101, 0b10001}; // {2A, 1A, 4A, 3A, EN}
    int step = 0;
    unsigned int microsPerStep = (SECS_PER_MIN * MICROS_PER_SEC * STEPSIZE) /
        (rpm * DEG_PER_REV);
    for (step = 0; step < steps; step++) {
        digitalWrite(stepperPins, 5, sequence[curStepState]);
        if (dir==0) curStepState = (curStepState + 1) % 4;
        else curStepState = (curStepState + 3) % 4;
        delayMicros(microsPerStep);
    }
}

void main(void) {
    pioInit();
    stepperInit();
    stepperSpin(1, 12000, 120); // Spin 60 revolutions at 120 rpm
}

```

9.5 BUS INTERFACES

A *bus interface* connects processors to memory and/or peripherals. In general, a bus interface supports one or more *bus masters* that can initiate read or write requests to the bus and one or more *slaves* that respond to the requests; processors are normally masters and memory and peripherals are slaves.

The *Advanced Microcontroller Bus Architecture* (AMBA) is an open standard bus interface for connecting components on a chip. Introduced by ARM in 1996, it has developed through multiple revisions to boost performance and features and has become a *de facto* standard for embedded microcontrollers. The *Advanced High-performance Bus* (AHB) is one of the AMBA standards. AHB-Lite is a simplified version of AHB that supports a single bus master. This section describes AHB-Lite to illustrate the characteristics of a typical bus interface and to show how to design memory and peripherals that interface to a standard bus.

AHB is an example of a point-to-point read bus, in contrast with older bus architectures that use a single shared data bus where each slave accesses the bus via a tristate driver. Using point-to-point links between each slave and the read multiplexer allows the bus to run faster and avoids wasting power when one slave turns on its driver before another has turned off.

9.5.1 AHB-Lite

Figure e9.45 shows a simple AHB-Lite bus connecting a processor (bus master) to RAM, ROM, and two peripherals (slaves). Observe that the bus is very similar to the one from Figure e9.1 except that the names have changed. The master provides a synchronous clock (HCLK) to all of the slaves and can reset the slaves by asserting HRESETn low. The master sends an address. The address decoder uses the most significant bits to generate the HSEL signal selecting which slave to access, and the slaves use the least significant bits to define the memory location or register. The master sends HWDATA for writes. Each slave reads onto its own HRDATA, and a multiplexer chooses the data from the selected slave.

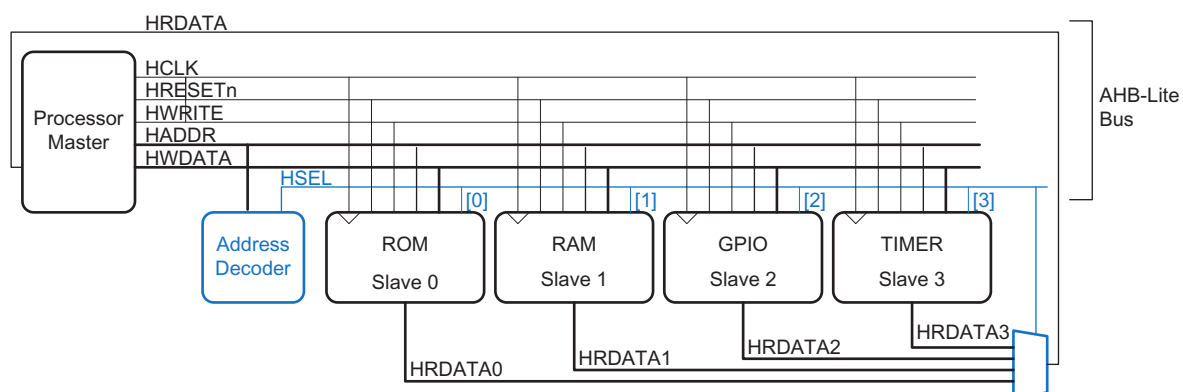


Figure e9.45 AHB-Lite bus

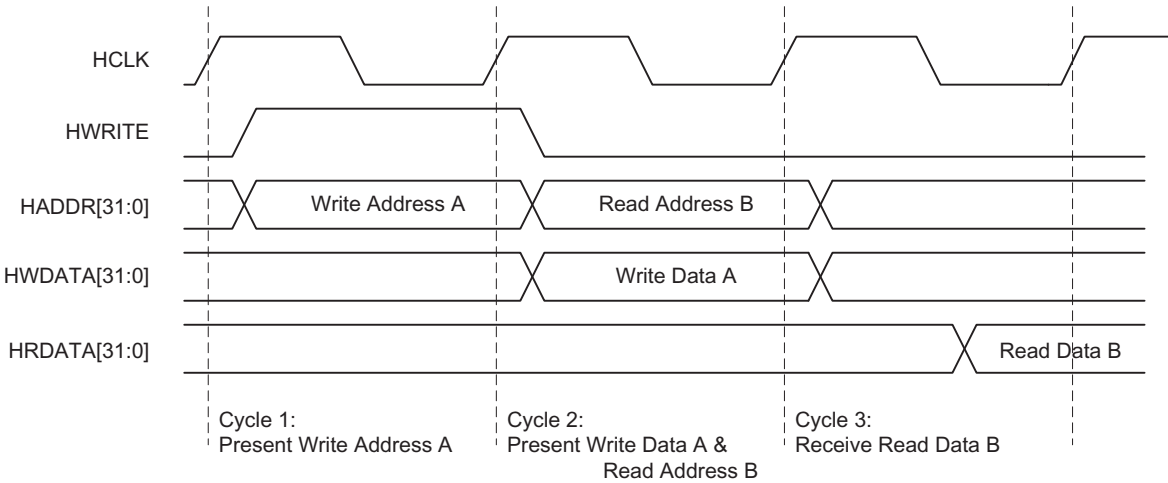


Figure e9.46 AHB-Lite transfer timing

The master sends a 32-bit address on one cycle and writes or reads data on the subsequent cycle. The write or read is called a *transfer*. For writes, the master raises HWRITE and sends the 32-bit HWDATA to write. For reads, the master lowers HWRITE and the slave responds with 32-bit HRDATA. Transfers can overlap so that the master can send the address of the next transfer while reading or writing data for the current transfer. Figure e9.46 illustrates the timing of the bus for a write followed immediately by a read. Observe how the data lags one cycle behind the address and how the two transfers partially overlap.

In this example, we assume the bus transfers a single 32-bit word at a time and that the slave responds in one clock cycle. AHB-Lite defines additional signals to specify the size of the transfer (8 – 1024 bits) and to transfer bursts of 4 to 16 elements. The master can also specify types of transfers, protection, and bus locking. Slaves can deassert HREADY to indicate that they need multiple clock cycles to respond, or can assert HRESP to indicate an error. Interested readers should consult the *AMBA 3 AHB-Lite Protocol Specification*, available online.

9.5.2 Memory and Peripheral Interface Example

This section illustrates connecting RAM, ROM, GPIO, and a timer to a processor over an AHB-Lite bus. Figure e9.47 shows a memory map for the system from Figure e9.45 with 128 KB of RAM and 64 KB of ROM. The GPIO controls 32 I/O pins. The 32-bit GPIO_DIR register controls whether each pin is an output (1) or an input (0). The 32-bit

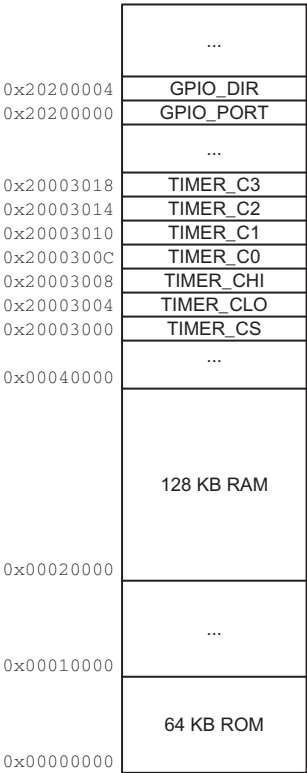


Figure e9.47 System memory map

GPIO_PORT register can be written to specify the value of outputs and read to return the values on the pins. The Timer module resembles the BCM2835 counter described in [Section 9.3.5](#), containing a 64-bit counter running at the HCLK frequency (TIMER_CHI:TIMER_CLO), four 32-bit compare channels (TIMER_C3:0), and a match register (TIMER_CS).

HDL Example e9.1 lists SystemVerilog code for the system. The decoder is based on the memory map. The memories and peripherals interface to the bus. Unnecessary signals are omitted; for example, the ROM ignores writes. The GPIO module also connects to 32 I/O pins that can behave as inputs or outputs.

HDL Example e9.1

```

module ahb_lite(input logic      HCLK,
               input logic      HRESETn,
               input logic [31:0] HADDR,
               input logic      HWRITE,
               input logic [31:0] HWDATA,
               output logic [31:0] HRDATA,
               inout tri [31:0] pins);

  logic [3:0] HSEL;
  logic [31:0] HRDATA0, HRDATA1, HRDATA2, HRDATA3;
  logic [31:0] pins_dir, pins_out, pins_in;
  logic [31:0] HADDRDEL;
  logic HWRITEDEL;

  // Delay address and write signals to align in time with data
  flop #(32) adrreg(HCLK, HADDR, HADDRDEL);
  flop #(1)   writereg(HCLK, HWRITE, HWRITEDEL);

  // Memory map decoding
  ahb_decoder dec(HADDRDEL, HSEL);
  ahb_mux mux(HSEL, HRDATA0, HRDATA1, HRDATA2, HRDATA3,
             HRDATA);

  // Memory and peripherals
  ahb_rom rom(HCLK, HSEL[0], HADDRDEL[15:2], HRDATA0);
  ahb_ram ram(HCLK, HSEL[1], HADDRDEL[16:2], HWRITEDEL,
             HWDATA, HRDATA1);
  ahb_gpio gpio(HCLK, HRESETn, HSEL[2], HADDRDEL[2],
              HWRITEDEL, HWDATA, HRDATA2, pins);
  ahb_timer timer(HCLK, HRESETn, HSEL[3], HADDRDEL[4:2],
                HWRITEDEL, HWDATA, HRDATA3);
endmodule

module ahb_decoder(input logic [31:0] HADDR,
                  output logic [3:0] HSEL);

  // Decode based on most significant bits of the address
  assign HSEL[0]=(HADDR[31:16]
    ==16'h0000); // 64KB ROM at 0x00000000 - 0x0000FFFF

  assign HSEL[1]=(HADDR[31:17]
    ==15'h0001); // 128KB RAM at 0x00020000 - 0x003FFFFF

  assign HSEL[2]=(HADDR[31:4]
    ==28'h2020000); // GPIO at 0x20200000 - 0x20200007

  assign HSEL[3]=(HADDR[31:8]
    ==24'h200030); // Timer at 0x20003000 - 0x2000301B
endmodule

module ahb_mux(input logic [3:0] HSEL,
               input logic [31:0] HRDATA0, HRDATA1, HRDATA2,
               HRDATA3,
               output logic [31:0] HRDATA);

  always_comb
  casez(HSEL)
    4'b???1: HRDATA <= HRDATA0;
    4'b??10: HRDATA <= HRDATA1;
    4'b?100: HRDATA <= HRDATA2;
    4'b1000: HRDATA <= HRDATA3;
  endcase
endmodule

module ahb_ram(input logic      HCLK,
               input logic      HSEL,
               input logic [16:2] HADDR,
               input logic      HWRITE,
               input logic [31:0] HWDATA,
               output logic [31:0] HRDATA);

  logic [31:0] ram[32767:0]; // 128KB RAM organized as 32K x 32 bits

  assign HRDATA = ram[HADDR]; // *** check addressing is correct

  always_ff @(posedge HCLK)
    if (HWRITE & HSEL) ram[HADDR] <= HWDATA;
endmodule

module ahb_rom(input logic      HCLK,
               input logic      HSEL,
               input logic [16:2] HADDR,
               output logic [31:0] HRDATA);

  logic [31:0] rom[16383:0]; // 64KB ROM organized as 16K x 32 bits

  // *** load ROM from disk file

  assign HRDATA = rom[HADDR]; // *** check addressing is correct
endmodule

```

```

module ahb_gpio(input logic HCLK,
                input logic HRESETn,
                input logic HSEL,
                input logic [2] HADDR,
                input logic HWRITE,
                input logic [31:0] HWDATA,
                output logic [31:0] HRDATA,
                output logic [31:0] pin_dir,
                output logic [31:0] pin_out,
                input logic [31:0] pin_in);

    logic [31:0] gpio[1:0]; // GPIO registers

    // write selected register
    always_ff @(posedge HCLK or negedge HRESETn)
        if (~HRESETn) begin
            gpio[0] <= 32'b0; // GPIO_PORT
            gpio[1] <= 32'b0; // GPIO_DIR
        end else if (HWRITE & HSEL)
            gpio[HADDR] <= HWDATA;

    // read selected register
    assign HRDATA = HADDR ? gpio[1] : pin_in;

    // send value and direction to I/O drivers
    assign pin_out = gpio[0];
    assign pin_dir = gpio[1];
endmodule

module ahb_timer(input logic HCLK,
                 input logic HRESETn,
                 input logic HSEL,
                 input logic [4:2] HADDR,
                 input logic HWRITE,
                 input logic [31:0] HWDATA,
                 output logic [31:0] HRDATA);

    logic [31:0] timers[6:0]; // timer registers
    logic [31:0] chi, clo; // next counter value
    logic [3:0] match, clr; // determine if counter matches
                          // compare reg

    // write selected register and update timers and match
    always_ff @(posedge HCLK or negedge HRESETn)
        if (~HRESETn) begin
            timers[0] <= 32'b0; // TIMER_CS
            timers[1] <= 32'b0; // TIMER_CLO
            timers[2] <= 32'b0; // TIMER_CHI
            timers[3] <= 32'b0; // TIMER_C0
            timers[4] <= 32'b0; // TIMER_C1
            timers[5] <= 32'b0; // TIMER_C2
            timers[6] <= 32'b0; // TIMER_C3
        end else begin
            timers[0] <= {28'b0, match};

            timers[1] <= (HWRITE & HSEL & HADDR == 3'b000) ?
                          HWDATA : clo;
            timers[2] <= (HWRITE & HSEL & HADDR == 3'b000) ?
                          HWDATA : chi;
            if (HWRITE & HSEL & HADDR == 3'b011) timers[3] <= HWDATA;
            if (HWRITE & HSEL & HADDR == 3'b100) timers[4] <= HWDATA;
            if (HWRITE & HSEL & HADDR == 3'b101) timers[5] <= HWDATA;
            if (HWRITE & HSEL & HADDR == 3'b110) timers[6] <= HWDATA;
        end

    // read selected register
    assign HRDATA = timers[HADDR];

    // increment 64-bit counter as pair of TIMER_CHI, TIMER_CLO
    assign {chi, clo} = {timers[2], timers[1]} + 1;

    // generate matches: set match bit when counter matches
    // compare register
    // clear bit when a 1 is written to that position of the match
    // register
    assign clr = (HWRITE & HSEL & HADDR == 3'b000 & HWDATA[3:0]);
    assign match[0] = ~clr[0] & (timers[0][0] |
                                (timers[1] == timers[3]));
    assign match[1] = ~clr[1] & (timers[0][1] |
                                (timers[1] == timers[4]));
    assign match[2] = ~clr[2] & (timers[0][2] |
                                (timers[1] == timers[5]));
    assign match[3] = ~clr[3] & (timers[0][3] |
                                (timers[1] == timers[6]));
endmodule

module gpio_pins(input logic [31:0] pin_dir, // 1 = output,
                // 0 = input
                input logic [31:0] pin_out, // value to drive
                // on outputs
                output logic [31:0] pin_in, // value read
                // from pins
                inout tri [31:0] pin); // tristate pins

    // Individual tristate control of each pin

    // No graceful way to control tristates on a per-bit basis in
    // SystemVerilog
    genvar i;
    generate
        for (i=0; i<32; i=i+1) begin: pinloop
            assign pin[i] = pin_dir[i] ? pin_out[i] : 1'bz;
        end
    endgenerate

    assign pin_in = pin;
endmodule

```

9.6 PC I/O SYSTEMS

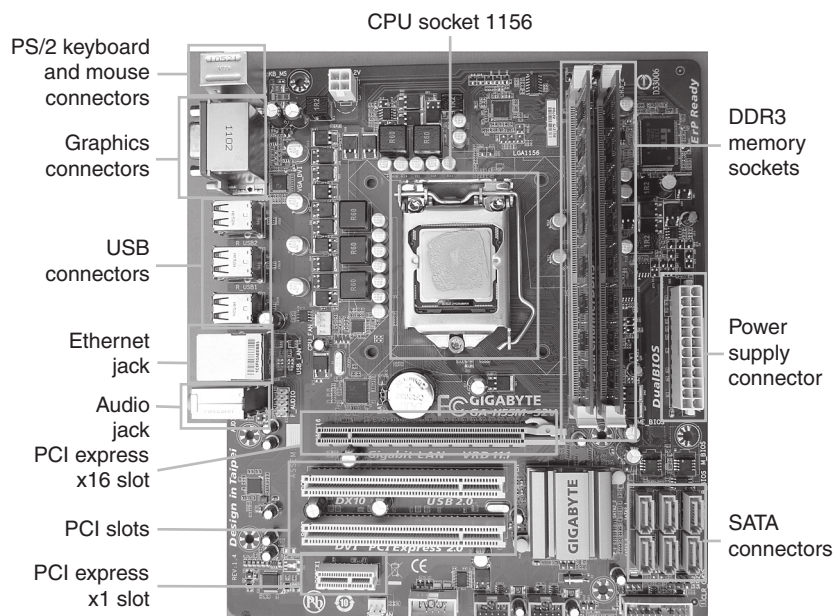
Personal computers (PCs) use a wide variety of I/O protocols for purposes including memory, disks, networking, internal expansion cards, and external devices. These I/O standards have evolved to offer very high performance and to make it easy for users to add devices. These attributes

come at the expense of complexity in the I/O protocols. This section explores the major I/O standards used in PCs and examines some options for connecting a PC to custom digital logic or other external hardware.

Figure e9.48 shows a PC motherboard for a Core i5 or i7 processor. The processor is packaged in a *land grid array* with 1156 gold-plated pads to supply power and ground to the processor and connect the processor to memory and I/O devices. The motherboard contains the DRAM memory module slots, a wide variety of I/O device connectors, and the power supply connector, voltage regulators, and capacitors. A pair of DRAM modules are connected over a DDR3 interface. External peripherals such as keyboards or webcams are attached over USB. High-performance expansion cards such as graphics cards connect over the PCI Express x16 slot, while lower-performance cards can use PCI Express x1 or the older PCI slots. The PC connects to the network using the Ethernet jack. The hard disk connects to a SATA port. The remainder of this section gives an overview of the operation of each of these I/O standards.

One of the major advances in PC I/O standards has been the development of high-speed serial links. Until recently, most I/O was built around parallel links consisting of a wide data bus and a clock signal. As data rates increased, the difference in delay among the wires in the bus set a limit to how fast the bus could run. Moreover, busses connected to multiple devices suffer from transmission line problems such as reflections and different flight times to different loads. Noise can also corrupt the data. Point-to-point serial

Figure e9.48 Gigabyte GA-H55M-S2V motherboard



links eliminate many of these problems. The data is usually transmitted on a differential pair of wires. External noise that affects both wires in the pair equally is unimportant. The transmission lines are easy to properly terminate, so reflections are small (see Section A.8 on transmission lines). No explicit clock is sent; instead, the clock is recovered at the receiver by watching the timing of the data transitions. High-speed serial link design is a specialized subject, but good links can run faster than 10 Gb/s over copper wires and even faster along optical fibers.

9.6.1 USB

Until the mid-1990s, adding a peripheral to a PC took some technical savvy. Adding expansion cards required opening the case, setting jumpers to the correct position, and manually installing a device driver. Adding an RS-232 device required choosing the right cable and properly configuring the baud rate, and data, parity, and stop bits. The *Universal Serial Bus* (USB), developed by Intel, IBM, Microsoft, and others, greatly simplified adding peripherals by standardizing the cables and software configuration process. Billions of USB peripherals are now sold each year.

USB 1.0 was released in 1996. It uses a simple cable with four wires: 5 V, GND, and a differential pair of wires to carry data. The cable is impossible to plug in backward or upside down. It operates at up to 12 Mb/s. A device can pull up to 500 mA from the USB port, so keyboards, mice, and other peripherals can get their power from the port rather than from batteries or a separate power cable.

USB 2.0, released in 2000, upgraded the speed to 480 Mb/s by running the differential wires much faster. With the faster link, USB became practical for attaching webcams and external hard disks. Flash memory sticks with a USB interface also replaced floppy disks as a means of transferring files between computers.

USB 3.0, released in 2008, further boosted the speed to 5 Gb/s. It uses the same shape connector, but the cable has more wires that operate at very high speed. It is well suited to connecting high-performance hard disks. At about the same time, USB added a Battery Charging Specification that boosts the power supplied over the port to speed up charging mobile devices.

The simplicity for the user comes at the expense of a much more complex hardware and software implementation. Building a USB interface from the ground up is a major undertaking. Even writing a simple device driver is moderately complex.

9.6.2 PCI and PCI Express

The *Peripheral Component Interconnect* (PCI) bus is an expansion bus standard developed by Intel that became widespread around 1994. It was

used to add expansion cards such as extra serial or USB ports, network interfaces, sound cards, modems, disk controllers, or video cards. The 32-bit parallel bus operates at 33 MHz, giving a bandwidth of 133 MB/s.

The demand for PCI expansion cards has steadily declined. More standard ports such as Ethernet and SATA are now integrated into the motherboard. Many devices that once required an expansion card can now be connected over a fast USB 2.0 or 3.0 link. And video cards now require far more bandwidth than PCI can supply.

Contemporary motherboards often still have a small number of PCI slots, but fast devices like video cards are now connected via *PCI Express* (PCIe). PCI Express slots provide one or more lanes of high-speed serial links. In PCIe 3.0, each lane operates at up to 8 Gb/s. Most motherboards provide an x16 slot with 16 lanes giving a total of 16 GB/s of bandwidth to data-hungry devices such as video cards.

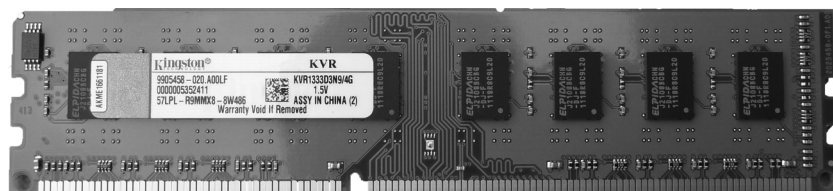
9.6.3 DDR3 Memory

DRAM connects to the microprocessor over a parallel bus. In 2015, the present standard is DDR3, a third generation of double-data rate memory bus operating at 1.5 V. Typical motherboards now come with two DDR3 channels so they can simultaneously access two banks of memory modules. DDR4 is emerging, operating at 1.2V and higher speed.

Figure e9.49 shows a 4 GB DDR3 dual inline memory module (DIMM). The module has 120 contacts on each side, for a total of 240 connections, including a 64-bit data bus, a 16-bit time-multiplexed address bus, control signals, and numerous power and ground pins. In 2015, DIMMs typically carry 1–16 GB of DRAM. Memory capacity has been doubling approximately every 2–3 years.

DRAM presently operates at a clock rate of 100–266 MHz. DDR3 operates the memory bus at four times the DRAM clock rate. Moreover, it transfers data on both the rising and falling edges of the clock. Hence, it sends 8 words of data for each memory clock. At 64 bits/word, this corresponds to 6.4–17 GB/s of bandwidth. For example, DDR3-1600 uses a 200 MHz memory clock and an 800 MHz I/O clock to send 1600 million words/sec, or 12800 MB/s. Hence, the modules are

Figure e9.49 DDR3 memory module



also called PC3-12800. Unfortunately, DRAM latency remains high, with a roughly 50 ns lag from a read request until the arrival of the first word of data.

9.6.4 Networking

Computers connect to the Internet over a network interface running the *Transmission Control Protocol and Internet Protocol* (TCP/IP). The physical connection may be an Ethernet cable or a wireless Wi-Fi link.

Ethernet is defined by the IEEE 802.3 standard. It was developed at Xerox Palo Alto Research Center (PARC) in 1974. It originally operated at 10 Mb/s (called 10 Mbit Ethernet), but now is commonly found at 100 Mbit (Mb/s) and 1 Gbit (Gb/s) running on Category 5 cables containing four twisted pairs of wires. 10 Gbit Ethernet running on fiber optic cables is increasingly popular for servers and other high-performance computing, and 100 Gbit Ethernet is emerging.

Wi-Fi is the popular name for the IEEE 802.11 wireless network standard. It operates in the 2.4 and 5 GHz unlicensed wireless bands, meaning that the user doesn't need a radio operator's license to transmit in these bands at low power. [Table e9.11](#) summarizes the capabilities of three generations of Wi-Fi; the emerging 802.11ac standard promises to push wireless data rates beyond 1 Gb/s. The increasing performance comes from advancing modulation and signal processing, multiple antennas, and wider signal bandwidths.

9.6.5 SATA

Internal hard disks require a fast interface to a PC. In 1986, Western Digital introduced the *Integrated Drive Electronics* (IDE) interface, which evolved into the *AT Attachment* (ATA) standard. The standard uses a bulky 40 or 80-wire ribbon cable with a maximum length of 18" to send data at 16–133 MB/s.

Table e9.11 802.11 Wi-Fi protocols

Protocol	Release	Frequency Band (GHz)	Data Rate (Mb/s)	Range (m)
802.11b	1999	2.4	5.5–11	35
802.11g	2003	2.4	6–54	38
802.11n	2009	2.4/5	7.2–150	70
802.11ac	2013	5	433+	variable

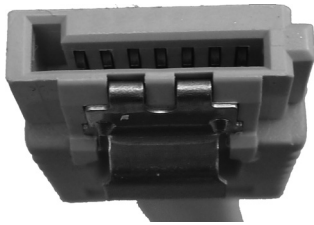


Figure e9.50 SATA cable

ATA has been supplanted by Serial ATA (SATA), which uses high-speed serial links to run at 1.5, 3, or 6 Gb/s over a more convenient 7-conductor cable shown in Figure e9.50. The fastest solid-state drives in 2015 exceed 500 MB/s of bandwidth, taking full advantage of SATA.

A related standard is Serial Attached SCSI (SAS), an evolution of the parallel SCSI (Small Computer System Interface). SAS offers performance comparable to SATA and supports longer cables; it is common in server computers.

9.6.6 Interfacing to a PC

All of the PC I/O standards described so far are optimized for high performance and ease of attachment but are difficult to implement in hardware. Engineers and scientists often need a way to connect a PC to external circuitry, such as sensors, actuators, microcontrollers, or FPGAs. The serial connection described in Section 9.3.4.2 is sufficient for a low-speed connection to a microcontroller with a UART. This section describes two more means: data acquisition systems, and USB links.

9.6.6.1 Data Acquisition Systems

Data Acquisition Systems (DAQs) connect a computer to the real world using multiple channels of analog and/or digital I/O. DAQs are now commonly available as USB devices, making them easy to install. National Instruments (NI) is a leading DAQ manufacturer.

High-performance DAQ prices tend to run into the thousands of dollars, mostly because the market is small and has limited competition. Fortunately, NI sells their handy myDAQ system at a student discount price of \$200 including their LabVIEW software. Figure e9.51 shows a myDAQ. It has two analog channels capable of input and output

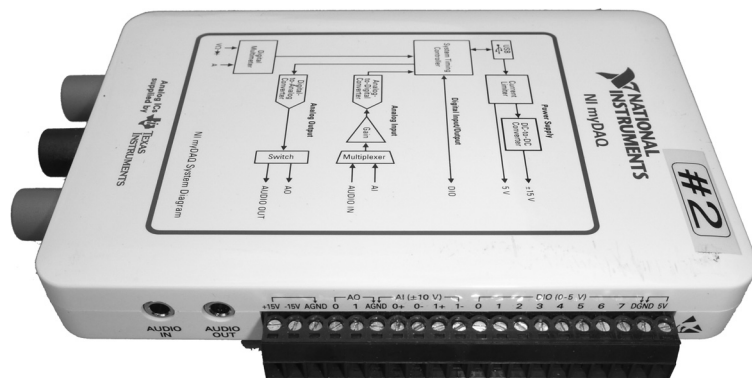


Figure e9.51 NI myDAQ

at 200 ksamples/sec with a 16-bit resolution and ± 10 V dynamic range. These channels can be configured to operate as an oscilloscope and signal generator. It also has eight digital input and output lines compatible with 3.3 and 5 V systems. Moreover, it generates 5, 15, and -15 V power supply outputs and includes a digital multimeter capable of measuring voltage, current, and resistance. Thus, the myDAQ can replace an entire bench of test and measurement equipment while simultaneously offering automated data logging.

Most NI DAQs are controlled with LabVIEW, NI's graphical language for designing measurement and control systems. Some DAQs can also be controlled from C programs using the LabWindows environment, from Microsoft .NET applications using the Measurement Studio environment, or from Matlab using the Data Acquisition Toolbox.

9.6.6.2 USB Links

An increasing variety of products now provide simple, inexpensive digital links between PCs and external hardware over USB. These products contain predeveloped drivers and libraries, allowing the user to easily write a program on the PC that blasts data to and from an FPGA or microcontroller.

FTDI is a leading vendor for such systems. For example, the FTDI C232HM-DDHSL USB to Multi-Protocol Synchronous Serial Engine (MPSSSE) cable shown in Figure e9.52 provides a USB jack at one end and, at the other end, an SPI interface operating at up to 30 Mb/s, along with 3.3 V power and four general purpose I/O pins. Figure e9.53 shows an example of connecting a PC to an FPGA using the cable. The cable can optionally supply 3.3 V power to the FPGA. The three SPI pins connect to an FPGA slave device like the one from Example e9.4. The figure also shows one of the GPIO pins used to drive an LED.

The PC requires the D2XX dynamically linked library driver to be installed. You can then write a C program using the library to send data over the cable.



Figure e9.52 FTDI USB to MPSSSE cable

(© 2012 by FTDI; reprinted with permission.)

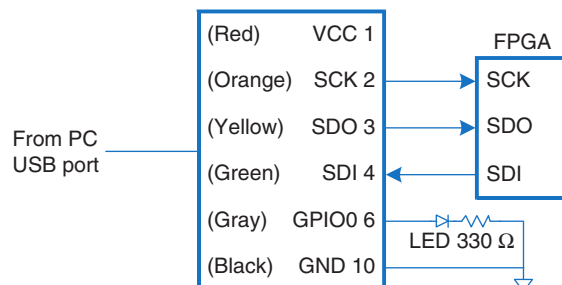


Figure e9.53 C232HM-DDHSL USB to MPSESE interface from PC to FPGA

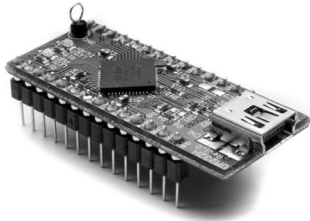


Figure e9.54 FTDI UM232H module

(© 2012 by FTDI; reprinted with permission.)

If an even faster connection is required, the FTDI UM232H module shown in [Figure e9.54](#) links a PC's USB port to an 8-bit synchronous parallel interface operating up to 40 MB/s.

9.7 SUMMARY

Most processors use memory-mapped I/O to communicate with the real world. Microcontrollers offer a range of basic peripherals including general-purpose, serial, and analog I/O and timers. PCs and advanced microcontrollers support more complex I/O standards including USB, Ethernet, and SATA.

This chapter has provided many specific examples of I/O using the Raspberry Pi. Embedded system designers continually encounter new processors and peripherals. The general principal for simple embedded I/O is to consult the datasheet to identify the peripherals that are available and which pins and memory-mapped I/O registers are involved. Then it is usually straightforward to write a simple device driver that initializes the peripheral and then transmits or receives data.

For more complex standards such as USB, writing a device driver is a highly specialized undertaking best done by an expert with detailed knowledge of the device and the USB protocol stack. Casual designers should select a processor that comes with proven device drivers and example code for the devices of interest.

Digital System Implementation



A.1 INTRODUCTION

This appendix introduces practical issues in the design of digital systems. The material is not necessary for understanding the rest of the book, however, it seeks to demystify the process of building real digital systems. Moreover, we believe that the best way to understand digital systems is to build and debug them yourself in the laboratory.

Digital systems are usually built using one or more chips. One strategy is to connect together chips containing individual logic gates or larger elements such as arithmetic/logical units (ALUs) or memories. Another is to use programmable logic, which contains generic arrays of circuitry that can be programmed to perform specific logic functions. Yet a third is to design a custom integrated circuit containing the specific logic necessary for the system. These three strategies offer trade-offs in cost, speed, power consumption, and design time that are explored in the following sections. This appendix also examines the physical packaging and assembly of circuits, the transmission lines that connect the chips, and the economics of digital systems.

The rest of this chapter is available online as a downloadable PDF from the book's companion site: <http://booksite.elsevier.com/9780128000564>.

- A.1 Introduction
- A.2 74xx Logic
- A.3 Programmable Logic
- A.4 Application-Specific Integrated Circuits
- A.5 Data sheets
- A.6 Logic Families
- A.7 Packaging and Assembly
- A.8 Transmission Lines
- A.9 Economics

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Digital System Implementation



A.1 INTRODUCTION

This appendix introduces practical issues in the design of digital systems. The material is not necessary for understanding the rest of the book, however, it seeks to demystify the process of building real digital systems. Moreover, we believe that the best way to understand digital systems is to build and debug them yourself in the laboratory.

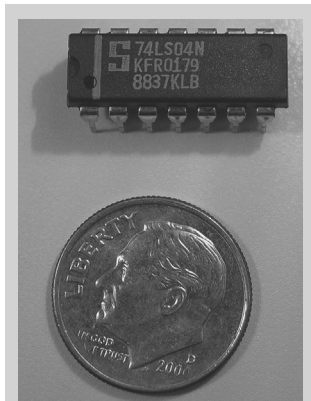
Digital systems are usually built using one or more chips. One strategy is to connect together chips containing individual logic gates or larger elements such as arithmetic/logical units (ALUs) or memories. Another is to use programmable logic, which contains generic arrays of circuitry that can be programmed to perform specific logic functions. Yet a third is to design a custom integrated circuit containing the specific logic necessary for the system. These three strategies offer trade-offs in cost, speed, power consumption, and design time that are explored in the following sections. This appendix also examines the physical packaging and assembly of circuits, the transmission lines that connect the chips, and the economics of digital systems.

A.2 74xx LOGIC

In the 1970s and 1980s, many digital systems were built from simple chips, each containing a handful of logic gates. For example, the 7404 chip contains six NOT gates, the 7408 contains four AND gates, and the 7474 contains two flip-flops. These chips are collectively referred to as *74xx-series* logic. They were sold by many manufacturers, typically for 10 to 25 cents per chip. These chips are now largely obsolete, but they are still handy for simple digital systems or class projects, because they are so inexpensive and easy to use. 74xx-series chips are commonly sold in 14-pin *dual inline packages* (DIPs).

- A.1 [Introduction](#)
- A.2 [74xx Logic](#)
- A.3 [Programmable Logic](#)
- A.4 [Application-Specific Integrated Circuits](#)
- A.5 [Data Sheets](#)
- A.6 [Logic Families](#)
- A.7 [Packaging and Assembly](#)
- A.8 [Transmission Lines](#)
- A.9 [Economics](#)

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	



74LS04 inverter chip in a 14-pin dual in-line package. The part number is on the first line. LS indicates the logic family (see Section A.6). The N suffix indicates a DIP package. The large S is the logo of the manufacturer, Signetics. The bottom two lines of gibberish are codes indicating the batch in which the chip was manufactured.

A.2.1 Logic Gates

Figure eA.1 shows the pinout diagrams for a variety of popular 74xx-series chips containing basic logic gates. These are sometimes called *small-scale integration* (SSI) chips, because they are built from a few transistors. The 14-pin packages typically have a notch at the top or a dot on the top left to indicate orientation. Pins are numbered starting with 1 in the upper left and going counterclockwise around the package. The chips need to receive power ($V_{DD} = 5\text{ V}$) and ground ($GND = 0\text{ V}$) at pins 14 and 7, respectively. The number of logic gates on the chip is determined by the number of pins. Note that pins 3 and 11 of the 7421 chip are not connected (NC) to anything. The 7474 flip-flop has the usual D , CLK , and Q terminals. It also has a complementary output, $\overline{Q}$. Moreover, it receives asynchronous set (also called preset, or $\overline{PRE}$) and reset (also called clear, or $\overline{CLR}$) signals. These are active low; in other words, the flop sets when $\overline{PRE} = 0$, resets when $\overline{CLR} = 0$, and operates normally when $\overline{PRE} = \overline{CLR} = 1$.

A.2.2 Other Functions

The 74xx series also includes somewhat more complex logic functions, including those shown in Figures eA.2 and eA.3. These are called *medium-scale integration* (MSI) chips. Most use larger packages to accommodate more inputs and outputs. Power and ground are still provided at the upper right and lower left, respectively, of each chip. A general functional description is provided for each chip. See the manufacturer's data sheets for complete descriptions.

A.3 PROGRAMMABLE LOGIC

Programmable logic consists of arrays of circuitry that can be configured to perform specific logic functions. We have already introduced three forms of programmable logic: programmable read only memories (PROMs), programmable logic arrays (PLAs), and field programmable gate arrays (FPGAs). This section shows chip implementations for each of these. Configuration of these chips may be performed by blowing on-chip fuses to connect or disconnect circuit elements. This is called *one-time programmable* (OTP) logic because, once a fuse is blown, it cannot be restored. Alternatively, the configuration may be stored in a memory that can be reprogrammed at will. Reprogrammable logic is convenient in the laboratory because the same chip can be reused during development.

A.3.1 PROMs

As discussed in Section 5.5.7, PROMs can be used as lookup tables. A 2^N -word $\times$ M -bit PROM can be programmed to perform any combinational function of N inputs and M outputs. Design changes simply involve

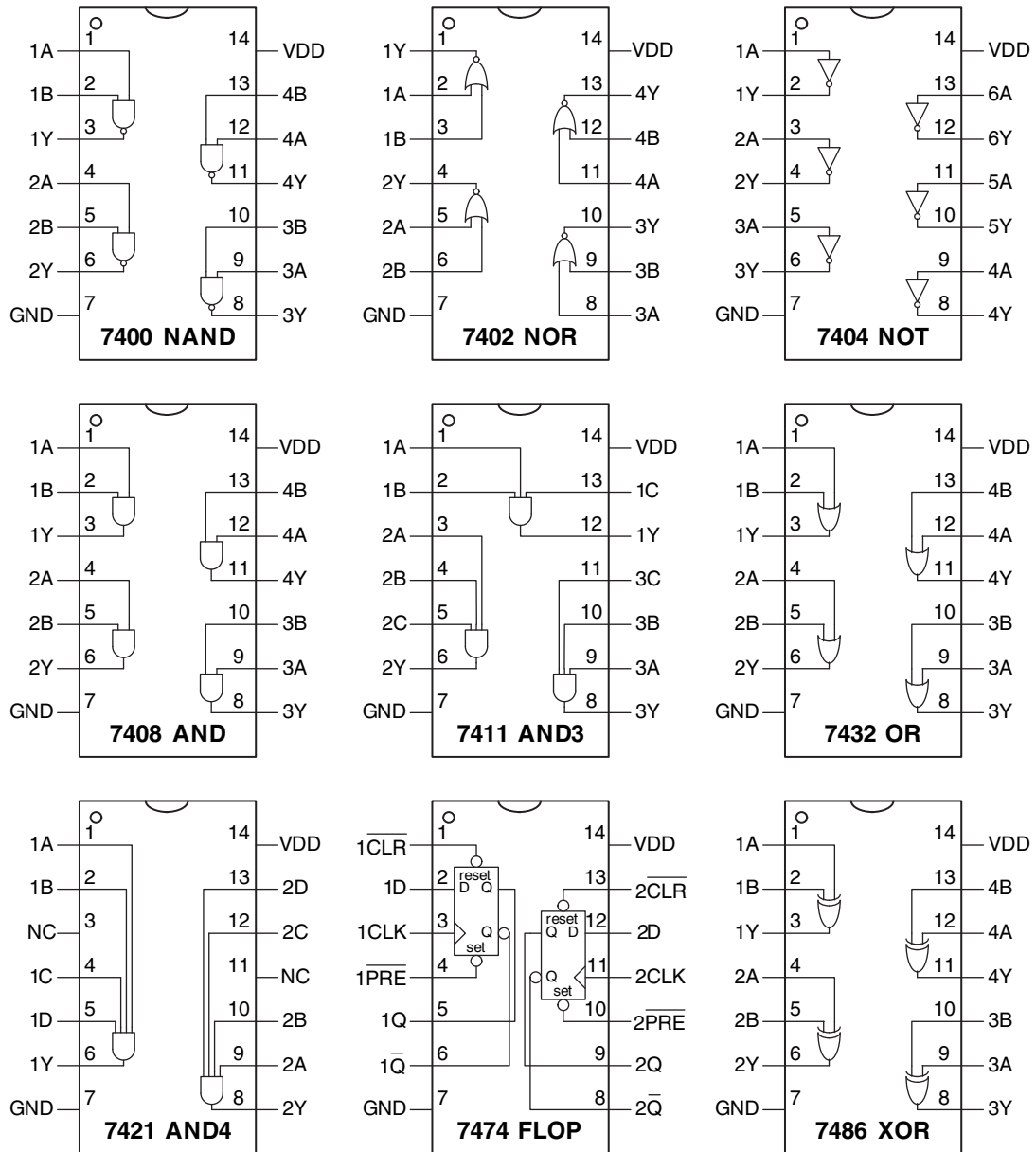
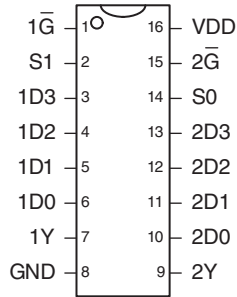


Figure eA.1 Common 74xx-series logic gates

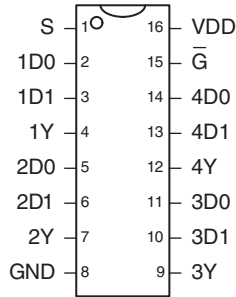
**74153 4:1 Mux****Two 4:1 Multiplexers**

D_{3:0}: data
S_{1:0}: select
Y: output
Gb: enable

```

always_comb
  if (1Gb) 1Y = 0;
  else 1Y = 1D[S];
always_comb
  if (2Gb) 2Y = 0;
  else 2Y = 2D[S];

```

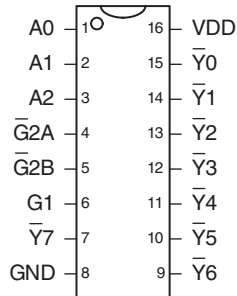
**74157 2:1 Mux****Four 2:1 Multiplexers**

D_{1:0}: data
S: select
Y: output
Gb: enable

```

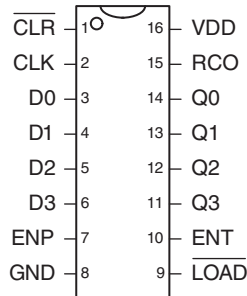
always_comb
  if (Gb) 1Y = 0;
  else 1Y = S ? 1D[1] : 1D[0];
  if (Gb) 2Y = 0;
  else 2Y = S ? 2D[1] : 2D[0];
  if (Gb) 3Y = 0;
  else 3Y = S ? 3D[1] : 3D[0];
  if (Gb) 4Y = 0;
  else 4Y = S ? 4D[1] : 4D[0];

```

**74138 3:8 Decoder****3:8 Decoder**

A_{2:0}: address
Y_{7:0}: output
G1: active high enable
G2: active low enables

G1	G2A	G2B	A2:0	Y7:0
0	x	x	xxx	11111111
1	1	x	xxx	11111111
1	0	1	xxx	11111111
1	0	0	000	11111110
1	0	0	001	11111101
1	0	0	010	11111011
1	0	0	011	11110111
1	0	0	100	11101111
1	0	0	101	11011111
1	0	0	110	10111111
1	0	0	111	01111111

**74161/163 Counter****4-bit Counter**

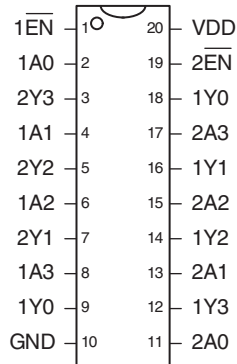
CLK: clock
Q_{3:0}: counter output
D_{3:0}: parallel input
CLRb: async reset (161)
sync reset (163)
LOADb: load Q from D
ENP, ENT: enables
RCO: ripple carry out

```

always_ff @(posedge CLK) // 74163
  if (~CLRb) Q <= 4'b0000;
  else if (~LOADb) Q <= D;
  else if (ENP & ENT) Q <= Q+1;

assign RCO = (Q == 4'b1111) & ENT;

```

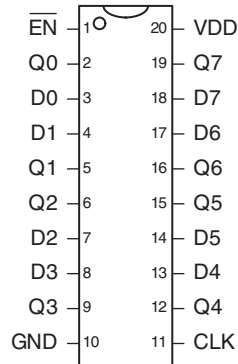
**74244 Tristate Buffer****8-bit Tristate Buffer**

A_{3:0}: input
Y_{3:0}: output
ENb: enable

```

assign 1Y =
  1ENb ? 4'bzzzz : 1A;
assign 2Y =
  2ENb ? 4'bzzzz : 2A;

```

**74377 Register****8-bit Enableable Register**

CLK: clock
D_{7:0}: data
Q_{7:0}: output
ENb: enable

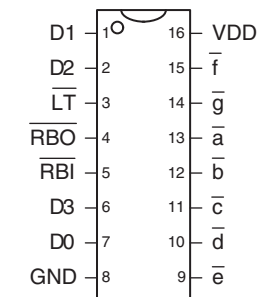
```

always_ff @(posedge CLK)
  if (~ENb) Q <= D;

```

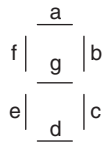
Note: SystemVerilog variable names cannot start with numbers, but the names in the example code in Figure A.2 are chosen to match the manufacturer's data sheet.

Figure eA.2 Medium-scale integration chips

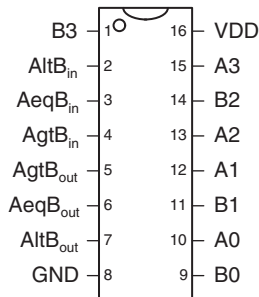


7447 7-Segment Decoder

7-segment Display Decoder
 D_{3:0}: data
 a...f: segments (low = ON)
 LTb: light test
 RBIf: ripple blanking in
 RBOb: ripple blanking out



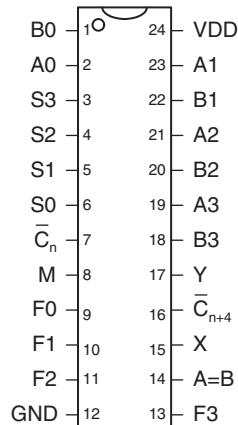
RBO	LT	RBI	D3:0	a	b	c	d	e	f	g
0	x	x	x	1	1	1	1	1	1	1
1	0	x	x	0	0	0	0	0	0	0
x	1	0	0000	1	1	1	1	1	1	1
1	1	1	0000	0	0	0	0	0	0	1
1	1	1	0001	1	0	0	1	1	1	1
1	1	1	0010	0	0	1	0	0	1	0
1	1	1	0011	0	0	0	0	1	1	0
1	1	1	0100	1	0	0	1	1	0	0
1	1	1	0101	0	1	0	0	1	0	0
1	1	1	0110	1	1	0	0	0	0	0
1	1	1	0111	0	0	0	1	1	1	1
1	1	1	1000	0	0	0	0	0	0	0
1	1	1	1001	0	0	0	1	1	0	0
1	1	1	1010	1	1	1	0	0	1	0
1	1	1	1011	1	1	0	0	1	1	0
1	1	1	1100	1	0	1	1	1	0	0
1	1	1	1101	0	1	1	0	1	0	0
1	1	1	1110	0	0	0	1	1	1	1
1	1	1	1111	0	0	0	0	0	0	0



7485 Comparator

4-bit Comparator
 A_{3:0}, B_{3:0}: data
 rel_{in}: input relation
 rel_{out}: output relation

```
always_comb
  if (A > B | (A == B & AgtBin)) begin
    AgtBout = 1; AeqBout = 0; AltBout = 0;
  end
  else if (A < B | (A == B & AltBin)) begin
    AgtBout = 0; AeqBout = 0; AltBout = 1;
  end else begin
    AgtBout = 0; AeqBout = 1; AltBout = 0;
  end
```



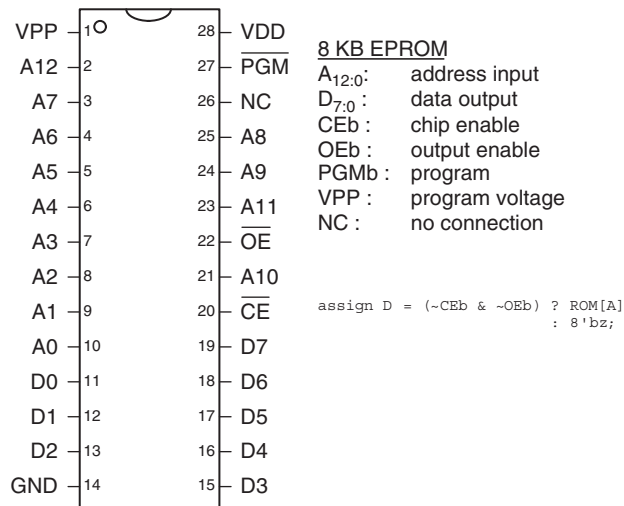
74181 ALU

4-bit ALU
 A_{3:0}, B_{3:0}: inputs
 Y_{3:0}: output
 F_{3:0}: function select
 M: mode select
 Cb_n: carry in
 Cb_{nplus4}: carry out
 AeqB: equality (in some modes)
 X, Y: carry lookahead
 adder outputs

```
always_comb
  case (F)
    0000: Y = M ? ~A : A + ~Cbn;
    0001: Y = M ? ~(A | B) : A + B + ~Cbn;
    0010: Y = M ? (~A) & B : A + ~B + ~Cbn;
    0011: Y = M ? 4'b0000 : 4'b1111 + ~Cbn;
    0100: Y = M ? ~(A & B) : A + (A & ~B) + ~Cbn;
    0101: Y = M ? ~B : (A | B) + (A & ~B) + ~Cbn;
    0110: Y = M ? A ^ B : A - B - Cbn;
    0111: Y = M ? A & ~B : (A & ~B) - Cbn;
    1000: Y = M ? ~A + B : A + (A & B) + ~Cbn;
    1001: Y = M ? ~(A ^ B) : A + B + ~Cbn;
    1010: Y = M ? B : (A | ~B) + (A & B) + ~Cbn;
    1011: Y = M ? A & B : (A & B) + ~Cbn;
    1100: Y = M ? 1 : A + A + ~Cbn;
    1101: Y = M ? A | ~B : (A | B) + A + ~Cbn;
    1110: Y = M ? A | B : (A | ~B) + A + ~Cbn;
    1111: Y = M ? A : A - Cbn;
  endcase
```

Figure eA.3 More medium-scale integration (MSI) chips

Figure eA.4 2764 8KB EPROM



replacing the contents of the PROM rather than rewiring connections between chips. Lookup tables are useful for small functions but become prohibitively expensive as the number of inputs grows.

For example, the classic 2764 8-KB (64-Kb) erasable PROM (EPROM) is shown in Figure eA.4. The EPROM has 13 address lines to specify one of the 8K words and 8 data lines to read the byte of data at that word. The chip enable and output enable must both be asserted for data to be read. The maximum propagation delay is 200 ps. In normal operation, $\overline{PGM} = 1$ and VPP is not used. The EPROM is usually programmed on a special programmer that sets $\overline{PGM} = 0$, applies 13 V to VPP , and uses a special sequence of inputs to configure the memory.

Modern PROMs are similar in concept but have much larger capacities and more pins. Flash memory is the cheapest type of PROM, selling for about \$0.30 per gigabyte in 2015. Prices have historically declined by 30 to 40% per year.

A.3.2 PLAs

As discussed in Section 5.6.1, PLAs contain AND and OR planes to compute any combinational function written in sum-of-products form. The AND and OR planes can be programmed using the same techniques for PROMs. A PLA has two columns for each input and one column for each output. It has one row for each minterm. This organization is more efficient than a PROM for many functions, but the array still grows excessively large for functions with numerous I/Os and minterms.

Many different manufacturers have extended the basic PLA concept to build *programmable logic devices* (PLDs) that include registers. The 22V10

is one of the most popular classic PLDs. It has 12 dedicated input pins and 10 outputs. The outputs can come directly from the PLA or from clocked registers on the chip. The outputs can also be fed back into the PLA. Thus, the 22V10 can directly implement FSMs with up to 12 inputs, 10 outputs, and 10 bits of state. The 22V10 costs about \$2 in quantities of 100. PLDs have been rendered mostly obsolete by the rapid improvements in capacity and cost of FPGAs.

A.3.3 FPGAs

As discussed in Section 5.6.2, FPGAs consist of arrays of configurable *logic elements (LEs)*, also called *configurable logic blocks (CLBs)*, connected together with programmable wires. The LEs contain small lookup tables and flip-flops. FPGAs scale gracefully to extremely large capacities, with thousands of lookup tables. Xilinx and Altera are two of the leading FPGA manufacturers.

Lookup tables and programmable wires are flexible enough to implement any logic function. However, they are an order of magnitude less efficient in speed and cost (chip area) than hard-wired versions of the same functions. Thus, FPGAs often include specialized blocks, such as memories, multipliers, and even entire microprocessors.

Figure eA.5 shows the design process for a digital system on an FPGA. The design is usually specified with a hardware description language (HDL), although some FPGA tools also support schematics. The design is then simulated. Inputs are applied and compared against expected outputs to *verify* that the logic is correct. Usually some debugging is required. Next, logic *synthesis* converts the HDL into Boolean functions. Good synthesis tools produce a schematic of the functions, and the prudent designer examines these schematics, as well as any warnings produced during synthesis, to ensure that the desired logic was produced. Sometimes sloppy coding leads to circuits that are much larger than intended or to circuits with asynchronous logic. When the synthesis results are good, the FPGA tool *maps* the functions onto the LEs of a specific chip. The *place and route* tool determines which functions go in which lookup tables and how they are wired together. Wire delay increases with length, so critical circuits should be placed close together. If the design is too big to fit on the chip, it must be reengineered. *Timing analysis* compares the timing constraints (e.g., an intended clock speed of 100 MHz) against the actual circuit delays and reports any errors. If the logic is too slow, it may have to be redesigned or pipelined differently. When the design is correct, a file is generated specifying the contents of all the LEs and the programming of all the wires on the FPGA. Many FPGAs store this *configuration* information in static RAM that must be reloaded each time the FPGA is

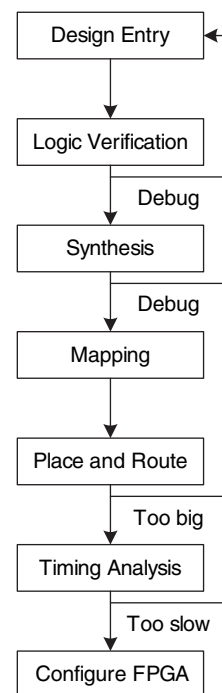


Figure eA.5 FPGA design flow

turned on. The FPGA can download this information from a computer in the laboratory, or can read it from a nonvolatile ROM when power is first applied.

Example eA.1 FPGA TIMING ANALYSIS

Alyssa P. Hacker is using an FPGA to implement an M&M sorter with a color sensor and motors to put red candy in one jar and green candy in another. Her design is implemented as an FSM, and she is using a Cyclone IV FPGA. According to the data sheet, the FPGA has the timing characteristics shown in Table eA.1.

Alyssa would like her FSM to run at 100 MHz. What is the maximum number of LEs on the critical path? What is the fastest speed at which her FSM could possibly run?

Solution: At 100 MHz, the cycle time, T_c , is 10 ns. Alyssa uses Equation 3.14 to figure out the minimum combinational propagation delay, t_{pd} , at this cycle time:

$$t_{pd} \leq 10 \text{ ns} - (0.199 \text{ ns} + 0.076 \text{ ns}) = 9.725 \text{ ns} \quad (\text{A.1})$$

With a combined LE and wire delay of $381 \text{ ps} + 246 \text{ ps} = 627 \text{ ps}$, Alyssa's FSM can use at most 15 consecutive LEs ($9.725/0.627$) to implement the next-state logic.

The fastest speed at which an FSM will run on this Cyclone IV FPGA is when it is using a single LE for the next state logic. The minimum cycle time is

$$T_c \geq 381 \text{ ps} + 199 \text{ ps} + 76 \text{ ps} = 656 \text{ ps} \quad (\text{A.2})$$

Therefore, the maximum frequency is 1.5 GHz.

Table eA.1 Cyclone IV timing

Name	Value (ps)
t_{pcq}	199
t_{setup}	76
t_{hold}	0
t_{pd} (per LE)	381
t_{wire} (between LEs)	246
t_{skew}	0

Altera advertises the Cyclone IV FPGA with 14,400 LEs for \$25 in 2015. In large quantities, medium-sized FPGAs typically cost several dollars. The largest FPGAs cost hundreds or even thousands of dollars.

The cost has declined at approximately 30% per year, so FPGAs are becoming extremely popular.

A.4 APPLICATION-SPECIFIC INTEGRATED CIRCUITS

Application-specific integrated circuits (ASICs) are chips designed for a particular purpose. Graphics accelerators, network interface chips, and cell phone chips are common examples of ASICs. The ASIC designer places transistors to form logic gates and wires the gates together. Because the ASIC is hardwired for a specific function, it is typically several times faster than an FPGA and occupies an order of magnitude less chip area (and hence cost) than an FPGA with the same function. However, the *masks* specifying where transistors and wires are located on the chip cost hundreds of thousands of dollars to produce. The fabrication process usually requires 6 to 12 weeks to manufacture, package, and test the ASICs. If errors are discovered after the ASIC is manufactured, the designer must correct the problem, generate new masks, and wait for another batch of chips to be fabricated. Hence, ASICs are suitable only for products that will be produced in large quantities and whose function is well defined in advance.

Figure eA.6 shows the ASIC design process, which is similar to the FPGA design process of Figure eA.5. Logic verification is especially important because correction of errors after the masks are produced is expensive. Synthesis produces a *netlist* consisting of logic gates and connections between the gates; the gates in this netlist are placed, and the wires are routed between gates. When the design is satisfactory, masks are generated and used to fabricate the ASIC. A single speck of dust can ruin an ASIC, so the chips must be tested after fabrication. The fraction of manufactured chips that work is called the *yield*; it is typically 50 to 90%, depending on the size of the chip and the maturity of the manufacturing process. Finally, the working chips are placed in packages, as will be discussed in Section A.7.

A.5 DATA SHEETS

Integrated circuit manufacturers publish *data sheets* that describe the functions and performance of their chips. It is essential to read and understand the data sheets. One of the leading sources of errors in digital systems comes from misunderstanding the operation of a chip.

Data sheets are usually available from the manufacturer's Web site. If you cannot locate the data sheet for a part and do not have clear documentation from another source, don't use the part. Some of the entries in the data sheet may be cryptic. Often the manufacturer publishes data books containing data sheets for many related parts. The beginning of

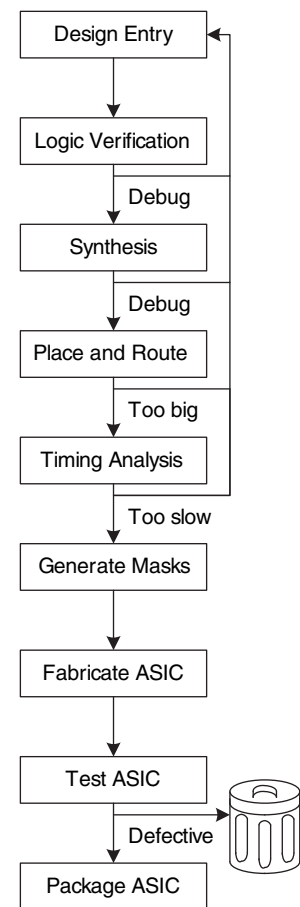


Figure eA.6 ASIC design flow

the data book has additional explanatory information. This information can usually be found on the Web with a careful search.

This section dissects the Texas Instruments (TI) data sheet for a 74HC04 inverter chip. The data sheet is relatively simple but illustrates many of the major elements. TI still manufactures a wide variety of 74xx-series chips. In the past, many other companies built these chips too, but the market is consolidating as the sales decline.

Figure eA.7 shows the first page of the data sheet. Some of the key sections are highlighted in blue. The title is SN54HC04, SN74HC04 HEX INVERTERS. HEX INVERTERS means that the chip contains six inverters. SN indicates that TI is the manufacturer. Other manufacture codes include MC for Motorola and DM for National Semiconductor. You can generally ignore these codes, because all of the manufacturers build compatible 74xx-series logic. HC is the logic family (high speed CMOS). The logic family determines the speed and power consumption of the chip, but not the function. For example, the 7404, 74HC04, and 74LS04 chips all contain six inverters, but they differ in performance and cost. Other logic families are discussed in Section A.6. The 74xx chips operate across the commercial or industrial temperature range (0 to 70 °C or –40 to 85 °C, respectively), whereas the 54xx chips operate across the military temperature range (–55 to 125 °C) and sell for a higher price but are otherwise compatible.

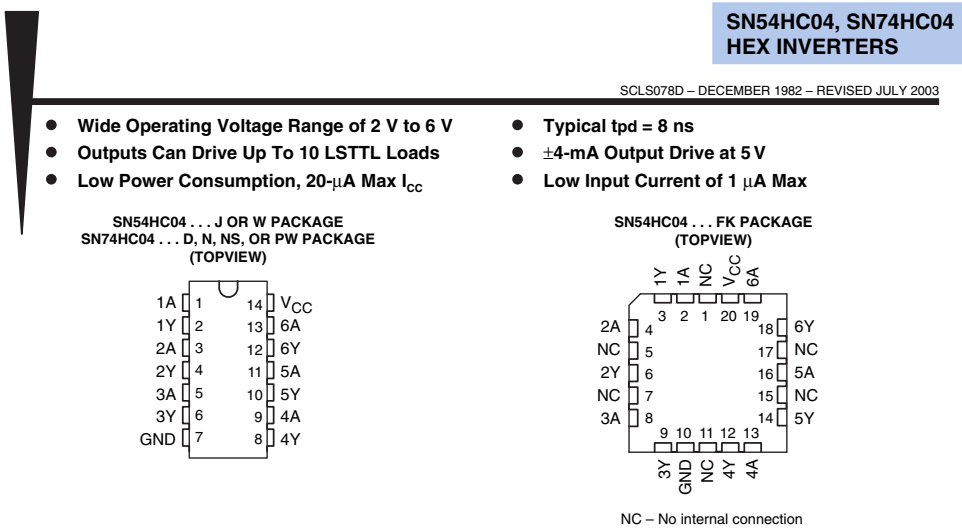
The 7404 is available in many different packages, and it is important to order the one you intended when you make a purchase. The packages are distinguished by a suffix on the part number. N indicates a *plastic dual inline package (PDIP)*, which fits in a breadboard or can be soldered in through-holes in a printed circuit board. Other packages are discussed in Section A.7.

The function table shows that each gate inverts its input. If A is HIGH (H), Y is LOW (L) and vice versa. The table is trivial in this case but is more interesting for more complex chips.

Figure eA.8 shows the second page of the data sheet. The logic diagram indicates that the chip contains inverters. The *absolute maximum* section indicates conditions beyond which the chip could be destroyed. In particular, the power supply voltage (V_{CC} , also called V_{DD} in this book) should not exceed 7 V. The continuous output current should not exceed 25 mA. The *thermal resistance* or impedance, θ_{JA} , is used to calculate the temperature rise caused by the chip's dissipating power. If the *ambient* temperature in the vicinity of the chip is T_A and the chip dissipates P_{chip} , then the temperature on the chip itself at its *junction* with the package is

$$T_J = T_A + P_{\text{chip}} \theta_{JA} \quad (\text{A.3})$$

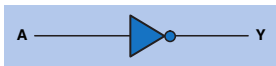
For example, if a 7404 chip in a plastic DIP package is operating in a hot box at 50 °C and consumes 20 mW, the junction temperature will climb



SN54HC04, SN74HC04 HEX INVERTERS

SCLS078D – DECEMBER 1982 – REVISED JULY 2003

logic diagram (positive logic)



absolute maximum ratings over operating free-air temperature range (unless otherwise noted)†

Supply voltage range, V_{CC}	–0.5 V to 7 V
Input clamp current, I_{IK} ($V_I < 0$ or $V_I > V_{CC}$) (see Note 1)	±20 mA
Output clamp current, I_{OK} ($V_O < 0$ or $V_O > V_{CC}$) (see Note 1)	±20 mA
Continuous output current, I_O ($V_O = 0$ to V_{CC})	±25 mA
Continuous current through V_{CC} or GND	±50 mA
Package thermal impedance, θ_{JA} (see Note 2): D package	86° C/W
N package	80° C/W
NS package	76° C/W
PW package	131° C/W
Storage temperature range, T_{stg}	–65° C to 150° C

† Stresses beyond those listed under "absolute maximum ratings" may cause permanent damage to the device. These are stress ratings only, and functional operation of the device at these or any other conditions beyond those indicated under "recommended operating conditions" is not implied. Exposure to absolute-maximum-rated conditions for extended periods may affect device reliability.

NOTES: 1. The input and output voltage ratings may be exceeded if the input and output current ratings are observed.
2. The package thermal impedance is calculated in accordance with JESD 51-7.

recommended operating conditions (see Note 3)

			SN54HC04			SN74HC04			UNIT
			MIN	NOM	MAX	MIN	NOM	MAX	
V _{CC}	Supply voltage		2	5	6	2	5	6	V
V _{IH}	High-level input voltage	V _{CC} = 2 V	1.5			1.5			V
		V _{CC} = 4.5 V	3.15			3.15			
		V _{CC} = 6 V	4.2			4.2			
V _{IL}	Low-level input voltage	V _{CC} = 2 V	0.5			0.5			V
		V _{CC} = 4.5 V	1.35			1.35			
		V _{CC} = 6 V	1.8			1.8			
V _I	Input voltage		0	V _{CC}		0	V _{CC}		V
V _O	Output voltage		0	V _{CC}		0	V _{CC}		V
Δt/Δv	Input transition rise/fall time	V _{CC} = 2 V	1000			1000			ns
		V _{CC} = 4.5 V	500			500			
		V _{CC} = 6 V	400			400			
T _A	Operating free-air temperature		−55	125		−40	85		°C

NOTE 3: All unused inputs of the device must be held at V_{CC} or GND to ensure proper device operation. Refer to the TI application report, *Implications of Slow or Floating CMOS Inputs*, literature number SCBA004.



TEXAS
INSTRUMENTS

POST OFFICE BOX 655303 • DALLAS, TEXAS 75265

Figure eA.8 7404 data sheet page 2

to $50^{\circ}\text{C} + 0.02\text{ W} \times 80^{\circ}\text{C}/\text{W} = 51.6^{\circ}\text{C}$. Internal power dissipation is seldom important for 74xx-series chips, but it becomes important for modern chips that dissipate tens of watts or more.

The *recommended operating conditions* define the environment in which the chip should be used. Within these conditions, the chip should meet specifications. These conditions are more stringent than the absolute maximums. For example, the power supply voltage should be between 2 and 6 V. The input logic levels for the HC logic family depend on V_{DD} . Use the 4.5 V entries when $V_{DD} = 5\text{ V}$, to allow for a 10% droop in the power supply caused by noise in the system.

Figure eA.9 shows the third page of the data sheet. The *electrical characteristics* describe how the device performs when used within the recommended operating conditions if the inputs are held constant. For example, if $V_{CC} = 5\text{ V}$ (and droops to 4.5 V) and the output current I_{OH}/I_{OL} does not exceed $20\text{ }\mu\text{A}$, $V_{OH} = 4.4\text{ V}$ and $V_{OL} = 0.1\text{ V}$ in the worst case. If the output current increases, the output voltages become less ideal, because the transistors on the chip struggle to provide the current. The HC logic family uses CMOS transistors that draw very little current. The current into each input is guaranteed to be less than 1000 nA and is typically only 0.1 nA at room temperature. The *quiescent* power supply current (I_{DD}) drawn while the chip is idle is less than $20\text{ }\mu\text{A}$. Each input has less than 10 pF of capacitance.

The *switching characteristics* define how the device performs when used within the recommended operating conditions if the inputs change. The *propagation delay*, t_{pd} , is measured from when the input passes through $0.5 V_{CC}$ to when the output passes through $0.5 V_{CC}$. If V_{CC} is nominally 5 V and the chip drives a capacitance of less than 50 pF , the propagation delay will not exceed 24 ns (and typically will be much faster). Recall that each input may present 10 pF , so the chip cannot drive more than five identical chips at full speed. Indeed, stray capacitance from the wires connecting chips cuts further into the useful load. The *transition time*, also called the rise/fall time, is measured as the output transitions between $0.1 V_{CC}$ and $0.9 V_{CC}$.

Recall from Section 1.8 that chips consume both *static* and *dynamic power*. Static power is low for HC circuits. At 85°C , the maximum quiescent supply current is $20\text{ }\mu\text{A}$. At 5 V , this gives a static power consumption of 0.1 mW . The dynamic power depends on the capacitance being driven and the switching frequency. The 7404 has an internal power dissipation capacitance of 20 pF per inverter. If all six inverters on the 7404 switch at 10 MHz and drive external loads of 25 pF , then the dynamic power given by Equation 1.4 is $\frac{1}{2}(6)(20\text{ pF} + 25\text{ pF})(5^2)(10\text{ MHz}) = 33.75\text{ mW}$ and the maximum total power is 33.85 mW .

SN54HC04, SN74HC04 HEX INVERTERS

SCLS078D – DECEMBER 1982 – REVISED JULY 2003

electrical characteristics over recommended operating free-air temperature range (unless otherwise noted)

PARAMETER	TEST CONDITIONS	V _{CC}	T _A = 25 °C			SN54HC04		SN74HC04		UNIT
			MIN	TYP	MAX	MIN	MAX	MIN	MAX	
V _{OH}	V _I = V _{IH} or V _{IL}	I _{OH} = -20 μA	2 V	1.9	1.998	1.9		1.9		V
			4.5 V	4.4	4.499	4.4		4.4		
			6 V	5.9	5.999	5.9		5.9		
			4.5 V	3.98	4.3	3.7		3.84		
			6 V	5.48	5.8	5.2		5.34		
V _{OL}	V _I = V _{IH} or V _{IL}	I _{OL} = 20 μA	2 V		0.002	0.1		0.1		V
			4.5 V		0.001	0.1		0.1		
			6 V		0.001	0.1		0.1		
			4.5 V		0.17	0.26		0.4		
			6 V		0.15	0.26		0.4		
I _I	V _I = V _{CC} or 0	6 V		±0.1	±100		±1000		±1000	nA
I _{CC}	V _I = V _{CC} or 0, I _O = 0	6 V			2		40		20	μA
C _I		2 V to 6 V			3		10		10	pF

switching characteristics over recommended operating free-air temperature range, CL = 50 pF (unless otherwise noted) (see Figure 1)

PARAMETER	FROM (INPUT)	TO (OUTPUT)	V _{CC}	T _A = 25 °C			SN54HC04		SN74HC04		UNIT
				MIN	TYP	MAX	MIN	MAX	MIN	MAX	
t _{pd}	A	Y	2 V		45	95		145		120	ns
			4.5 V		9	19		29		24	
			6 V		8	16		25		20	
t _t		Y	2 V		38	75		110		95	ns
			4.5 V		8	15		22		19	
			6 V		6	13		19		16	

operating characteristics, T_A = 25 °C

PARAMETER	TEST CONDITIONS	TYP	UNIT
C _{pd} Power dissipation capacitance per inverter	No load	20	pF

A.6 LOGIC FAMILIES

The 74xx-series logic chips have been manufactured using many different technologies, called *logic families*, that offer different speed, power, and logic level trade-offs. Other chips are usually designed to be compatible with some of these logic families. The original chips, such as the 7404, were built using bipolar transistors in a technology called *Transistor-Transistor Logic (TTL)*. Newer technologies add one or more letters after the 74 to indicate the logic family, such as 74LS04, 74HC04, or 74AHCT04. [Table eA.2](#) summarizes the most common 5-V logic families.

Advances in bipolar circuits and process technology led to the *Schottky (S)* and *Low-Power Schottky (LS)* families. Both are faster than TTL. Schottky draws more power, whereas Low-Power Schottky draws less. *Advanced Schottky (AS)* and *Advanced Low-Power Schottky (ALS)* have improved speed and power compared to S and LS. *Fast (F)* logic is faster and draws less power than AS. All of these families provide more current for LOW outputs than for HIGH outputs and hence have

Table eA.2 Typical specifications for 5-V logic families

Characteristic	Bipolar / TTL						CMOS		CMOS / TTL Compatible	
	TTL	S	LS	AS	ALS	F	HC	AHC	HCT	AHCT
t_{pd} (ns)	22	9	12	7.5	10	6	21	7.5	30	7.7
V_{IH} (V)	2	2	2	2	2	2	3.15	3.15	2	2
V_{IL} (V)	0.8	0.8	0.8	0.8	0.8	0.8	1.35	1.35	0.8	0.8
V_{OH} (V)	2.4	2.7	2.7	2.5	2.5	2.5	3.84	3.8	3.84	3.8
V_{OL} (V)	0.4	0.5	0.5	0.5	0.5	0.5	0.33	0.44	0.33	0.44
I_{OH} (mA)	0.4	1	0.4	2	0.4	1	4	8	4	8
I_{OL} (mA)	16	20	8	20	8	20	4	8	4	8
I_{IL} (mA)	1.6	2	0.4	0.5	0.1	0.6	0.001	0.001	0.001	0.001
I_{IH} (mA)	0.04	0.05	0.02	0.02	0.02	0.02	0.001	0.001	0.001	0.001
I_{DD} (mA)	33	54	6.6	26	4.2	15	0.02	0.02	0.02	0.02
C_{Pd} (pF)	n/a						20	12	20	14
cost* (US \$)	obsolete	0.63	0.25	0.53	0.32	0.22	0.12	0.12	0.12	0.12

*Per unit in quantities of 1000 for the 7408 from Texas Instruments in 2012.

asymmetric logic levels. They conform to the “TTL” logic levels: $V_{IH} = 2\text{ V}$, $V_{IL} = 0.8\text{ V}$, $V_{OH} > 2.4\text{ V}$, and $V_{OL} < 0.5\text{ V}$.

As CMOS circuits matured in the 1980s and 1990s, they became popular because they draw very little power supply or input current. The *High Speed CMOS (HC)* and *Advanced High Speed CMOS (AHC)* families draw almost no static power. They also deliver the same current for HIGH and LOW outputs. They conform to the “CMOS” logic levels: $V_{IH} = 3.15\text{ V}$, $V_{IL} = 1.35\text{ V}$, $V_{OH} > 3.8\text{ V}$, and $V_{OL} < 0.44\text{ V}$. Unfortunately, these levels are incompatible with TTL circuits, because a TTL HIGH output of 2.4 V may not be recognized as a legal CMOS HIGH input. This motivates the use of *High Speed TTL-compatible CMOS (HCT)* and *Advanced High Speed TTL-compatible CMOS (AHCT)*, which accept TTL input logic levels and generate valid CMOS output logic levels. These families are slightly slower than their pure CMOS counterparts. All CMOS chips are sensitive to *electrostatic discharge (ESD)* caused by static electricity. Ground yourself by touching a large metal object before handling CMOS chips, lest you zap them.

The 74xx-series logic is inexpensive. The newer logic families are often cheaper than the obsolete ones. The LS family is widely available and robust and is a popular choice for laboratory or hobby projects that have no special performance requirements.

The 5-V standard collapsed in the mid-1990s, when transistors became too small to withstand the voltage. Moreover, lower voltage offers lower power consumption. Now 3.3, 2.5, 1.8, 1.2, and even lower voltages are commonly used. The plethora of voltages raises challenges in communicating between chips with different power supplies. [Table eA.3](#) lists some of the low-voltage logic families. Not all 74xx parts are available in all of these logic families.

All of the low-voltage logic families use CMOS transistors, the workhorse of modern integrated circuits. They operate over a wide range of V_{DD} , but the speed degrades at lower voltage. *Low-Voltage CMOS (LVC)* logic and *Advanced Low-Voltage CMOS (ALVC)* logic are commonly used at 3.3, 2.5, or 1.8 V. LVC withstands inputs up to 5.5 V, so it can receive inputs from 5-V CMOS or TTL circuits. *Advanced Ultra-Low-Voltage CMOS (AUC)* is commonly used at 2.5, 1.8, or 1.2 V and is exceptionally fast. Both ALVC and AUC withstand inputs up to 3.6 V, so they can receive inputs from 3.3-V circuits.

FPGAs often offer separate voltage supplies for the internal logic, called the *core*, and for the input/output (I/O) pins. As FPGAs have advanced, the core voltage has dropped from 5 to 3.3, 2.5, 1.8, and 1.2 V to save power and avoid damaging the very small transistors. FPGAs have configurable I/Os that can operate at many different voltages, so as to be compatible with the rest of the system.

Table eA.3 Typical specifications for low-voltage logic families

V_{dd} (V)	LVC			ALVC			AUC		
	3.3	2.5	1.8	3.3	2.5	1.8	2.5	1.8	1.2
t_{pd} (ns)	4.1	6.9	9.8	2.8	3	?	1.8	2.3	3.4
V_{IH} (V)	2	1.7	1.17	2	1.7	1.17	1.7	1.17	0.78
V_{IL} (V)	0.8	0.7	0.63	0.8	0.7	0.63	0.7	0.63	0.42
V_{OH} (V)	2.2	1.7	1.2	2	1.7	1.2	1.8	1.2	0.8
V_{OL} (V)	0.55	0.7	0.45	0.55	0.7	0.45	0.6	0.45	0.3
I_O (mA)	24	8	4	24	12	12	9	8	3
I_I (mA)	0.02			0.005			0.005		
I_{DD} (mA)	0.01			0.01			0.01		
C_{pd} (pF)	10	9.8	7	27.5	23	?	17	14	14
cost (US \$)	0.17			0.20			not available		

*Delay and capacitance not available at the time of writing.

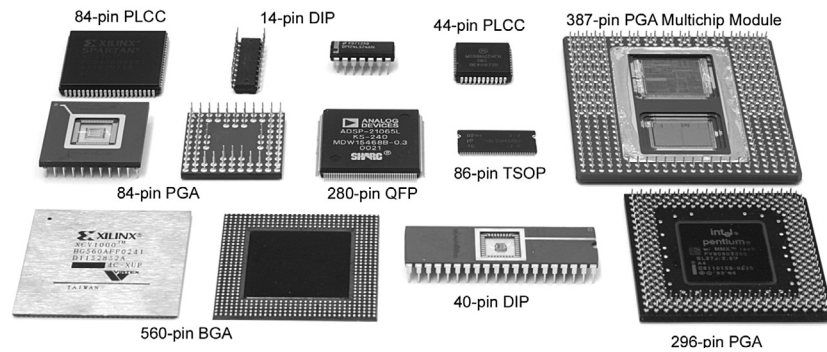
A.7 PACKAGING AND ASSEMBLY

Integrated circuits are typically placed in *packages* made of plastic or ceramic. The packages serve a number of functions, including connecting the tiny metal I/O pads of the chip to larger pins in the package for ease of connection, protecting the chip from physical damage, and spreading the heat generated by the chip over a larger area to help with cooling. The packages are placed on a breadboard or printed circuit board and wired together to assemble the system.

Packages

Figure eA.10 shows a variety of integrated circuit packages. Packages can be generally categorized as *through-hole* or *surface mount* (SMT). Through-hole packages, as their name implies, have pins that can be inserted through holes in a printed circuit board or into a socket. *Dual inline packages* (DIPs) have two rows of pins with 0.1-inch spacing between pins. *Pin grid arrays* (PGAs) support more pins in a smaller package by placing the pins under the package. SMT packages are soldered directly to the surface of a printed circuit board without using holes. Pins on SMT parts are called *leads*. The *thin small outline package* (TSOP) has two rows of closely spaced leads (typically 0.02-inch spacing). *Plastic leaded chip carriers* (PLCCs) have J-shaped leads on all four

Figure eA.10 Integrated circuit packages



sides, with 0.05-inch spacing. They can be soldered directly to a board or placed in special sockets. *Quad flat packs (QFPs)* accommodate a large number of pins using closely spaced legs on all four sides. *Ball grid arrays (BGAs)* eliminate the legs altogether. Instead, they have hundreds of tiny solder balls on the underside of the package. They are carefully placed over matching pads on a printed circuit board, then heated so that the solder melts and joins the package to the underlying board.

Breadboards

DIPs are easy to use for prototyping, because they can be placed in a *breadboard*. A breadboard is a plastic board containing rows of sockets, as shown in [Figure eA.11](#). All five holes in a row are connected together. Each pin of the package is placed in a hole in a separate row. Wires can be placed in adjacent holes in the same row to make connections to the pin. Breadboards often provide separate columns of connected holes running the height of the board to distribute power and ground.

[Figure eA.11](#) shows a breadboard containing a majority gate built with a 74LS08 AND chip and a 74LS32 OR chip. The schematic of the circuit is shown in [Figure eA.12](#). Each gate in the schematic is labeled with the chip (08 or 32) and the pin numbers of the inputs and outputs (see [Figure eA.1](#)). Observe that the same connections are made on the breadboard. The inputs are connected to pins 1, 2, and 5 of the 08 chip, and the output is measured at pin 6 of the 32 chip. Power and ground are connected to pins 14 and 7, respectively, of each chip, from the vertical power and ground columns that are attached to the banana plug receptacles, V_b and V_a. Labeling the schematic in this way and checking off connections as they are made is a good way to reduce the number of mistakes made during breadboarding.

Unfortunately, it is easy to accidentally plug a wire in the wrong hole or have a wire fall out, so breadboarding requires a great deal of care (and usually some debugging in the laboratory). Breadboards are suited only to prototyping, not production.

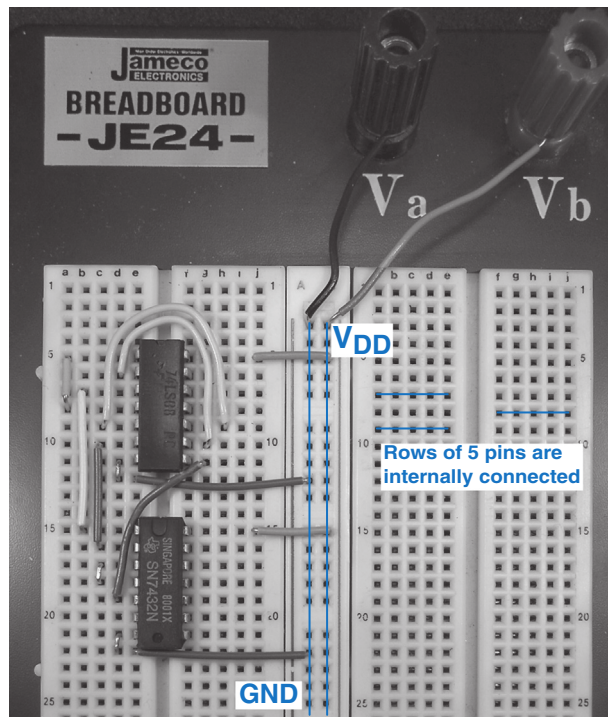


Figure eA.11 Majority circuit on breadboard

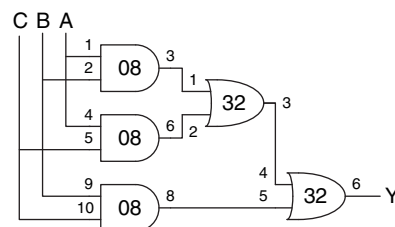
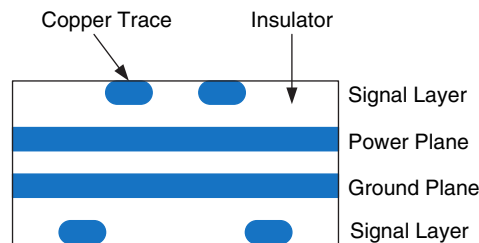


Figure eA.12 Majority gate schematic with chips and pins identified

Printed Circuit Boards

Instead of breadboarding, chip packages may be soldered to a *printed circuit board (PCB)*. The PCB is formed of alternating layers of conducting copper and insulating epoxy. The copper is etched to form wires called *traces*. Holes called *vias* are drilled through the board and plated with metal to connect between layers. PCBs are usually designed with *computer-aided design (CAD)* tools. You can etch and drill your own simple boards in the laboratory, or you can send the board design to a specialized factory for inexpensive mass production. Factories have turnaround times of days (or weeks, for cheap mass production runs) and typically charge a few hundred dollars in setup fees and a few dollars per board for moderately complex boards built in large quantities.

Figure eA.13 Printed circuit board cross-section



PCB traces are normally made of copper because of its low resistance. The traces are embedded in an insulating material, usually a green, fire-resistant plastic called FR4. A PCB also typically has copper power and ground layers, called *planes*, between signal layers. Figure eA.13 shows a cross-section of a PCB. The signal layers are on the top and bottom, and the power and ground planes are embedded in the center of the board. The power and ground planes have low resistance, so they distribute stable power to components on the board. They also make the capacitance and inductance of the traces uniform and predictable.

Figure eA.14 shows a PCB for a 1970s vintage Apple II+ computer. At the top is a 6502 microprocessor. Beneath are six 16-Kb ROM chips forming 12 KB of ROM containing the operating system. Three rows of eight 16-Kb DRAM chips provide 48 KB of RAM. On the right are several rows of 74xx-series logic for memory address decoding and other functions. The lines between chips are traces that wire the chips together. The dots at the ends of some of the traces are vias filled with metal.

Putting It All Together

Most modern chips with large numbers of inputs and outputs use SMT packages, especially QFPs and BGAs. These packages require a printed circuit board rather than a breadboard. Working with BGAs is especially challenging because they require specialized assembly equipment. Moreover, the balls cannot be probed with a voltmeter or oscilloscope during debugging in the laboratory, because they are hidden under the package.

In summary, the designer needs to consider packaging early on to determine whether a breadboard can be used during prototyping and whether BGA parts will be required. Professional engineers rarely use breadboards when they are confident of connecting chips together correctly without experimentation.

A.8 TRANSMISSION LINES

We have assumed so far that wires are *equipotential* connections that have a single voltage along their entire length. Signals actually propagate along wires at the speed of light in the form of electromagnetic waves. If the wires are short enough or the signals change slowly, the equipotential

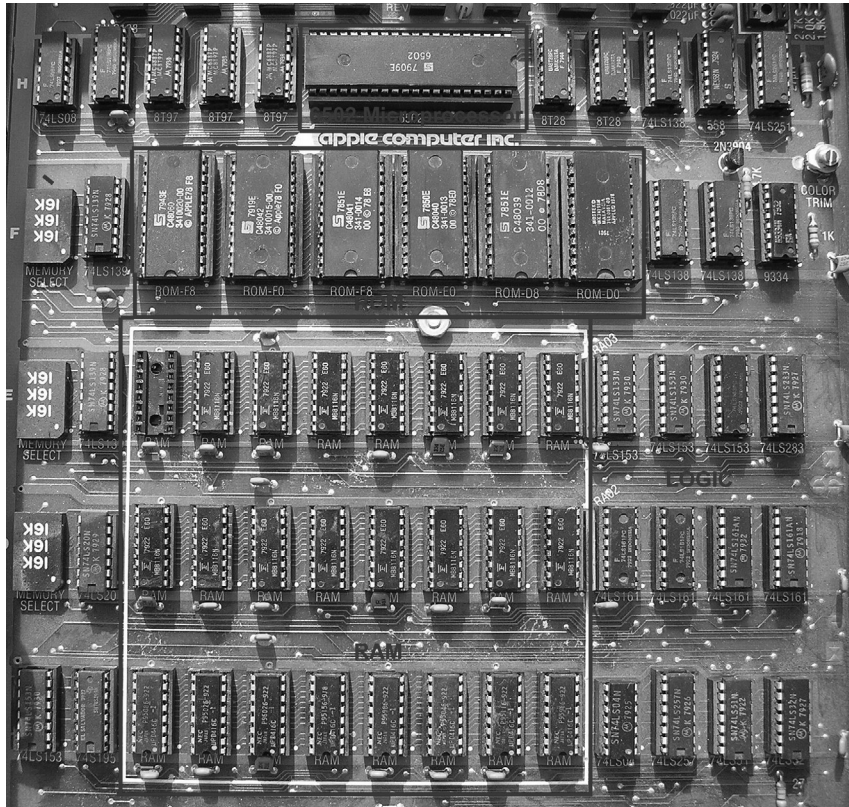


Figure eA.14 Apple II+ circuit board

assumption is good enough. When the wire is long or the signal is very fast, the *transmission time* along the wire becomes important to accurately determine the circuit delay. We must model such wires as *transmission lines*, in which a wave of voltage and current propagates at the speed of light. When the wave reaches the end of the line, it may reflect back along the line. The reflection may cause noise and odd behaviors unless steps are taken to limit it. Hence, the digital designer must consider transmission line behavior to accurately account for the delay and noise effects in long wires.

Electromagnetic waves travel at the speed of light in a given medium, which is fast but not instantaneous. The speed of light, v , depends on the *permittivity*, ϵ , and *permeability*, μ , of the medium¹: $v = \frac{1}{\sqrt{\mu\epsilon}} = \frac{1}{\sqrt{LC}}$.

¹ The capacitance, C , and inductance, L , of a wire are related to the permittivity and permeability of the physical medium in which the wire is located.

The speed of light in free space is $v = c = 3 \times 10^8$ m/s. Signals in a PCB travel at about half this speed, because the FR4 insulator has four times the permittivity of air. Thus, PCB signals travel at about 1.5×10^8 m/s, or 15 cm/ns. The time delay for a signal to travel along a transmission line of length l is

$$t_d = \frac{l}{v} \quad (\text{A.4})$$

The *characteristic impedance* of a transmission line, Z_0 (pronounced “Z-naught”), is the ratio of voltage to current in a wave traveling along the line: $Z_0 = V/I$. It is *not* the resistance of the wire (a good transmission line in a digital system typically has negligible resistance). Z_0 depends on the inductance and capacitance of the line (see the derivation in [Section A.8.7](#)) and typically has a value of 50 to 75 Ω .

$$Z_0 = \sqrt{\frac{L}{C}} \quad (\text{A.5})$$

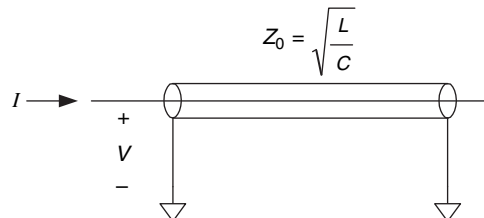
[Figure eA.15](#) shows the symbol for a transmission line. The symbol resembles a *coaxial cable* with an inner signal conductor and an outer grounded conductor like that used in television cable wiring.

The key to understanding the behavior of transmission lines is to visualize the wave of voltage propagating along the line at the speed of light. When the wave reaches the end of the line, it may be absorbed or reflected, depending on the termination or load at the end. Reflections travel back along the line, adding to the voltage already on the line. Terminations are classified as matched, open, short, or mismatched. The following sections explore how a wave propagates along the line and what happens to the wave when it reaches the termination.

A.8.1 Matched Termination

[Figure eA.16](#) shows a transmission line of length l with a *matched termination*, which means that the load impedance, Z_L , is equal to the characteristic impedance, Z_0 . The transmission line has a characteristic impedance of 50 Ω . One end of the line is connected to a voltage source

Figure eA.15 Transmission line symbol



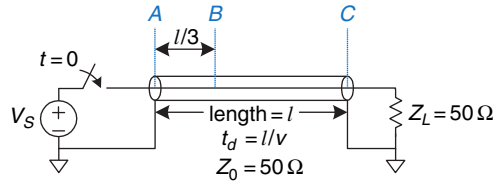


Figure eA.16 Transmission line with matched termination

through a switch that closes at time $t = 0$. The other end is connected to the $50\ \Omega$ matched load. This section analyzes the voltages and currents at points A, B, and C—at the beginning of the line, one-third of the length along the line, and at the end of the line, respectively.

Figure eA.17 shows the voltages at points A, B, and C over time. Initially, there is no voltage or current flowing in the transmission line, because the switch is open. At time $t = 0$, the switch closes, and the voltage source launches a wave with voltage $V = V_S$ along the line. This is called the *incident wave*. Because the characteristic impedance is Z_0 , the wave has current $I = V_S/Z_0$. The voltage reaches the beginning of the line (point A) immediately, as shown in Figure eA.17(a). The wave propagates along the line at the speed of light. At time $t_d/3$, the wave reaches point B. The voltage at this point abruptly rises from 0 to V_S , as shown in Figure eA.17(b). At time t_d , the incident wave reaches point C at the end of the line, and the voltage rises there too. All of the current, I , flows into the resistor, Z_L , producing a voltage across the resistor of $Z_L I = Z_L (V_S/Z_0) = V_S$ because $Z_L = Z_0$. This voltage is consistent with the wave flowing along the transmission line. Thus, the wave is *absorbed* by the load impedance, and the transmission line reaches its *steady state*.

In steady state, the transmission line behaves like an ideal equipotential wire because it is, after all, just a wire. The voltage at all points along the line must be identical. Figure eA.18 shows the steady-state equivalent model of the circuit in Figure eA.16. The voltage is V_S everywhere along the wire.

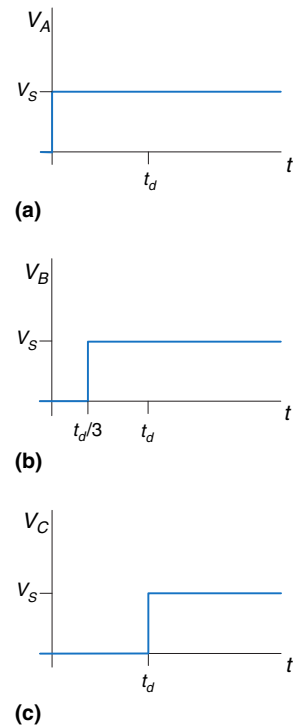


Figure eA.17 Voltage waveforms for Figure eA.16 at points A, B, and C

Example eA.2 TRANSMISSION LINE WITH MATCHED SOURCE AND LOAD TERMINATIONS

Figure eA.19 shows a transmission line with matched source and load impedances Z_S and Z_L . Plot the voltage at nodes A, B, and C versus time. When does the system reach steady-state, and what is the equivalent circuit at steady-state?

Solution: When the voltage source has a source impedance Z_S in series with the transmission line, part of the voltage drops across Z_S , and the remainder propagates down the transmission line. At first, the transmission line behaves as an impedance

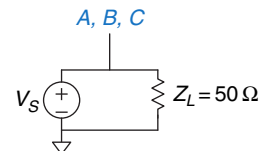
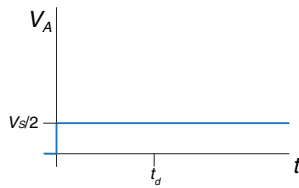
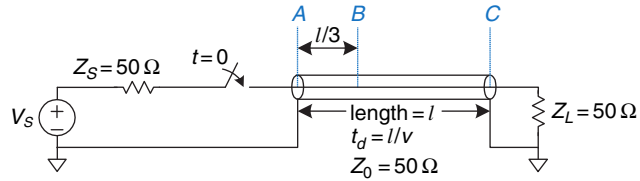
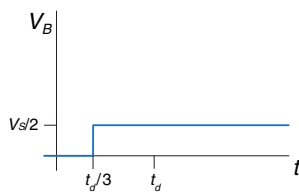


Figure eA.18 Equivalent circuit of Figure eA.16 at steady state

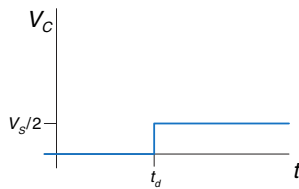
Figure eA.19 Transmission line with matched source and load impedances



(a)



(b)



(c)

Figure eA.20 Voltage waveforms for Figure eA.19 at points A, B, and C

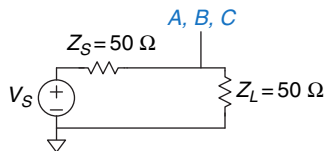


Figure eA.21 Equivalent circuit of Figure eA.19 at steady state

Z_0 , because the load at the end of the line cannot possibly influence the behavior of the line until a speed of light delay has elapsed. Hence, by the *voltage divider equation*, the incident voltage flowing down the line is

$$V = V_S \left(\frac{Z_0}{Z_0 + Z_S} \right) = \frac{V_S}{2} \quad (\text{A.6})$$

Thus, at $t = 0$, a wave of voltage, $V = \frac{V_S}{2}$, is sent down the line from point A. Again, the signal reaches point B at time $t_d/3$ and point C at t_d , as shown in Figure eA.20. All of the current is absorbed by the load impedance Z_L , so the circuit enters steady-state at $t = t_d$. In steady-state, the entire line is at $V_S/2$, just as the steady-state equivalent circuit in Figure eA.21 would predict.

A.8.2 Open Termination

When the load impedance is not equal to Z_0 , the termination cannot absorb all of the current, and some of the wave must be reflected. Figure eA.22 shows a transmission line with an open load termination. No current can flow through an open termination, so the current at point C must always be 0.

The voltage on the line is initially zero. At $t = 0$, the switch closes and a wave of voltage, $V = V_S \frac{Z_0}{Z_0 + Z_S} = \frac{V_S}{2}$, begins propagating down the line. Notice that this initial wave is the same as that of Example eA.2 and is independent of the termination, because the load at the end of the line cannot influence the behavior at the beginning until at least $2t_d$ has elapsed. This wave reaches point B at $t_d/3$ and point C at t_d as shown in Figure eA.23.

When the incident wave reaches point C, it cannot continue forward because the wire is open. It must instead reflect back toward the source. The reflected wave also has voltage $V = \frac{V_S}{2}$, because the open termination reflects the entire wave.

The voltage at any point is the sum of the incident and reflected waves. At time $t = t_d$, the voltage at point C is $V = \frac{V_S}{2} + \frac{V_S}{2} = V_S$. The reflected wave reaches point B at $5t_d/3$ and point A at $2t_d$. When it reaches point A,

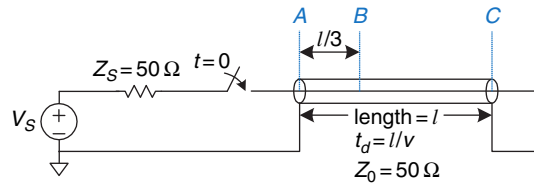


Figure eA.22 Transmission line with open load termination

the wave is absorbed by the source termination impedance that matches the characteristic impedance of the line. Thus, the system reaches steady state at time $t = 2t_d$, and the transmission line becomes equivalent to an equipotential wire with voltage V_S and current $I = 0$.

A.8.3 Short Termination

Figure eA.24 shows a transmission line terminated with a short circuit to ground. Thus, the voltage at point C must always be 0.

As in the previous examples, the voltages on the line are initially 0. When the switch closes, a wave of voltage, $V = \frac{V_S}{2}$, begins propagating down the line (Figure eA.25). When it reaches the end of the line, it must reflect with opposite polarity. The reflected wave, with voltage $V = -\frac{V_S}{2}$, adds to the incident wave, ensuring that the voltage at point C remains 0. The reflected wave reaches the source at time $t = 2t_d$ and is absorbed by the source impedance. At this point, the system reaches steady state, and the transmission line is equivalent to an equipotential wire with voltage $V = 0$.

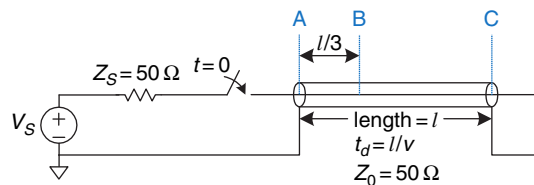
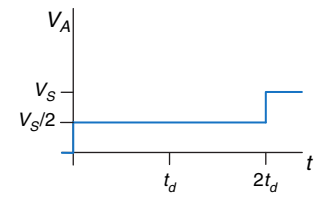
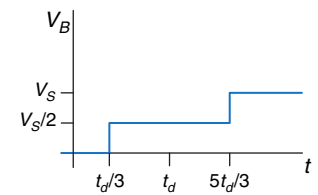


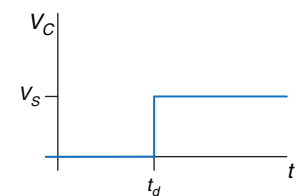
Figure eA.24 Transmission line with short termination



(a)



(b)



(c)

Figure eA.23 Voltage waveforms for Figure eA.22 at points A, B, and C

A.8.4 Mismatched Termination

The termination impedance is said to be *mismatched* when it does not equal the characteristic impedance of the line. In general, when an incident wave reaches a mismatched termination, part of the wave is absorbed and part is reflected. The reflection coefficient k_r indicates the fraction of the incident wave V_i that is reflected: $V_r = k_r V_i$.

Section A.8.8 derives the reflection coefficient using conservation of current arguments. It shows that, when an incident wave flowing along

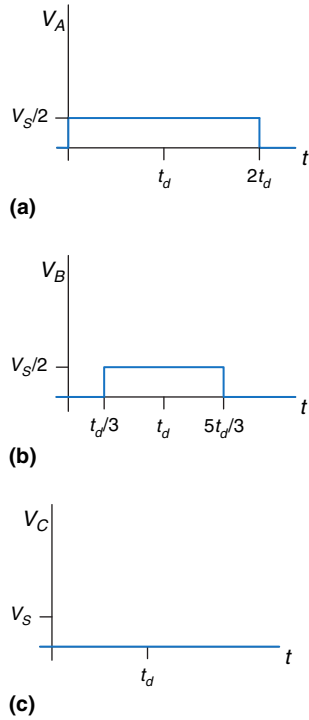


Figure eA.25 Voltage waveforms for Figure eA.24 at points A, B, and C

a transmission line of characteristic impedance Z_0 reaches a termination impedance Z_T at the end of the line, the reflection coefficient is

$$k_r = \frac{Z_T - Z_0}{Z_T + Z_0} \quad (\text{A.7})$$

Note a few special cases. If the termination is an open circuit ($Z_T = \infty$), $k_r = 1$, because the incident wave is entirely reflected (so the current out the end of the line remains zero). If the termination is a short circuit ($Z_T = 0$), $k_r = -1$, because the incident wave is reflected with negative polarity (so the voltage at the end of the line remains zero). If the termination is a matched load ($Z_T = Z_0$), $k_r = 0$, because the incident wave is completely absorbed.

Figure eA.26 illustrates reflections in a transmission line with a *mismatched load termination* of $75\ \Omega$. $Z_T = Z_L = 75\ \Omega$, and $Z_0 = 50\ \Omega$, so $k_r = 1/5$. As in previous examples, the voltage on the line is initially 0. When the switch closes, a wave of voltage $V = \frac{V_S}{2}$ propagates down the line, reaching the end at $t = t_d$. When the incident wave reaches the termination at the end of the line, one fifth of the wave is reflected, and the remaining four fifths flows into the load impedance. Thus, the reflected wave has a voltage $V = \frac{V_S}{2} \times \frac{1}{5} = \frac{V_S}{10}$. The total voltage at point C is the sum of the incoming and reflected voltages, $V_C = \frac{V_S}{2} + \frac{V_S}{10} = \frac{3V_S}{5}$. At $t = 2t_d$, the reflected wave reaches point A, where it is absorbed by the matched $50\ \Omega$ termination, Z_S . Figure eA.27 plots the voltages and currents along the line. Again, note that, in steady state (in this case at time $t > 2t_d$), the transmission line is equivalent to an equipotential wire, as shown in Figure eA.28. At steady state, the system acts like a voltage divider, so

$$V_A = V_B = V_C = V_S \left(\frac{Z_L}{Z_L + Z_S} \right) = V_S \left(\frac{75\ \Omega}{75\ \Omega + 50\ \Omega} \right) = \frac{3V_S}{5}$$

Reflections can occur at both ends of the transmission line. Figure eA.29 shows a transmission line with a source impedance, Z_S , of $450\ \Omega$ and an open termination at the load. The reflection coefficients at the load and source, k_{rL} and k_{rS} , are 1 and $4/5$, respectively. In this case, waves reflect off both ends of the transmission line until a steady state is reached.

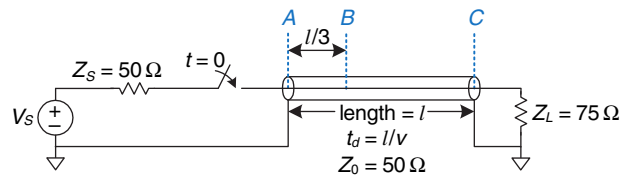


Figure eA.26 Transmission line with mismatched termination

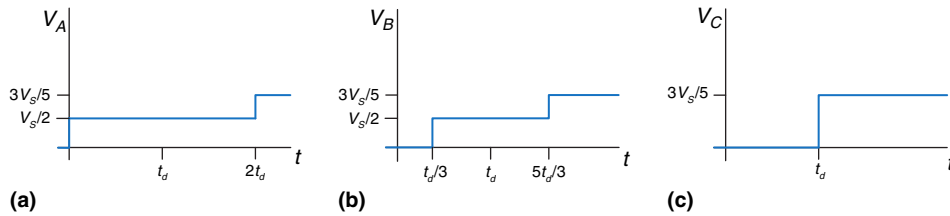


Figure eA.27 Voltage waveforms for Figure eA.26 at points A, B, and C

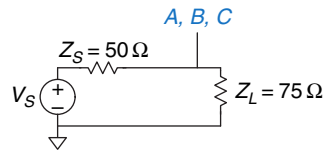


Figure eA.28 Equivalent circuit of Figure eA.26 at steady state

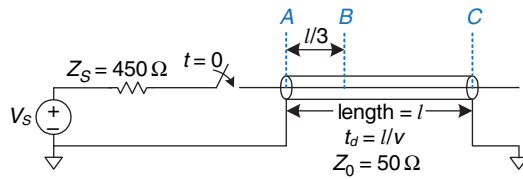


Figure eA.29 Transmission line with mismatched source and load terminations

The *bounce diagram* shown in Figure eA.30 helps visualize reflections off both ends of the transmission line. The horizontal axis represents distance along the transmission line, and the vertical axis represents time, increasing downward. The two sides of the bounce diagram represent the source and load ends of the transmission line, points A and C. The incoming and reflected signal waves are drawn as diagonal lines between points A and C. At time $t = 0$, the source impedance and transmission line behave as a voltage divider, launching a voltage wave of $\frac{V_s}{10}$ from point A toward point C. At time $t = t_d$, the signal reaches point C and is completely reflected ($k_{rL} = 1$). At time $t = 2t_d$, the reflected wave of $\frac{V_s}{10}$ reaches point A and is reflected with a reflection coefficient, $k_{rS} = 4/5$, to produce a wave of $\frac{2V_s}{25}$ traveling toward point C, and so forth.

The voltage at a given time at any point on the transmission line is the sum of all the incident and reflected waves. Thus, at time $t = 1.1t_d$, the voltage at point C is $\frac{V_s}{10} + \frac{V_s}{10} = \frac{V_s}{5}$. At time $t = 3.1t_d$, the voltage at point C is $\frac{V_s}{10} + \frac{V_s}{10} + \frac{2V_s}{25} + \frac{2V_s}{25} = \frac{9V_s}{25}$, and so forth. Figure eA.31 plots the voltages

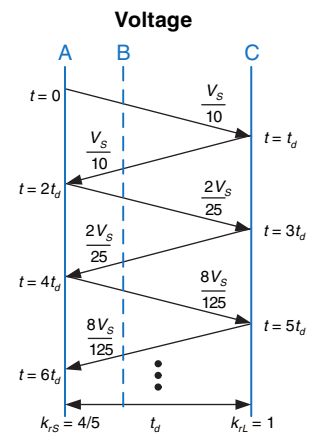


Figure eA.30 Bounce diagram for Figure eA.29

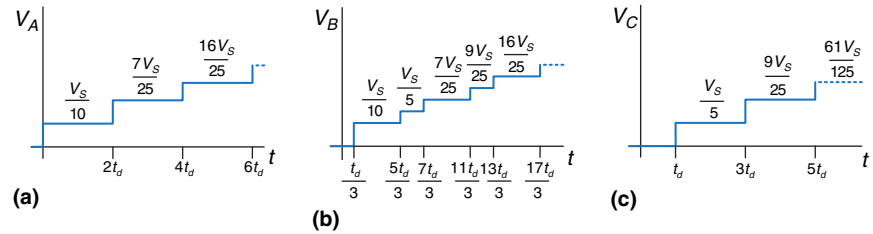


Figure eA.31 Voltage and current waveforms for Figure eA.29

against time. As t approaches infinity, the voltages approach steady state with $V_A = V_B = V_C = V_S$.

A.8.5 When to Use Transmission Line Models

Transmission line models for wires are needed whenever the wire delay, t_d , is longer than a fraction (e.g., 20%) of the edge rates (rise or fall times) of a signal. If the wire delay is shorter, it has an insignificant effect on the propagation delay of the signal, and the reflections dissipate while the signal is transitioning. If the wire delay is longer, it must be considered in order to accurately predict the propagation delay and waveform of the signal. In particular, reflections may distort the digital characteristic of a waveform, resulting in incorrect logic operations.

Recall that signals travel on a PCB at about 15 cm/ns. For TTL logic, with edge rates of 10 ns, wires must be modeled as transmission lines only if they are longer than 30 cm ($10 \text{ ns} \times 15 \text{ cm/ns} \times 20\%$). PCB traces are usually less than 30 cm, so most traces can be modeled as ideal equipotential wires. In contrast, many modern chips have edge rates of 2 ns or less, so traces longer than about 6 cm (about 2.5 inches) must be modeled as transmission lines. Clearly, use of edge rates that are crisper than necessary just causes difficulties for the designer.

Breadboards lack a ground plane, so the electromagnetic fields of each signal are nonuniform and difficult to model. Moreover, the fields interact with other signals. This can cause strange reflections and crosstalk between signals. Thus, breadboards are unreliable above a few megahertz.

In contrast, PCBs have good transmission lines with consistent characteristic impedance and velocity along the entire line. As long as they are terminated with a source or load impedance that is matched to the impedance of the line, PCB traces do not suffer from reflections.

A.8.6 Proper Transmission Line Terminations

There are two common ways to properly terminate a transmission line, shown in Figure eA.32. In *parallel termination*, the driver has a low impedance ($Z_S \approx 0$). A load resistor Z_L with impedance Z_0 is placed in parallel

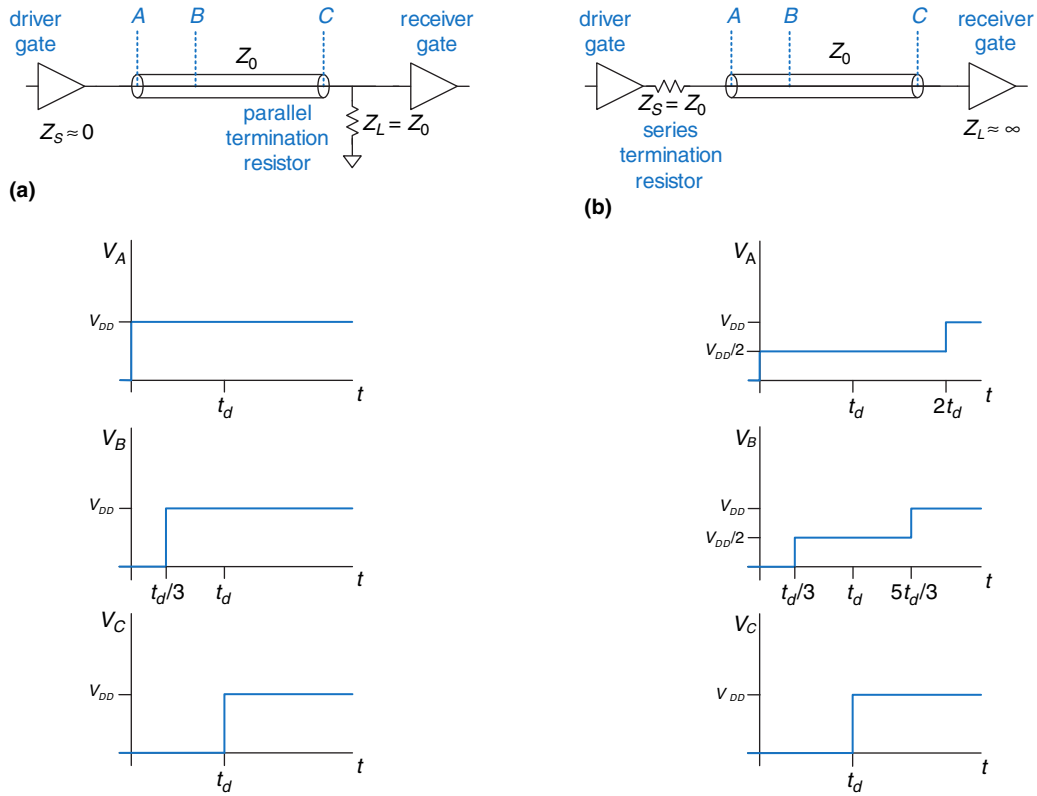


Figure eA.32 Termination schemes: (a) parallel, (b) series

with the load (between the input of the receiver gate and ground). When the driver switches from 0 to V_{DD} , it sends a wave with voltage V_{DD} down the line. The wave is absorbed by the matched load termination, and no reflections take place. In *series termination*, a source resistor Z_S is placed in series with the driver to raise the source impedance to Z_0 . The load has a high impedance ($Z_L \approx \infty$). When the driver switches, it sends a wave with voltage $V_{DD}/2$ down the line. The wave reflects at the open circuit load and returns, bringing the voltage on the line up to V_{DD} . The wave is absorbed at the source termination. Both schemes are similar in that the voltage at the receiver transitions from 0 to V_{DD} at $t = t_d$, just as one would desire. They differ in power consumption and in the waveforms that appear elsewhere along the line. Parallel termination dissipates power continuously through the load resistor when the line is at a high voltage. Series termination dissipates no DC power, because the load is an open circuit. However, in series terminated lines, points near the middle of the transmission line initially see a voltage of $V_{DD}/2$,

until the reflection returns. If other gates are attached to the middle of the line, they will momentarily see an illegal logic level. Therefore, series termination works best for *point-to-point* communication with a single driver and a single receiver. Parallel termination is better for a *bus* with multiple receivers, because receivers at the middle of the line never see an illegal logic level.

A.8.7 Derivation of Z_0^*

Z_0 is the ratio of voltage to current in a wave propagating along a transmission line. This section derives Z_0 ; it assumes some previous knowledge of resistor-inductor-capacitor (RLC) circuit analysis.

Imagine applying a step voltage to the input of a semi-infinite transmission line (so that there are no reflections). Figure eA.33 shows the semi-infinite line and a model of a segment of the line of length dx . R , L , and C , are the values of resistance, inductance, and capacitance per unit length. Figure eA.33(b) shows the transmission line model with a resistive component, R . This is called a *lossy* transmission line model, because energy is dissipated, or lost, in the resistance of the wire. However, this loss is often negligible, and we can simplify analysis by ignoring the resistive component and treating the transmission line as an *ideal* transmission line, as shown in Figure eA.33(c).

Voltage and current are functions of time and space throughout the transmission line, as given by Equations eA.8 and eA.9.

$$\frac{\partial}{\partial x} V(x, t) = L \frac{\partial}{\partial t} I(x, t) \quad (\text{A.8})$$

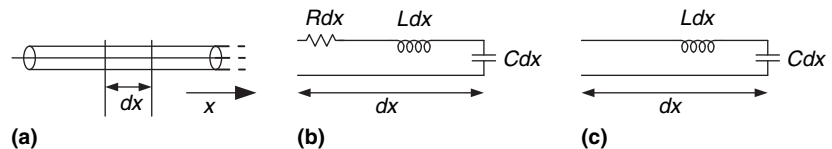
$$\frac{\partial}{\partial x} I(x, t) = C \frac{\partial}{\partial t} V(x, t) \quad (\text{A.9})$$

Taking the space derivative of Equation eA.8 and the time derivative of Equation eA.9 and substituting gives Equation eA.10, the *wave equation*.

$$\frac{\partial^2}{\partial x^2} V(x, t) = LC \frac{\partial^2}{\partial t^2} V(x, t) \quad (\text{A.10})$$

Z_0 is the ratio of voltage to current in the transmission line, as illustrated in Figure eA.34(a). Z_0 must be independent of the length of the line, because the behavior of the wave cannot depend on things at a distance. Because it is independent of length, the impedance must still equal Z_0 after the addition of a small amount of transmission line, dx , as shown in Figure eA.34(b).

Figure eA.33 Transmission line models: (a) semi-infinite cable, (b) lossy, (c) ideal



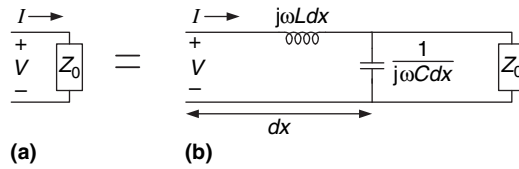


Figure eA.34 Transmission line model: (a) for entire line and (b) with additional length, dx

Using the impedances of an inductor and a capacitor, we rewrite the relationship of Figure eA.34 in equation form:

$$Z_0 = j\omega Ldx + [Z_0 || (1/(j\omega Cdx))] \quad (\text{A.11})$$

Rearranging, we get

$$Z_0^2(j\omega C) - j\omega L + \omega^2 Z_0 LCdx = 0 \quad (\text{A.12})$$

Taking the limit as dx approaches 0, the last term vanishes and we find that

$$Z_0 = \sqrt{\frac{L}{C}} \quad (\text{A.13})$$

A.8.8 Derivation of the Reflection Coefficient*

The reflection coefficient k_r is derived using conservation of current. Figure eA.35 shows a transmission line with characteristic impedance Z_0 and load impedance Z_L . Imagine an incident wave of voltage V_i and current I_i . When the wave reaches the termination, some current I_L flows through the load impedance, causing a voltage drop V_L . The remainder of the current reflects back down the line in a wave of voltage V_r and current I_r . Z_0 is the ratio of voltage to current in waves propagating along the line, so $\frac{V_i}{I_i} = \frac{V_r}{I_r} = Z_0$.

The voltage on the line is the sum of the voltages of the incident and reflected waves. The current flowing in the positive direction on the line is the difference between the currents of the incident and reflected waves.

$$V_L = V_i + V_r \quad (\text{A.14})$$

$$I_L = I_i - I_r \quad (\text{A.15})$$

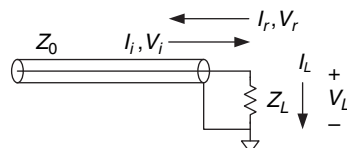


Figure eA.35 Transmission line showing incoming, reflected, and load voltages and currents

Using Ohm's law and substituting for I_L , I_i , and I_r in Equation eA.15, we get

$$\frac{V_i + V_r}{Z_L} = \frac{V_i}{Z_0} - \frac{V_r}{Z_0} \quad (\text{A.16})$$

Rearranging, we solve for the reflection coefficient, k_r :

$$\frac{V_r}{V_i} = \frac{Z_L - Z_0}{Z_L + Z_0} = k_r \quad (\text{A.17})$$

A.8.9 Putting It All Together

Transmission lines model the fact that signals take time to propagate down long wires because the speed of light is finite. An ideal transmission line has uniform inductance L and capacitance C per unit length and zero resistance. The transmission line is characterized by its characteristic impedance Z_0 and delay t_d which can be derived from the inductance, capacitance, and wire length. The transmission line has significant delay and noise effects on signals whose rise/fall times are less than about $5t_d$. This means that, for systems with 2 ns rise/fall times, PCB traces longer than about 6 cm must be analyzed as transmission lines to accurately understand their behavior.

A digital system consisting of a gate driving a long wire attached to the input of a second gate can be modeled with a transmission line as shown in Figure eA.36. The voltage source, source impedance Z_S , and switch model the first gate switching from 0 to 1 at time 0. The driver gate cannot supply infinite current; this is modeled by Z_S . Z_S is usually small for a logic gate, but a designer may choose to add a resistor in series with the gate to raise Z_S and match the impedance of the line. The input to the second gate is modeled as Z_L . CMOS circuits usually have little input current, so Z_L may be close to infinity. The designer may also choose to add a resistor in parallel with the second gate, between the gate input and ground, so that Z_L matches the impedance of the line.

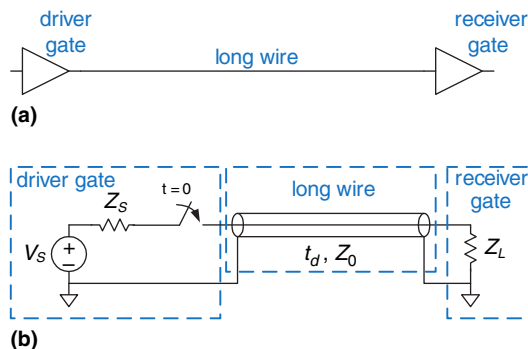


Figure eA.36 Digital system modeled with transmission line

When the first gate switches, a wave of voltage is driven onto the transmission line. The source impedance and transmission line form a voltage divider, so the voltage of the incident wave is

$$V_i = V_s \frac{Z_0}{Z_0 + Z_s} \quad (\text{A.18})$$

At time t_d , the wave reaches the end of the line. Part is absorbed by the load impedance, and part is reflected. The reflection coefficient k_r indicates the portion that is reflected: $k_r = V_r/V_i$, where V_r is the voltage of the reflected wave and V_i is the voltage of the incident wave.

$$k_r = \frac{Z_L - Z_0}{Z_L + Z_0} \quad (\text{A.19})$$

The reflected wave adds to the voltage already on the line. It reaches the source at time $2t_d$, where part is absorbed and part is again reflected. The reflections continue back and forth, and the voltage on the line eventually approaches the value that would be expected if the line were a simple equipotential wire.

A.9 ECONOMICS

Although digital design is so much fun that some of us would do it for free, most designers and companies intend to make money. Therefore, economic considerations are a major factor in design decisions.

The cost of a digital system can be divided into *nonrecurring engineering costs* (NRE), and *recurring costs*. NRE accounts for the cost of designing the system. It includes the salaries of the design team, computer and software costs, and the costs of producing the first working unit. The fully loaded cost of a designer in the United States in 2015 (including salary, health insurance, retirement plan, and a computer with design tools) was roughly \$200,000 per year, so design costs can be significant. Recurring costs are the cost of each additional unit; this includes components, manufacturing, marketing, technical support, and shipping.

The sales price must cover not only the cost of the system but also other costs such as office rental, taxes, and salaries of staff who do not directly contribute to the design (such as the janitor and the CEO). After all of these expenses, the company should still make a profit.

Example eA.3 BEN TRIES TO MAKE SOME MONEY

Ben Bitdiddle has designed a crafty circuit for counting raindrops. He decides to sell the device and try to make some money, but he needs help deciding what implementation to use. He decides to use either an FPGA or an ASIC. The

development kit to design and test the FPGA costs \$1500. Each FPGA costs \$17. The ASIC costs \$600,000 for a mask set and \$4 per chip.

Regardless of what chip implementation he chooses, Ben needs to mount the packaged chip on a printed circuit board (PCB), which will cost him \$1.50 per board. He thinks he can sell 1000 devices per month. Ben has coerced a team of bright undergraduates into designing the chip for their senior project, so it doesn't cost him anything to design.

If the sales price has to be twice the cost (100% profit margin), and the product life is 2 years, which implementation is the better choice?

Solution: Ben figures out the total cost for each implementation over 2 years, as shown in Table eA.4. Over 2 years, Ben plans on selling 24,000 devices, and the total cost is given in Table eA.4 for each option. If the product life is only two years, the FPGA option is clearly superior. The per-unit cost is $\$445,500/24,000 = \18.56 , and the sales price is \$37.13 per unit to give a 100% profit margin. The ASIC option would have cost $\$732,000/24,000 = \30.50 and would have sold for \$61 per unit.

Table eA.4 ASIC vs FPGA costs

Cost	ASIC	FPGA
NRE	\$600,000	\$1500
chip	\$4	\$17
PCB	\$1.50	\$1.50
TOTAL	$\$600,000 + (24,000 \times \$5.50)$ $= \$732,000$	$\$1500 + (24,000 \times \$18.50)$ $= \$445,500$
per unit	\$30.50	\$18.56

Example eA.4 BEN GETS GREEDY

After seeing the marketing ads for his product, Ben thinks he can sell even more chips per month than originally expected. If he were to choose the ASIC option, how many devices per month would he have to sell to make the ASIC option more profitable than the FPGA option?

Solution: Ben solves for the minimum number of units, N , that he would need to sell in 2 years:

$$\$600,000 + (N \times \$5.50) = \$1500 + (N \times \$18.50)$$

Solving the equation gives $N = 46,039$ units, or 1919 units per month. He would need to almost double his monthly sales to benefit from the ASIC solution.

Example eA.5 BEN GETS LESS GREEDY

Ben realizes that his eyes have gotten too big for his stomach, and he doesn't think he can sell more than 1000 devices per month. But he does think the product life can be longer than 2 years. At a sales volume of 1000 devices per month, how long would the product life have to be to make the ASIC option worthwhile?

Solution: If Ben sells more than 46,039 units in total, the ASIC option is the best choice. So, Ben would need to sell at a volume of 1000 per month for at least 47 months (rounding up), which is almost 4 years. By then, his product is likely to be obsolete.

Chips are usually purchased from a distributor rather than directly from the manufacturer (unless you are ordering tens of thousands of units). Digikey (www.digikey.com) is a leading distributor that sells a wide variety of electronics. Jameco (www.jameco.com) and All Electronics (www.allelectronics.com) have eclectic catalogs that are competitively priced and well suited to hobbyists.

ARM Instructions

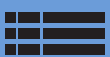
B

- B.1 Data-processing Instructions
- B.2 Memory Instructions
- B.3 Branch Instructions
- B.4 Miscellaneous Instructions
- B.5 Condition Flags

This appendix summarizes ARMv4 instructions used in this book. Condition encodings are given in Table 6.3.

B.1 DATA-PROCESSING INSTRUCTIONS

Standard data-processing instructions use the encoding in Figure B.1. The 4-bit *cmd* field specifies the type of instruction as given in Table B.1. When the *S*-bit is 1, the status register is updated with the condition flags produced by the instruction. The *I*-bit and bits 4 and 7 specify one of three encodings for the second source operand, *Src2*, as described in Section 6.4.2. The *cond* field specifies which condition codes to check, as given in Section 6.3.2.

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

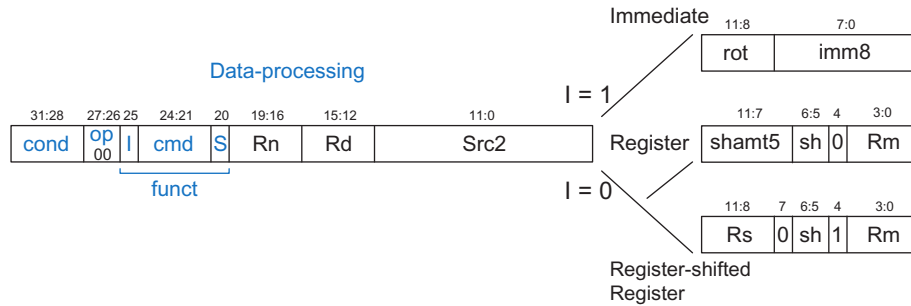


Figure B.1 Data-processing instruction encodings

Table B.1 Data-processing instructions

cmd	Name	Description	Operation
0000	AND Rd, Rn, Src2	Bitwise AND	$Rd \leftarrow Rn \& Src2$
0001	EOR Rd, Rn, Src2	Bitwise XOR	$Rd \leftarrow Rn \wedge Src2$
0010	SUB Rd, Rn, Src2	Subtract	$Rd \leftarrow Rn - Src2$
0011	RSB Rd, Rn, Src2	Reverse Subtract	$Rd \leftarrow Src2 - Rn$
0100	ADD Rd, Rn, Src2	Add	$Rd \leftarrow Rn + Src2$
0101	ADC Rd, Rn, Src2	Add with Carry	$Rd \leftarrow Rn + Src2 + C$
0110	SBC Rd, Rn, Src2	Subtract with Carry	$Rd \leftarrow Rn - Src2 - \bar{C}$
0111	RSC Rd, Rn, Src2	Reverse Sub w/ Carry	$Rd \leftarrow Src2 - Rn - \bar{C}$
1000 ($S = 1$)	TST Rd, Rn, Src2	Test	Set flags based on $Rn \& Src2$
1001 ($S = 1$)	TEQ Rd, Rn, Src2	Test Equivalence	Set flags based on $Rn \wedge Src2$
1010 ($S = 1$)	CMP Rn, Src2	Compare	Set flags based on $Rn - Src2$
1011 ($S = 1$)	CMN Rn, Src2	Compare Negative	Set flags based on $Rn + Src2$
1100	ORR Rd, Rn, Src2	Bitwise OR	$Rd \leftarrow Rn Src2$
1101 $I = 1$ OR ($instr_{11:4} = 0$)	Shifts: MOV Rd, Src2	Move	$Rd \leftarrow Src2$
$I = 0$ AND ($sh = 00$; $instr_{11:4} \neq 0$)	LSL Rd, Rm, Rs/shamt5	Logical Shift Left	$Rd \leftarrow Rm \ll Src2$
$I = 0$ AND ($sh = 01$)	LSR Rd, Rm, Rs/shamt5	Logical Shift Right	$Rd \leftarrow Rm \gg Src2$

(continued)

Table B.1 Data-processing instructions—Cont'd

cmd	Name	Description	Operation
$I = 0$ AND ($sh = 10$)	ASR Rd, Rm, Rs/shamt5	Arithmetic Shift Right	$Rd \leftarrow Rm \gg \gg Src2$
$I = 0$ AND ($sh = 11$; $instr_{11:7}, 4 = 0$)	RRX Rd, Rm, Rs/shamt5	Rotate Right Extend	$\{Rd, C\} \leftarrow \{C, Rd\}$
$I = 0$ AND ($sh = 11$; $instr_{11:7} \neq 0$)	ROR Rd, Rm, Rs/shamt5	Rotate Right	$Rd \leftarrow Rn \text{ ror } Src2$
1110	BIC Rd, Rn, Src2	Bitwise Clear	$Rd \leftarrow Rn \& \sim Src2$
1111	MVN Rd, Rn, Src2	Bitwise NOT	$Rd \leftarrow \sim Rn$

NOP (no operation) is typically encoded as 0xE1A000, which is equivalent to MOV R0, R0.

B.1.1 Multiply Instructions

Multiply instructions use the encoding in Figure B.2. The 3-bit *cmd* field specifies the type of multiply, as given in Table B.2.

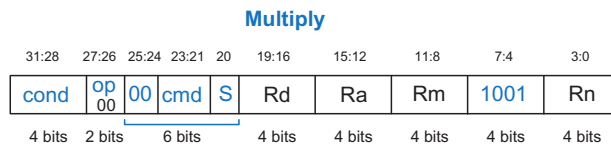


Figure B.2 Multiply instruction encoding

Table B.2 Multiply instructions

cmd	Name	Description	Operation
000	MUL Rd, Rn, Rm	Multiply	$Rd \leftarrow Rn \times Rm$ (low 32 bits)
001	MLA Rd, Rn, Rm, Ra	Multiply Accumulate	$Rd \leftarrow (Rn \times Rm) + Ra$ (low 32 bits)
100	UMULL Rd, Rn, Rm, Ra	Unsigned Multiply Long	$\{Rd, Ra\} \leftarrow Rn \times Rm$ (all 64 bits, Rm/Rn unsigned)
101	UMLAL Rd, Rn, Rm, Ra	Unsigned Multiply Accumulate Long	$\{Rd, Ra\} \leftarrow (Rn \times Rm) + \{Rd, Ra\}$ (all 64 bits, Rm/Rn unsigned)
110	SMULL Rd, Rn, Rm, Ra	Signed Multiply Long	$\{Rd, Ra\} \leftarrow Rn \times Rm$ (all 64 bits, Rm/Rn signed)
111	SMLAL Rd, Rn, Rm, Ra	Signed Multiply Accumulate Long	$\{Rd, Ra\} \leftarrow (Rn \times Rm) + \{Rd, Ra\}$ (all 64 bits, Rm/Rn signed)

B.2 MEMORY INSTRUCTIONS

The most common memory instructions (LDR, STR, LDRB, and STRB) operate on words or bytes and are encoded with $op = 01$. Extra memory instructions operating on halfwords or signed bytes are encoded with $op = 00$ and have less flexibility generating $Src2$. The immediate offset is only 8 bits and the register offset cannot be shifted. LDRB and LDRH zero-extend the bits to fill a word, while LDRSB and LDRSH sign-extend the bits. Also see memory indexing modes in Section 6.3.6.

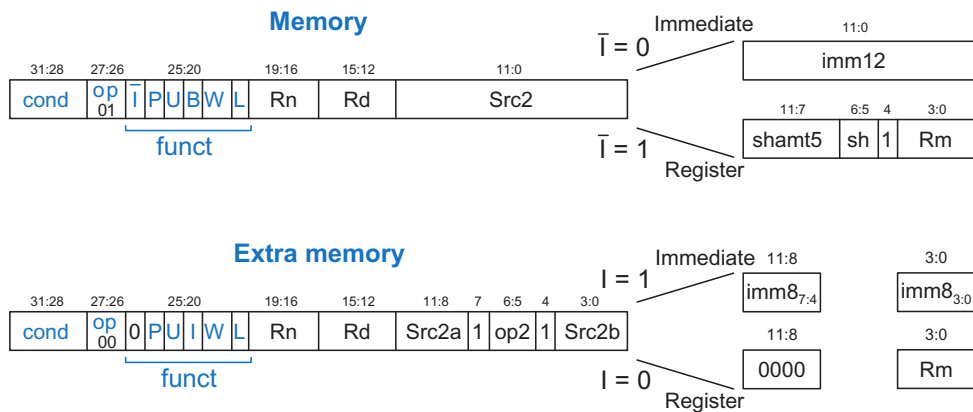


Figure B.3 Memory instruction encodings

Table B.3 Memory instructions

op	B	op2	L	Name	Description	Operation
01	0	N/A	0	STR Rd, [Rn, ±Src2]	Store Register	Mem[Adr] ← Rd
01	0	N/A	1	LDR Rd, [Rn, ±Src2]	Load Register	Rd ← Mem[Adr]
01	1	N/A	0	STRB Rd, [Rn, ±Src2]	Store Byte	Mem[Adr] ← Rd _{7:0}
01	1	N/A	1	LDRB Rd, [Rn, ±Src2]	Load Byte	Rd ← Mem[Adr] _{7:0}
00	N/A	01	0	STRH Rd, [Rn, ±Src2]	Store Halfword	Mem[Adr] ← Rd _{15:0}
00	N/A	01	1	LDRH Rd, [Rn, ±Src2]	Load Halfword	Rd ← Mem[Adr] _{15:0}
00	N/A	10	1	LDRSB Rd, [Rn, ±Src2]	Load Signed Byte	Rd ← Mem[Adr] _{7:0}
00	N/A	11	1	LDRSH Rd, [Rn, ±Src2]	Load Signed Half	Rd ← Mem[Adr] _{15:0}

B.3 BRANCH INSTRUCTIONS

Figure B.4 shows the encoding for branch instructions (B and BL) and Table B.4 describes their operation.

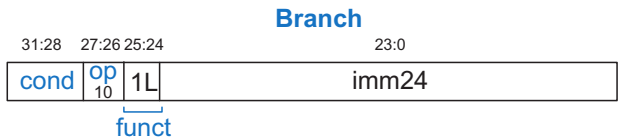


Figure B.4 Branch instruction encoding

Table B.4 Branch instructions

L	Name	Description	Operation
0	B label	Branch	$PC \leftarrow (PC+8)+imm24 \ll 2$
1	BL label	Branch with Link	$LR \leftarrow (PC+8) - 4; PC \leftarrow (PC+8)+imm24 \ll 2$

B.4 MISCELLANEOUS INSTRUCTIONS

The ARMv4 instruction set includes the following miscellaneous instructions. Consult the ARM Architecture Reference Manual for details.

Instructions	Description	Purpose
LDM, STM	Load/store multiple	Save and recall registers in subroutine calls
SWP / SWPB	Swap (byte)	Atomic load and store for process synchronization
LDRT, LDRBT, STRT, STRBT	Load/store word/byte with translation	Allow operating system to access memory in user virtual memory space
SWI ¹	Software Interrupt	Create an exception, often used to call the operating system
CDP, LDC, MCR, MRC, STC	Coprocessor access	Communicate with optional coprocessor
MRS, MSR	Move from/to status register	Save status register during exceptions

¹ SWI was renamed SVC (supervisor call) in ARMv7.

B.5 CONDITION FLAGS

Condition flags are changed by data-processing instructions with $S = 1$ in the machine code. All instructions except `CMP`, `CMN`, `TEQ`, and `TST` must have an “S” appended to the instruction mnemonic to make $S = 1$. [Table B.5](#) shows which condition flags are affected by each instruction.

Table B.5 Instructions that affect condition flags

Type	Instructions	Condition Flags
Add	<code>ADDS</code> , <code>ADCS</code>	N, Z, C, V
Subtract	<code>SUBS</code> , <code>SBCS</code> , <code>RSBS</code> , <code>RSCS</code>	N, Z, C, V
Compare	<code>CMP</code> , <code>CMN</code>	N, Z, C, V
Shifts	<code>ASRS</code> , <code>LSLS</code> , <code>LSRS</code> , <code>RORS</code> , <code>RRXS</code>	N, Z, C
Logical	<code>ANDS</code> , <code>ORRS</code> , <code>EORS</code> , <code>BICS</code>	N, Z, C
Test	<code>TEQ</code> , <code>TST</code>	N, Z, C
Move	<code>MOVS</code> , <code>MVNS</code>	N, Z, C
Multiply	<code>MULS</code> , <code>MLAS</code> , <code>SMLALS</code> , <code>SMULLS</code> , <code>UMLALS</code> , <code>UMULLS</code>	N, Z

C Programming

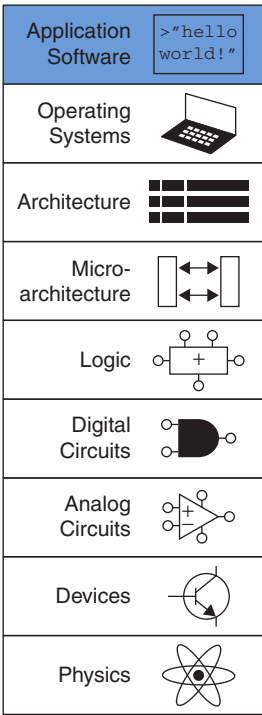


C.1 INTRODUCTION

The overall goal of this book is to give a picture of how computers work on many levels, from the transistors by which they are constructed all the way up to the software they run. The first five chapters of this book work up through the lower levels of abstraction, from transistors to gates to logic design. Chapters 6 through 8 jump up to architecture and work back down to microarchitecture to connect the hardware with the software. This Appendix on C programming fits logically between Chapters 5 and 6, covering C programming as the highest level of abstraction in the text. It motivates the architecture material and links this book to programming experience that may already be familiar to the reader. This material is placed in the Appendix so that readers may easily cover or skip it depending on previous experience.

The rest of this chapter is available online as a downloadable PDF from the book's companion site: <http://booksite.elsevier.com/9780128000564>.

- C.1 Introduction
- C.2 Welcome to C
- C.3 Compilation
- C.4 Variables
- C.5 Operators
- C.6 Function Calls
- C.7 Control-Flow Statements
- C.8 More Data Types
- C.9 Standard Libraries
- C.10 Compiler and Command Line Options
- C.11 Common Mistakes



C Programming



C.1 INTRODUCTION

The overall goal of this book is to give a picture of how computers work on many levels, from the transistors by which they are constructed all the way up to the software they run. The first five chapters of this book work up through the lower levels of abstraction, from transistors to gates to logic design. Chapters 6 through 8 jump up to architecture and work back down to micro-architecture to connect the hardware with the software. This Appendix on C programming fits logically between Chapters 5 and 6, covering C programming as the highest level of abstraction in the text. It motivates the architecture material and links this book to programming experience that may already be familiar to the reader. This material is placed in the Appendix so that readers may easily cover or skip it depending on previous experience.

Programmers use many different languages to tell a computer what to do. Fundamentally, computers process instructions in *machine language* consisting of 1's and 0's, as is explored in Chapter 6. But programming in machine language is tedious and slow, leading programmers to use more abstract languages to get their meaning across more efficiently. Table eC.1 lists some examples of languages at various levels of abstraction.

One of the most popular programming languages ever developed is called C. It was created by a group including Dennis Ritchie and Brian Kernighan at Bell Laboratories between 1969 and 1973 to rewrite the UNIX operating system from its original assembly language. By many measures, C (including a family of closely related languages such as C++, C#, and Objective C) is the most widely used language in existence. Its popularity stems from a number of factors including its:

- ▶ Availability on a tremendous variety of platforms, from supercomputers down to embedded microcontrollers
- ▶ Relative ease of use, with a huge user base

C.1	Introduction
C.2	Welcome to C
C.3	Compilation
C.4	Variables
C.5	Operators
C.6	Function Calls
C.7	Control-Flow Statements
C.8	More Data Types
C.9	Standard Libraries
C.10	Compiler and Command Line Options
C.11	Common Mistakes

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	



Dennis Ritchie, 1941–2011



Brian Kernighan, 1942–

C was formally introduced in 1978 by Brian Kernighan and Dennis Ritchie's classic book, *The C Programming Language*. In 1989, the American National Standards Institute (ANSI) expanded and standardized the language, which became known as ANSI C, Standard C, or C89. Shortly thereafter, in 1990, this standard was adopted by the International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC). ISO/IEC updated the standard in 1999 to what is called C99, which we will be discussing in this text.

Table eC.1 Languages at roughly decreasing levels of abstraction

Language	Description
Matlab	Designed to facilitate heavy use of math functions
Perl	Designed for scripting
Python	Designed to emphasize code readability
Java	Designed to run securely on any computer
C	Designed for flexibility and overall system access, including device drivers
Assembly Language	Human-readable machine language
Machine Language	Binary representation of a program

- ▶ Moderate level of abstraction providing higher productivity than assembly language, yet giving the programmer a good understanding of how the code will be executed
- ▶ Suitability for generating high performance programs
- ▶ Ability to interact directly with the hardware

This chapter is devoted to C programming for a variety of reasons. Most importantly, C allows the programmer to directly access addresses in memory, illustrating the connection between hardware and software emphasized in this book. C is a practical language that all engineers and computer scientists should know. Its uses in many aspects of implementation and design – e.g., software development, embedded systems programming, and simulation – make proficiency in C a vital and marketable skill.

The following sections describe the overall syntax of a C program, discussing each part of the program including the header, function and variable declarations, data types, and commonly used functions provided in libraries. Chapter 9 (available as a web supplement, see Preface) describes a hands-on application by using C to program an ARM-based Raspberry Pi computer.

SUMMARY

- ▶ **High-level programming:** High-level programming is useful at many levels of design, from writing analysis or simulation software to programming microcontrollers that interact with hardware.
- ▶ **Low-level access:** C code is powerful because, in addition to high-level constructs, it provides access to low-level hardware and memory.

C.2 WELCOME TO C

A C program is a text file that describes operations for the computer to perform. The text file is *compiled*, converted into a machine-readable format, and run or *executed* on a computer. [C Code Example eC.1](#) is a simple C program that prints the phrase “Hello world!” to the *console*, the computer screen. C programs are generally contained in one or more text files that end in “.c”. Good programming style requires a file name that indicates the contents of the program – for example, this file could be called `hello.c`.

C Code Example eC.1 SIMPLE C PROGRAM

```
// Write "Hello world!" to the console
#include <stdio.h>

int main(void){
    printf("Hello world!\n");
}
```

Console Output

```
Hello world!
```

C is the language used to program such ubiquitous systems as Linux, Windows, and iOS. C is a powerful language because of its direct access to hardware. As compared with other high level languages, for example Perl and Matlab, C does not have as much built-in support for specialized operations such as file manipulation, pattern matching, matrix manipulation, and graphical user interfaces. It also lacks features to protect the programmer from common mistakes, such as writing data past the end of an array. Its power combined with its lack of protection has assisted hackers who exploit poorly written software to break into computer systems.

C.2.1 C Program Dissection

In general, a C program is organized into one or more functions. Every program must include the `main` function, which is where the program starts executing. Most programs use other functions defined elsewhere in the C code and/or in a library. The overall sections of the `hello.c` program are the header, the `main` function, and the body.

Header: `#include <stdio.h>`

The header includes the *library functions* needed by the program. In this case, the program uses the `printf` function, which is part of the standard I/O library, `stdio.h`. See [Section C.9](#) for further details on C’s built-in libraries.

Main function: `int main(void)`

All C programs must include exactly one `main` function. Execution of the program occurs by running the code inside `main`, called the *body* of `main`. Function syntax is described in [Section C.6](#). The body of a function contains a sequence of *statements*. Each statement ends with a semicolon. `int` denotes that the `main` function outputs, or *returns*, an integer result that indicates whether the program ran successfully.

While this chapter provides a fundamental understanding of C programming, entire texts are written that describe C in depth. One of our favorites is the classic text *The C Programming Language* by Brian Kernighan and Dennis Ritchie, the developers of C. This text gives a concise description of the nuts and bolts of C. Another good text is *A Book on C* by Al Kelley and Ira Pohl.

Body: `printf("Hello world!\n");`

The body of this `main` function contains one statement, a call to the `printf` function, which prints the phrase “Hello world!” followed by a newline character indicated by the special sequence “`\n`”. Further details about I/O functions are described in [Section C.9.1](#).

All programs follow the general format of the simple `hello.c` program. Of course, very complex programs may contain millions of lines of code and span hundreds of files.

C.2.2 Running a C Program

C programs can be run on many different machines. This *portability* is another advantage of C. The program is first compiled on the desired machine using the *C compiler*. Slightly different versions of the C compiler exist, including *cc* (C compiler), or *gcc* (GNU C compiler). Here we show how to compile and run a C program using *gcc*, which is freely available for download. It runs directly on Linux machines and is accessible under the Cygwin environment on Windows machines. It is also available for many embedded systems such as the ARM-based Raspberry Pi. The general process described below of C file creation, compilation, and execution is the same for any C program.

1. Create the text file, for example `hello.c`.
2. In a terminal window, change to the directory that contains the file `hello.c` and type `gcc hello.c` at the command prompt.
3. The compiler creates an executable file. By default, the executable is called `a.out` (or `a.exe` on Windows machines).
4. At the command prompt, type `./a.out` (or `./a.exe` on Windows) and press return.
5. “Hello world!” will appear on the screen.

SUMMARY

- ▶ `filename.c`: C program files are typically named with a `.c` extension.
- ▶ `main`: Each C program must have exactly one `main` function.
- ▶ `#include`: Most C programs use functions provided by built-in libraries. These functions are used by writing `#include <library.h>` at the top of the C file.
- ▶ `gcc filename.c`: C files are converted into an executable using a compiler such as the GNU compiler (*gcc*) or the C compiler (*cc*).
- ▶ **Execution**: After compilation, C programs are executed by typing `./a.out` (or `./a.exe`) at the command line prompt.

C.3 COMPILATION

A compiler is a piece of software that reads a program in a high-level language and converts it into a file of machine code called an executable. Entire textbooks are written on compilers, but we describe them here briefly. The overall operation of the compiler is to (1) preprocess the file by including referenced libraries and expanding macro definitions, (2) ignore all unnecessary information such as comments, (3) translate the high-level code into simple instructions native to the processor that are represented in binary, called machine language, and (4) compile all the instructions into a single binary executable that can be read and executed by the computer. Each machine language is specific to a given processor, so a program must be compiled specifically for the system on which it will run. For example, the ARM machine language is covered in Chapter 6 in detail.

C.3.1 Comments

Programmers use comments to describe code at a high-level and clarify code function. Anyone who has read uncommented code can attest to their importance. C programs use two types of comments: Single-line comments begin with `//` and terminate at the end of the line; multiple-line comments begin with `/*` and end with `*/`. While comments are critical to the organization and clarity of a program, they are ignored by the compiler.

```
// This is an example of a one-line comment.  
/* This is an example  
   of a multi-line comment. */
```

A comment at the top of each C file is useful to describe the file's author, creation and modification dates, and purpose. The comment below could be included at the top of the `hello.c` file.

```
// hello.c  
// 1 Jan 2015 Sarah_Harris@hmc.edu, David_Harris@hmc.edu  
//  
// This program prints "Hello world!" to the screen
```

C.3.2 #define

Constants are named using the `#define` directive and then used by name throughout the program. These globally defined constants are also called *macros*. For example, suppose you write a program that allows at most 5 user guesses, you can use `#define` to identify that number.

```
#define MAXGUESSES 5
```

Number constants in C default to decimal but can also be hexadecimal (prefix "0x") or octal (prefix "0"). Binary constants are not defined in C99 but are supported by some compilers (prefix "0b"). For example, the following assignments are equivalent:

```
char x = 37;
char x = 0x25;
char x = 045;
```

Globally defined constants eradicate *magic numbers* from a program. A magic number is a constant that shows up in a program without a name. The presence of magic numbers in a program often introduces tricky bugs – for example, when the number is changed in one location but not another.

The `#` indicates that this line in the program will be handled by the *pre-processor*. Before compilation, the preprocessor replaces each occurrence of the identifier `MAXGUESSES` in the program with 5. By convention, `#define` lines are located at the top of the file and identifiers are written in all capital letters. By defining constants in one location and then using the identifier in the program, the program remains consistent, and the value is easily modified – it need only be changed at the `#define` line instead of at each line in the code where the value is needed.

C Code Example eC.2 shows how to use the `#define` directive to convert inches to centimeters. The variables `inch` and `cm` are declared to be `float` to indicate they represent single-precision floating point numbers. If the conversion factor (`INCH2CM`) were used throughout a large program, having it declared using `#define` obviates errors due to typos (for example, typing 2.53 instead of 2.54) and makes it easy to find and change (for example, if more significant digits were required).

C Code Example eC.2 USING `#define` TO DECLARE CONSTANTS

```
// Convert inches to centimeters
#include <stdio.h>

#define INCH2CM 2.54

int main(void) {
    float inch = 5.5;    // 5.5 inches
    float cm;

    cm = inch * INCH2CM;
    printf("%f inches = %f cm\n", inch, cm);
}
```

Console Output

```
5.500000 inches = 13.970000 cm
```

C.3.3 `#include`

Modularity encourages us to split programs across separate files and functions. Commonly used functions can be grouped together for easy reuse. Variable declarations, defined values, and function definitions located in a *header file* can be used by another file by adding the `#include` preprocessor directive. *Standard libraries* that provide commonly used functions are accessed in this way. For example, the following line is required to use the functions defined in the standard input/output (I/O) library, such as `printf`.

```
#include <stdio.h>
```

The “.h” postfix of the include file indicates it is a header file. While `#include` directives can be placed anywhere in the file before the included

functions, variables, or identifiers are needed, they are conventionally located at the top of a C file.

Programmer-created header files can also be included by using quotation marks (" ") around the file name instead of brackets (< >). For example, a user-created header file called `myfunctions.h` would be included using the following line.

```
#include "myfunctions.h"
```

At compile time, files specified in brackets are searched for in system directories. Files specified in quotes are searched for in the same local directory where the C file is found. If the user-created header file is located in a different directory, the path of the file relative to the current directory must be included.

SUMMARY

- ▶ **Comments:** C provides single-line comments (`//`) and multi-line comments (`/* */`).
- ▶ **#define NAME val:** the `#define` directive allows an identifier (NAME) to be used throughout the program. Before compilation, all instances of NAME are replaced with val.
- ▶ **#include:** `#include` allows common functions to be used in a program. For built-in libraries, include the following line at the top of the code: `#include <library.h>` To include a user-defined header file, the name must be in quotes, listing the path relative to the current directory as needed: i.e., `#include "other/myFuncs.h"`.

C.4 VARIABLES

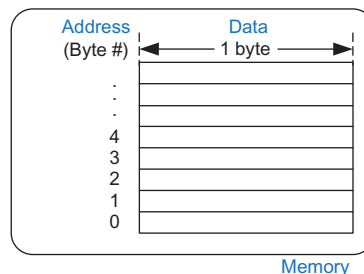
Variables in C programs have a type, name, value, and memory location. A variable declaration states the type and name of the variable. For example, the following declaration states that the variable is of type `char` (which is a 1-byte type), and that the variable name is `x`. The compiler decides where to place this 1-byte variable in memory.

```
char x;
```

C views memory as a group of consecutive bytes, where each byte of memory is assigned a unique number indicating its location or *address*, as shown in [Figure eC.1](#). A variable occupies one or more bytes of memory, and the address of multiple-byte variables is indicated by the lowest numbered byte. The type of a variable indicates whether to interpret the byte(s) as an integer, floating point number, or other type. The rest of this section describes C's primitive data types, the declaration of global and local variables, and the initialization of variables.

Variable names are case sensitive and can be of your choosing. However, the name may not be any of C's reserved words (i.e., `int`, `while`, etc.), may not start with a number (i.e., `int 1x;` is not a valid declaration), and may not include special characters such as `\`, `*`, `?`, or `-`. Underscores (`_`) are allowed.

Figure eC.1 C's view of memory



C.4.1 Primitive Data Types

C has a number of primitive, or built-in, data types available. They can be broadly characterized as integers, floating-point variables, and characters. An integer represents a two's complement or unsigned number within a finite range. A floating-point variable uses IEEE floating point representation to describe real numbers with a finite range and precision. A character can be viewed as either an ASCII value or an 8-bit integer.¹ Table eC.2 lists the size and range of each primitive type. Integers may be 16, 32, or 64 bits. They use two's complement unless qualified as unsigned.

Table eC.2 Primitive data types and sizes

Type	Size (bits)	Minimum	Maximum
char	8	$-2^{-7} = -128$	$2^7 - 1 = 127$
unsigned char	8	0	$2^8 - 1 = 255$
short	16	$-2^{15} = -32,768$	$2^{15} - 1 = 32,767$
unsigned short	16	0	$2^{16} - 1 = 65,535$
long	32	$-2^{31} = -2,147,483,648$	$2^{31} - 1 = 2,147,483,647$
unsigned long	32	0	$2^{32} - 1 = 4,294,967,295$
long long	64	-2^{63}	$2^{63} - 1$
unsigned long	64	0	$2^{64} - 1$
int	machine-dependent		
unsigned int	machine-dependent		
float	32	$\pm 2^{-126}$	$\pm 2^{127}$
double	64	$\pm 2^{-1023}$	$\pm 2^{1022}$

¹ Technically, the C99 standard defines a character as “a bit representation that fits in a byte,” without requiring a byte to be 8 bits. However, current systems define a byte as 8 bits.

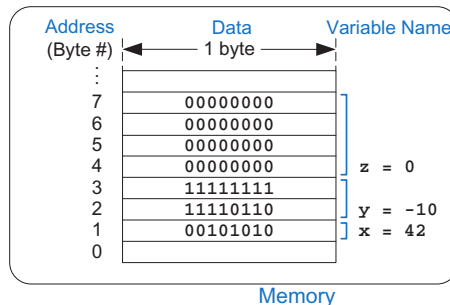


Figure eC.2 Variable storage in memory for C Code Example eC.3

The size of the `int` type is machine dependent and is generally the native word size of the machine. For example, on a 32-bit ARM processor, the size of an `int` or unsigned `int` is 32 bits. Floating point numbers may be 32- or 64-bit single or double precision. Characters are 8 bits.

C Code Example eC.3 shows the declaration of variables of different types. As shown in Figure eC.2, `x` requires one byte of data, `y` requires two, and `z` requires four. The program decides where these bytes are stored in memory, but each type always requires the same amount of data. For illustration, the addresses of `x`, `y`, and `z` in this example are 1, 2, and 4. Variable names are case-sensitive, so, for example, the variable `x` and the variable `X` are two different variables. (But it would be very confusing to use both in the same program!)

C Code Example eC.3 EXAMPLE DATA TYPES

```
// Examples of several data types and their binary representations
unsigned char x = 42;      // x = 00101010
short y = -10;            // y = 11111111 11110110
unsigned long z = 0;      // z = 00000000 00000000 00000000 00000000
```

C.4.2 Global and Local Variables

Global and local variables differ in where they are declared and where they are visible. A global variable is declared outside of all functions, typically at the top of a program, and can be accessed by all functions. Global variables should be used sparingly because they violate the principle of modularity, making large programs more difficult to read. However, a variable accessed by many functions can be made global.

A local variable is declared inside a function and can only be used by that function. Therefore, two functions can have local variables with the same names without interfering with each other. Local variables are declared at the beginning of a function. They cease to exist when the function ends and are recreated when the function is called again. They do not retain their value from one invocation of a function to the next.

The machine-dependent nature of the `int` data type is a blessing and a curse. On the bright side, it matches the native word size of the processor so it can be fetched and manipulated efficiently. On the down side, programs using `ints` may behave differently on different computers. For example, a banking program might store the number of cents in your bank account as an `int`. When compiled on a 64-bit PC, it will have plenty of range for even the wealthiest entrepreneur. But if it is ported to a 16-bit microcontroller, it will overflow for accounts exceeding \$327.67, resulting in unhappy and poverty-stricken customers.

The *scope* of a variable is the context in which it can be used. For example, for a local variable, its scope is the function in which it is declared. It is out of scope everywhere else.

C Code Examples eC.4 and eC.5 compare programs using global versus local variables. In C Code Example eC.4, the global variable `max` can be accessed by any function. Using a local variable, as shown in C Code Example eC.5, is the preferred style because it preserves the well-defined interface of modularity.

C Code Example eC.4 GLOBAL VARIABLES

```
// Use a global variable to find and print the maximum of 3 numbers
int max;                // global variable holding the maximum value

void findMax(int a, int b, int c) {
    max = a;
    if (b > max) {
        if (c > b) max = c;
        else     max = b;
    } else if (c > max) max = c;
}

void printMax(void) {
    printf("The maximum number is: %d\n", max);
}

int main(void) {
    findMax(4, 3, 7);
    printMax();
}
```

C Code Example eC.5 LOCAL VARIABLES

```
// Use local variables to find and print the maximum of 3 numbers

int getMax(int a, int b, int c) {
    int result = a; // local variable holding the maximum value

    if (b > result) {
        if (c > b) result = c;
        else     result = b;
    } else if (c > result) result = c;

    return result;
}

void printMax(int m) {
    printf("The maximum number is: %d\n", m);
}

int main(void) {
    int max;

    max = getMax(4, 3, 7);
    printMax(max);
}
```

C.4.3 Initializing Variables

A variable needs to be *initialized* – assigned a value – before it is read. When a variable is declared, the correct number of bytes is reserved for that variable in memory. However, the memory at those locations retains whatever value it had last time it was used, essentially a random value. Global and local variables can be initialized either when they are declared or within the body of the program. [C Code Example eC.3](#) shows variables initialized at the same time they are declared. [C Code Example eC.4](#) shows how variables are initialized before their use, but after declaration; the global variable `max` is initialized by the `getMax` function before it is read by the `printMax` function. Reading from uninitialized variables is a common programming error, and can be tricky to debug.

SUMMARY

- ▶ **Variables:** Each variable is defined by its data type, name, and memory location. A variable is declared as `datatype name`.
- ▶ **Data types:** A data type describes the size (number of bytes) and representation (interpretation of the bytes) of a variable. [Table eC.2](#) lists C's built-in data types.
- ▶ **Memory:** C views memory as a list of bytes. Memory stores variables and associates each variable with an address (byte number).
- ▶ **Global variables:** Global variables are declared outside of all functions and can be accessed anywhere in the program.
- ▶ **Local variables:** Local variables are declared within a function and can be accessed only within that function.
- ▶ **Variable initialization:** Each variable must be initialized before it is read. Initialization can happen either at declaration or afterward.

C.5 OPERATORS

The most common type of statement in a C program is an *expression*, such as

```
y = a + 3;
```

An expression involves operators (such as `+` or `*`) acting on one or more operands, such as variables or constants. C supports the operators shown in [Table eC.3](#), listed by category and in order of decreasing precedence. For example, multiplicative operators take precedence over additive

Table eC.3 Operators listed by decreasing precedence

Category	Operator	Description	Example
Unary	++	post-increment	a++; // a = a+1
	--	post-decrement	x--; // x = x-1
	&	memory address of a variable	x = &y; // x = the memory // address of y
	~	bitwise NOT	z = ~a;
	!	Boolean NOT	!x
	-	negation	y = -a;
	++	pre-increment	++a; // a = a+1
	--	pre-decrement	--x; // x = x-1
	(type)	casts a variable to (type)	x = (int)c; // cast c to an // int and assign it to x
	sizeof()	size of a variable or type in bytes	long int y; x = sizeof(y); // x = 4
Multiplicative	*	multiplication	y = x * 12;
	/	division	z = 9 / 3; // z = 3
	%	modulo	z = 5 % 2; // z = 1
Additive	+	addition	y = a + 2;
	-	subtraction	y = a - 2;
Bitwise Shift	<<	bitshift left	z = 5 << 2; // z = 0b00010100
	>>	bitshift right	x = 9 >> 3; // x = 0b00000001
Relational	==	equals	y == 2
	!=	not equals	x != 7
	<	less than	y < 12
	>	greater than	val > max
	<=	less than or equal	z <= 2
	>=	greater than or equal	y >= 10

(continued)

Table eC.3 Operators listed by decreasing precedence—Cont'd

Category	Operator	Description	Example
Bitwise	&	bitwise AND	<code>y = a & 15;</code>
	^	bitwise XOR	<code>y = 2 ^ 3;</code>
		bitwise OR	<code>y = a b;</code>
Logical	&&	Boolean AND	<code>x && y</code>
		Boolean OR	<code>x y</code>
Ternary	? :	ternary operator	<code>y = x ? a : b; // if x is TRUE, // y=a, else y=b</code>
Assignment	=	assignment	<code>x = 22;</code>
	+=	addition and assignment	<code>y += 3; // y = y + 3</code>
	-=	subtraction and assignment	<code>z -= 10; // z = z - 10</code>
	*=	multiplication and assignment	<code>x *= 4; // x = x * 4</code>
	/=	division and assignment	<code>y /= 10; // y = y / 10</code>
	%=	modulo and assignment	<code>x %= 4; // x = x % 4</code>
	>>=	bitwise right-shift and assignment	<code>x >>= 5; // x = x >> 5</code>
	<<=	bitwise left-shift and assignment	<code>x <<= 2; // x = x << 2</code>
	&=	bitwise AND and assignment	<code>y &= 15; // y = y & 15</code>
	=	bitwise OR and assignment	<code>x = y; // x = x y</code>
	^=	bitwise XOR and assignment	<code>x ^= y; // x = x ^ y</code>

operators. Within the same category, operators are evaluated in the order that they appear in the program.

Unary operators, also called monadic operators, have a single operand. Ternary operators have three operands, and all others have two. The ternary operator (from the Latin *ternarius* meaning consisting of three) chooses the second or third operand depending on whether the first value is TRUE (nonzero) or FALSE (zero), respectively. [C Code Example eC.6](#) shows how to compute `y = max(a, b)` using the ternary operator, along with an equivalent but more verbose `if/else` statement.

The Truth, the Whole Truth, and Nothing But the Truth
C considers a variable to be TRUE if it is nonzero and FALSE if it is zero. Logical and ternary operators, as well as control-flow statements such as `if` and `while`, depend on the truth of a variable. Relational and logical operators produce a result that is 1 when TRUE or 0 when FALSE.

C Code Example eC.6 (a) TERNARY OPERATOR, AND (b) EQUIVALENT `if/else` STATEMENT

```
(a) y = (a > b) ? a : b; // parentheses not necessary, but makes it clearer
(b) if (a > b) y = a;
    else      y = b;
```

Simple assignment uses the `=` operator. C code also allows for compound assignment, that is, assignment after a simple operation such as addition (`+=`) or multiplication (`*=`). In compound assignments, the variable on the left side is both operated on and assigned the result. [C Code Example eC.7](#) shows these and other C operations. Binary values in the comments are indicated with the prefix `0b`.

C Code Example eC.7 OPERATOR EXAMPLES

Expression	Result	Notes
<code>44 / 14</code>	3	Integer division truncates
<code>44 % 14</code>	2	44 mod 14
<code>0x2C && 0xE //0b101100 && 0b1110</code>	1	Logical AND
<code>0x2C 0xE //0b101100 0b1110</code>	1	Logical OR
<code>0x2C & 0xE //0b101100 & 0b1110</code>	0xC (0b001100)	Bitwise AND
<code>0x2C 0xE //0b101100 0b1110</code>	0x2E (0b101110)	Bitwise OR
<code>0x2C ^ 0xE //0b101100 ^ 0b1110</code>	0x22 (0b100010)	Bitwise XOR
<code>0xE << 2 //0b1110 << 2</code>	0x38 (0b111000)	Left shift by 2
<code>0x2C >> 3 //0b101100 >> 3</code>	0x5 (0b101)	Right shift by 3
<code>x = 14; x += 2;</code>	x=16	
<code>y = 0x2C; // y = 0b101100 y &= 0xF; // y &= 0b1111</code>	y=0xC (0b001100)	
<code>x = 14; y = 44; y = y + x++;</code>	x=15, y=58	Increment x after using it
<code>x = 14; y = 44; y = y + ++x;</code>	x=15, y=59	Increment x before using it

C.6 FUNCTION CALLS

Modularity is key to good programming. A large program is divided into smaller parts called functions that, similar to hardware modules, have well-defined inputs, outputs, and behavior. [C Code Example eC.8](#) shows the `sum3` function. The function declaration begins with the return type, `int`, followed by the name, `sum3`, and the inputs enclosed within parentheses (`int a, int b, int c`). Curly braces `{}` enclose the body of the function, which may contain zero or more statements. The `return` statement indicates the value that the function should return to its caller; this can be viewed as the output of the function. A function can only return a single value.

C Code Example eC.8 `sum3` FUNCTION

```
// Return the sum of the three input variables
int sum3(int a, int b, int c) {
    int result = a + b + c;
    return result;
}
```

After the following call to `sum3`, `y` holds the value 42.

```
int y = sum3(10, 15, 17);
```

Although a function may have inputs and outputs, neither is required. [C Code Example eC.9](#) shows a function with no inputs or outputs. The keyword `void` before the function name indicates that nothing is returned. `void` between the parentheses indicates that the function has no input arguments.

C Code Example eC.9 FUNCTION `printPrompt` WITH NO INPUTS OR OUTPUTS

```
// Print a prompt to the console
void printPrompt(void)
{
    printf("Please enter a number from 1-3:\n");
}
```

Nothing between the parentheses also indicates no input arguments. So, in this case we could have written:

```
void printPrompt()
```

A function must be declared in the code before it is called. This may be done by placing the called function earlier in the file. For this reason, `main` is often placed at the end of the C file after all the functions it calls. Alternatively, a function *prototype* can be placed in the program before the function is defined. The function prototype is the first line of

With careful ordering of functions, prototypes may be unnecessary. However, they are unavoidable in certain cases, such as when function `f1` calls `f2` and `f2` calls `f1`. It is good style to place prototypes for all of a program's functions near the beginning of the C file or in a header file.

As with variable names, function names are case sensitive, cannot be any of C's reserved words, may not contain special characters (except underscore `_`), and cannot start with a number. Typically function names include a verb to indicate what they do.

Be consistent in how you capitalize your function and variable names so you don't have to constantly look up the correct capitalization. Two common styles are to camelCase, in which the initial letter of each word after the first is capitalized like the humps of a camel (e.g., `printPrompt`), or to use underscores between words (e.g., `print_prompt`). We have unscientifically observed that reaching for the underscore key exacerbates carpal tunnel syndrome (my pinky finger twinges just thinking about the underscore!) and hence prefer camelCase. But the most important thing is to be consistent in style within your organization.

the function, declaring the return type, function name, and function inputs. For example, the function prototypes for the functions in [C Code Examples eC.8](#) and [eC.9](#) are:

```
int sum3(int a, int b, int c);
void printPrompt(void);
```

[C Code Example eC.10](#) shows how function prototypes are used. Even though the functions themselves are after `main`, the function prototypes at the top of the file allow them to be used in `main`.

C Code Example eC.10 FUNCTION PROTOTYPES

```
#include <stdio.h>

// function prototypes
int sum3(int a, int b, int c);
void printPrompt(void);

int main(void)
{
    int y = sum3(10, 15, 20);

    printf("sum3 result: %d\n", y);
    printPrompt();
}

int sum3(int a, int b, int c) {
    int result = a+b+c;
    return result;
}

void printPrompt(void) {
    printf("Please enter a number from 1-3:\n");
}
```

Console Output

```
sum3 result: 45
Please enter a number from 1-3:
```

The `main` function is always declared to return an `int`, which conveys to the operating system the reason for program termination. A zero indicates normal completion, while a nonzero value signals an error condition. If `main` reaches the end without encountering a `return` statement, it will automatically return 0. Most operating systems do not automatically inform the user of the value returned by the program.

C.7 CONTROL-FLOW STATEMENTS

C provides control-flow statements for conditionals and loops. Conditionals execute a statement only if a condition is met. A loop repeatedly executes a statement as long as a condition is met.

C.7.1 Conditional Statements

if, if/else, and switch/case statements are conditional statements commonly used in high-level languages including C.

if Statements

An if statement executes the statement immediately following it when the expression in parentheses is TRUE (i.e., nonzero). The general format is:

```
if (expression)
    statement
```

C Code Example eC.11 shows how to use an if statement in C. When the variable `aintBroke` is equal to 1, the variable `dontFix` is set to 1. A block of multiple statements can be executed by placing curly braces `{}` around the statements, as shown in C Code Example eC.12.

Curly braces, `{}`, are used to group one or more statements into a *compound statement* or *block*.

C Code Example eC.11 if STATEMENT

```
int dontFix = 0;

if (aintBroke == 1)
    dontFix = 1;
```

C Code Example eC.12 if STATEMENT WITH A BLOCK OF CODE

```
// If amt >= $2, prompt user and dispense candy
if (amt >= 2) {
    printf("Select candy.\n");
    dispenseCandy = 1;
}
```

if/else Statements

if/else statements execute one of two statements depending on a condition, as shown below. When the expression in the if statement is TRUE, `statement1` is executed. Otherwise, `statement2` is executed.

```
if (expression)
    statement1
else
    statement2
```

C Code Example eC.6(b) gives an example if/else statement in C. The code sets `y` equal to `a` if `a` is greater than `b`; otherwise `y = b`.

switch/case Statements

switch/case statements execute one of several statements depending on the conditions, as shown in the general format below.

```
switch (variable) {  
    case (expression1): statement1 break;  
    case (expression2): statement2 break;  
    case (expression3): statement3 break;  
    default:             statement4  
}
```

For example, if `variable` is equal to `expression2`, execution continues at `statement2` until the keyword `break` is reached, at which point it exits the `switch/case` statement. If no conditions are met, the `default` executes.

If the keyword `break` is omitted, execution begins at the point where the condition is `TRUE` and then falls through to execute the remaining cases below it. This is usually not what you want and is a common error among beginning C programmers.

[C Code Example eC.13](#) shows a `switch/case` statement that, depending on the variable `option`, determines the amount of money `amt` to be disbursed. A `switch/case` statement is equivalent to a series of nested `if/else` statements, as shown by the equivalent code in [C Code Example eC.14](#).

C Code Example eC.13 switch/case STATEMENT

```
// Assign amt depending on the value of option  
switch (option) {  
    case 1:  amt = 100; break;  
    case 2:  amt = 50;  break;  
    case 3:  amt = 20;  break;  
    case 4:  amt = 10;  break;  
    default: printf("Error: unknown option.\n");  
}
```

C Code Example eC.14 NESTED if/else STATEMENT

```
// Assign amt depending on the value of option  
if (option == 1) amt = 100;  
else if (option == 2) amt = 50;  
else if (option == 3) amt = 20;  
else if (option == 4) amt = 10;  
else printf("Error: unknown option.\n");
```

C.7.2 Loops

while, do/while, and for loops are common loop constructs used in many high-level languages including C. These loops repeatedly execute a statement as long as a condition is satisfied.

while Loops

while loops repeatedly execute a statement until a condition is not met, as shown in the general format below.

```
while (condition)
    statement
```

The while loop in [C Code Example eC.15](#) computes the factorial of $9 = 9 \times 8 \times 7 \times \dots \times 1$. Note that the condition is checked before executing the statement. In this example, the statement is a compound statement or block, so curly braces are required.

C Code Example eC.15 while LOOP

```
// Compute 9! (the factorial of 9)
int i = 1, fact = 1;

// multiply the numbers from 1 to 9
while (i < 10) { // while loops check the condition first
    fact *= i;
    i++;
}
```

do/while Loops

do/while loops are like while loops but the condition is checked only after the statement is executed once. The general format is shown below. The condition is followed by a semi-colon.

```
do
    statement
while (condition);
```

The do/while loop in [C Code Example eC.16](#) queries a user to guess a number. The program checks the condition (if the user's number is equal to the correct number) only after the body of the do/while loop executes once. This construct is useful when, as in this case, something must be done (for example, the guess retrieved from the user) before the condition is checked.

C Code Example eC.16 do/while LOOP

```
// Query user to guess a number and check it against the correct number.
#define MAXGUESSES 3
#define CORRECTNUM 7
int guess, numGuesses = 0;
```



```
do {
    printf("Guess a number between 0 and 9. You have %d more guesses.\n",
          (MAXGUESSES-numGuesses));
    scanf("%d", &guess);    // read user input
    numGuesses++;
} while ( (numGuesses < MAXGUESSES) & (guess != CORRECTNUM) );
// do loop checks the condition after the first iteration

if (guess == CORRECTNUM)
    printf("You guessed the correct number!\n");
```

for Loops

for loops, like while and do/while loops, repeatedly execute a statement until a condition is not satisfied. However, for loops add support for a *loop variable*, which typically keeps track of the number of loop executions. The general format of the for loop is

```
for (initialization; condition; loop operation)
    statement
```

The initialization code executes only once, before the for loop begins. The condition is tested at the beginning of each iteration of the loop. If the condition is not TRUE, the loop exits. The loop operation executes at the end of each iteration. [C Code Example eC.17](#) shows the factorial of 9 computed using a for loop.

C Code Example eC.17 for LOOP

```
// Compute 9!
int i;    // loop variable
int fact = 1;

for (i=1; i<10; i++)
    fact *= i;
```

Whereas the while and do/while loops in [C Code Examples eC.15](#) and [eC.16](#) include code for incrementing and checking the loop variable *i* and *numGuesses*, respectively, the for loop incorporates those statements into its format. A for loop could be expressed equivalently, but less conveniently, as

```
initialization;
while (condition) {
    statement
    loop operation;
}
```

SUMMARY

- ▶ **Control-flow statements:** C provides control-flow statements for conditional statements and loops.
- ▶ **Conditional statements:** Conditional statements execute a statement when a condition is TRUE. C includes the following conditional statements: if, if/else, and switch/case.
- ▶ **Loops:** Loops repeatedly execute a statement until a condition is FALSE. C provides while, do/while, and for loops.

C.8 MORE DATA TYPES

Beyond various sizes of integers and floating-point numbers, C includes other special data types including pointers, arrays, strings, and structures. These data types are introduced in this section along with dynamic memory allocation.

C.8.1 Pointers

A pointer is the address of a variable. C Code Example eC.18 shows how to use pointers. salary1 and salary2 are variables that can contain integers, and ptr is a variable that can hold the address of an integer. The compiler will assign arbitrary locations in RAM for these variables depending on the runtime environment. For the sake of concreteness, suppose this program is compiled on a 32-bit system with salary1 at addresses 0x70-73, salary2 at addresses 0x74-77, and ptr at 0x78-7B. Figure eC.3 shows memory and its contents after the program is executed.

In a variable declaration, a star (*) before a variable name indicates that the variable is a pointer to the declared type. In using a pointer variable, the * operator dereferences a pointer, returning the value stored at the

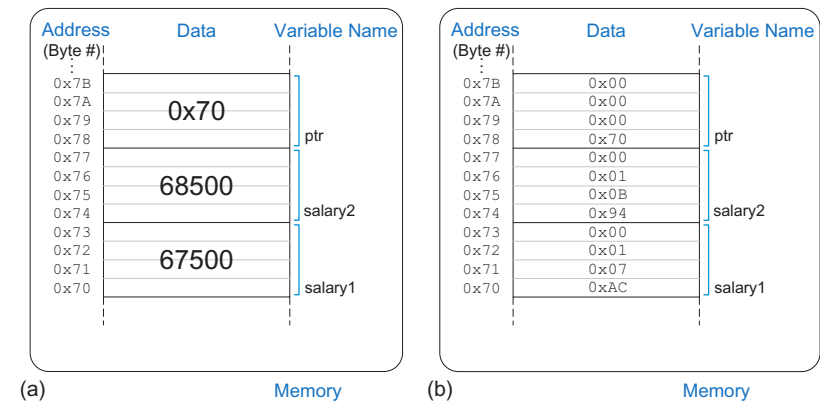


Figure eC.3 Contents of memory after C Code Example eC.18 executes shown (a) by value and (b) by byte using little-endian memory

Dereferencing a pointer to a non-existent memory location or an address outside of the range accessible by the program will usually cause a program to crash. The crash is often called a *segmentation fault*.

indicated memory address contained in the pointer. The & operator is pronounced “address of,” and it produces the memory address of the variable being referenced.

Pointers are particularly useful when a function needs to modify a variable, instead of just returning a value. Because functions can’t modify their

C Code Example eC.18 POINTERS

```
// Example pointer manipulations
int salary1, salary2; // 32-bit numbers
int *ptr;             // a pointer specifying the address of an int variable

salary1 = 67500;      // salary1 = $67,500 = 0x000107AC
ptr = &salary1;       // ptr = 0x0070, the address of salary1
salary2 = *ptr + 1000; /* dereference ptr to give the contents of address 70 = $67,500,
                       then add $1,000 and set salary2 to $68,500 */
```

inputs directly, a function can make the input a pointer to the variable. This is called passing an input variable *by reference* instead of *by value*, as shown in prior examples. C Code Example eC.19 gives an example of passing x by reference so that quadruple can modify the variable directly.

C Code Example eC.19 PASSING AN INPUT VARIABLE BY REFERENCE

```
// Quadruple the value pointed to by a
#include <stdio.h>

void quadruple(int *a)
{
    *a = *a * 4;
}

int main(void)
{
    int x = 5;

    printf("x before: %d\n", x);
    quadruple(&x);
    printf("x after: %d\n", x);
    return 0;
}
```

Console Output

```
x before: 5
x after: 20
```

A pointer to address 0 is called a *null pointer* and indicates that the pointer is not actually pointing to meaningful data. It is written as NULL in a program.

C.8.2 Arrays

An array is a group of similar variables stored in consecutive addresses in memory. The elements are numbered from 0 to $N-1$, where N is the length of the array. C Code Example eC.20 declares an array variable called `scores` that holds the final exam scores for three students. Memory space is reserved for three `longs`, that is, $3 \times 4 = 12$ bytes. Suppose the `scores` array starts at address `0x40`. The address of the 1st element (i.e., `scores[0]`) is `0x40`, the 2nd element is `0x44`, and the 3rd element is `0x48`, as shown in Figure eC.4. In C, the array variable, in this case `scores`, is a pointer to the 1st element. It is the programmer’s responsibility not to access elements beyond the end of the array. C has no internal bounds checking, so a program that writes beyond the end of an array will compile fine but may stomp on other parts of memory when it runs.

C Code Example eC.20 ARRAY DECLARATION

```
long scores[3];    // array of three 4-byte numbers
```

The elements of an array can be initialized either at declaration using curly braces `{ }`, as shown in C Code Example eC.21, or individually in the body of the code, as shown in C Code Example eC.22. Each element of an array is accessed using brackets `[]`. The contents of memory containing the array are shown in Figure eC.4. Array initialization using curly braces `{ }` can only be performed at declaration, and not afterward. `for` loops are commonly used to assign and read array data, as shown in C Code Example eC.23.

C Code Example eC.21 ARRAY INITIALIZATION AT DECLARATION USING { }

```
long scores[3]={93, 81, 97}; // scores[0]=93; scores[1]=81; scores[2]=97;
```

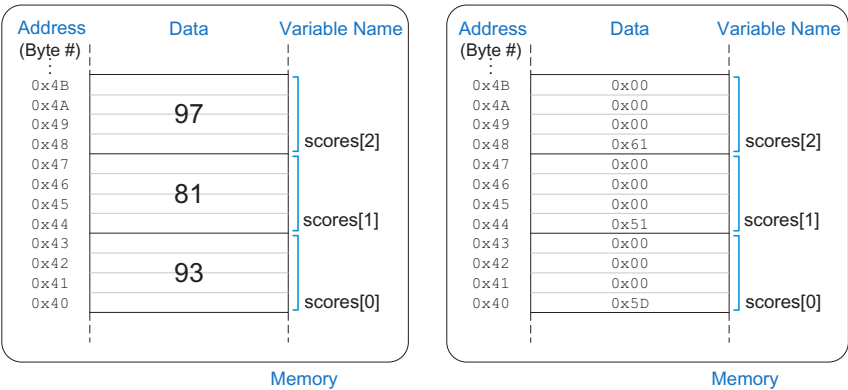


Figure eC.4 `scores` array stored in memory

C Code Example eC.22 ARRAY INITIALIZATION USING ASSIGNMENT

```
long scores[3];

scores[0] = 93;
scores[1] = 81;
scores[2] = 97;
```

C Code Example eC.23 ARRAY INITIALIZATION USING A for LOOP

```
// User enters 3 student scores into an array
long scores[3];
int i, entered;

printf("Please enter the student's 3 scores.\n");
for (i=0; i<3; i++) {
    printf("Enter a score and press enter.\n");
    scanf("%d", &entered);
    scores[i] = entered;
}
printf("Scores: %d %d %d\n", scores[0], scores[1], scores[2]);
```

When an array is declared, the length must be constant so that the compiler can allocate the proper amount of memory. However, when the array is passed to a function as an input argument, the length need not be defined because the function only needs to know the address of the beginning of the array. [C Code Example eC.24](#) shows how an array is passed to a function. The input argument `arr` is simply the address of the 1st element of an array. Often the number of elements in an array is also passed as an input argument. In a function, an input argument of type `int[]` indicates that it is an array of integers. Arrays of any type may be passed to a function.

C Code Example eC.24 PASSING AN ARRAY AS AN INPUT ARGUMENT

```
// Initialize a 5-element array, compute the mean, and print the result.
#include <stdio.h>

// Returns the mean value of an array (arr) of length len
float getMean(int arr[], int len) {
    int i;
    float mean, total = 0;

    for (i=0; i < len; i++)
        total += arr[i];

    mean = total / len;
    return mean;
}
```

```
int main(void) {
    int data[4] = {78, 14, 99, 27};
    float avg;

    avg = getMean(data, 4);

    printf("The average value is: %f.\n", avg);
}
```

Console Output

The average value is: 54.500000.

An array argument is equivalent to a pointer to the beginning of the array. Thus, `getMean` could also have been declared as

```
float getMean(int *arr, int len);
```

Although functionally equivalent, `datatype[]` is the preferred method for passing arrays as input arguments because it more clearly indicates that the argument is an array.

A function is limited to a single output, i.e., return variable. However, by receiving an array as an input argument, a function can essentially output more than a single value by changing the array itself. [C Code Example eC.25](#) sorts an array from lowest to highest and leaves the result in the same array. The three function prototypes below are equivalent. The length of an array in a function declaration (i.e., `int vals[100]`) is ignored.

```
void sort(int *vals, int len);
void sort(int vals[], int len);
void sort(int vals[100], int len);
```

C Code Example eC.25 PASSING AN ARRAY AND ITS LENGTH AS INPUTS

```
// Sort the elements of the array vals of length len from lowest to highest
void sort(int vals[], int len)
{
    int i, j, temp;

    for (i=0; i<len; i++) {
        for (j=i+1; j<len; j++) {
            if (vals[i] > vals[j]) {
                temp = vals[i];
                vals[i] = vals[j];
                vals[j] = temp;
            }
        }
    }
}
```

Arrays may have multiple dimensions. [C Code Example eC.26](#) uses a two-dimensional array to store the grades across eight problem sets for ten students. Recall that initialization of array values using `{}` is only allowed at declaration.

C Code Example eC.26 TWO-DIMENSIONAL ARRAY INITIALIZATION

```
// Initialize 2-D array at declaration
int grades[10][8] = { {100, 107, 99, 101, 100, 104, 109, 117},
                     {103, 101, 94, 101, 102, 106, 105, 110},
                     {101, 102, 92, 101, 100, 107, 109, 110},
                     {114, 106, 95, 101, 100, 102, 102, 100},
                     {98, 105, 97, 101, 103, 104, 109, 109},
                     {105, 103, 99, 101, 105, 104, 101, 105},
                     {103, 101, 100, 101, 108, 105, 109, 100},
                     {100, 102, 102, 101, 102, 101, 105, 102},
                     {102, 106, 110, 101, 100, 102, 120, 103},
                     {99, 107, 98, 101, 109, 104, 110, 108} };
```

[C Code Example eC.27](#) shows some functions that operate on the 2-D `grades` array from [C Code Example eC.26](#). Multi-dimensional arrays used as input arguments to a function must define all but the first dimension. Thus, the following two function prototypes are acceptable:

```
void print2dArray(int arr[10][8]);
void print2dArray(int arr[][8]);
```

C Code Example eC.27 OPERATING ON MULTI-DIMENSIONAL ARRAYS

```
#include <stdio.h>

// Print the contents of a 10x8 array
void print2dArray(int arr[10][8])
{
    int i, j;

    for (i=0; i<10; i++) {          // for each of the 10 students
        printf("Row %d\n", i);
        for (j=0; j<8; j++) {
            printf("%d ", arr[i][j]); // print scores for all 8 problem sets
        }
        printf("\n");
    }
}

// Calculate the mean score of a 10x8 array
float getMean(int arr[10][8])
{
    int i, j;
    float mean, total = 0;

    // get the mean value across a 2D array
    for (i=0; i<10; i++) {
```

```

    for (j=0; j<8; j++) {
        total += arr[i][j];    // sum array values
    }
}
mean = total/(10*8);
printf("Mean is: %f\n", mean);
return mean;
}

```

Note that because an array is represented by a pointer to the initial element, C cannot copy or compare arrays using the `=` or `==` operators. Instead, you must use a loop to copy or compare each element one at a time.

C.8.3 Characters

A character (`char`) is an 8-bit variable. It can be viewed either as a two's complement number between -128 and 127 or as an ASCII code for a letter, digit, or symbol. ASCII characters can be specified as a numeric value (in decimal, hexadecimal, etc.) or as a printable character enclosed in single quotes. For example, the letter A has the ASCII code `0x41`, `B=0x42`, etc. Thus `'A' + 3` is `0x44`, or `'D'`. Table 6.5 lists the ASCII character encodings, and Table eC.4 lists characters used to indicate formatting or special characters. Formatting codes include carriage return (`\r`), newline (`\n`), horizontal tab (`\t`), and the end of a string (`\0`). `\r` is shown for completeness but is rarely used in C programs. `\r` returns the carriage (location of typing) to the beginning (left) of the line, but any text that was there is overwritten. `\n`, instead, moves the location of typing to the beginning of a new line.² The `NULL` character (`'\0'`) indicates the end of a text string and is discussed next in Section C.8.4.

Table eC.4 Special characters

Special Character	Hexadecimal Encoding	Description
<code>\r</code>	<code>0x0D</code>	carriage return
<code>\n</code>	<code>0x0A</code>	new line
<code>\t</code>	<code>0x09</code>	tab
<code>\0</code>	<code>0x00</code>	terminates a string
<code>\\</code>	<code>0x5C</code>	backslash
<code>\"</code>	<code>0x22</code>	double quote
<code>\'</code>	<code>0x27</code>	single quote
<code>\a</code>	<code>0x07</code>	bell

² Windows text files use `\r\n` to represent end-of-line while UNIX-based systems use `\n`, which can cause nasty bugs when moving text files between systems.

The term “carriage return” originates from typewriters that required the carriage, the contraption that holds the paper, to move to the right in order to allow typing to begin at the left side of the page.

A carriage return lever, shown on the left in the figure below, is pressed so that the carriage would both move to the right and advance the paper by one line, called a line feed.



A Remington electric typewriter used by Winston Churchill. (<http://cwr.iwm.org.uk/server/show/conMediaFile.71979>)

C strings are called *null terminated* or *zero terminated* because the length is determined by looking for a zero at the end. In contrast, languages such as Pascal use the first byte to specify the string length, up to a maximum of 255 characters. This byte is called the *prefix byte* and such strings are called *P-strings*. An advantage of null-terminated strings is that the length can be arbitrarily great. An advantage of P-strings is that the length can be determined immediately without having to inspect all of the characters of the string.

C.8.4 Strings

A string is an array of characters used to store a piece of text of bounded but variable length. Each character is a byte representing the ASCII code for that letter, number, or symbol. The size of the array determines the maximum length of the string, but the actual length of the string could be shorter. In C, the length of the string is determined by looking for the null terminator (ASCII value 0x00) at the end of the string.

C Code Example eC.28 shows the declaration of a 10-element character array called `greeting` that holds the string "Hello!". For concreteness, suppose `greeting` starts at memory address 0x50. Figure eC.5 shows the contents of memory from 0x50 to 0x59 holding the string "Hello!". Note that the string only uses the first seven elements of the array, even though ten elements are allocated in memory.

C Code Example eC.28 STRING DECLARATION

```
char greeting[10] = "Hello!";
```

C Code Example eC.29 shows an alternate declaration of the string `greeting`. The pointer `greeting` holds the address of the 1st element of a 7-element array comprised of each of the characters in "Hello!" followed by the null terminator. The code also demonstrates how to print strings by using the `%s` format code.

C Code Example eC.29 ALTERNATE STRING DECLARATION

```
char *greeting = "Hello!";
printf("greeting: %s", greeting);
```

Console Output

```
greeting: Hello!
```

Unlike primitive variables, a string cannot be set equal to another string using the equals operator, `=`. Each element of the character array must be individually copied from the source string to the target string. This is true for any array. C Code Example eC.30 copies one string, `src`, to another, `dst`. The sizes of the arrays are not needed, because the end of the `src` string is indicated by the null terminator. However, `dst` must be large enough so that you don't stomp on other data. `strcpy` and other string manipulation functions are available in C's built-in libraries (see Section C.9.4).

C Code Example eC.30 COPYING STRINGS

```
// Copy the source string, src, to the destination string, dst
void strcpy(char *dst, char *src)
{
```

```
int i = 0;

do {
    dst[i] = src[i];    // copy characters one byte at a time
} while (src[i++] != 0; // until the null terminator is found
}
```

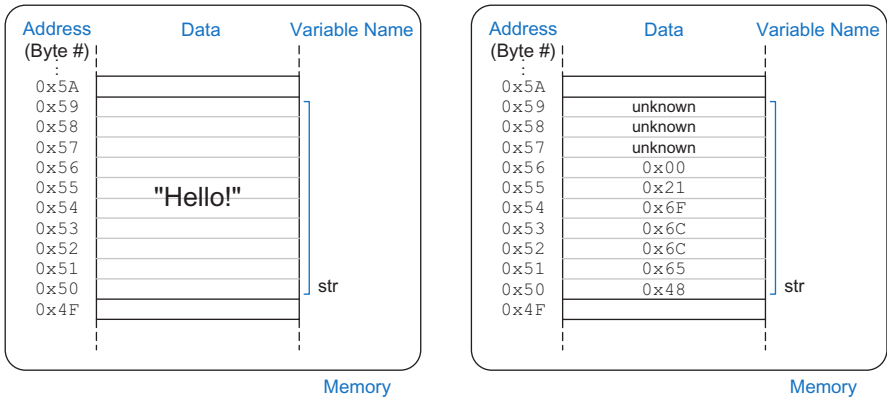


Figure eC.5 The string “Hello!” stored in memory

C.8.5 Structures

In C, structures are used to store a collection of data of various types. The general format of a structure declaration is

```
struct name {
    type1 element1;
    type2 element2;
    ...
};
```

where `struct` is a keyword indicating that it is a structure, `name` is the structure tag name, and `element1` and `element2` are members of the structure. A structure may have any number of members. C Code Example eC.31 shows how to use a structure to store contact information. The program then declares a variable `c1` of type `struct contact`.

C Code Example eC.31 STRUCTURE DECLARATION

```
struct contact {
    char name[30];
    int phone;
    float height; // in meters
};

struct contact c1;
```

```
strcpy(c1.name, "Ben Bitdiddle");  
c1.phone = 7226993;  
c1.height = 1.82;
```

Just like built-in C types, you can create arrays of structures and pointers to structures. [C Code Example eC.32](#) creates an array of contacts.

C Code Example eC.32 ARRAY OF STRUCTURES

```
struct contact classlist[200];  
classlist[0].phone = 9642025;
```

It is common to use pointers to structures. C provides the *member access operator* `->` to dereference a pointer to a structure and access a member of the structure. [C Code Example eC.33](#) shows an example of declaring a pointer to a struct contact, assigning it to point to the 42nd element of `classlist` from [C Code Example eC.32](#), and using the member access operator to set a value in that element.

C Code Example eC.33 ACCESSING STRUCTURE MEMBERS USING POINTERS AND `->`

```
struct contact *cptr;  
cptr = &classlist[42];  
cptr->height = 1.9; // equivalent to: (*cptr).height = 1.9;
```

Structures can be passed as function inputs or outputs by value or by reference. Passing by value requires the compiler to copy the entire structure into memory for the function to access. This can require a large amount of memory and time for a big structure. Passing by reference involves passing a pointer to the structure, which is more efficient. The function can also modify the structure being pointed to rather than having to return another structure. [C Code Example eC.34](#) shows two versions of the `stretch` function that makes a contact 2 cm taller. `stretchByReference` avoids copying the large structure twice.

C Code Example eC.34 PASSING STRUCTURES BY VALUE OR BY NAME

```
struct contact stretchByValue(struct contact c)  
{  
    c.height += 0.02;  
    return c;  
}
```

```

void stretchByReference(struct contact *cptr)
{
    cptr->height += 0.02;
}

int main(void)
{
    struct contact George;

    George.height = 1.4; // poor fellow has been stooped over
    George = stretchByValue(George); // stretch for the stars
    stretchByReference(&George);    // and stretch some more
}

```

C.8.6 typedef

C also allows you to define your own names for data types using the `typedef` statement. For example, writing `struct contact` becomes tedious when it is often used, so we can define a new type named `contact` and use it as shown in [C Code Example eC.35](#).

C Code Example eC.35 CREATING A CUSTOM TYPE USING `typedef`

```

typedef struct contact {
    char name[30];
    int phone;
    float height; // in meters
} contact;      // defines contact as shorthand for "struct contact"

contact c1;     // now we can declare the variable as type contact

```

`typedef` can be used to create a new type occupying the same amount of memory as a primitive type. [C Code Example eC.36](#) defines `byte` and `bool` as 8-bit types. The `byte` type may make it clearer that the purpose of `pos` is to be an 8-bit number rather than an ASCII character. The `bool` type indicates that the 8-bit number is representing `TRUE` or `FALSE`. These types make a program easier to read than if one simply used `char` everywhere.

C Code Example eC.36 `typedef` `byte` AND `bool`

```

typedef unsigned char byte;
typedef char bool;
#define TRUE 1
#define FALSE 0

byte pos = 0x45;
bool loveC = TRUE;

```

C Code Example eC.37 illustrates defining a 3-element vector and a 3×3 matrix type using arrays.

C Code Example eC.37 typedef vector AND matrix

```
typedef double vector[3];
typedef double matrix[3][3];

vector a = {4.5, 2.3, 7.0};
matrix b = {{3.3, 4.7, 9.2}, {2.5, 4, 9}, {3.1, 99.2, 88}};
```

C.8.7 Dynamic Memory Allocation*

In all the examples thus far, variables have been declared *statically*; that is, their size is known at compile time. This can be problematic for arrays and strings of variable size because the array must be declared large enough to accommodate the largest size the program will ever see. An alternative is to *dynamically* allocate memory at run time when the actual size is known.

The `malloc` function from `stdlib.h` allocates a block of memory of a specified size and returns a pointer to it. If not enough memory is available, it returns a NULL pointer instead. For example, the following code allocates 10 shorts ($10 \times 2 = 20$ bytes). The `sizeof` operator returns the size of a type or variable in bytes.

```
// dynamically allocate 20 bytes of memory
short *data = malloc(10*sizeof(short));
```

C Code Example eC.38 illustrates dynamic allocation and de-allocation. The program accepts a variable number of inputs, stores them in a dynamically allocated array, and computes their average. The amount of memory necessary depends on the number of elements in the array and the size of each element. For example, if an `int` is a 4-byte variable and 10 elements are needed, 40 bytes are dynamically allocated. The `free` function de-allocates the memory so that it could later be used for other purposes. Failing to de-allocate dynamically allocated data is called a *memory leak* and should be avoided.

C Code Example eC.38 DYNAMIC MEMORY ALLOCATION AND DE-ALLOCATION

```
// Dynamically allocate and de-allocate an array using malloc and free
#include <stdlib.h>

// Insert getMean function from C Code Example eC.24.

int main(void) {
    int len, i;
```

```

int *nums;

printf("How many numbers would you like to enter?  ");
scanf("%d", &len);
nums = malloc(len*sizeof(int));
if (nums == NULL) printf("ERROR: out of memory.\n");
else {
    for (i=0; i<len; i++) {
        printf("Enter number:  ");
        scanf("%d", &nums[i]);
    }
    printf("The average is %f\n", getMean(nums, len));
}
free(nums);
}

```

C.8.8 Linked Lists*

A *linked list* is a common data structure used to store a variable number of elements. Each element in the list is a structure containing one or more data fields and a link to the next element. The first element in the list is called the *head*. Linked lists illustrate many of the concepts of structures, pointers, and dynamic memory allocation.

C Code Example eC.39 describes a linked list for storing computer user accounts to accommodate a variable number of users. Each user has a user name, a password, a unique user identification number (UID), and a field indicating whether they have administrator privileges. Each element of the list is of type `userL`, containing all of this user information along with a link to the next element in the list. A pointer to the head of the list is stored in a global variable called `users`, and is initially set to `NULL` to indicate that there are no users.

The program defines functions to insert, delete, and find a user and to count the number of users. The `insertUser` function allocates space for a new list element and adds it to the head of the list. The `deleteUser` function scans through the list until the specified UID is found and then removes that element, adjusting the link from the previous element to skip the deleted element and freeing the memory occupied by the deleted element. The `findUser` function scans through the list until the specified UID is found and returns a pointer to that element, or `NULL` if the UID is not found. The `numUsers` function counts the number of elements in the list.

C Code Example eC.39 LINKED LIST

```

#include <stdlib.h>
#include <string.h>

typedef struct userL {

```

```
char uname[80];    // user name
char passwd[80];   // password
int uid;           // user identification number
int admin;         // 1 indicates administrator privileges
struct userL *next;
} userL;

userL *users = NULL;

void insertUser(char *uname, char *passwd, int uid, int admin) {
    userL *newUser;

    newUser = malloc(sizeof(userL)); // create space for new user
    strcpy(newUser->uname, uname);    // copy values into user fields
    strcpy(newUser->passwd, passwd);
    newUser->uid = uid;
    newUser->admin = admin;
    newUser->next = users;             // insert at start of linked list
    users = newUser;
}

void deleteUser(int uid) { // delete first user with given uid
    userL *cur = users;
    userL *prev = NULL;

    while (cur != NULL) {
        if (cur->uid == uid) { // found the user to delete
            if (prev == NULL) users = cur->next;
            else prev->next = cur->next;
            free(cur);
            return; // done
        }
        prev = cur; // otherwise, keep scanning through list
        cur = cur->next;
    }
}

userL *findUser(int uid) {
    userL *cur = users;

    while (cur != NULL) {
        if (cur->uid == uid) return cur;
        else cur = cur->next;
    }
    return NULL;
}

int numUsers(void) {
    userL *cur = users;
    int count = 0;

    while (cur != NULL) {
        count++;
        cur = cur->next;
    }
    return count;
}
```

SUMMARY

- ▶ **Pointers:** A pointer holds the address of a variable.
- ▶ **Arrays:** An array is a list of similar elements declared using square brackets [].
- ▶ **Characters:** `char` types can hold small integers or special codes for representing text or symbols.
- ▶ **Strings:** A string is an array of characters ending with the null terminator `0x00`.
- ▶ **Structures:** A structure stores a collection of related variables.
- ▶ **Dynamic memory allocation:** `malloc` is a built-in functions for allocating memory as the program runs. `free` de-allocates the memory after use.
- ▶ **Linked Lists:** A linked list is a common data structure for storing a variable number of elements.

C.9 STANDARD LIBRARIES

Programmers commonly use a variety of standard functions, such as printing and trigonometric operations. To save each programmer from having to write these functions from scratch, C provides *libraries* of frequently used functions. Each library has a header file and an associated object file, which is a partially compiled C file. The header file holds variable declarations, defined types, and function prototypes. The object file contains the functions themselves and is linked at compile-time to create the executable. Because the library function calls are already compiled into an object file, compile time is reduced. [Table eC.5](#) lists some of the most frequently used C libraries, and each is described briefly below.

C.9.1 `stdio`

The standard input/output library `stdio.h` contains commands for printing to a console, reading keyboard input, and reading and writing files. To use these functions, the library must be included at the top of the C file:

```
#include <stdio.h>
```

`printf`

The *print formatted* statement `printf` displays text to the console. Its required input argument is a string enclosed in quotes `" "`. The string contains text and optional commands to print variables. Variables to be printed are listed after the string and are printed using format codes shown in [Table eC.6](#). [C Code Example eC.40](#) gives a simple example of `printf`.

Table eC.5 Frequently used C libraries

C Library Header File	Description
stdio.h	Standard input/output library. Includes functions for printing or reading to/from the screen or a file (printf, fprintf and scanf, fscanf) and to open and close files (fopen and fclose).
stdlib.h	Standard library. Includes functions for random number generation (rand and srand), for dynamically allocating or freeing memory (malloc and free), terminating the program early (exit), and for conversion between strings and numbers (atoi, atol, and atof).
math.h	Math library. Includes standard math functions such as sin, cos, asin, acos, sqrt, log, log10, exp, floor, and ceil.
string.h	String library. Includes functions to compare, copy, concatenate, and determine the length of strings.

Table eC.6 printf format codes for printing variables

Code	Format
%d	Decimal
%u	Unsigned decimal
%x	Hexadecimal
%o	Octal
%f	Floating point number (float or double)
%e	Floating point number (float or double) in scientific notation (e.g., 1.56e7)
%c	Character (char)
%s	String (null-terminated array of characters)

C Code Example eC.40 PRINTING TO THE CONSOLE USING printf

```
// Simple print function
#include <stdio.h>

int num = 42;
```

```
int main(void) {  
    printf("The answer is %d.\n", num);  
}
```

Console Output:

The answer is 42.

Floating point formats (`floats` and `doubles`) default to printing six digits after the decimal point. To change the precision, replace `%f` with `%w.df`, where `w` is the minimum width of the number, and `d` is the number of decimal places to print. Note that the decimal point is included in the width count. In [C Code Example eC.41](#), `pi` is printed with a total of four characters, two of which are after the decimal point: 3.14. `e` is printed with a total of eight characters, three of which are after the decimal point. Because it only has one digit before the decimal point, it is padded with three leading spaces to reach the requested width. `c` should be printed with five characters, three of which are after the decimal point. But it is too wide to fit, so the requested width is overridden while retaining the three digits after the decimal point.

C Code Example eC.41 FLOATING POINT NUMBER FORMATS FOR PRINTING

```
// Print floating point numbers with different formats  
float pi = 3.14159, e = 2.7182, c = 2.998e8;  
printf("pi = %4.2f\ne = %8.3f\nc = %5.3f\n", pi, e, c);
```

Console Output:

```
pi = 3.14  
e =    2.718  
c = 299800000.000
```

Because `%` and `\` are used in print formatting, to print these characters themselves, you must use the special character sequences shown in [C Code Example eC.42](#).

C Code Example eC.42 PRINTING `%` AND `\` USING `printf`

```
// How to print % and \ to the console  
printf("Here are some special characters: %% \\ \n");
```

Console Output:

Here are some special characters: % \

scanf

The `scanf` function reads text typed on the keyboard. It uses format codes in the same way as `printf`. [C Code Example eC.43](#) shows how to use `scanf`. When the `scanf` function is encountered, the program waits until the user types a value before continuing execution. The arguments to `scanf` are a string indicating one or more format codes and pointers to the variables where the results should be stored.

C Code Example eC.43 READING USER INPUT FROM THE KEYBOARD WITH `scanf`

```
// Read variables from the command line
#include <stdio.h>

int main(void)
{
    int a;
    char str[80];
    float f;

    printf("Enter an integer.\n");
    scanf("%d", &a);
    printf("Enter a floating point number.\n");
    scanf("%f", &f);
    printf("Enter a string.\n");
    scanf("%s", str); // note no & needed: str is a pointer
}
```

File Manipulation

Many programs need to read and write files, either to manipulate data already stored in a file or to log large amounts of information. In C, the file must first be opened with the `fopen` function. It can then be read or written with `fscanf` or `fprintf` in a way analogous to reading and writing to the console. Finally, it should be closed with the `fclose` command.

The `fopen` function takes as arguments the file name and a *print mode*. It returns a *file pointer* of type `FILE*`. If `fopen` is unable to open the file, it returns `NULL`. This might happen when one tries to read a nonexistent file or write a file that is already opened by another program. The modes are:

"w": Write to a file. If the file exists, it is overwritten.

"r": Read from a file.

"a": Append to the end of an existing file. If the file doesn't exist, it is created.

[C Code Example eC.44](#) shows how to open, print to, and close a file. It is good practice to always check if the file was opened successfully and to provide an error message if it was not. The `exit` function will be discussed in Section C.9.2. The `fprintf` function is like `printf` but it also takes the file pointer as an input argument to know which file to write. `fclose` closes the file, ensuring that all of the information is actually written to disk and freeing up file system resources.

C Code Example eC.44 PRINTING TO A FILE USING `fprintf`

```
// Write "Testing file write." to result.txt
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fptr;

    if ((fptr = fopen("result.txt", "w")) == NULL) {
        printf("Unable to open result.txt for writing.\n");
        exit(1); // exit the program indicating unsuccessful execution
    }
    fprintf(fptr, "Testing file write.\n");
    fclose(fptr);
}
```

It is idiomatic to open a file and check if the file pointer is `NULL` in a single line of code, as shown in [C Code Example eC.44](#). However, you could just as easily separate the functionality into two lines:

```
fptr = fopen("result.txt", "w");
if (fptr == NULL)
    ...
```

[C Code Example eC.45](#) illustrates reading numbers from a file named `data.txt` using `fscanf`. The file must first be opened for reading. The program then uses the `feof` function to check if it has reached the end of the file. As long as the program is not at the end, it reads the next number and prints it to the screen. Again, the program closes the file at the end to free up resources.

C Code Example eC.45 READING INPUT FROM A FILE USING `fscanf`

```
#include <stdio.h>

int main(void)
{
    FILE *fptr;
    int data;

    // read in data from input file
    if ((fptr = fopen("data.txt", "r")) == NULL) {
        printf("Unable to read data.txt\n");
        exit(1);
    }
}
```

```
while (!feof(fptr)) { // check that the end of the file hasn't been reached
    fscanf(fptr, "%d", &data);
    printf("Read data: %d\n", data);
}

fclose(fptr);
}
```

data.txt

25 32 14 89

Console Output:

Read data: 25
Read data: 32
Read data: 14
Read data: 89

Other Handy stdio Functions

The `sprintf` function prints characters into a string, and `sscanf` reads variables from a string. The `fgetc` function reads a single character from a file, while `fgets` reads a complete line into a string.

`fscanf` is rather limited in its ability to read and parse complex files, so it is often easier to `fgets` one line at a time and then digest that line using `sscanf` or with a loop that inspects characters one at a time using `fgetc`.

C.9.2 `stdlib`

The standard library `stdlib.h` provides general purpose functions including random number generation (`rand` and `srand`), dynamic memory allocation (`malloc` and `free`, already discussed in Section C.8.7), exiting the program early (`exit`), and number format conversions. To use these functions, add the following line at the top of the C file.

```
#include <stdlib.h>
```

`rand` and `srand`

`rand` returns a pseudo-random integer. Pseudo-random numbers have the statistics of random numbers but follow a deterministic pattern starting with an initial value called the *seed*. To convert the number to a particular range, use the modulo operator (%) as shown in [C Code Example eC.46](#) for a range of 0 to 9. The values `x` and `y` will be random but they will be the same each time this program runs. Sample console output is given below the code.

C Code Example eC.46 RANDOM NUMBER GENERATION USING `rand`

```
#include <stdlib.h>
int x, y;

x = rand();      // x = a random integer
y = rand() % 10; // y = a random number from 0 to 9
printf("x = %d, y = %d\n", x, y);
```

Console Output:

```
x = 1481765933, y = 3
```

A programmer creates a different sequence of random numbers each time a program runs by changing the seed. This is done by calling the `srand` function, which takes the seed as its input argument. As shown in [C Code Example eC.47](#), the seed itself must be random, so a typical C program assigns it by calling the `time` function, that returns the current time in seconds.

C Code Example eC.47 SEEDING THE RANDOM NUMBER GENERATOR USING `srand`

```
// Produce a different random number each run
#include <stdlib.h>
#include <time.h> // needed to call time()

int main(void)
{
    int x;

    srand(time(NULL)); // seed the random number generator
    x = rand() % 10;    // random number from 0 to 9
    printf("x = %d\n", x);
}
```

For historical reasons, the `time` function usually returns the current time in seconds relative to January 1, 1970 00:00 UTC. UTC stands for Coordinated Universal Time, which is the same as Greenwich Mean Time (GMT). This date is just after the UNIX operating system was created by a group at Bell Labs, including Dennis Ritchie and Brian Kernighan, in 1969. Similar to New Year's Eve parties, some UNIX enthusiasts hold parties to celebrate significant values returned by `time`. For example, on February 1, 2009 at 23:31:30 UTC, `time` returned 1,234,567,890. In the year 2038, 32-bit UNIX clocks will overflow into the year 1901.

`exit`

The `exit` function terminates a program early. It takes a single argument that is returned to the operating system to indicate the reason for termination. 0 indicates normal completion, while nonzero conveys an error condition.

Format Conversion: `atoi`, `atol`, `atof`

The standard library provides functions for converting ASCII strings to integers, long integers, or doubles using `atoi`, `atol`, and `atof`, respectively, as shown in [C Code Example eC.48](#). This is particularly useful

when reading in mixed data (a mix of strings and numbers) from a file or when processing numeric command line arguments, as described in [Section C.10.3](#).

C Code Example eC.48 FORMAT CONVERSION

```
// Convert ASCII strings to ints, longs, and floats
#include <stdlib.h>

int main(void)
{
    int x;
    long int y;
    double z;

    x = atoi("42");
    y = atol("833");
    z = atof("3.822");

    printf("x = %d\ty = %d\tz = %f\n", x, y, z);
}
```

Console Output:

```
x = 42   y = 833   z = 3.822000
```

C.9.3 math

The math library `math.h` provides commonly used math functions such as trigonometry functions, square root, and logs. [C Code Example eC.49](#) shows how to use some of these functions. To use math functions, place the following line in the C file:

```
#include <math.h>
```

C Code Example eC.49 MATH FUNCTIONS

```
// Example math functions
#include <stdio.h>
#include <math.h>

int main(void) {
    float a, b, c, d, e, f, g, h;

    a = cos(0);           // 1, note: the input argument is in radians
    b = 2 * acos(0);       // pi (acos means arc cosine)
    c = sqrt(144);         // 12
    d = exp(2);            // e^2 = 7.389056,
    e = log(7.389056);     // 2 (natural logarithm, base e)
```

```
f = log10(1000);    // 3 (log base 10)
g = floor(178.567); // 178, rounds to next lowest whole number
h = pow(2, 10);     // computes 2 raised to the 10th power

printf("a = %.0f, b = %f, c = %.0f, d = %.0f, e = %.2f, f = %.0f, g = %.2f, h = %.2f\n",
       a, b, c, d, e, f, g, h);
}
```

Console Output:

```
a = 1, b = 3.141593, c = 12, d = 7, e = 2.00, f = 3, g = 178.00, h = 1024.00
```

C.9.4 string

The string library `string.h` provides commonly used string manipulation functions. Key functions include:

```
// copy src into dst and return dst
char *strcpy(char *dst, char *src);

// concatenate (append) src to the end of dst and return dst
char *strcat(char *dst, char *src);

// compare two strings. Return 0 if equal, nonzero otherwise
int strcmp(char *s1, char *s2);

// return the length of str, not including the null termination
int strlen(char *str);
```

C.10 COMPILER AND COMMAND LINE OPTIONS

Although we have introduced relatively simple C programs, real-world programs can consist of tens or even thousands of C files to enable modularity, readability, and multiple programmers. This section describes how to compile a program spread across multiple C files and shows how to use compiler options and command line arguments.

C.10.1 Compiling Multiple C Source Files

Multiple C files are compiled into a single executable by listing all file names on the compile line as shown below. Remember that the group of C files still must contain only one `main` function, conventionally placed in a file named `main.c`.

```
gcc main.c file2.c file3.c
```

C.10.2 Compiler Options

Compiler options allow the programmer to specify such things as output file names and formats, optimizations, etc. Compiler options are not

Table eC.7 Compiler options

Compiler Option	Description	Example
-o outfile	specifies output file name	gcc -o hello hello.c
-S	create assembly language output file (not executable)	gcc -S hello.c this produces hello.s
-v	verbose mode – prints the compiler results and processes as compilation completes	gcc -v hello.c
-Olevel	specify the optimization level (level is typically 0 through 3), producing faster and/or smaller code at the expense of longer compile time	gcc -O3 hello.c
--version	list the version of the compiler	gcc --version
--help	list all command line options	gcc --help
-Wall	print all warnings	gcc -Wall hello.c

standardized, but [Table eC.7](#) lists ones that are commonly used. Each option is typically preceded by a dash (-) on the command line, as shown. For example, the "-o" option allows the programmer to specify an output file name other than the a.out default. A plethora of options exist; they can be viewed by typing `gcc --help` at the command line.

C.10.3 Command Line Arguments

Like other functions, `main` can also take input variables. However, unlike other functions, these arguments are specified at the command line. As shown in [C Code Example eC.50](#), `argc` stands for *argument count*, and it denotes the number of arguments on the command line. `argv` stands for *argument vector*, and it is an array of the strings found on the command line. For example, suppose the program in [C Code Example eC.50](#) is compiled into an executable called `testargs`. When the lines below are typed at the command line, `argc` has the value 4, and the array `argv` has the values `{"/testargs", "arg1", "25", "lastarg!"}`. Note that the executable name is counted as the 1st argument. The console output after typing this command is shown below [C Code Example eC.50](#).

```
gcc -o testargs testargs.c
./testargs arg1 25 lastarg!
```

Programs that need numeric arguments may convert the string arguments to numbers using the functions in `stdlib.h`.

C Code Example eC.50 COMMAND LINE ARGUMENTS

```
// Print command line arguments
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i;

    for (i=0; i<argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
}
```

Console Output:

```
argv[0] = ./testargs
argv[1] = arg1
argv[2] = 25
argv[3] = lastarg!
```

C.11 COMMON MISTAKES

As with any programming language, you are almost certain to make errors while you write nontrivial C programs. Below are descriptions of some common mistakes made when programming in C. Some of these errors are particularly troubling because they compile but do not function as the programmer intended.

C Code Mistake eC.1 MISSING & IN scanf

Erroneous Code	Corrected Code:
<pre>int a; printf("Enter an integer:\t"); scanf("%d", a); // missing & before a</pre>	<pre>int a; printf("Enter an integer:\t"); scanf("%d", &a);</pre>

C Code Mistake eC.2 USING = INSTEAD OF == FOR COMPARISON

Erroneous Code	Corrected Code
<pre>if (x=1) // always evaluates as TRUE printf("Found!\n");</pre>	<pre>if (x==1) printf("Found!\n");</pre>

Debugging skills are acquired with practice, but here are a few hints.

- Fix bugs starting with the first error indicated by the compiler. Later errors may be downstream effects of this error. After fixing that bug, recompile and repeat until all bugs (at least those caught by the compiler!) are fixed.
- When the compiler says a valid line of code is in error, check the code above it (i.e., for missing semicolons or braces).
- When needed, split up complicated statements into multiple lines.
- Use `printf` to output intermediate results.
- When a result doesn't match expectations, start debugging the code at the *first* place it deviates from expectations.
- Look at all compiler warnings. While some warnings can be ignored, others may alert you to more subtle code errors that will compile but not run as intended.

C Code Mistake eC.3 INDEXING PAST LAST ELEMENT OF ARRAY

Erroneous Code

```
int array[10];
array[10] = 42;    // index is 0-9
```

Corrected Code

```
int array[10];
array[9] = 42;
```

C Code Mistake eC.4 USING = IN #define STATEMENT

Erroneous Code

```
// replaces NUM with "= 4" in code
#define NUM = 4
```

Corrected Code

```
#define NUM 4
```

C Code Mistake eC.5 USING AN UNINITIALIZED VARIABLE

Erroneous Code

```
int i;
if (i == 10) // i is uninitialized
    ...
```

Corrected Code

```
int i = 10;
if (i == 10)
    ...
```

C Code Mistake eC.6 NOT INCLUDING PATH OF USER-CREATED HEADER FILES

Erroneous Code

```
#include "myfile.h"
```

Corrected Code

```
#include "othercode\myfile.h"
```

C Code Mistake eC.7 USING LOGICAL OPERATORS (!, ||, &&) INSTEAD OF BITWISE (~, |, &)

Erroneous Code

```
char x=!5;    // logical NOT: x = 0
char y=5||2;  // logical OR:  y = 1
char z=5&&2;  // logical AND: z = 1
```

Corrected Code

```
char x=~5;    // bitwise NOT: x = 0b11111010
char y=5|2;   // bitwise OR:  y = 0b00000111
char z=5&2;   // logical AND: z = 0b00000000
```

C Code Mistake eC.8 FORGETTING break IN A switch/case STATEMENT**Erroneous Code**

```
char x = 'd';
...
switch (x) {
    case 'u': direction = 1;
    case 'd': direction = 2;
    case 'l': direction = 3;
    case 'r': direction = 4;
    default: direction = 0;
}
// direction = 0
```

Corrected Code

```
char x = 'd';
...
switch (x) {
    case 'u': direction = 1; break;
    case 'd': direction = 2; break;
    case 'l': direction = 3; break;
    case 'r': direction = 4; break;
    default: direction = 0;
}
// direction = 2
```

C Code Mistake eC.9 MISSING CURLY BRACES {}**Erroneous Code**

```
if (ptr == NULL) // missing curly braces
    printf("Unable to open file.\n");
exit(1);        // always executes
```

Corrected Code

```
if (ptr == NULL) {
    printf("Unable to open file.\n");
    exit(1);
}
```

C Code Mistake eC.10 USING A FUNCTION BEFORE IT IS DECLARED**Erroneous Code**

```
int main(void)
{
    test();
}

void test(void)
{
    ...
}
```

Corrected Code

```
void test(void)
{
    ...
}

int main(void)
{
    test();
}
```

C Code Mistake eC.11 DECLARING A LOCAL AND GLOBAL VARIABLE WITH THE SAME NAME**Erroneous Code**

```
int x = 5;    // global declaration of x
int test(void)
{
    int x = 3; // local declaration of x
    ...
}
```

Corrected Code

```
int x = 5;    // global declaration of x
int test(void)
{
    int y = 3; // local variable is y
    ...
}
```

C Code Mistake eC.12 TRYING TO INITIALIZE AN ARRAY WITH {} AFTER DECLARATION**Erroneous Code**

```
int scores[3];
scores = {93, 81, 97}; // won't compile
```

Corrected Code

```
int scores[3] = {93, 81, 97};
```

C Code Mistake eC.13 ASSIGNING ONE ARRAY TO ANOTHER USING =**Erroneous Code**

```
int scores[3] = {88, 79, 93};
int scores2[3];
scores2 = scores;
```

Corrected Code

```
int scores[3] = {88, 79, 93};
int scores2[3];
for (i=0; i<3; i++)
    scores2[i] = scores[i];
```

C Code Mistake eC.14 FORGETTING THE SEMI-COLON AFTER A do/while LOOP**Erroneous Code**

```
int num;
do {
    num = getNum();
} while (num < 100)    // missing ;
```

Corrected Code

```
int num;
do {
    num = getNum();
} while (num < 100);
```

C Code Mistake eC.15 USING COMMAS INSTEAD OF SEMICOLONS IN for LOOP**Erroneous Code**

```
for (i=0, i < 200, i++)
    ...
```

Corrected Code

```
for (i=0; i < 200; i++)
    ...
```

C Code Mistake eC.16 INTEGER DIVISION INSTEAD OF FLOATING POINT DIVISION**Erroneous Code**

```
// integer (truncated) division occurs when
// both arguments of division are integers
float x = 9 / 4; // x = 2.0
```

Corrected Code

```
// at least one of the arguments of
// division must be a float to
// perform floating point division
float x = 9.0 / 4; // x = 2.25
```

C Code Mistake eC.17 WRITING TO AN UNINITIALIZED POINTER

Erroneous Code

```
int *y = 77;
```

Corrected Code

```
int x, *y = &x;  
*y = 77;
```

C Code Mistake eC.18 GREAT EXPECTATIONS (OR LACK THEREOF)

A common beginner error is to write an entire program (usually with little modularity) and expect it to work perfectly the first time. For non-trivial programs, writing modular code and testing the individual functions along the way are essential. Debugging becomes exponentially harder and more time-consuming with complexity.

Another common error is lacking expectations. When this happens, the programmer can only verify that the code produces a result, not that the result is correct. Testing a program with known inputs and expected results is critical in verifying functionality.

This appendix has focused on using C on a system such as a personal computer. Chapter 9 (available as a web supplement) describes how C is used to program an ARM-based Raspberry Pi computer that can be used in embedded systems. Microcontrollers are usually programmed in C because the language provides nearly as much low-level control of the hardware as assembly language, yet is much more succinct and faster to write.

Index

Note: Page numbers in *italics* indicate figures, tables and text boxes; page numbers preceded by “e” refer to online material.

0, 8, 22. *See also* LOW, FALSE
 1, 8, 22. *See also* HIGH, TRUE
 32-bit datapath, 386
 32-bit instructions, 329
 64-bit architecture, 360
 74xx series logic, 533.e1–533.e5
 parts
 2:1 mux (74157), 533.e4
 3:8 decoder (74138), 533.e4
 4:1 mux (74153), 533.e4
 AND (7408), 533.e3
 AND3 (7411), 533.e3
 AND4 (7421), 533.e3
 counter (74161, 74163), 533.e4
 FLOP (7474), 533.e1, 533.e3
 NAND (7400), 533.e3
 NOR (7402), 533.e3
 NOT (7404), 533.e1
 OR (7432), 533.e3
 register (74377), 533.e4
 tristate buffer (74244), 533.e4
 XOR (7486), 533.e3
 #define, 541.e5–541.e6
 #include, 541.e6–541.e7. *See also*
 Standard libraries

A

ABI. *See* Application Binary Interface (ABI)
 Abstraction, 4–5
 digital. *See* Digital abstraction
 Accumulator, 367

Acorn Computer Group, 296, 472
 Acorn RISC Machine, 350
 Active low, 74–75
 A/D conversion, 531.e31–531.e32
 Ad hoc testing, 452
 ADCs. *See* Analog-to-digital converters (ADCs)
 ADD, 297, 536
 Adders, 239–246
 carry propagate, 240
 carry-lookahead, 241
 full, 56, 240
 half, 240
 HDL for, 184, 200, 450
 prefix, 243
 ripple-carry, 240
 Addition, 14–15, 17–18, 235, 239–246,
 297. *See also* Adders
 binary, 14–15
 floating point, 259
 signed binary, 15–17
 Address. *See also* Memory
 physical, 509–513
 translation, 509–512
 virtual, 508. *See also* Virtual
 memory
 Addressing modes, ARM, 336
 base, 336
 immediate, 336
 PC-relative, 336
 register, 336
 Advanced High-performance Bus
 (AHB), 531.e54
 Advanced Micro Devices
 (AMD), 296

Advanced microarchitecture, 456–470
 branch prediction. *See* Branch
 prediction
 deep pipelines. *See* Deep pipelines
 heterogeneous multiprocessors. *See*
 Heterogeneous
 multiprocessors
 homogeneous multiprocessors. *See*
 Homogeneous
 multiprocessors
 micro-operations. *See* Micro-
 operations
 multiprocessors. *See* Multiprocessors
 multithreading. *See* Multithreading
 out-of-order processor. *See* Out-of-
 order processor
 register renaming. *See* Register
 renaming
 single instruction multiple data. *See*
 Single instruction multiple
 data (SIMD)
 superscalar processor. *See*
 Superscalar processor
 Advanced Microcontroller Bus
 Architecture (AMBA), 531.e54
 Advanced RISC Machines,
 472
 AHB. *See* Advanced High-performance
 Bus (AHB)
 AHB-Lite bus, 531.e54–531.e55
 Altera FPGA, 274–279
 ALU. *See* Arithmetic/logical unit (ALU)
 ALU Decoder, 398–400
 ALUControl, 248–250, 392, 395
 ALUOp, 398

- ALUResult*, 392–397
- ALUSrc*, 396
- AMAT. *See* Average memory access time (AMAT)
- AMBA. *See* Advanced Microcontroller Bus Architecture (AMBA)
- AMD. *See* Advanced Micro Devices (AMD)
- AMD64, 368
- Amdahl, Gene, 492
- Amdahl's Law, 492
- American Standard Code for Information Interchange (ASCII), 315–316, 541.e8, 541.e27–541.e28
- Analog I/O, 531.e25–531.e32
 - A/D conversion, 531.e31–531.e32
 - D/A conversion, 531.e25–531.e28
 - Pulse-width modulation (PWM), 531.e28–531.e31
- Analog-to-digital converters (ADCs), 531.e25, 531.e27, 531.e31–531.e32
- Analytical engine, 7–8
- AND gate, 20–22, 179
 - chips (7408, 7411, 7421), 533.e3
 - truth table, 20, 22
 - using CMOS transistors, 32–33
- AND, 303–304
- AND-OR (AO) gate, 46
- Anode, 27
- Antidependence, 464
- Application Binary Interface (ABI), 320
- Application-specific integrated circuits (ASICs), 533.e9
- Architectural state, 338, 364
 - for ARM, 385–386
- Architecture, 295
 - assembly language, 296
 - instructions, 297–298
 - operands, 298–303
 - compiling, assembling, and loading, 339
 - assembling, 342–343
 - compilation, 340–341
 - linking, 343–344
 - loading, 344–345
 - memory map, 339–340
 - evolution of ARM architecture, 350
 - 64-bit architecture, 360
 - digital signal processors (DSPs), 352–356
 - floating-point instructions, 357–358
 - power-saving and security instructions, 358
 - SIMD instructions, 358–360
 - Thumb instruction set, 351–352
- machine language, 329
 - addressing modes, 336
 - branch instructions, 334–335
 - data-processing instructions, 329–333
 - interpreting, 336–337
 - memory instructions, 333–334
 - stored program, 337–338
- odds and ends, 345
 - exceptions, 347–350
 - loading literals, 345–346
 - NOP, 346
- programming, 303
 - branching, 308–309
 - conditional statements, 309–312
 - condition flags, 306–308
 - function calls, 317–329
 - getting loopy, 312–313
 - logical and arithmetic instructions, 303–306
 - memory, 313–317
- x86 architecture, 360
 - big picture, 368
 - instruction encoding, 364–367
 - instructions, 364
 - operands, 362–363
 - peculiarities, 367–368
 - registers, 362
 - status flags, 363–364
- Arguments, 317–319, 541.e15
 - pass by reference, 541.e22
 - pass by value, 541.e22
- Arithmetic
 - ARM instructions, 303–306
 - circuits, 239–255
 - C operators, 541.e11–541.e13
 - HDL operators, 185
- Arithmetic/logical unit (ALU), 248–251, 392
 - implementation of, 249
 - in processor, 392–430
- ARM architecture, evolution of, 296, 350
 - 64-bit architecture, 360
 - digital signal processing (DSP) instructions, 352–356
 - floating-point instructions, 357–358
 - power-saving and security instructions, 358
 - SIMD instructions, 358–360
 - Thumb instruction set, 351–352
- ARM instructions, 295–369, 535–540
 - branch instructions, 308–309, 539
 - condition flags, 306–308, 540
 - data-processing instructions, 303–306, 535–537
 - logical instructions, 303–304
 - multiply instructions, 305–306, 537
 - shift instructions, 304–305
- formats
 - addressing modes, 336
 - branch instructions, 334
 - data-processing instructions, 329–333
 - interpreting, 336–337
 - memory instructions, 333–335
 - stored program, 337–338
- instruction set, 295
- memory instructions, 301–303, 313–317, 538
- miscellaneous instructions, 345–346, 539
- ARM Microcontroller Development Kit (MDK-ARM), 297
- ARM microprocessor, 385
 - data memory, 385–388
 - instruction memory, 385–388
 - multicycle, 406–425
 - pipelined, 425–433
 - program counter, 385–388
 - register file, 385–388
 - single-cycle, 390–406, 443–456
 - state elements of, 385–388
- ARM processors, 470
- ARM registers, 299–300
 - program counter, 308, 338, 386–387
 - register file, 386–387
 - register set, 299–300
- ARM single-cycle HDL, 443–456
 - building blocks, 449–452
 - controller, 443
 - datapath, 443
 - testbench, 452–456
- ARM7, 472, 473
- ARM9, 474
- ARM9E, 472
- ARMv3 architecture, 472
- ARMv4 instruction set, 295, 539
- ARMv7 instruction, 472
- Arrays, 313–317, 541.e23–541.e29
 - accessing, 313–317, 541.e23

- bytes and characters, 315–317, 541.
e27–541.e29
 - comparison or assignment of, 541.e28
 - declaration, 314–317, 541.e23
 - indexing, 314–317, 541.e23–541.e27
 - initialization, 541.e23–541.e24
 - as input argument, 541.e24–541.e25
 - multi-dimension, 541.e26–541.e27
 - ASCII. *See* American Standard Code for Information Interchange (ASCII)
 - ASICs. *See* Application-specific integrated circuits (ASICs)
 - ASR, 304
 - Assembler, 339, 541.e44
 - Assembling, 342–343
 - Assembly language, ARM, 295–350, 535–540
 - instructions, 297–350, 535–540
 - operands, 297–303
 - translating high-level code to, 339–345
 - translating machine language to, 337
 - Assembly language, x86. *See* x86
 - Associativity
 - in Boolean algebra, 62, 63
 - in caches, 493, 498–500
 - Astable circuits, 119
 - Asymmetric multiprocessors. *See* Heterogeneous multiprocessors
 - Asynchronous circuits, 120–123
 - Asynchronous resettable flip-flops
 - definition, 116
 - HDL, 194–196
 - Asynchronous serial link, 531.e17, 531.e17. *See also* Universal Asynchronous Receiver Transmitter (UART)
 - AT Attachment (ATA), 531.e61–531.e62
 - Average memory access time (AMAT), 491, 504
- B**
- B, 308–309, 334–336, 396–397
 - Babbage, Charles, 7
 - Banked registers, 348–349
 - Base addressing, 336
 - Baud rate, 531.e17–531.e19
 - BCD. *See* Binary coded decimal (BCD)
 - BCM2835, 531.e3, 531.e4–531.e5, 531.e8, 531.e9, 531.e19
 - timer, 531.e23
 - Behavioral modeling, 173–174
 - Benchmarks, 389
 - BEQ, 309
 - Biased exponent, 257
 - BIC (bit clear), 303–304
 - big.LITTLE, 469
 - Big-endian memory, 303
 - Big-endian order, 178
 - Binary addition, 14–15. *See also* Adders, Addition
 - Binary coded decimal (BCD), 258
 - Binary encoding, 125–126, 129–131
 - for divide-by-3 counter, 129–131
 - for traffic light FSM, 125–126
 - Binary numbers
 - signed, 15–19
 - unsigned, 9–11
 - Binary to decimal conversion, 10, 10–11
 - Binary to hexadecimal conversion, 12
 - Bipolar junction transistors, 26
 - Bipolar motor drive, 531.e50
 - Bipolar signaling, 531.e18
 - Bipolar stepper motor, 531.e51, 531.e52–531.e53
 - AIRPAX LB82773-M1, 531.e51, 531.e51
 - direct drive current, 531.e52
 - Bistable element, 109
 - Bit, 8
 - dirty, 506
 - least significant, 13, 14
 - most significant, 13, 14
 - sign, 16
 - use, 502
 - valid, 496
 - Bit cells, 264–269
 - DRAM, 266–267
 - ROM, 268–270
 - SRAM, 267
 - Bit swizzling, 188
 - Bitline, 264
 - Bitwise operators, 177–179
 - BL (branch and link), 318
 - Block, 493
 - Block offset, 500–501
 - Block size (*b*), 493, 500–501
 - Blocking and nonblocking assignments, 199–200, 205–209
 - BLT. *See* Branch if less than (BLT)
 - BlueSMiRF silver module, 531.e42–531.e43, 531.e42
 - Bluetooth wireless communication, 531.e42–531.e43
 - BlueSMiRF silver module, 531.e42–531.e43
 - classes, 531.e42
 - BNE, 310
 - Boole, George, 8
 - Boolean algebra, 60–66
 - axioms, 61
 - equation simplification, 65–66
 - theorems, 61–64
 - Boolean equations, 58–60
 - product-of-sums form, 60
 - sum-of-products form, 58–60
 - Boolean logic, 8. *See also* Boolean algebra, Logic gates
 - Boolean theorems, 61–64
 - associativity, 63
 - combining, 62
 - commutativity, 63
 - complements, 62
 - consensus, 62, 64
 - covering, 62
 - De Morgan's, 63–64
 - distributivity, 63
 - idempotency, 62
 - identity, 62
 - involution, 62
 - null element, 62
 - Branch if less than (BLT), 334–335
 - Branch instructions, 308–309
 - ARM instructions, 539, 539
 - Branch misprediction penalty, 438, 459
 - Branch prediction, 459–461
 - Branch target address (BTA), 334–335
 - Branch target buffer, 459
 - Branching, 308–309, 334–336
 - conditional, 309
 - unconditional, 309
 - Breadboards, 533.e18–533.e19
 - BTA. *See* Branch target address (BTA)
 - Bubble, 20, 63
 - pushing, 63–64, 71–73
 - Bubble, in pipeline, 435–436
 - Buffers, 20
 - lack of, 117
 - tristate, 74–75
 - Bugs, 175
 - in C code, 541.e45–541.e49
 - Bus, 56
 - tristate, 75

- Bus interfaces, 531.e54–531.e57
 - AHB-Lite, 531.e54–531.e55
 - memory and peripheral interface example, 531.e55–531.e57
 - Bypassing, 432. *See also* Forwarding
 - Byte, 13–14, 315–317. *See also* Characters
 - least significant, 13–14
 - most significant, 13–14
 - Byte offset, 495
 - Byte-addressable memory, 301–302
 - big-endian, 302–303
 - little-endian, 303
- C**
- C programming, 541.e1–541.e49
 - common mistakes. *See* Common mistakes in C
 - compiler. *See* Compiler, i_Hlt414277118n C
 - conditional statements. *See* Conditional statements
 - control-flow statements. *See* Control-flow statements
 - data types. *See* Data types
 - executing a program, 541.e4
 - function calls. *See* Function calls
 - loops. *See* Loops
 - operators. *See* Operators
 - simple program, 541.e3–541.e4
 - standard libraries. *See* Standard libraries
 - variables. *See* Variables in C
 - Caches, 489–508
 - address fields
 - block offset, 500–501
 - byte offset, 495
 - set bits, 495
 - tag, 495
 - advanced design, 503–507
 - evolution of, in ARM, 507
 - multiple level, 504
 - organizations, 502
 - direct mapped, 494–498
 - fully associative, 499–500
 - multi-way set associative, 498–499
 - parameters
 - block, 493
 - block size, 493, 500–501
 - capacity (C), 492–493
 - degree of associativity (N), 499
 - number of sets (S), 493
 - performance of
 - hit, 490–492
 - hit rate, 491–492
 - miss, 480–492, 505
 - capacity, 505
 - compulsory, 505
 - conflict, 498, 505
 - penalty, 500
 - miss rate, 491–492
 - reducing, 505–506
 - miss rate *vs.* cache parameters, 505–506
 - replacement policy, 502–503
 - status bits
 - dirty bit (D), 506
 - use bit (U), 502
 - valid bit (V), 496
 - write policy, 506–507
 - write-back, 506–507
 - write-through, 506–507
 - CAD. *See* Computer-aided design (CAD)
 - Callee, 317
 - Callee save rule, 324
 - Callee-saved registers, 323
 - Caller save rule, 324
 - Caller-saved registers, 323
 - Canonical form. *See* Sum-of-products (SOP) form, Product-of-sums (POS) form
 - Capacitors, 28
 - Capacity, of cache, 492–493
 - Capacity miss, 505
 - Carry propagate adder (CPA). *See* Carry-lookahead adder (CLA); Prefix adders; Ripple-carry adder
 - Carry-lookahead adder (CLA), 241–243, 242
 - case statement, in HDL, 201–203. *See also* Switch/case statement
 - casez, case?, in HDL, 205
 - Cathode, 27
 - Cathode ray tube (CRT), 531.e36. *See also* VGA (Video Graphics Array) monitor
 - horizontal blanking interval, 531.e36
 - vertical blanking interval, 531.e36
 - Character LCDs, 531.e33–531.e36
 - Characters (char), 315–317, 541.e8, 541.e27
 - arrays. *See also* Strings
 - C type, 541.e27
 - Chips, 28
 - multiprocessors, 468
 - Chopper constant current drive, 531.e51
 - Circuits
 - 74xx series. *See* 74xx series logic
 - application-specific integrated (ASICs), 533.e9
 - astable, 119
 - asynchronous, 120, 122–123
 - combinational. *See* Combinational logic
 - definition of, 55
 - delay, 88–92
 - glitches in, 92–95
 - multiple-output, 68
 - priority, 68
 - sequential. *See* Sequential logic
 - synchronous, 122–123
 - synchronous sequential, 120–123
 - synthesized, 176, 179, 181
 - timing, 88–95, 141–151
 - CISC. *See* Complex Instruction Set Computer (CISC) architectures
 - CLBs. *See* Configurable logic blocks (CLBs)
 - Clock cycles per instruction (CPI), 390
 - Clock period, 142, 390
 - Clock skew, 148–151
 - Clustered multiprocessors, 470
 - cmd field, 330, 535, 537
 - CMOS. *See* Complementary Metal-Oxide-Semiconductor Logic (CMOS)
 - CMP, 402
 - Combinational composition, 56
 - Combinational logic, 174
 - design, 55–106
 - Boolean algebra, 60–66
 - Boolean equations, 58–60
 - building blocks, 83–88, 239–255
 - delays, 88–92
 - don't cares, 81–82
 - Karnaugh maps (K-maps), 75–83
 - multilevel, 66–73
 - precedence, 58
 - timing, 88–95
 - two-level, 69

- X (contention). *See* Contention (X)
 - X (don't cares). *See* Don't care (X)
 - Z (floating). *See* Floating (Z)
 - HDLs. *See* Hardware description languages (HDLs)
 - Combining theorem, 62
 - Command line arguments, 541.
e44–541.e45
 - Comments
 - in ARM assembly, 297–298
 - in C, 297–298, 541.e5
 - in SystemVerilog, 180
 - in VHDL, 180
 - Common mistakes in C, 541.e45–541.e49
 - Comparators, 246–248
 - Comparison
 - in hardware. *See* Comparators;
 - Arithmetic/logical unit (ALU)
 - processor performance, 424–425
 - using ALU, 251
 - Compiler, in C, 339–345, 541.e4–541.e5, 541.e43–541.e44
 - Complementary Metal-Oxide-Semiconductor gates (CMOS), 26–34
 - Complements theorem, 62
 - Complex instruction set computer (CISC) architectures, 298, 361, 458
 - Complexity management, 4–7
 - digital abstraction, 4–5
 - discipline, 5–6
 - hierarchy, 6–7
 - modularity, 6–7
 - regularity, 6–7
 - Compulsory miss, 505
 - Computer-aided design (CAD), 71, 129
 - Concurrent signal assignment statement, 179, 183–184, 193, 200–206
 - cond* field, 306–307, 330, 535
 - Condition flags, 306–308
 - ARM instructions, 540, 540
 - Condition mnemonics, 307
 - Conditional assignment, 181–182
 - Conditional branches, 308–309
 - Conditional Logic, 398–400, 413–415
 - Conditional operator, 181–182
 - Conditional signal assignments, 181–182
 - Conditional statements, 309
 - in ARM assembly
 - if, 309–310
 - if/else, 310–311
 - switch/case, 311–312
 - in C, 541.e17–541.e18
 - if, 541.e17–541.e18
 - if/else, 541.e17
 - switch/case, 541.e17–541.e18
 - in HDL, 194, 201–205
 - case, 201–203
 - casez, case?, 205
 - if, if/else, 202–205
 - Configurable logic blocks (CLBs), 275, 533.e7. *See also* Logic elements (LEs)
 - Conflict miss, 505
 - Consensus theorem, 62, 64
 - Constants
 - in ARM assembly, 300–301. *See also* Immediates
 - in C, 541.e5–541.e6
 - Contamination delay, 88–92. *See also* Short path
 - Contention (x), 73–74
 - Context switching, 467
 - Continuous assignment statements, 179, 193, 200, 206
 - Control hazard, 432, 437–440
 - Control signals, 91, 249
 - Control unit, 386. *See also* ALU
 - Decoder, Main Decoder
 - of multicycle ARM processor, 413–423
 - of pipelined ARM processor, 430
 - of single-cycle ARM processor, 397–401
 - Control-flow statements
 - conditional statements. *See* Conditional statements
 - loops. *See* Loops
 - CoreMark, 389
 - Cortex-A7 and -A15, 475
 - Cortex-A9, 475
 - Counters, 260–261
 - divide-by-3, 130
 - Covering theorem, 62
 - CPA. *See* Carry propagate adder (CPA)
 - CPI. *See* Clock cycles per instruction (CPI); Cycles per instruction (CPI)
 - Critical path, 89–92, 402
 - Cross-coupled inverters, 109, 110
 - bistable operation of, 110
 - CRT. *See* Cathode ray tube (CRT)
 - Current Program Status Register (CPSR), 306, 324, 347
 - Cycle time. *See* Clock period
 - Cycles per instruction (CPI), 390, 424
 - Cyclic paths, 120
 - Cyclone IV FPGA, 275–279
- D**
- D flip-flops. *See* Flip-flops
 - D latch. *See* La_Hlt414277505tches
 - D/A conversion, 531.e25–531.e28
 - DACs. *See* Digital-to-analog converters (DACs)
 - DAQs. *See* Data Acquisition Systems (DAQs)
 - Data Acquisition Systems (DAQs), 531.
e62–531.e63
 - myDAQ, 531.e62–531.e63
 - Data hazard, 432–436
 - HDL for, 455
 - Data memory, 387–388
 - Data segment, 340
 - Data sheets, 533.e9–533.e14
 - Data types, 541.e21–541.e35
 - arrays. *See* Arrays
 - characters. *See* Characters (char)
 - dynamic memory allocation. *See* Dynamic memory allocation (malloc, free)
 - linked list. *See* Linked list
 - pointers. *See* Pointers
 - strings. *See* Strings
 - structures. *See* Structures (struct)
 - typedef, 541.e31–541.e32
 - Datapath
 - multicycle ARM processor, 406–413
 - B instruction, 412–413
 - LDR instruction, 407–410
 - STR instruction, 411–412
 - pipelined ARM processor, 428–430
 - single-cycle ARM processor, 390
 - B instruction, 396–397

- Datapath (*Continued*)
 - LDR instruction, 391–394
 - STR instruction, 394–396
 - Data-processing instructions, 536
 - ARM instructions, 329–333, 396–397, 535–537
 - encodings, 536
 - DC motors, 531.e43, 531.e44–531.e48
 - H-bridge, 531.e44, 531.e45
 - shaft encoder, 531.e43–531.e44
 - DC transfer characteristics, 24–26. *See also* Direct current (DC) transfer characteristics, Noise margins
 - DDR. *See* Double-data rate memory (DDR)
 - De Morgan, Augustus, 63
 - De Morgan's theorem, 63–64
 - DE-9 cable, 531.e19
 - Decimal numbers, 9
 - Decimal to binary conversion, 11
 - Decimal to hexadecimal conversion, 13
 - Decode stage, 425
 - Decoders
 - definition of, 86–87
 - HDL for
 - behavioral, 202–203
 - parameterized, 219
 - logic using, 87–88
 - Seven-segment. *See* Seven-segment display decoder
 - Deep pipelines, 457
 - Delaymicros function, 531.e24
 - Delays, logic gates. *See also* Propagation delay
 - in HDL (simulation only), 188–189
 - DeleteUser function, 541.e33
 - Dennard, Robert, 266
 - Destination register (rd or rt), 393, 409
 - Device driver, 531.e3, 531.e6–531.e8
 - Device under test (DUT), 220
 - Dhrystone, 389
 - Dice, 28
 - Dielectric, 28
 - Digital abstraction, 4–5, 7–9, 22–26
 - Digital circuits. *See* Logic
 - Digital signal processors (DSPs), 352–356, 469
 - Digital system implementation, 533.e1–533.e35
 - 74xx series logic. *See* 74xx series logic
 - application-specific integrated circuits (ASICs), 533.e9
 - assembly of, 533.e17–533.e20
 - breadboards, 533.e18–533.e19
 - data sheets, 533.e9–533.e14
 - economics, 533.e33–533.e35
 - logic families, 533.e15–533.e17
 - packaging, 533.e17–533.e20
 - printed circuit boards, 533.e19–533.e20
 - programmable logic, 533.e2–533.e9
 - Digital-to-analog converters (DACs), 531.e25–531.e28
 - DIMM. *See* Dual inline memory module (DIMM)
 - Diodes, 27–28
 - p-n junction, 28
 - DIPs. *See* Dual-inline packages (DIPs)
 - Direct current (DC) transfer characteristics, 24, 25
 - Direct mapped cache, 494–498, 495
 - Direct voltage drive, 531.e51
 - Dirty bit (D), 506
 - Discipline
 - dynamic, 142–151. *See also* Timing analysis
 - static, 142–151. *See also* Noise margins
 - Discrete-valued variables, 7
 - Distributivity theorem, 63
 - Divide-by-3 counter
 - design of, 129–131
 - HDL for, 210–211
 - Divider, 254–255
 - Division
 - circuits, 254–255
 - Do/while loops, in C, 541.e19–541.e20
 - Don't care (X), 69, 81–83, 205
 - Dopant atoms, 27
 - Double, C type, 541.e8–541.e9
 - Double-data rate memory (DDR), 268, 531.e60–531.e61
 - Double-precision formats, 258
 - DRAM. *See* Dynamic random access memory (DRAM)
 - DSPs. *See* Digital signal processors (DSPs)
 - Dual inline memory module (DIMM), 531.e60
 - Dual-inline packages (DIPs), 28, 533.e1, 533.e17
 - Dynamic branch predictors, 459
 - Dynamic data segment, 340
 - Dynamic discipline, 142–151. *See also* Timing analysis
 - Dynamic memory allocation (malloc, free), 541.e32–541.e33
 - in ARM memory map, 340
 - Dynamic power, 34
 - Dynamic random access memory (DRAM), 266–267, 487–490, 519, 531.e58, 531.e60, 531.e61
- ## E
- EasyPIO, 531.e6
 - Economics, 533.e33
 - Edge-triggered flip-flop. *See* Flip-flops
 - EEPROM. *See* Electrically erasable programmable read only memory (EEPROM)
 - EFLAGS register, 363
 - Electrically erasable programmable read only memory (EEPROM), 270
 - Embedded I/O (input/output) systems, 531.e3–531.e32
 - analog I/O, 531.e25–531.e32
 - A/D conversion, 531.e31–531.e32
 - D/A conversion, 531.e25–531.e28
 - digital I/O, 531.e8–531.e11
 - general-purpose I/O (GPIO), 531.e8–531.e11
 - interrupts, 531.e32
 - LCDs. *See* Liquid Crystal Displays (LCDs)
 - microcontroller peripherals, 531.e32–531.e53
 - motors. *See* Motors
 - serial I/O, 531.e11–531.e23. *See also* Serial I/O
 - timers, 531.e23–531.e24
 - VGA monitor. *See* VGA (Video Graphics Array) monitor
 - Enabled flip-flops, 115–116
 - Enabled registers, 196–197. *See also* Flip-flops
 - EOR (XOR), 303–304
 - EPROM. *See* Erasable programmable read only memory (EPROM)

Equality comparator, 247
 Equation minimization
 using Boolean algebra, 65–66
 using Karnaugh maps. *See* Karnaugh maps (K-maps)
 Erasable programmable read only memory (EPROM), 270, 533.e6
 Ethernet, 531.e61
 Exceptions, 346–350
 banked registers, 348–349
 exception-related instructions, 349–350
 exception vector table, 347–348
 execution modes and privilege levels, 347
 handler, 340, 349
 start-up, 350
 Execution time, 389
 exit, 541.e41
 Extended instruction pointer (EIP), 362
 ExtImm, 408

F

factorial function call, 326
 stack during, 327
 Factoring state machines, 134–136
 False, 8, 20, 35, 58, 60, 74, 111, 112, 113, 116, 124, 196
 Fast Fourier Transform (FFT), 352
 FDIV. *See* Floating-point division (FDIV)
 FFT. *See* Fast Fourier Transform (FFT)
 Field programmable gate arrays (FPGAs), 274–279, 531.e14, 531.e38, 531.e63, 533.e7–533.e9
 driving VGA cable, 531.e38
 in SPI interface, 531.e13–531.e16
 File manipulation, in C, 541.e38–541.e40
 Finite state machines (FSMs), 123–141, 209–213, 413, 417
 complete multicycle control, 424
 deriving from circuit, 137–140
 divide-by-3 FSM, 129–131, 210–211
 factoring, 134–136, 136
 in HDL, 209–213
 LE configuration for, 277–279
 Mealy FSM, 132–134
 Moore FSM, 132–134
 snail/pattern recognizer FSM, 132–134, 212–213
 state encodings, 129–131. *See also* Binary encoding, One-cold encoding, One-hot encoding
 state transition diagram, 124, 125
 traffic light FSM, 123–129
 Fixed-point numbers, 255–256
 Flags, 250
 Flash memory, 270. *See also* Solid state drive (SSD)
 Flip-flops, 114–118, 193–197. *See also* Registers
 back-to-back, 145, 152–157, 197. *See also* Synchronizers
 comparison with latches, 118
 enabled, 115–116
 HDL for, 451. *See also* Registers
 metastable state of. *See* Metastability
 register, 114–115
 resettable, 116
 scannable, 262–263
 shift register, 261–263
 transistor count, 114, 117
 transistor-level, 116–117
 Float, C type, 541.e6–541.e9
 print formats of, 541.e36–541.e37
 Floating (Z), 74–75
 in HDLs, 186–188
 Floating output node, 117
 Floating point division (FDIV) bug, 175
 Floating-gate transistor, 270. *See also* Flash memory
 Floating-point division (FDIV), 259
 Floating-point instructions, ARM, 357–358
 Floating-point numbers, 256–258
 addition, 259
 formats, single- and double-precision, 258
 in programming. *See* Double, C type; Float, C type
 rounding, 259
 special cases
 infinity, 258
 NaN, 258
 Floating-Point Status and Control Register (FPSCR), 358
 Floating-point unit (FPU), 259
 For loops, 312–313, 541.e20

Format conversion (atoi, atol, atof), 541.e41–541.e42
 Forwarding, 432–435. *See also* Hazards
 FPGAs. *See* Field programmable gate arrays (FPGAs)
 FPU. *See* Floating-point unit (FPU)
 FPSCR. *See* Floating-Point Status and Control Register (FPSCR)
 Frequency shift keying (FSK), 531.e42
 and GFSK waveforms, 531.e42
 Front porch, 531.e37
 FSK. *See* Frequency shift keying (FSK)
 FSMs. *See* Finite state machines (FSMs)
 Full adder, 56, 182, 184, 200, 240
 using always/process statement, 200
 Fully associative cache, 499–500
 funct field, 330, 333
 Function calls, 317, 541.e15–541.e16
 additional arguments and local variables, 328–329
 arguments, 319, 541.e15
 leaf, 324–326
 multiple registers, loading and storing, 322
 naming conventions, 541.e16
 with no inputs or outputs, 318, 541.e15
 nonleaf, 324–326
 preserved registers, 322–324
 prototypes, 541.e16
 recursive, 326–328
 return, 318–319, 541.e15
 stack, use of, 320–322. *See also* Stack
 Furber, Steve, 473
 Fuse-programmable ROM, 269–270

G

Gates
 AND, 20, 22, 128
 buffer, 20
 multiple-input, 21–22
 NAND, 21, 31
 NOR, 21–22, 111, 128
 NOT, 20
 OR, 21
 transistor-level. *See* Transistors
 XNOR, 21
 XOR, 21

General-purpose I/O (GPIO), 531.
 e8–531.e11
 switches and LEDs example, 531.e8
 Generate signal, 241, 243
 Genwaves function, 531.e27
 Glitches, 92–95
 Global data segment, 340
 GPIO. *See* General-purpose I/O (GPIO)
 Graphics accelerators, 469
 Graphics processing units (GPUs), 460
 Gray, Frank, 76
 Gray codes, 76
 Ground (GND), 22
 symbol for, 31

H

Half adder, 240, 240
 Hard disk, 490–491. *See also* Hard drive
 Hard drive, 490, 508. *See also* Hard disk, Solid state drive (SSD), Virtual memory
 Hardware description languages (HDLs), 443–456. *See also* SystemVerilog, VHSIC Hardware Description Language (VHDL)
 2:1 multiplexer, 452
 adder, 450
 capacity, 505
 combinational logic, 174, 198
 bitwise operators, 177–179
 blocking and nonblocking assignments, 205–209
 case statements, 201–202
 conditional assignment, 181–182
 delays, 188–189
 data memory, 455
 data types, 213–217
 history of, 174–175
 if statements, 202–205
 internal variables, 182–184
 numbers, 185
 operators and precedence, 184–185
 reduction operators, 180–181
 immediate extension, 451
 instruction memory, 455–456
 modules, 173–174
 parameterized modules, 217–220
 processor building blocks, 449–452
 register file, 450
 resettable flip-flop, 451
 resettable flip-flop with enable, 452
 sequential logic, 193–198, 209–213
 simulation and synthesis, 175–177
 single-cycle ARM processor, 443–456
 structural modeling, 190–193
 testbench, 220–224, 452–453
 top-level module, 454
 Hardware handshaking, 531.e18
 Hardware reduction, 70–71. *See also* Equation minimization
 Hazard unit, 432–435
 Hazards. *See also* Hazard unit
 control hazards, 432, 437–440
 data hazards, 432–436
 pipelined processor, 431–441
 read after write (RAW), 431, 464
 solving
 control hazards, 437–440
 forwarding, 432–434
 stalls, 435–436
 write after read (WAR), 464
 write after write (WAW), 465
 H-bridge control, 531.e45
 HDL. *See* Hardware description languages (HDLs), SystemVerilog; VHSIC Hardware Description Language (VHDL)
 Heap, 340
 Heterogeneous multiprocessors, 469–470
 Hexadecimal numbers, 11–13
 Hexadecimal to binary and decimal conversion, 11, 12
 Hierarchy, 6
 HIGH, 23. *See also* 1, ON
 High-level programming languages, 303, 541.e2
 compiling, assembling, and loading, 339–345
 translating into assembly, 300
 High-performance microprocessors, 456
 Hit, 490
 Hit rate, 491
 Hold time constraint, 142–148
 with clock skew, 149–151
 Hold time violations, 145, 146, 147–148, 150–151

Homogeneous multiprocessors, 468–469
 Hopper, Grace, 340

I

I/O. *See* Input/output (I/O) systems
 IA-32 architecture. *See* x86
 IA-64, 368
 ICs. *See* Integrated circuits (ICs)
 Idempotency theorem, 62
 Identity theorem, 62
 Idioms, 177
 if statements
 in ARM assembly, 309–310
 in C, 541.e17
 in HDL, 202–205
 if/else statements, 310, 541.e27
 in ARM assembly, 310–311
 in C, 541.e17–541.e18
 in HDL, 202–205
 ILP. *See* Instruction level parallelism (ILP)
 IM. *See* Instruction memory
 imm8 field, 330–331
 imm12 field, 333
 imm24 field, 334
 Immediate addressing, 336
 Immediate extension, 451
 Immediates, 300–301, 330–332, 345–346. *See also* Constants
 Implicit leading one, 257
 Information, amount of, 8
 Initializing
 arrays in C, 541.e23–541.e24
 variables in C, 541.e11
 Input/Output (I/O) systems, 531.
 e1–531.e64
 device driver, 531.e3, 531.e6–531.e8
 embedded I/O systems. *See* Embedded I/O (input/output) systems
 I/O registers, 531.e3
 memory-mapped I/O, 531.e1–531.e3
 personal computer I/O systems. *See* Personal computer (PC) I/O systems
 Input/output elements (IOEs), 275
 Institute of Electrical and Electronics Engineers (IEEE), 257–258

Instruction encoding, x86, 364–367, 366

Instruction formats, ARM, 328

- addressing modes, 336
- branch instructions, 334–335
- data-processing instructions, 329–333
- interpreting, 336–337
- memory instructions, 333–335
- stored program, 337–338

Instruction formats, x86, 364–367

Instruction level parallelism (ILP), 465, 467, 468

Instruction memory, 387, 427, 455

Instruction register (IR), 407, 414

Instruction set, 295

- for ARM, 386

Instruction set. *See also* Architecture

Instructions, x86, 360–368

Instructions, ARM, 295–360, 535–540

- branch instructions, 308–309, 539
- condition flags, 306–308, 540
- data-processing instructions, 535
- logical, 303–304, 536–537
- memory instructions, 301–303, 313–317, 333–334, 538
- miscellaneous instructions, 539
- multiply instructions, 305–306, 537
- shift instructions, 304–305

Instructions per cycle (IPC), 390

Integrated circuits (ICs), 533.e17

Intel. *See* x86

Intel processors, 360

Intel x86. *See* x86

Interrupts, 347, 531.e32

Invalid logic level, 186

Inverters, 20, 119, 178. *See also* NOT gate

- cross-coupled, 109, 110
- in HDL, 178, 199

An Investigation of the Laws of Thought (Boole), 8

Involvement theorem, 62

IOEs. *See* Input/output elements (IOEs)

IPC. *See* Instructions per cycle (IPC)

IR. *See* Instruction register (IR)

IRWrite, 407, 414

J

Java, 303. *See also* Language

K

Karnaugh, Maurice, 75

Karnaugh maps (K-maps), 75–84, 93–95, 126

- logic minimization using, 77–83
- prime implicants, 65, 77–81, 94–95
- seven-segment display decoder, 79–81
- with “don’t cares”, 81–82

Kilobit (Kb/Kbit), 14

Kilobyte (KB), 14

K-maps. *See* Karnaugh maps (K-maps)

L

LAB. *See* Logic array block (LAB)

Land grid array, 531.e58

Language. *See also* Instructions

- assembly, 296–303
- machine, 329–338
- mnemonic, 297

Last-in-first-out (LIFO) queue, 320.

- See also* Stack

Latches, 111–113

- comparison with flip-flops, 109, 118
- D, 113, 120
- SR, 111–113, 112
- transistor-level, 116–117

Latency, 157–160, 425, 435

Lattice, silicon, 27

LCDs. *See* Liquid crystal displays (LCDs)

LDR, 301–303, 313–317, 333–334, 391–394, 538

- critical paths for, 402

Leaf function, 324

Leakage current, 34

Least recently used (LRU) replacement, 502–503

- two-way associative cache with, 502–503, 503

Least significant bit (lsb), 13, 14

Least significant byte (LSB), 13, 14, 301

LEs. *See* Logic elements (LEs)

Level-sensitive latch. *See* La_Hlt414277542tches: D

LIFO. *See* Last-in-first-out (LIFO) queue

Line options, compiler and command, 341–343, 541.e43–541.e45

Linked list, 541.e33–541.e34

Linker, 340–341

Linking, 339

Linux, 531.e23–531.e24

Liquid crystal displays (LCDs), 531.e33–531.e36

Literal, 58, 96

- loading, 345–346

Little-endian bus order in HDL, 178

Little-endian memory addressing, 303

Load register instruction (LDR), 301–302

Loading literals, 345–346

Loads, 344–345

- base addressing of, 336

Local variables, 328–329

Locality, 488

Logic

- bubble pushing, 71–73
- combinational. *See* Combinational logic
- families, 25–26, 533.e15–533.e17, 533.e15, 533.e17
- gates. *See* Gates
- hardware reduction, 70–71
- multilevel. *See* Multilevel
- combinational logic
- programmable, 533.e2–533.e9
- sequential. *See* Sequential logic
- transistor-level. *See* Transistors
- two-level, 69

Logic array block (LAB), 276

Logic arrays, 271–280. *See also* Field

- programmable gate arrays (FPGAs), Programmable logic arrays (PLAs)
- transistor-level implementation, 279–280

Logic elements (LEs), 275–279

- of Cyclone IV, 276–277
- functions built using, 277–279

Logic families, 25–26, 533.e15–533.e17, 533.e15, 533.e17

- compatibility of, 26
- logic levels of, 25
- specifications, 533.e15, 533.e17

Logic gates, 19–22, 179, 533.e2

- AND. *See* AND gate
- AND-OR (AO) gate, 46
- with delays in HDL, 189
- multiple-input gates, 21–22
- NAND. *See* NAND gate
- NOR. *See* NOR gate

Logic gates (*Continued*)
 NOT. *See* NOT gate
 OR. *See* OR gate
 OR-AND-INVERT (OAI) gate, 46
 XNOR. *See* XNOR gate
 XOR. *See* XOR gate
 Logic levels, 22–26
 Logic simulation, 175–176
 Logic synthesis, 176–177, 176
 Logical instructions, 303–304
 Logical shifter, 251
 Lookup tables (LUTs), 270, 275–276
 Loops, 312–313, 541.e19–541.e20
 in ARM assembly
 for, 312–313
 while, 312
 in C
 do/while, 541.e19–541.e20
 for, 541.e20
 while, 541.e19
 Lovelace, Ada, 338
 LOW, 23. *See also* 0, FALSE
 Low Voltage CMOS Logic (LVCMOS), 25
 Low Voltage TTL Logic (LVTTTL), 25
 lsb. *See* Least significant bit (lsb)
 LSB. *See* Least significant byte (LSB)
 LSL, 304
 LSR, 304
 LUTs. *See* Lookup tables (LUTs)
 LVCMOS. *See* Low Voltage CMOS Logic (LVCMOS)
 LVTTTL. *See* Low Voltage TTL Logic (LVTTTL)

M

MAC. *See* Multiply-accumulate (MAC)
 Machine code. *See* Machine language
 Machine language, 329
 addressing modes, 336
 branch instructions, 334–335
 data-processing instructions, 329–333
 interpreting, 336–337
 memory instructions, 333–335
 stored program, 337–338, 338
 translating to assembly language, 337

Magnitude comparator, 247
 Main Decoder, 398–400, 400
 Main FSM, 413–423, 423
 main function in C, 541.e3
 Main memory, 489–491
 malloc function, 541.e32
 Mantissa, 257
 Master-slave flip-flop. *See* Flip-flops
 Masuoka, Fujio, 270
 math.h, C library, 541.e42–541.e43
 Max-delay constraint. *See* Setup time constraint
 Maxterms, 58
 MCUs. *See* Microcontroller units (MCUs)
 Mealy machines, 123, 123, 132–134
 state transition and output table, 134
 state transition diagrams, 133
 timing diagrams for, 135
 Mean time between failure (MTBF), 153–154
 Medium-scale integration (MSI) chips, 533.e2
 MemWrite, 394, 397
 Memory, 313. *See also* Memory arrays
 access time, 491
 addressing modes, 363
 area and delay, 267–268
 big-endian, 302
 byte-addressable, 301–303
 bytes and characters, 315–317
 HDL for, 272, 273, 455–456
 hierarchy, 490
 little-endian, 303
 logic using, 270–271
 main, 490
 operands in, 301–303
 physical, 509
 ports, 265–266
 protection, 515. *See also* Virtual memory
 types, 266–270
 DDR, 268
 DRAM, 266–267
 flash, 270
 register file, 268
 ROM, 268–270
 SRAM, 266
 virtual, 490. *See also* Virtual memory
 Memory address computation, 419
 data flow during, 419

Memory and peripheral interface, 531.e55–531.e57
 Memory arrays, 264–271. *See also* Memory
 bit cell, 264–270
 HDL for, 272, 273, 455–456
 logic using, 270–271
 organization, 264–265
 Memory hierarchy, 490–491
 Memory instructions, 301–303, 313–317, 333–334, 391–394
 encodings, 333–334, 538
 Memory interface, 487–488
 Memory map, ARM, 339–340, 531.e2
 Memory performance. *See* Average Memory Access Time (AMAT)
 Memory protection, 515
 Memory systems, 487
 ARM, 507–508
 performance analysis, 491–492
 x86, 531.e3
 Memory-mapped I/O, 531.e1–531.e3, 531.e7
 address decoder, 531.e1, 531.e2
 communicating with I/O devices, 531.e2
 hardware, 531.e2, 531.e2, 531.e3
 MemtoReg, 396, 397
 Metal-oxide-semiconductor field effect transistors (MOSFETs), 26
 switch models of, 30
 Metastability, 151–157
 metastable state, 110, 151
 resolution time, 151–152, 154–157
 synchronizers, 152–154
 Microarchitecture, 296, 385, 388–389.
 See also Architecture
 advanced. *See* Advanced microarchitecture
 architectural state. *See* Architectural state
 description of, 385–389
 design process, 386–388
 evolution of, 470–476
 HDL representation, 443–456
 generic building blocks, 449–452
 single-cycle processor, 444–449
 testbench, 452–456
 multicycle processor. *See* Multicycle ARM processor

- performance analysis, 389–390.
 - See also* Performance analysis
 - pipelined processor. *See* Pipelined ARM processor
 - real-world perspective, 470–476
 - single-cycle processor. *See* Single-cycle ARM processor
 - Microcontroller, 531.e3, 531.e25
 - Microcontroller peripherals, 531.
 - e32–531.e53
 - Bluetooth wireless communication, 531.e42–531.e43
 - character LCD, 531.e33–531.e36
 - control, 531.e35–531.e36
 - parallel interface, 531.e33
 - motor control, 531.e43–531.e53
 - VGA monitor, 531.e36–531.e42
 - Microcontroller units (MCUs), 531.e3
 - Micro-operations (micro-ops), 458–459
 - designers, 456
 - high-performance, 456
 - Microprocessors, 3, 13, 295
 - architectural state of, 338
 - Millions of instructions per second, 425
 - Min-delay constraint. *See* Hold time constraint
 - Minterms, 58
 - Miss, 490–492, 505
 - capacity, 505
 - compulsory, 505
 - conflict, 498, 505
 - Miss penalty, 500
 - Miss rate, 491–492
 - and access times, 492
 - Misses
 - cache, 490
 - capacity, 505
 - compulsory, 505
 - conflict, 505
 - page fault, 509–510
 - ModR/M byte, 366
 - Modularity, 6
 - Modules, in HDL
 - behavioral and structural, 173–174
 - parameterized modules, 217–220
 - Moore, Gordon, 30
 - Moore machines, 123, 132
 - state transition and output table, 134
 - state transition diagrams, 133
 - timing diagrams for, 135
 - Moore's law, 30
 - MOS transistors. *See* Metal-oxide-semiconductor field effect transistors (MOSFETs)
 - MOSFET. *See* Metal-oxide-semiconductor field effect transistors (MOSFETs)
 - Most significant bit (msb), 13, 14
 - Most significant byte (MSB), 13, 14, 301, 302
 - Motors
 - DC, 531.e43, 531.e44–531.e47
 - H-bridge, 531.e45–531.e46, 531.e45, 531.e46
 - servo, 531.e44, 531.e48–531.e49
 - stepper, 531.e44, 531.e49–531.e53
 - MOV, 301
 - MPSSE. *See* Multi-Protocol Synchronous Serial Engine (MPSSE)
 - msb. *See* Most significant bit (msb)
 - MSB. *See* Most significant byte (MSB)
 - MSI chips. *See* Medium-scale integration (MSI) chips
 - MTBF. *See* Mean time between failure (MTBF)
 - Multicycle ARM processor, 406
 - control, 413–421
 - datapath, 407–413
 - B instruction, 412–413
 - data-processing instructions, 412
 - LDR instruction, 407–410
 - STR instruction, 411–412
 - performance, 421–425
 - Multicycle microarchitectures, 388
 - Multilevel combinational logic, 69–73.
 - See also* Logic
 - Multilevel page tables, 516–518
 - Multiple-output circuit, 68–69
 - Multiplexers, 83–86
 - definition of, 83–84
 - HDL for
 - behavioral model of, 181–183
 - parameterized N-bit, 218–219
 - structural model of, 190–193
 - logic using, 84–86
 - symbol and truth table, 83
 - Multiplicand, 252–253
 - Multiplication. *See* Multiplier
 - Multiplier, 252–253
 - HDL for, 253
 - Multiply instructions, 305–306, 537, 537
 - Multiply and multiply-accumulate instructions, 355–356
 - Multiply-accumulate (MAC), 352, 356
 - Multiprocessors, 468–470
 - chip, 468
 - heterogeneous, 469–470
 - homogeneous, 468
 - Multi-Protocol Synchronous Serial Engine (MPSSE), 531.e63
 - Multithreaded processor, 467
 - Multithreading, 467–468
 - Mux. *See* Multiplexers
 - myDAQ, 531.e62–531.e63
- ## N
- NAND (7400), 533.e3
 - NAND gate, 21
 - CMOS, 31–32
 - Nested if/else statement, 311, 541.e18
 - Newton computer, 472
 - Nibbles, 13–14
 - nMOS transistors, 28–31, 29–30
 - Noise margins, 23–26, 23
 - calculating, 23–24
 - Nonarchitectural state, 386, 388
 - Nonblocking and blocking assignments, 199–200, 205–209
 - Nonleaf function calls, 324–326
 - Nonpreserved registers, 322–323, 326
 - NOP, 346, 431
 - NOR gate, 21–22, 63, 533.e3
 - chip (7402), 533.e3
 - CMOS, 32
 - pseudo-nMOS logic, 33
 - truth table, 22
 - Not a number (NaN), 258
 - NOT gate, 20
 - chip (7404), 533.e3
 - CMOS, 31
 - Noyce, Robert, 26
 - Null element theorem, 62
 - Number conversion
 - binary to decimal, 10–11
 - binary to hexadecimal, 12
 - decimal to binary, 11, 13
 - decimal to hexadecimal, 13
 - hexadecimal to binary and decimal, 11, 12
 - taking the two's complement, 16

Number systems, 9–19
 binary, 9–11, 10–11
 comparison of, 18–19, 19
 estimating powers of two, 14
 fixed-point, 255, 255–256
 floating-point, 256–259
 addition, 259, 260
 special cases, 258
 hexadecimal, 11–13, 12
 negative and positive, 15
 sign/magnitude, 15–16
 signed, 15–18
 two's complement, 16–18
 unsigned, 9–11

O

Odds and ends, 345
 exceptions, 346–350
 loading literals, 345–346
 NOP, 346

OFF, 26, 30

Offset, 302, 392, 408

Offset indexing, ARM, 314

ON, 26, 30

One-bit dynamic branch predictor, 460

One-cold encoding, 130

One-hot encoding, 129–131

One-time programmable (OTP), 533.e2

op field, 330

Opcode. *See op* field

Operands
 ARM, 298
 constants/immediates, 300–301
 memory, 301–303
 registers, 299
 register set, 300
 x86, 362–363, 363

Operation code. *See op* field

Operators
 in C, 541.e11–541.e14
 in HDL, 177–185
 bitwise, 177–181
 precedence, 185
 reduction, 180–181
 table of, 185
 ternary, 181–182

OR gate, 21

OR-AND-INVERT (OAI) gate, 46

ORR (OR), 303–304

OTP. *See* One-time programmable (OTP)

Out-of-order execution, 466

Out-of-order processor, 463–465

Output dependence, 465

Overflow
 with addition, 15
 detection, 250–251

Oxide, 28

P

Packages, chips, 533.e17–533.e18

Page fault, 509

Page number, 511

Page offset, 511

Page table, 510–513

Pages, 509

Paging, 516

Parallel I/O, 531.e11

Parallelism, 157–160

Parity gate. *See* XOR gate

Partial products, 252

Pass by reference, 541.e22

Pass by value, 541.e22

Pass gate. *See* Transmission gates

PC. *See* Program counter (PC)

PC Logic, 400

PCB. *See* Printed circuit boards (PCBs)

PCI. *See* Peripheral Component Interconnect (PCI)

PCI express (PCIe), 531.e60

PC-relative addressing, 335, 336

PCSrc, 394, 395–396, 440

PCWrite, 410

Perfect induction, proving theorems
 using, 64–65

Performance analysis, 389–390
 multicycle ARM processor, 422–424
 pipelined ARM processor, 425–428
 processor comparison, 424
 single-cycle ARM processor, 402

Performance Analysis, 389–390.
See also Average Memory Access Time (AMAT)

Peripheral Component Interconnect (PCI), 531.e59–531.e60

Peripherals devices. *See* Input/output (I/O) systems

Personal computer (PC) I/O systems,
 531.e57–531.e64
 data acquisition systems, 531.
 e62–531.e63
 DDR3 memory, 531.e60–531.e61
 networking, 531.e61
 PCI, 531.e59–531.e60
 SATA, 531.e61–531.e62
 USB, 531.e59, 531.e63–531.e64

Phase locked loop (PLL), 531.e39

Physical memory, 509

Physical page number (PPN), 511

Physical pages, 509

Pipelined ARM processor, 425–428
 abstract view of, 427
 control unit, 430
 datapath, 428–429
 description, 425–428
 hazards, 431–441
 performance analysis, 441–443
 throughput, 426

Pipelined microarchitecture. *See*
 Pipelined ARM processor

Pipelining, 158–160

PLAs. *See* Programmable logic arrays (PLAs)

Plastic leaded chip carriers (PLCCs),
 533.e17

Platters, 508

PLCCs. *See* Plastic leaded chip carriers (PLCCs)

PLDs. *See* Programmable logic devices (PLDs)

PLL. *See* Phase locked loop (PLL)

pMOS transistors, 28–31, 29

Pointers, 541.e21–541.e23, 541.e25,
 541.e28, 541.e30, 541.e32

POS. *See* Product-of-sums (POS) form

Positive edge-triggered flip-flop, 114

Post-indexed addressing, ARM, 314

Power consumption, 34–35

Power-saving and security instructions,
 358

PPN. *See* Physical page number (PPN)

Prefix adders, 243–245, 244

Prefix tree, 245

Pre-indexed addressing, ARM, 314

Preserved registers, 322–324, 323

Prime implicants, 65, 77

Printed circuit boards (PCBs), 533.
 e19–533.e20

`printf`, 541.e35–541.e37

Priority
 circuit, 68–69
 encoder, 102–103, 105
 Procedure calls. *See* Function calls
 Processor performance comparison, 442
 multicycle ARM processor, 424
 pipelined ARM processor, 442
 single-cycle processor, 405
 Processor-memory gap, 489
 Product-of-sums (POS) form, 60
 Program counter (PC), 308, 338, 387, 394
 Programmable logic arrays (PLAs), 67, 272–274, 533.e6–533.e7
 transistor-level implementation, 280
 Programmable logic devices (PLDs), 533.e6
 Programmable read only memories (PROMs), 269, 271, 533.e2–533.e6
 Programming
 in ARM, 303
 arrays. *See* Arrays
 branching. *See* Branching
 in C. *See* C programming
 conditional statements, 309–312
 condition flags, 306–308
 constants. *See* Constants; Immediates
 function calls. *See* Function calls
 getting loopy, 312–313
 logical and arithmetic instructions, 303–306
 loops. *See* Loops
 memory, 313–317
 shift instructions, 304–305
 PROMs. *See* Programmable read only memories (PROMs)
 Propagate signal, 241
 Propagation delay, 88–92. *See also* Critical path
 Pseudoinstructions, 346
 Pseudo-nMOS logic, 33–34, 33
 NOR gate, 33
 ROMs and PLAs, 279–280
 Pulse-Width Modulation (PWM), 531.e28–531.e31
 analog output with, 531.e30–531.e31
 duty cycle, 531.e28
 signal, 531.e28
 PWM. *See* Pulse-Width Modulation (PWM)

Q

Quiescent supply current, 34

R

Race conditions, 119–120, 120
 rand, 541.e40–541.e41
 Random access memory (RAM), 266–268, 271, 272
 Raspberry Pi, 531.e3–531.e4, 531.e5, 531.e6, 531.e32, 531.e48–531.e49
 RAW hazard. *See* Read after write (RAW) hazard
 Rd field, 330
 Read after write (RAW) hazard, 431, 464. *See also* Hazards
 Read only memory (ROM), 266, 268–270
 transistor-level implementation, 279–280
 Read/write head, 508
 ReadData bus, 393, 394
 Receiver gate, 22
 Recursive function calls, 326–328
 Reduced instruction set computer (RISC) architecture, 298, 458
 Reduction operators, 180–181
 Register file (RF)
 ARM register descriptions, 299
 HDL for, 449
 in pipelined ARM processor (write on falling edge), 428
 schematic, 268
 use in ARM processor, 387
 Register renaming, 465–467
 Register set, 300. *See also* Register file (RF)
 Registers. *See* ARM registers; Flip-flops; x86 registers
 loading and storing, 322
 preserved and nonpreserved, 322–324
 RegSrc, 402
 Regularity, 6
 RegWrite, 393, 433
 Replacement policies, 516

Resettable flip-flops, 116
 Resettable registers, 194–196
 Resolution time, 151–152. *See also* Metastability
 derivation of, 154–157
 Return value, 317
 RF. *See* Register file (RF)
 Ring oscillator, 119, 119
 Ripple-carry adder, 240, 240–241, 243
 RISC architecture. *See* Reduced instruction set computer (RISC) architecture
 Rising edge, 88
 Rm field, 330
 Rn field, 330
 ROM. *See* Read only memory (ROM)
 ROR, 304
 rot field, 330–331
 Rotations per minute (RPM), 531.e44
 Rotators, 251–252
 Rounding modes, 259
 RPM. *See* Rotations per minute (RPM)
 RS-232, 531.e18

S

Sampling, 141
 Sampling rate, 531.e25
 SATA. *See* Serial ATA (SATA)
 Saturated arithmetic, 353
 Scalar processor, 461–463, 460
 Scan chains, 262–263
 scanf, 541.e38
 Scannable flip-flop, 262–263
 Schematics, rules of drawing, 31, 67
 SCK. *See* Serial Clock (SCK)
 SDI. *See* Serial Data In (SDI)
 SDO. *See* Serial Data Out (SDO)
 SDRAM. *See* Synchronous dynamic random access memory (SDRAM)
 Segment descriptor, 367
 Segmentation, 367
 Selected signal assignment statements, 182
 Semiconductors, 27
 industry, sales, 3
 Sequencing overhead, 143–144, 149, 160, 442
 Sequential building blocks. *See* Sequential logic

- Sequential logic, 109–161, 259–263
 - counters, 260
 - finite state machines. *See* Finite state machines (FSMs)
 - flip-flops, 114–118. *See also* Registers
 - latches, 111–113
 - D, 113
 - SR, 111–113
 - registers. *See* Registers
 - shift registers, 261–263
 - timing of. *See* Timing analysis
- Serial ATA (SATA), 531.e62
- Serial Clock (SCK), 531.e12
- Serial communication, with PC, 531.e20
- Serial Data In (SDI), 531.e12
- Serial Data Out (SDO), 531.e12
- Serial I/O, 531.e11–531.e23
 - SPI. *See* Serial peripheral interface (SPI)
 - UART. *See* Universal Asynchronous Receiver Transmitter (UART)
- Serial Peripheral Interface (SPI), 531.
 - e11, 531.e12–531.e17
 - connection between PI and FPGA, 531.e14
 - ports
 - Serial Clock (SCK), 531.e12
 - Serial Data In (SDI), 531.e12
 - Serial Data Out (SDO), 531.e12
 - register fields in, 531.e13
 - slave circuitry and timing, 531.e15
 - waveforms, 531.e12
- Servo motor, 531.e44, 531.e48–531.e49
- Set bits, 495
- Setup time constraint, 142, 145–147
 - with clock skew, 148–150
- Seven-segment display decoder, 79–82
 - with don't cares, 82–83
 - HDL for, 201–202
- Shaft encoder, 531.e43, 531.e47–531.
 - e48, 531.e48
- Shift instructions, 304–305, 305
- Shift registers, 261–263
- Shifters, 251–252
- Short path, 89–92
- Sign bit, 16
- Sign extension, 18
- Sign/magnitude numbers, 15–16, 256
- Signed binary numbers, 15–19
- Signed multiplier, 217
- Silicon dioxide (SiO₂), 28
- Silicon lattice, 27
- SIMD. *See* Single instruction multiple data (SIMD)
- SIMD instructions, 358–360
 - simple function, 318
- Simple programmable logic devices (SPLDs), 274
- Simulation waveforms, 176
 - with delays, 189
- Single instruction multiple data (SIMD), 460, 472
- Single-cycle ARM processor, 390, 444
 - Conditional Logic, 447–448
 - control, 397–401
 - controller, 445
 - datapath, 390, 448–449
 - B instruction, 396–397
 - data-processing instructions, 395–396
 - LDR instruction, 391–394
 - STR instruction, 394–396
 - Decoder, 446
 - instructions, 402
 - performance, 402–405
- Single-cycle microarchitecture, 388
- Single-precision formats, 258. *See also* Floating-point numbers
- Skew. *See* Clock skew
- Slash notation, 56
- Slave latch, 114. *See also* Flip-flops
- Small-scale integration (SSI) chips, 533.
 - e2
- Solid state drive (SSD), 490. *See also* Flash memory, Hard drive
- SOP. *See* Sum-of-products (SOP) form
- Spatial locality, 488, 500–502
- Spatial parallelism, 157–158
- SPEC, 389
- SPECINT2000, 424
- SPI. *See* Serial Peripheral Interface (SPI)
- Squashing, 465
- SR latches, 111–113, 112
- SRAM. *See* Static random access memory (SRAM)
- srand, 541.e40–541.e41
- Src2 field, 330, 333
- SSI chips. *See* Small-scale integration (SSI) chips
- Stack, 320–329. *See also* Function calls
 - during recursive function call, 326–328
 - preserved registers, 322–324
- stack frame, 322, 328
- stack pointer (SP), 320
- storing additional arguments on, 328–329
- storing local variables on, 328–329
- Stalls, 435–436. *See also* Hazards
- Standard libraries, 541.e35–541.e43
 - math, 541.e42–541.e43
 - stdio, 541.e35–541.e40
 - file manipulation, 541.e38–541.e40
 - printf, 541.e35–541.e37
 - scanf, 541.e38
 - stdlib, 541.e40–541.e42
 - exit, 541.e41
 - format conversion (atoi, atol, atof), 541.e41–541.e42
 - rand, srand, 541.e40–541.e41
 - string, 541.e43
- State encodings, FSM, 129–131, 134. *See also* Binary encoding, One-cold encoding, One-hot encoding
- State machine circuit. *See* Finite state machines (FSMs)
- State variables, 109
- Static branch prediction, 459
- Static discipline, 24–26
- Static power, 34
- Static random access memory (SRAM), 266, 267, 519
- Status flags, 363. *See also* Condition flags
- stdio.h, C library, 541.e35–541.e40. *See also* Standard libraries
- stdlib.h, C library, 541.e40–541.e42. *See also* Standard libraries
- Stepper motors, 531.e44, 531.e49–531.
 - e53
 - bipolar stepper motor, 531.e49, 531.e50–531.e52
 - half-step drive, 531.e50, 531.e51
 - two-phase-on drive, 531.e50, 531.e51
 - wave drive, 531.e52–531.e53
- Stored program, 337–338
- STR, 394–396
- string.h, C library, 541.e43
- Strings, 316–317, 541.e28–541.e29. *See also* Characters (char)
- Structural modeling, 173–174, 190–193
- Structures (struct), 541.e29–541.e31

SUB, 297
 Substrate, 28–29
 Subtraction, 17, 246, 297
 Subtractor, 246–247
 Sum-of-products (SOP) form, 58–60
 Superscalar processor, 461–463
 Supervisor call (SVC) instruction, 349
 Supply voltage, 22. *See also* V_{DD}
 SVC. *See* Supervisor call (SVC) instruction
 Swap space, 516
 switch/case statements
 in ARM assembly, 311–312
 in C, 541.e17–541.e18
 in HDL. *See* case statement, in HDL
 Symbol table, 342, 343
 Symmetric multiprocessing (SMP), 468.
 See also Homogeneous multiprocessors
 Synchronizers, 152–154, 152–153
 Synchronous circuits, 122–123
 Synchronous dynamic random access memory (SDRAM), 268
 DDR, 268
 Synchronous logic, design, 119–123
 Synchronous resettable flip-flops, 116
 Synchronous sequential circuits,
 120–123, 122. *See also* Finite state machines (FSMs)
 timing specification. *See* Timing analysis
 SystemVerilog, 173–225. *See also*
 Hardware description languages (HDLs)
 accessing parts of busses, 188, 192
 bad synchronizer with blocking assignments, 209
 bit swizzling, 188
 blocking and nonblocking assignment, 199–200, 205–208
 case statements, 201–202, 205
 combinational logic using, 177–193, 198–208, 217–220
 comments, 180
 conditional assignment, 181–182
 data types, 213–217
 decoders, 202–203, 219
 delays (in simulation), 189
 divide-by-3 FSM, 210–211
 finite state machines (FSMs), 209–213
 Mealy FSM, 213

Moore FSM, 210, 212
 full adder, 184
 using always/process, 200
 using nonblocking assignments, 208
 history of, 175
 if statements, 202–205
 internal signals, 182–184
 inverters, 178, 199
 latches, 198
 logic gates, 177–179
 multiplexers, 181–183, 190–193, 218–219
 multiplier, 217
 numbers, 185–186
 operators, 185
 parameterized modules, 217–220
 $N:2^N$ decoder, 219
 N -bit multiplexers, 218–219
 N -input AND gate, 220
 priority circuit, 204
 using don't cares, 205
 reduction operators, 180–181
 registers, 193–197
 enabled, 196
 resettable, 194–196
 sequential logic using, 193–198, 209–213
 seven-segment display decoder, 201
 simulation and synthesis, 175–177
 structural models, 190–193
 synchronizer, 197
 testbench, 220–224
 self-checking, 222
 simple, 221
 with test vector file, 223–224
 tristate buffer, 187
 truth tables with undefined and floating inputs, 187, 188
 z's and x's, 186–188, 205

T

Tag, 495
 Taking the two's complement, 16–17
 Temporal locality, 488, 493–494, 497, 502
 Temporal parallelism, 158–159
 Temporary registers, 299

Ternary operators, 181, 541.e13
 Testbench, 452–456
 Testbenches, HDLs, 220–224
 self-checking, 221–222
 simple, 220–221
 with testvectors, 222–224
 Text Segment, 340, 344
 Thin small outline package (TSOP), 533.e17
 Thread level parallelism (TLP), 467
 Threshold voltage, 29
 Throughput, 157–160, 388, 425, 468
 Thumb instruction set, 351–352
 Timers, 531.e23–531.e24
 Timing
 of combinational logic, 88–95
 delay. *See* Contamination delay;
 Propagation delay
 glitches. *See* Glitches
 of sequential logic, 141–157
 analysis. *See* Timing analysis
 clock skew. *See* Clock skew
 dynamic discipline, 141–142
 metastability. *See* Metastability
 resolution time. *See* Resolution time
 time
 system timing. *See* Timing analysis
 Timing analysis, 141–151
 calculating cycle time. *See* Setup time constraint
 with clock skew. *See* Clock skew
 hold time constraint. *See* Hold time constraint
 max-delay constraint. *See* Setup time constraint
 min-delay constraint. *See* Hold time constraint
 multicycle processor, 424
 pipelined processor, 441
 setup time constraint. *See* Setup time constraint
 single-cycle processor, 405
 TLB. *See* Translation lookaside buffer (TLB)
 TLP. *See* Thread level parallelism (TLP)
 Transistors, 26–34
 bipolar, 26
 CMOS, 26–33
 gates made from, 31–34
 latches and flip-flops, 116–117
 MOSFETs, 26

Transistors (*Continued*)
 nMOS, 28–34, 29–33
 pMOS, 28–34, 29–33
 pseudo-nMOS, 33–34
 ROMs and PLAs, 279–280
 transmission gate, 33
 Transistor-Transistor Logic (TTL),
 25–26, 533.e15–533.e16
 Translating and starting a program,
 339
 Translation lookaside buffer (TLB),
 514–515
 Transmission Control Protocol and
 Internet Protocol (TCP/IP), 531.
 e61
 Transmission gates, 33
 Transmission lines, 533.e20–533.e33
 characteristic impedance (Z_0), 533.
 e30–533.e31
 derivation of, 533.e30–533.e31
 matched termination, 533.e22–533.
 e24
 mismatched termination, 533.
 e25–533.e28
 open termination, 533.e24–533.e25
 reflection coefficient (k_r), 533.
 e31–533.e32
 derivation of, 533.e31–533.e32
 series and parallel terminations, 533.
 e28–533.e30
 short termination, 533.e25
 when to use, 533.e28
 Transparent latch. *See* Latches: D
 Traps, 347
 Tristate buffer, 74–75, 187
 HDL for, 186–187
 multiplexer built using, 84–85,
 91–93
 True, 8, 20–22, 58–59, 70, 74,
 111–112, 116, 129, 176, 180,
 205
 Truth tables, 20
 ALU decoder, 399, 404
 with don't cares, 69, 81–83, 205
 multiplexer, 83
 seven-segment display decoder, 79
 SR latch, 111, 112
 with undefined and floating inputs,
 187–188
 TSOP. *See* Thin small outline package
 (TSOP)

TTL. *See* Transistor-Transistor Logic
 (TTL)
 Two's complement numbers, 16–18
 Two-bit dynamic branch predictor, 460
 Two-cycle latency of LDR, 435
 Two-level logic, 69
 typedef, 541.e31–541.e32

U

UART. *See* Universal Asynchronous
 Receiver Transmitter (UART)
 Unconditional branches, 308, 309
 Undefined instruction exception, 347
 Unicode, 315
 Unit under test (UUT), 220
 Unity gain points, 24
 Universal Asynchronous Receiver
 Transmitter (UART), 531.
 e17–531.e23
 hardware handshaking, 531.e18
 Universal Serial Bus (USB), 270, 531.
 e18, 531.e59
 USB 1.0, 531.e59
 USB 2.0, 531.e59
 USB 3.0, 531.e59
 Unsigned multiplier, 217, 252–253
 Unsigned numbers, 18
 Upton, Eben, 531.e4
 USB. *See* Universal Serial Bus (USB)
 USB links, 531.e63–531.e64
 FTDI, 531.e63
 UM232H module, 531.e64
 Use bit (U), 502

V

Valid bit (V), 496
 Variables in C, 541.e7–541.e11
 global and local, 541.e9–541.e10
 initializing, 541.e11
 primitive data types, 541.e8–541.e9
 V_{CC} , 23. *See also* Supply voltage, V_{DD}
 V_{DD} , 22, 23. *See also* Supply voltage
 Vector processor, 460
 Verilog. *See* SystemVerilog

Very High Speed Integrated Circuits
 (VHSIC), 175. *See also* VHSIC
 Hardware Description Language
 (VHDL)
 VGA (Video Graphics Array) monitor,
 531.e36–531.e42
 connector pinout, 531.e37
 driver for, 531.e39–531.e42
 VHDL. *See* VHSIC Hardware
 Description Language (VHDL)
 VHSIC. *See* Very High Speed Integrated
 Circuits (VHSIC)
 VHSIC Hardware Description Language
 (VHDL), 173–175
 accessing parts of busses, 188, 192
 bad synchronizer with blocking
 assignments, 209
 bit swizzling, 188
 blocking and nonblocking assignment,
 199–200, 205–208
 case statements, 201–202, 205
 combinational logic using, 177–193,
 198–208, 217–220
 comments, 180
 conditional assignment, 181–182
 data types, 213–217
 decoders, 202–203, 219
 delays (in simulation), 189
 divide-by-3 FSM, 210–211
 finite state machines (FSMs),
 209–213
 Mealy FSM, 213
 Moore FSM, 210, 212
 full adder, 184
 using always/process, 200
 using nonblocking assignments,
 208
 history of, 175
 if statements, 202
 internal signals, 182–184
 inverters, 178, 199
 latches, 198
 logic gates, 177–179
 multiplexer, 181–183, 190–193,
 218–219
 multiplier, 217
 numbers, 185–186
 operators, 185
 parameterized modules, 217–220
 $N:2^N$ decoder, 219
 N -bit multiplexers, 218, 219
 N -input AND gate, 220, 220

priority circuit, 204
 reduction operators, 180–181
 using don't cares, 205
 reduction operators, 180–181
 registers, 193–197
 enabled, 196
 resettable, 194–196
 sequential logic using, 193–198,
 209–213
 seven-segment display decoder,
 201
 simulation and synthesis, 175–177
 structural models, 190–193
 synchronizer, 197
 testbench, 220–224
 self-checking, 222
 simple, 221
 with test vector file, 223–224
 tristate buffer, 187
 truth tables with undefined and
 floating inputs, 187, 188
 z's and x's, 186–188, 205
 Video Graphics Array (VGA). *See* VGA
 (Video Graphics Array) monitor
 Virtual address, 509
 space, 515
 Virtual memory, 490, 508–518
 address translation, 509–512
 cache terms comparison, 510
 memory protection, 515
 multilevel page tables, 516–518
 page fault, 509–510
 page number, 511
 page offset, 511
 pages, 509
 page table, 512–513

replacement policies, 516
 translation lookaside buffer (TLB),
 514–515
 write policy, 506–507
 Virtual page number (VPN), 512
 Virtual pages, 509
 V_{SS}, 23

W

Wafers, 28
 Wait for event (WFE) instruction, 358
 Wait for interrupt (WFI) instruction, 358
 Wall, Larry, 20
 WAR hazard. *See* Write after read
 (WAR) hazard
 WAW hazard. *See* Write after write
 (WAW) hazard
 Weak pull-up, 33
 Weird number, 18
 WFE. *See* Wait for event (WFE)
 instruction
 WFI. *See* Wait for interrupt (WFI)
 instruction
 while loops, 312, 541.e19
 White space, 180
 Whitmore, Georgiana, 7
 Wi-Fi, 531.e61
 Wilson, Sophie, 472
 Wire, 67
 Wireless communication, Bluetooth,
 531.e42–531.e43
 Wordline, 264

Write after read (WAR) hazard, 464.
 See also Hazards
 Write after write (WAW) hazard,
 464–465
 Write policy, 506–507
 write-back, 506–507
 write-through, 506–507

X

X. *See* Contention (x); Don't care (X)
 x86
 architecture, 360–368, 362
 big picture, 368
 branch conditions, 366
 instruction encoding, 364–367
 instructions, 364–367
 memory addressing modes,
 363
 operands, 362–363
 peculiarities, 368
 registers, 362
 status flags, 363
 Xilinx FPGA, 275
 XNOR gate, 21–22
 XOR gate, 21

Z

Z. *See* Floating (Z)